

前言 / Introduction

Introduction(引言)

PART I · Foundations: The Three Pillars

1. Why Claude Code Alone Isn't Enough
2. The Anatomy of Claude Code
3. Skills, Subagents, and Hooks: The Three-Pillar Framework

PART II · Skills: Claude's Extensible Knowledge System

4. Skills Fundamentals: The SKILL.md Anatomy
5. Writing Your First Skill: A Hands-On Build
6. Skill Description Engineering: The Art of Triggering

Claude Code Architect · 双语版

Source: Andrei, Patrick. *Claude Code Architect: Master SKILL.md, Subagents, and Hooks That Transform Claude Code Into a Fully Engineered, Production-Ready System*. 2026. · 中文逐句对照翻译 · 共 242 页

INTRODUCTION · 引言

You are holding a book about a tool that did not exist three years ago. Claude Code shipped to general availability in early 2025, and within twelve months it had transformed how a substantial slice of working software engineers spend their days. The transformation is still in progress. Most engineers using Claude Code today are using it the way most people use a new tool: by typing requests at it and accepting whatever it produces. The handful of engineers who have moved past that posture, who have learned to engineer their environment rather than merely use it, are working at a different level of leverage entirely.

你手中这本书所讲的那个工具,三年前还不存在。Claude Code 于 2025 年初正式发布(GA),并在短短十二个月内,改变了相当一部分在软件工程师的日常工作方式。这一变革至今仍在持续。如今大多数使用 Claude Code 的工程师,用它的方式正和大多数人对新工具的态度一样:敲一段请求,然后接受它吐出的任何结果。而少数已经跨过这道坎的工程师——他们不再只是"用"工具,而是学会"工程化"自己的环境——所发挥出的杠杆效率,根本不在同一个量级。

This book is for the engineers who want to make that move. The ones who have noticed that their session quality varies wildly day to day. The ones who have wished they could teach Claude their team's conventions just once and have those conventions stick. The ones who have watched Claude make the same kind of mistake three times in one afternoon and wondered whether there is a way to make it stop. The ones who suspect that the tool is capable of more than they are getting from it, and who are tired of suspecting and want to start engineering.

这本书写给的就是想迈出这一步的工程师:那些察觉到自己每次会话的质量日复一日忽高忽低的人;那些希望能"教 Claude 一次团队规约,就让它一直守规矩"的人;那些眼见 Claude 一个下午犯三次同类错误、想知道有没有办法让它停下来的人;那些已经隐隐感觉这工具能发挥出远超当下表现的威力,却厌倦了只是怀疑、想要开始动手工程化的人。

The argument of this book is that Claude Code, properly understood, is not a chat interface. It is a programmable substrate with three distinct customization mechanisms, and the engineers who learn to use those mechanisms in combination produce results that engineers using the chat interface alone cannot match. The three mechanisms have specific names. Skills are units of reusable capability that teach Claude how to do specific kinds of work consistently. Subagents are isolated specialist contexts that handle particular roles in parallel. Hooks are deterministic shell commands that fire at lifecycle moments and impose mechanical guarantees the model cannot evade. Each of these is documented in Anthropic's official materials. None of them, individually, is the secret. The secret, if there is one, is in their composition.

本书的核心论点是:真正理解之后,Claude Code 不是一个聊天界面。它是一个可编程的底层平台(substrate),自带三种截然不同的定制机制;而学会把这三种机制组合使用的工程师,所能产出的成果,是"只用聊天界面"的工程师无论如何也达不到的。这三种机制各有其名。**Skills(技能)**是可复用的能力单元,教 Claude 如何一致地完成某类工作。**Subagents(子代理)**是隔离的专家上下文,并行处理特定角色。**Hooks(钩子)**是在生命周期节点触发的确定性 shell 命令,用来施加模型无法规避的机械性约束。这三者,每一项在 Anthropic 官方文档里都有介绍。但单独任何一项,都不是秘诀所在。如果有秘诀,那秘诀在它们的**组合**。

What this book teaches, across twenty-two chapters, is the architecture for that composition. Part One lays out the framework. Parts Two through

全书二十二章,讲的就是这套"组合"的架构。第一部分铺陈整体框架;第二至

Four go deep into each pillar. Part Five shows the composed architecture at work in two full case studies, one for a senior engineer at a small product company and one for a solo SaaS founder running an entire business out of a single Claude Code installation. Part Six closes with the practical concerns of shipping, sharing, and evolving an environment over time. The four appendices give you reference material to come back to.

四部分深入每一根支柱;第五部分通过两个完整案例展示组合架构的实战——一个面向小型产品公司的高级工程师,一个用一套 Claude Code 撑起整个生意的独立 SaaS 创始人;第六部分收束于发布、共享、长期演进这些工程化议题;四个附录则是你回头查阅的参考材料。

A short word about who this book is not for. If you are looking for a quick introduction to Claude Code, this is not it. The official Anthropic quickstart is better for that purpose, and it is free. If you are looking for a tour of every feature in the product, this is not that either. The book covers the three customization pillars in depth and treats other features only when they intersect with the three pillars. If what you want is a comprehensive feature catalog, the official documentation is more current than any book can be, and it should be your first stop.

简单说一句"这本书不适合谁"。如果你在找一份 Claude Code 的快速入门,这本书不是——官方的 Anthropic quickstart 更适合这个目的,而且免费。如果你想要一份"产品所有特性"的导览,这本也不是。本书深入讲的是"三根定制支柱",只在与其它特性与这三根支柱相交时才会顺带提及;如果你想要的是"完整的特性目录",官方文档永远比任何书都新,那应该是你的第一站。

What this book offers, that the documentation does not, is a coherent way of thinking about how the pieces fit together. The documentation explains each piece accurately. It does not, and cannot, explain why the three pieces sit at different layers of the architecture, what kinds of failures each pillar prevents, in what order to build them, or how to compose them into a working environment. That synthesis is what this book exists to provide. The pieces are public. The architecture is what you are paying for.

本书提供、而文档没提供的,是一种"这些碎片到底如何拼起来"的连贯思维方式。文档能精确解释每一块;但它无法、也无意去解释:为什么这三块在架构里位于不同层级?每根支柱各防住了哪类失败?应该按什么顺序去搭?又该怎样把它们组合出一个"能跑起来"的环境?本书

存在的全部意义,就是提供这种"综合"。那些碎片是公开的;你愿意为这本书付费,买的是这套架构。

A note on how to read the book. The chapters are designed to be read in order, because each pillar builds on the conceptual vocabulary established in the previous one. A reader who jumps directly to the hooks chapters before reading the skills chapters will find the hooks discussion harder than it needs to be, because the deterministic-versus-probabilistic framing that organizes the hooks treatment is set up earlier. That said, after a first read, the book is meant to be returned to non-linearly. The appendices are reference material. The case studies are templates to come back to when you are designing your own environment. Many readers will read the book once cover to cover, then keep it within reach as they build, opening it to specific sections when specific design questions surface.

关于怎么读这本书,多说一句:各章是按顺序写的——因为每根支柱都建立在前面那一根所确立的"概念词汇"之上。如果一位读者跳过 skills 直接去看 hooks,他/她会觉得 hook 那几章比实际更难看懂——因为贯穿 hook 部分的"确定性 vs 概率性"那套框架,是前面就搭好的。当然,在第一遍之后,这本书就是一本"可以非线性回头翻"的书。附录是参考材料;案例研究是"等你设计自己的环境时回来翻的模板"。许多读者会先把整本书通读一遍,然后把它放在伸手够得到的地方;真正开始搭建的时候,遇到具体的设计问题再翻开对应的小节。

A note about pronouns. The hypothetical developer throughout the book is referred to with feminine pronouns. This is a deliberate choice and is meant to be read as one. Software engineering is more diverse than its

关于代词的说明:全书那位假想中的"开发者",统一使用她。这是一项经过思考的、有意为之的选择,读到这里请按其本意来读。软件工程比它通常被描绘的代词所指代的群体,要更多元。

conventional pronouns suggest, and a book written in 2026 ought to reflect that reality. Readers who find this jarring may want to take a moment to consider why. The book itself proceeds without further comment.

一本写于 2026 年的书,理应反映这一现实。如果有读者觉得这种写法有些刺眼,不妨花一点时间想想"为什么";本书对此不再赘述。

A note about the present moment. This book is being written and shipped at a particular point in the evolution of AI-assisted development tooling. The specific features the book describes are accurate as of early 2026. Some of those features will change. New features will appear. The framework, however, is meant to be durable, because it is rooted in the structure of engineering work itself rather than in the specifics of any particular tool. A reader returning to this book in 2028 will find that some of the syntactic details have moved on, but that the architectural principles still apply, because the underlying division of capability, specialization, and enforcement is a property of the work and not of the tool.

关于"此刻"的几句话:本书写作与面世,正逢 AI 辅助开发工具演化的某个特定节点。本书描述的那些具体特性,在 2026 年初是准确的;其中一些会变,新特性也会出现。然而,这套框架的意图是耐久的——因为它根植于"工程工作本身的结构",而不是任何特定工具的细节。等 2028 年回头再翻这本书的读者,会看到:一些语法细节已经过时了;但架构层面的原则仍然成立——因为"能力的划分、专门化的划分、强制的划分"那套底层结构,是"工作"本身的属性,不是"工具"的属性。

Now to the work. The first chapter opens at the moment most engineers experience as the trigger for taking their tooling more seriously: a Friday afternoon, three different kinds of failure in quick succession, and the dawning recognition that the way they have been using the tool is not going to scale to the work they actually need to do.

现在开工。第一章,会从大多数工程师"开始认真对待自己工具"那一刻切入:一个周五的下午,接连发生的、互不相同的几类失败,以及"我一

直在用的方式,撑不起我真正要做的工作"这一正在浮出水面的认知。

PART ONE · 第一部分

Foundations: The Three Pillars of Claude Code Architecture

基础:Claude Code 架构的三根支柱

The gap between using Claude Code and engineering with Claude Code.

"使用 Claude Code"与"用 Claude Code 做工程"之间的距离。

CHAPTER ONE · 第一章

Why Claude Code Alone Isn't Enough

为什么单靠 Claude Code 远远不够

"Tools are not merely aids to the hand. They are extensions of the mind, and the mind that fails to shape them is shaped by them instead."

— Marshall McLuhan, paraphrased

"工具绝不仅仅是手的延伸,更是心智的延伸;不去塑造工具的心智,反过来必被工具所塑造。"

—— 马歇尔·麦克卢汉(转述)

There is a specific moment, familiar to anyone who has lived with Claude Code for more than a few weeks, when the honeymoon ends. The tool still feels magical. The responses are still sharp. But the reader begins to notice something quieter underneath the surface: the same instructions are being typed again. The same three warnings are being issued before every refactor. The same linting step is being requested after every edit. The same four sentences of project context are being pasted into the first message of every new session. The magic is still there, but it has started to feel like a magic that requires a great deal of bookkeeping.

凡是跟 Claude Code 相处过几周以上的人,都会熟悉一个特定时刻——蜜月期结束的那一刻。工具依然令人惊艳,响应依然敏锐。但在表面之下,读者开始察觉到一些更安静的迹象:同一段指令在反复输入;同一套三次警告在每次重构前都重新出现;同一步 lint 在每次编辑后都被重新请求;同一段四句长的项目背景在每个新会话开头被原封不动地粘贴。魔力依然在,但它已经开始变成一种需要大量"簿记工作"才能维持的魔力。

This book was written for that moment. Not the moment of discovery, when Claude Code first opens in a terminal and writes a working function from a vague sentence. Not the moment of curiosity, when the reader begins to explore what the tool can and cannot do. The moment this book was written for is the moment of friction. The moment when the gap between using the tool and being mastered by the tool becomes visible, and when the reader begins to suspect, correctly, that there must be a better way.

这本书就为那个时刻而写。不是"发现"那一刻——Claude Code 第一次在终端里被打开、从一句模糊的话写出能跑的函数;也不是"好奇"那一刻——读者开始试探这工具能做什么、不能做什么。本书写的是摩擦那一刻:当你看清了"用工具"和"被工具所用"之间的鸿沟,并开始正确地怀疑——一定还有更好的方式。

The better way is not a better prompt. It is not a longer `CLAUDE.md` file. It is not a collection of tricks. The better way is a shift in how the reader relates to the tool itself: from a user of Claude Code to an engineer of Claude Code. This shift is the central promise of the book, and this chapter is where that promise gets defined.

那条更优的路,不是写一段更好的 prompt,也不是把 `CLAUDE.md` 写得更长,更不是堆一摞小技巧。所谓更优,是关系本身的转变:从 Claude Code 的使用者,变成 Claude Code 的工程师。这一转变,是本书最核心的承诺;而这一章,正是把那个承诺明确下来的地方。

Consider a concrete scene. A developer sits down on a Tuesday morning to continue work on a medium-sized web application. She opens Claude Code. The first thing she types, as she has typed roughly forty times before, is a reminder that this codebase uses TypeScript with strict mode, that components should be written as functional components with hooks, that tests are written in Vitest rather than Jest, that commit messages follow the conventional commits format, and that any changes to the authentication layer require her personal review. She types this reminder because she has learned, through painful experience, that if she does not type it, Claude Code will occasionally produce perfectly reasonable code that violates every one of those rules.

请设想一个具体场景。一位开发者在某个周二上午坐下,继续开发一个中等规模的 Web 应用。她打开 Claude Code,敲下的第一段话——和之前大约四十次一样——是重新提醒一遍:这个代码库使用 TypeScript 的 strict 模式;组件必须写成带 hooks 的函数式组件;测试用 Vitest,不要用 Jest;commit message 遵守 conventional commits 规范;鉴权层的任何修改必须经她本人复核。她之所以要打这一遍,是因为她吃过太大亏才学到的:不打这一段,Claude Code 偶尔会写出一段"看起来很合理、却把所有规则都违反一遍"的代码。

She then asks Claude Code to add a new feature. The code appears. It is, as always, impressive. She reads it. She notices that the test file is missing, because she forgot to ask for it. She asks for tests. The tests appear.

接着她让 Claude Code 加一个新功能。代码出来了,一如既往地漂亮。她通读一遍,发现测试文件缺了——因为她刚才忘了开口要。她补上"给我测试",测试出来了。

She notices that one test uses a matcher that exists in Jest but not Vitest. She corrects Claude, who apologizes and fixes it. She commits the change. She notices, twenty minutes later, that her commit message does not follow the conventional commits format, because she wrote it herself in a hurry. She amends the commit.

她发现,其中一个测试用到的 matcher 只在 Jest 里有,Vitest 没有。她纠正 Claude,Claude 道了歉、改掉了。她提交了改动。二十分钟后,她注意到自己的 commit message 并不符合 conventional commits 规范——那是她刚才匆忙之下自己写的。她又补了一次 commit。

The Working Developer's Shape · 普通开发者的早晨

This is a good developer. She is getting real work done. Claude Code is, objectively, saving her hours. And yet, look closely at the shape of her morning. It is full of tiny, avoidable frictions. Instructions she typed that should have been remembered. Checks she ran that should have fired

automatically. Specialists she consulted in her own head, one after another, because her tool did not know how to consult them for her. She is doing the work that a well-engineered environment should be doing on her behalf.

这是一位很靠谱的开发者的。她确实产出真实的工作。客观地讲,Claude Code 替她省下了大量时间。然而,若细细打量她上午的工作流,你会看到:处处都是细碎的、本可避免的摩擦——本该被记住的指令,本该自动触发的检查,本该由"专家"来回答的问题——这些她都得在自己脑子里一个个处理,因为她的工具不知道如何替她去请那些"专家"。她正在替那个本应被工程化好的环境,做着它本该做的事。

Now imagine the same morning in a different configuration of the same tool. She opens Claude Code. She does not type a reminder about TypeScript or Vitest or commit messages, because the environment already knows. When she asks for a new feature, the code appears with tests, because a skill for writing features in this codebase has been installed and knows that tests are part of the definition of done. Before anything is written to disk, a pre-edit hook silently checks the file path against the codebase's protected paths. After the code is written, a post-edit hook runs the linter and the type checker, and if they fail, Claude Code sees the failure and corrects the code before returning control to her. When she is ready to commit, a pre-commit hook runs the full test suite in the background, and a subagent dedicated to commit messages produces a conventional commit message from the diff. She reviews the message, accepts it, and moves on.

现在,请想象同一个上午、同一套工具,但环境配置完全不同。她打开 Claude Code,却不再需要重复提醒 TypeScript、Vitest、commit message 这些——因为环境已经知道了。她提一个新功能,代码连同测试一起出现,因为已经安装了一个"为本代码库写功能"的 skill,它知道"完成"里包含测试。文件落到磁盘之前,有一个 pre-edit hook 默默核对路径,确认没碰受保护目录。代码写完后,有一个 post-edit hook 跑 linter 和类型检查器;若失败,Claude Code 自己看到失败,先改好,再把控制权交还给她。她准备提交时,一个 pre-commit hook 在后台跑完整套测试;一个专门负责 commit message 的 subagent 根据 diff 写出一段 conventional commit 信息。她浏览一遍,通过,然后继续。

The work done in this second configuration is the same work. The tool is the same tool. What has changed is the scaffolding around it. The developer has stopped being the carrier of every rule and every check, and has become, instead, the supervisor of a small, patient, consistent system that carries those rules and checks on her behalf.

两种配置下,做的事情是同一件事,工具是同一个工具。变的,是工具周围的那层脚手架。开发者不再背负每一条规则、每一道检查;她变成了一个小型、耐心、始终如一的系统的监督者,由那个系统替她背着这些规则和检查。

The distance between those two mornings is the distance between using Claude Code and engineering with it. And that distance is not a matter of talent, intelligence, or time spent with the tool. It is a matter of knowledge. The second developer knows something the first one does not. She knows that Claude Code is not a chat interface to a language model. She knows that Claude Code is a programmable environment, and she knows the three systems that make the programming possible.

这两个上午之间的距离,就是"使用 Claude Code"和"用 Claude Code 做工程"之间的距离。而这段距离,与天赋、智商、与工具相处的时间都无关。它是认知的差距。第二位开发者知道一些第一位不知道的事:Claude Code 不是通往语言模型的聊天界面;Claude Code 是一个可编程的环境;并且,她了解让这种编程得以成立的那三套系统。

that Claude Code is not a chat interface to a language model. She knows that Claude Code is a programmable environment, and she knows the three systems that make the programming possible.

知道 Claude Code 不是一个通往语言模型的聊天界面;知道 Claude Code 是一个 *可编程的环境*;并且,她了解让这种编程得以成立的那三套系统。

Those three systems are Skills, Subagents, and Hooks. Every power-user setup, every company-wide Claude Code deployment, every open-source configuration pack that developers are quietly building and trading in private Slacks right now, is built out of some combination of these three primitives. They are the primitives of Claude Code mastery. They are also, surprisingly, the primitives that the overwhelming majority of Claude Code users have never consciously touched.

这三套系统,就是 Skills、Subagents、Hooks。每一个高阶用户的配置、每一次企业级的 Claude Code 部署、每一份开发者们正在悄悄搭起来、并在私有 Slack 群里互通有无的开源配置包,都是由这三种原语的某种组合搭建出来的。它们是掌握 Claude Code 的"原语"。而令人惊讶的是:绝大多数 Claude Code 的用户,从未有意识地触碰过它们中的任何一个。

This is the strange fact at the center of the tool. Anthropic shipped Claude Code with all three of these systems built in from an early stage. The documentation exists. The configuration files are well-designed. The examples in the official repositories are good. And yet, walking into any developer forum or any Claude Code community channel, the overwhelming pattern of conversation is about prompt wording. About trying different `CLAUDE.md` phrasings. About sharing clever one-liners. The three systems that could transform a workflow go almost entirely unmentioned, because they sit one level below the surface, and almost no one has taken the time to descend.

这就是这个工具中央那个反常的事实。Anthropic 在 Claude Code 早期就把这三套系统全部内置了。文档在;配置文件设计得不错;官方仓库里的样例也写得到位。然而,随便走进任何一个开发者论坛或任何一个 Claude Code 社区频道,话题压倒性地围着 prompt 措辞打转——尝试不同的 `CLAUDE.md` 写法,分享巧妙的一行 prompt。其实可以彻底改变工作流的那三套系统,几乎完全无人提及,因为它们藏在表面之下整整一层,几乎没人愿意花时间潜下去看一眼。

Three Systems, One Architecture · 三个系统,一套架构

The reader is about to descend. But first, it is worth naming the three systems clearly, so that every subsequent chapter has a stable vocabulary to refer back to.

读者即将潜下去。但在进入正文之前,有必要把这三套系统的名字讲清楚——这样后续每一章都会有一个稳定可援引的词汇表。

A Skill is a reusable capability. It is a folder containing a `SKILL.md` file, and possibly supporting scripts or reference material, that teaches Claude how to perform a specific task consistently. When a user asks Claude to perform that task, Claude recognizes the request, reads the skill, and carries it out according to the skill's definition. Skills are how a developer stops repeating themselves. They are the mechanism by which institutional knowledge enters the environment and stays there.

Skill(技能)是一种可复用的能力。它是一个目录,里面有一个 `SKILL.md` 文件,可能还附带支持脚本或参考材料,用来教 Claude 如何一致地完成一项特定任务。当用户请 Claude 做这件事时,Claude 识别出请求,读取该 skill,按其定义执行。Skill,是开发者停止重复自己的方式;也是组织知识进入这个环境、并长期留存的机制。

A Subagent is a specialized parallel worker. It is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window. The main Claude session delegates work to a

Subagent(子代理)是一个专门化、可并行的工作者。它是一个独立的 Claude 进程,有自己的系统提示、受限的工具集,以及隔离的上下文窗口。主会话把工作委派给一个

subagent, the subagent performs the work, and only the final result comes back. Subagents are how a single developer becomes the orchestrator of a small team of specialists without paying the cognitive cost of switching roles. A code-reviewer subagent reviews code. A test-writer subagent writes tests. A security-auditor subagent audits security. They do not interrupt each other, they do not pollute each other's thinking, and they do not consume the main session's context window.

subagent, subagent 执行, 只把最终结果回传。Subagent, 是一个人如何"在不付认知切换成本的前提下"变成一支小型专家团队编排者的方式。一个 code-reviewer subagent 负责审查代码; 一个 test-writer subagent 负责写测试; 一个 security-auditor subagent 负责安全审计。它们互不打断、互不污染思考、也不消耗主会话的上下文窗口。

A Hook is a deterministic event trigger. It is a shell command, bound to a specific moment in the Claude Code lifecycle, that executes automatically whenever that moment occurs. A hook fires when a tool is about to be used, or just after it was used, or when a session starts, or when it ends. Hooks are how a probabilistic system gets wrapped in deterministic edges. They are how the developer stops trusting Claude to remember to run the tests and starts trusting the hook that always runs the tests whether Claude remembers or not.

Hook(钩子)是一个确定性的事件触发器。它是一条 shell 命令, 绑定到 Claude Code 生命周期中的某个具体时刻, 只要该时刻发生, 它就自动执行。工具被调用之前、之后、会话开始、会话结束——hook 都可以触发。Hook, 是"如何用一个确定性的外壳把一个概率性的系统包起来"的方式。它把"信任 Claude 自己记得跑测试"换成"信任那个无论 Claude 记不记得都会跑测试的 hook"。

Each of these systems, on its own, is useful. A skill saves the developer from typing the same instructions twice. A subagent saves the context window from drowning in noise. A hook saves the workflow from the failure mode of forgetting. But the real power, the power that this book is built around, does not come from the three systems in isolation. It comes from the composition. It comes from building an environment in which all three work together, reinforcing each other, filling each other's gaps, turning Claude Code into something that is less like a tool the developer uses and more like a development environment that the developer has engineered to fit the shape of her work.

这三套系统, 每一套单拿出来都很有用: Skill 让你不必打两遍同样的指令; Subagent 让主会话的上下文窗口不被噪声淹没; Hook 让工作流从"忘记"这种失败模式中解放出来。但真正的力量——也是本书所围绕的核心——并非来自任何一套独立使用, 而是来自**组合**。来自"让三者协同工作、互相补位、互相加固", 把 Claude Code 从"一个开发者使用的工具"变成"一个开发者按自己工作形状工程化出来的开发环境"。

That phrase, fully engineered, is not an exaggeration. By the time the reader has finished this book, the reader will have installed skills that teach Claude how to write code the way the reader wants it written. The reader will have defined subagents that handle specialized work without disturbing the main session. The reader will have configured hooks that enforce the boundaries and the rituals of the reader's workflow without any conscious effort. The three systems will be humming together, quietly, behind every prompt the reader types.

所谓"被完整工程化", 绝非夸张。当你读完这本书时, 你将装上若干 skill, 教 Claude 按你想要的风格写代码; 定义若干 subagent, 让它们处理专项工作而不打扰主会话; 配置好若干 hook, 让工作流的边界和仪式感无须刻意维护、就自动生效。三套系统, 会在你每敲下一句 prompt 的背后, 安静地一起运转。

Engineering as Platform · 把工程做成"平台"

There is a word for this in software engineering: platform. A platform is the set of conventions, tools, and automations that shape how work is done within an organization, so that individual engineers do not have to carry all of that shaping in their own heads. What this book teaches, in the end, is how to build a personal platform on top of Claude Code. Not a company platform, necessarily, though the same techniques scale. A personal platform. One developer, one machine, one Claude Code installation, and a set of skills, subagents, and hooks that together make the environment feel as if it was built specifically for the way that one developer thinks.

软件工程里有一个词来形容这件事:平台(platform)。所谓平台,就是一套约定、工具、自动化机制,它们共同塑造了一个组织里"工作是怎样完成的",从而让单个工程师不必把这些塑造规则都装在自己脑子里。本书最后要教的就是:如何在 Claude Code 之上搭一个"个人平台"。不一定非要是公司级平台——虽然这套技术同样可以放大到公司级。是个人平台。一位开发者,一台机器,一套 Claude Code 安装,再加上一组 skills、subagents、hooks——让整个环境看起来仿佛就是为这一位开发者的思维方式量身打造的。

This is not a small thing to promise. And it is worth being honest about what it costs. Building a personal Claude Code platform requires an investment. Not a large one, but a real one. The developer has to understand the tool well enough to see which frictions are worth automating and which are not. The developer has to be willing to write configuration as carefully as she writes production code, because the configuration is, in effect, production code for her own workflow. The developer has to treat her `.claude` directory with the same respect she treats her source code, versioning it, reviewing changes, and improving it over time.

这并不是一个轻飘飘的承诺。所以有必要坦白它的代价。搭一个个人 Claude Code 平台,需要投入——不是巨额,但必须真实。开发者需要足够了解这个工具,才能看清哪些摩擦值得自动化、哪些不值得。她需要愿意像写生产代码那样认真地写配置——因为那份配置,在事实上,就是她自己工作流的生产代码。她需要像对待源代码一样对待自己的 `.claude` 目录:版本化、审阅变更、持续迭代。

In exchange for that investment, the developer gets something that compounds. Every skill installed pays dividends for every future session in which that skill fires. Every subagent defined saves the context window on every future task that would otherwise have polluted it. Every hook installed protects every future action from the failure mode it was designed to prevent. A year of thoughtful configuration turns into an environment that is noticeably more productive than any colleague's environment who has not made the same investment. Two years of it turns into a compounding lead that becomes difficult to close.

作为对这份投入的回报,开发者会得到一种复利。每装一个 skill,都会在它今后触发的每一个 session 里产生收益。每定义一个 subagent,都会在今后每一个本会污染上下文的任务中省下主会话的窗口。每配置一个 hook,都会在今后每一个动作中预先挡掉它专门针对的那种失败。一年的深思熟虑的配置,会让这个环境比"没做同样投入"的同事的环境显著高产;两年下来,这种复利就会变成一道难以追平的领先。

This is the pitch of the book. It is not a pitch about tricks. It is a pitch about building a platform. The reader who follows the path laid out in these chapters will finish with a platform of her own, and with the understanding of how to extend that platform indefinitely as her work changes and as Claude Code itself evolves.

这就是本书的卖点。它不是在兜售"小技巧",而是在兜售"搭一个平台"。沿着这些章节铺好的路径走下来的读者,最终会拥有自己的平台,以及一种能力:随着工作变化、随着 Claude Code 自身演化,持续、无限制地扩展那个平台。

The rest of Part One is about laying the foundations for that journey. Chapter Two takes the reader under the hood of Claude Code, because no serious engineering is possible without an accurate mental model of the

第一部分剩下的内容,都在为这段旅程打地基。第二章会带你掀开 Claude Code 的引擎盖,看它内部是怎么运转的——因为没有准确的心智模型,严肃的工程化无从谈起。

system being engineered. Chapter Three introduces the three-pillar framework formally, explains how the pillars map to the real problems of daily development, and establishes the composition principle that will guide every subsequent part of the book. By the end of Part One, the reader will have the vocabulary and the mental model needed to understand why the rest of the book says what it says.

正在被工程化的那个"系统",到底是什么样的。第二章之后,第三章会正式介绍"三支柱"框架,讲清这三根支柱是怎么映射回日常开发的真实问题,并奠定将贯穿全书后半段的"组合原则"。读完第一部分,你将拥有阅读后续内容所必需的词汇和心智模型,理解为什么后面的章节要这么写、这么说。

It is worth pausing here on a question the reader may already be asking. If the three systems are so valuable, why are they underused? The question has a few honest answers, and each of them is worth naming, because naming them helps the reader avoid the fate of the developers who have not benefited from what the tool already offers.

这里值得停下来,回答一个读者可能已经在心里冒出来的问题:既然三套系统这么有价值,为什么用的人这么少?这个问题有几个诚实的答案,每一个都值得点出——因为点出它们,能帮你避开"本可以受惠于工具却没受惠"的开发者的命运。

Why So Few Have Done It · 为什么做的人这么少

The first answer is visibility. Prompts are visible. A developer sees her own prompts every day, and she sees the prompts that other developers share in public channels, and she forms an intuition that prompting is where the action is. Configuration is invisible. A developer does not see another developer's `.claude` directory by default. It does not show up in screencasts. It does not appear in most tutorials. It is the dark matter of Claude Code: the thing that holds the visible workflow together, but that the visible workflow never calls attention to. The natural result is that developers imitate what they can see, and what they can see is mostly prompts.

第一个答案是可见性。Prompt 是可见的:开发者每天都能看到自己写的 prompt,也能在公共频道看到别人分享的 prompt,从而形成一个直觉——"功夫在 prompt"。配置是不可见的:开发者默认看不到别人的 `.claude` 目录;它不会出现在录屏里;它也不会出现在大多数教程里。它就是 Claude Code 的"暗物质":那根把可见的工作流撑起来的梁,但可见的工作流从不主动提到它。自然的结果就是——开发者模仿她能看见的东西,而她能看见的,主要是 prompt。

The second answer is that configuration feels like overhead. A developer who is trying to ship a feature does not want to stop shipping features in order to design a skill library. The investment looks like a distraction from the work, and the returns are deferred. This feeling is real, and it is also wrong. The returns are not deferred by very long. A well-designed skill pays for itself within a week of normal use, because it runs automatically for every matching task and saves its entire cost every time. A hook pays for itself the first time it prevents a failure that would have taken an hour to clean up. The problem is not that the returns are slow; it is that the returns are distributed across many future sessions, rather than concentrated in the session where the investment was made, so they are psychologically harder to see.

第二个答案是配置看上去像额外负担。一个正忙着交付功能的开发者,不想为了设计一套 skill 库而停下来。投入看起来像是工作的"分心",回报又像是延迟到未来的。这种感觉是真实的,但也是错的。回报并不会真的"延迟很久":一个设计良好的 skill,正常使用一周内就能收回成本,因为它会为每一个匹配的任务自动运行,每次都把全部成本省下来;一个 hook,在它第一次阻止了一次本来要花一小时收拾的故障时,就已经回本。问题不在于回报慢;问题在于回报分散在许多未来的 session 里,而非集中在"投入发生"的那个 session 里——所以从心理上更难被看见。

The third answer is cultural. The habit of configuring tools carefully used to be widespread in software engineering, and it has eroded.

第三个答案是文化。"认真配置工具"这个习惯,曾经在软件工程里普遍存在,而今已经逐渐消退。

Developers customize their editors, but they do not usually customize their terminals, their shells, or their build systems with the same care that was once standard. Claude Code is a tool that rewards that older discipline. It is designed to be bent and shaped, and it gives back, in long-term productivity, whatever the developer is willing to put in during the shaping. Readers who bring the older discipline back with them into this new tool will find themselves at a significant advantage over those who do not.

开发者会定制自己的编辑器,但已经不太用同样的心力去定制终端、shell、构建系统——而这在过去是标配。Claude Code 是一个会回报那种"老派严谨"习惯的工具。它生来就允许被弯折、被塑造;你愿意在塑造上投入多少,它就在长期生产力上以相应的形式回馈你多少。把那份老派严谨重新带回到这个新工具里的读者,会显著领先于没带回来的人。

One more anchor is worth placing before this chapter ends. Consider what Claude Code would feel like if all three pillars were ignored. A developer in that situation is using Claude Code as a conversational interface to a language model. She gets good outputs when she writes good prompts. She gets bad outputs when she writes bad prompts. Her productivity rises and falls with her attention, her memory, and her patience. Now consider the same developer after six months of thoughtful configuration. Her productivity no longer rises and falls in the same way, because the environment has absorbed much of the burden that used to sit on her attention. The ceiling of what she can produce in a day has risen, because the floor has also risen. On her worst days, the environment carries her. On her best days, the environment amplifies her. The variance has fallen and the average has climbed. This is, in the end, the simplest case for engineering rather than merely using: a reduction in variance and an increase in average, across every future session the developer will ever run.

在本章结束之前,值得再立一个锚点。请设想:三根支柱全部被忽略时,Claude Code 用起来是什么感觉。那位开发者把它当成了一个通往语言模型的对话界面:prompt 写得好,产出就好;prompt 写得差,产出就差;她的生产力,完全随着她的注意力、记忆力和耐心上下波动。再设想:同一位开发者,在经过六个月的认真配置之后,她的生产力已经不再以同样的方式起伏了——因为环境把她原本要背负的许多东西都吸走了。她一天能产出的"上限"被抬高了,因为"下限"也被抬高了。状态最差的日子里,环境托着她;状态最好的日子里,环境放大了她。方差下降了,均值上升了。说到底,"工程化"而不是"只是用",最简洁的论据就是这一点:在她未来要跑的每一个 session 里,方差降低、均值提高。

There is a deeper argument to be made, too, about what this kind of configuration does to a developer over time. A developer who has spent six months shaping her Claude Code environment has, in the course of doing so, answered a series of questions about her own work that she might otherwise never have answered explicitly. What is my preferred format for a pull request description? What counts as a complete definition of done for a new feature in my codebase? Which files should never be edited without manual confirmation, and why? What does a good commit message look like in this project, precisely? Every skill, subagent, and hook the developer builds is an answer to one of these questions, captured in a form the tool can use. The side effect of building the environment is that the developer ends up with a much clearer articulation of how she actually wants to work than she had before she started. The environment is a mirror of the developer's craft, and the craft becomes sharper by being mirrored. This is a

还有一个更深的论点:这种配置,长期看,会对一个开发者本身做些什么。花六个月去塑造自己 Claude Code 环境的开发者,在塑造的过程中,实际上回答了一连串关于自己工作的、她以前可能永远不会明确作答的问题——"我偏好的 PR 描述格式是什么?"、"在我的代码库里,一个新功能的'完成'到底包括哪些?哪些文件绝对不能不经我手就改,为什么?"、"在这个项目里,一段好的 commit message 究竟长什么样?"每一个她搭建起来的 skill、subagent、hook,都是这些问题的某一条答案,只是以工具能使用的形式固化下来。搭建环境的副作用是:

开发者最终对自己"到底想怎么工作"有了一份比开始之前清晰得多的表达。环境是开发者手艺的一面镜子,而手艺因被映照而变得更锐利。这是一种

benefit that never appears on any productivity chart, but it is one of the most valuable outcomes of the whole exercise, and readers who go through the work will recognize it when it arrives.

它从来不会出现在任何"生产力图表"上,却是整件事最有价值的成果之一——走过这段路的读者,会在它真正到来的那一刻认出它。

The remaining frustration of the Tuesday morning described at the start of this chapter is not an inherent property of working with Claude Code. It is a property of working with Claude Code without having engineered the environment around it. That environment, the one the second developer was enjoying, is buildable by anyone. The building is what this book is about. The next step in the building is understanding the machine we are about to shape, and that is the subject of the chapter that follows.

本章开头描述的那个"周二上午的沮丧",并不是 Claude Code 本身带来的固有属性——它是"用 Claude Code、却没在它周围搭建起工程化环境"才带来的属性。那个环境,也就是第二位开发者正在享受的那个,任何人都能搭建。搭建的过程,就是本书的全部内容。搭建的下一步,是理解我们即将去塑造的那台机器——而那正是下一章的主题。

CHAPTER TWO · 第二章

The Anatomy of Claude Code · Claude Code 的解剖

"If I had an hour to solve a problem, I would spend the first fifty-five minutes understanding the problem, and the last five solving it."

— attributed to Albert Einstein

"如果给我一个小时解决一道难题,我会花前五十五分钟弄清问题本身,最后五分钟再去解题。"

—— 通常归于阿尔伯特·爱因斯坦

To engineer something, one must first see it clearly. This chapter is about seeing Claude Code clearly. Not as a conversational interface, which is how it feels from the outside, but as a layered system with a specific shape, a specific set of moving parts, and a specific vocabulary for the relationships between those parts. Without this clarity, every later chapter in the book will feel like a collection of incantations. With it, every later chapter will feel like the deliberate, ordinary application of a few simple principles to a machine whose behavior is already understood.

要工程化一个东西,首先得先把它看清楚。这一章,就是要把 Claude Code 看清——不是从外面看过去那种"像聊天界面"的观感,而是把它当成一个分层的系统:有它特定的形状、特定的运动部件、特定的词汇来描述这些部件之间的关系。少了这份"看清",后面每一章都会像一堆咒语的合集;有了它,后面每一章都会变成:对一台"我们其实已经懂得其行为"的机器,几条简单原则的从容而普通的应用。

The first thing to understand about Claude Code is that, from the moment a session begins, a small but crucial amount of context is being assembled on the user's behalf, every single turn, before the user ever sees a response. The user types a prompt. That prompt is not sent in isolation to the language model. It is sent as part of a carefully constructed bundle. The bundle includes a system prompt that defines who Claude is and what it is authorized to do within this environment. It includes a description of the tools that Claude is allowed to call, including the file system tools, the shell, the web, and whatever other tools the installation has made available. It includes contextual information about the project, typically harvested from files like `CLAUDE.md` or from explicit references the user has made. It includes, when relevant, the definitions of the skills and the subagents that apply to this session, and it includes the configuration that tells Claude which hooks will fire at which points. Only after that entire bundle is constructed is the user's prompt appended, and the whole assembly is sent to the language model for a response.

关于 Claude Code,第一件要理解的事是:从一次会话开始的那一刻起,在用户看到任何一次响应之前,每一轮都会为用户那边"代为装配"出一小撮、但又至关重要的上下文。用户敲下一句 prompt,那句 prompt 并不是"孤身"被送到语言模型那里去的;它是作为一个被精心构造好的"包袱"的一部分被送过去的。这个包袱里包括:一段系统提示(system prompt),定义 Claude 是谁、在本环境里被授权做什么;一份工具清单,告诉模型它可以调用哪些工具——包括文件系统、shell、web,以及安装时给它配置进来的其他工具;关于本项目的上下文信息,通常取自 `CLAUDE.md` 这类文件,或者用户显式给出的引用;在相关的情况下,还会包含本 session 适用的若干 skill 与 subagent 的定义,以及那份"告诉 Claude 在哪些时刻会触发哪些 hook"的配置。直到这一整包都装配完毕,用户的 prompt 才会被追加到末尾,然后把整包一起送到语言模型那里去,等一次响应。

This matters because it means that the user's prompt is not the sole cause of Claude's behavior. The user's prompt is one ingredient among many, and in many cases, it is not even the most important ingredient. The system prompt is more important. The `CLAUDE.md` content is often more

这件事之所以重要,是因为它意味着:用户的 prompt,并不是 Claude 行为的唯一成因。Prompt 只是众多配料中的一种,很多时候,甚至不是最重要的一种。系统提示更重要。`CLAUDE.md` 的内容,通常更

important. The skills that are recognized as applicable are often more important. A developer who believes that the only way to change Claude's behavior is to change the wording of her prompts is a developer operating with a faulty mental model of where behavior comes from. The model is mostly not in the prompt. The model is in the bundle.

重要。被识别为"当前适用"的那些 skill,通常还更重要。一个以为"想改 Claude 的行为只能改 prompt 措辞"的开发者,她对"行为到底来自哪里"的心智模型,从根本上就错了。模型不在 prompt 里——模型在那整包"装配好的上下文"里。

The second thing to understand is that Claude Code is agentic. The word *agentic* has been overused in the last two years to the point of fatigue, but in this specific context it has a precise and useful meaning. An agent, in the sense that matters here, is a language model that has been given the ability to call tools, observe the results of those tool calls, and decide, based on those observations, what to do next. Claude Code is an agent in exactly this sense. When a user asks Claude Code to add a new feature to a codebase, Claude does not produce the code in a single shot and then stop. Claude produces a plan, inspects files to verify assumptions about the codebase, reads surrounding code to understand conventions, writes a first draft of the new code, sometimes runs tests or type checks, observes any failures, and adjusts the code in response. This loop of

propose, act, observe, adjust is the beating heart of the tool, and every subsequent system in this book is ultimately a way of shaping what happens inside that loop.

第二件要理解的事是: Claude Code 是智能体式(agentic)的。 *agentic* 这词在最近两年被用得太多,滥到让人疲劳;但是在本书的具体语境里,它有一个精确且有用的含义。这里的"agent"(智能体),是指一个语言模型——它被赋予了一种能力:调用工具、观察这些工具调用的结果、并基于这些观察决定下一步做什么。 Claude Code 正是这种意义上的 agent。当用户让 Claude Code 给代码库加一个新功能, Claude 不会"一把梭哈"地把代码写完就停;它会先列计划,检查文件以核实对代码库的假设,读周围的代码以理解约定,写新代码的第一稿,有时还会跑测试或类型检查,观察有没有失败,再据此调整。这条"提议—行动—观察—调整"的循环,就是这款工具的"心脏";本书后面讲的所有系统,归根结底都是"如何塑造这条循环内部所发生的事"。

The Loop · 那个循环

The loop is worth naming because it will be referred to constantly. The loop has three phases per iteration. In the first phase, Claude thinks, which is to say, the language model produces text that constitutes its next reasoning step. In the second phase, Claude acts, which is to say, Claude chooses a tool to call and provides the arguments to that tool. In the third phase, Claude observes, which is to say, the result of the tool call is read back into the context window, and Claude decides whether it is done or whether another iteration is needed. Think, act, observe. Think, act, observe. The loop runs until Claude decides, based on the current state of the context window, that the task is complete.

值得给这个循环起个名字,因为后面会反复提到它。每迭代一次,这个循环包含三个阶段。第一阶段: Claude 思考——模型生成一段文本,代表它下一步的推理动作。第二阶段: Claude 行动——它选一个要调用的工具,并给出这个工具的参数。第三阶段: Claude 观察——工具调用的结果被读回上下文窗口, Claude 据此判断:这件事是否已经做完了,还是需要再迭代一次。思考,行动,观察;思考,行动,观察……这个循环一直跑,直到 Claude 在当前上下文窗口的状态下判断:任务已经完成。

This loop is the reason subagents exist, the reason hooks exist, and the reason skills matter. Subagents are a way of saying, this part of the loop is a different kind of work, and it deserves its own isolated loop with different rules. Hooks are a way of saying, whenever the act phase is about to call this specific tool, run this shell command first, or after, to enforce

正是这个循环,回答了"为什么要有 subagent、为什么要有 hook、为什么 skill 重要"。 Subagent,是用来表达"循环中的这一段是另一类工作,值得给它一条独立的、规则不同的循环"; Hook,是用来表达"无论何时,只要行动阶段即将调用这个具体工具,先(或后)跑这条 shell 命令,以施加某种

something deterministic that the model alone cannot be trusted to enforce. Skills are a way of saying, this kind of task recurs often enough that it deserves a standing set of instructions, so that when the loop encounters it, the right behavior is already present in the context bundle.

确定性——而这种确定性,单靠模型本身是没法被信赖它会执行的"。 Skill,是用来表达"这类任务重复出现的频率,已经足以值得为它常驻一套指令;这样当循环遇到它时,正确的行为就预先存在于那包上下文里"。

The third thing to understand is the file system. Claude Code is not a disembodied agent floating above a project. It runs inside a specific working directory, and it is deeply aware of the structure of that directory. A set of conventions controls what Claude sees and how Claude configures itself. The most important of these conventions is the `.claude` directory. The `.claude` directory sits at the root of a project, or in the user's home folder, or in both, and it is where configuration lives. Skills live in `.claude/skills/` as subfolders. Subagent definitions live in

`.claude/agents/` as files. Hook definitions live in the settings file at `.claude/settings.json`, or in the corresponding user-level settings file. The structure of this directory is, in effect, the source code of the developer's Claude Code environment.

第三件要理解的事,是文件系统。Claude Code 不是一个"漂浮"在项目上方的无实体 agent;它跑在一个具体的工作目录里,并深度感知该目录的结构。一组约定决定了 Claude 能看到什么、它如何为自己做配置。其中最重要的一个约定,是 `.claude` 这个目录。它可以放在一个项目的根目录下,也可以放在用户的 home 目录下,或者两边都放——配置就住在那里。Skill 住在 `.claude/skills/` 下的子目录里;subagent 的定义住在 `.claude/agents/` 下的文件里;hook 的定义住在 `.claude/settings.json` (或者对应的 user-level 配置文件)里。这个目录的结构,事实上就是"开发者那个 Claude Code 环境"的源代码。

There is a hierarchy at play. User-level configuration, which lives in the home directory, applies to every Claude Code session the user opens, on every project. Project-level configuration, which lives in the `.claude` directory at the root of a specific project, applies only when Claude Code is opened inside that project. When both are present, the project-level configuration generally takes precedence for project-specific concerns, while the user-level configuration provides a stable personal baseline. A developer who works on five different projects does not need to redefine her personal commit-message skill five times. She defines it once at the user level, and it travels with her. When a specific project needs a project-specific skill, a pre-commit hook that runs that project's specific test suite, or a subagent trained on that codebase's peculiar conventions, those live in the project's `.claude` directory and are committed to source control alongside the code itself.

这里存在一个层级。User-level 配置(放在 home 目录里)作用于这个用户在每一个项目上打开的每一次 Claude Code 会话。Project-level 配置(放在某个具体项目根目录下的 `.claude` 里)只在 Claude Code 是在该项目里被打开时才生效。两者都存在时,通常 project-level 在"项目特有"的事务上具有更高优先级,而 user-level 提供一个稳定的"个人基线"。一个同时在做五个项目的开发者,不需要把"个人 commit message skill"重新定义五遍——在 user 级别定义一次,它会跟着她走。当某个项目需要"项目特有的 skill"、需要"跑该项目特定测试套件的 pre-commit hook"、或需要"针对该代码库独特规约训练过的 subagent"时,这些就放在该项目的 `.claude` 目录里,并随源代码一起被提交进版本控制。

Where Configuration Lives · 配置住在哪里

This last point deserves emphasis. The project-level `.claude` directory is part of the codebase. It is as much part of the codebase as the source files, the package manifest, or the README. It should be committed, reviewed in pull requests, and evolved deliberately. When a developer joins a team, the team's `.claude` directory travels to her automatically when she clones the

这最后一点值得多花点笔墨强调:project-level 的 `.claude` 目录,是代码库本身的一部分。它和源代码、package manifest、README 一样,都属于"代码库"。它应当被提交进 git、在 PR 中被评审、有意识地演进。当一位新开发者加入团队,她一 clone 仓库,团队的 `.claude` 就自动跟着她来到她本机——

repository. She inherits the team's skills, the team's subagents, and the team's hooks at the moment she starts working. She does not have to be told, orally, what the team's conventions are, because the conventions are encoded as configuration that Claude enforces for her. This is how a well-engineered environment spreads. Not through wikis. Not through onboarding videos. Through version-controlled configuration that the tool itself honors on the first session.

她一 clone 下来,团队的 skills、subagents、hooks 就同时落到了她手上。她不需要别人口头告诉她团队的规约是什么——那些规约,已经以"由 Claude 来替她执行"的配置形式编码好了。这就是"被工程化好的环境"如何扩散:不是通过 wiki,不是通过 onboarding 视频,而是通过"被版本管理、且被工具自身在第一个 session 就尊重"的配置。

The fourth thing to understand is `CLAUDE.md`. This file is a special convention that Claude Code looks for automatically, and whose contents are added to the system context at the start of every session. `CLAUDE.md` is the place for information that applies to every interaction within a project. It is where one writes, this codebase uses TypeScript with strict mode; tests are in Vitest; authentication changes require careful review. `CLAUDE.md` is not a skill, and it is not a substitute for one. A skill is targeted; it activates when its description matches the user's intent. `CLAUDE.md` is ambient; its contents are always loaded. The right things to put in `CLAUDE.md` are things that need to be true at all times. The wrong things to put there are things that only matter when a specific kind of work is being done, because those things should be skills instead, and should be activated only when they are needed.

第四件要理解的事,是 `CLAUDE.md`。这个文件是 Claude Code 自动寻找的特定约定:它的内容,会在每个 session 启动时,被加进 system context 里。`CLAUDE.md` 是写"对项目内每一次交互都生效"信息的地方。比如:"本代码库使用 TypeScript 的 strict 模式;测试用 Vitest;鉴权改动需要仔细 review"。`CLAUDE.md` 不是一个 skill,也不是 skill 的替代品。Skill 是**靶向的**——只有当它的 description 命中用户意图时,它才会激活;`CLAUDE.md` 是**环境性的**——它总是被加载。适合放进 `CLAUDE.md` 的,是"任何时候都需要为真"的事;不适合放进去的,是"只有在做某一类工作时才有意义"的事——后者应该写成 skill,只在被需要时才激活。

This distinction, ambient versus targeted, is one of the design questions the reader will be making over and over again throughout the book. When a piece of knowledge belongs in `CLAUDE.md`, and when it belongs in a skill, is a judgment call. A rule of thumb that will serve well: if the knowledge is relevant to almost every action in the project, put it in `CLAUDE.md`. If the knowledge is relevant only to a specific kind of action, put it in a skill, and write the skill's description so that the skill fires when that kind of action is being requested. This rule will be refined as the book proceeds, but it is a good starting place.

这条"环境性 vs 靶向"的区分,是读者在整本书里会反复做出的一个设计判断。"这条知识该放 `CLAUDE.md`,还是该放 skill 里",是一个判断题。一条很好用的经验法则:如果这条知识对项目里几乎所有动作都相关,放 `CLAUDE.md`;如果只对某一类动作相关,放 skill,并把 skill 的 description 写好,使得"该类动作被请求时"它能被触发。这条法则在本书后续会逐步精化,但作为出发点,够用了。

The fifth thing to understand is the context window. Every model has a finite context window, and every session spends that window progressively as it runs. The context window contains the system prompt, the assembled bundle of skills and subagents and hooks that apply to this session, the `CLAUDE.md` content, the history of the conversation so far, and the outputs of every tool call Claude has made. The context window fills up. When it fills up, the quality of the model's responses degrades, because the model has to

第五件要理解的事,是**上下文窗口(context window)**。每个模型都有一个有限的上下文窗口;每次会话,在它运行的过程中,都会把那个窗口一点一点地花掉。上下文窗口里装着的,包括:系统提示、为本次 session 装配好的 skills / subagents / hooks、`CLAUDE.md` 的内容、到目前为止的对话历史,以及 Claude 调用过的每一个工具的输出。上下文窗口会满。满的时候,模型回答的质量就会下降,因为模型要在更多噪声里费力地去定位真正相关的信息。

work harder to find the relevant information among the noise. Eventually, the context window fills up entirely, and something has to give.

到最后,模型不得不在大量噪声里苦搜,等窗口彻底塞满,总得有东西得让位。

Context as a Scarce Resource · 把上下文视作稀缺资源

This is why subagents matter so much. A subagent, when it runs, has its own fresh context window. The main session dispatches a task to the subagent, the subagent does the work in its own private context, and only the subagent's final output is added back to the main session's context.

The dozens of tool calls the subagent may have made, the files it may have read, the drafts it may have discarded, none of it enters the main session. The main session stays clean. The developer who designs subagents well is, in effect, designing a context budget, a plan for what deserves to live in the main session's precious memory and what should be delegated to a subprocess that will return only its conclusion.

这就是 subagent 如此重要的原因。一个 subagent 跑起来时,自带着一个全新、干净的上下文窗口。主会话把任务派给 subagent;subagent 在自己的私有上下文里干完活;只有 subagent 的"最终输出"会被加回主会话的上下文。subagent 内部做过的那几十次工具调用、它读过的文件、它丢掉过的草稿,统统不进主会话。主会话因此保持干净。一个把 subagent 设计得好的开发者,事实上是在设计一份"上下文预算"——一份计划:什么值得住进主会话宝贵的记忆里、什么应该外包给一个只回传结论的子进程。

The sixth thing to understand is the lifecycle. A Claude Code session is not a single event. It has stages. There is the session start, when the environment is assembled and the initial context is loaded. There is the prompt submission phase, when the user's input is appended to the context. There is the tool use phase, when Claude decides to call a tool; this phase is composed of an approval step, the actual tool call, and the observation of its result. There is the stop phase, when Claude decides the task is done. There is the session end, when the process terminates. Each of these stages is a moment at which something can be made to happen. The hook system is precisely the mechanism by which a developer attaches behavior to these moments. When a developer wants something to happen before every tool use, she attaches a PreToolUse hook. When she wants something to happen after Claude finishes speaking, she attaches a Stop hook. The lifecycle is the grid of available moments, and hooks are the pins that attach behavior to the grid.

第六件要理解的事,是生命周期。一次 Claude Code 会话并不是一个"单一事件";它有阶段。SessionStart——环境被装配、初始上下文被加载。Prompt 提交阶段——用户的输入被追加进上下文。Tool 调用阶段——Claude 决定调用一个工具;这一阶段由三步组成:批准、实际调用、观察结果。Stop 阶段——Claude 判定任务完成。SessionEnd——进程终止。每一个阶段,都是"可以发生点什么"的时刻。Hook 系统,正是开发者用来把行为挂到这些时刻上去的机制。想在每次工具调用之前发生点什么,就挂一个 PreToolUse hook;想在 Claude 说完话后发生点什么,就挂一个 Stop hook。生命周期是"可挂载时刻"组成的一张网格,hook 是把行为挂到网格上的那根钉子。

A seventh observation, and the last of the anatomy, concerns the way Claude Code thinks about safety. Every tool use is, by default, a point of trust. The language model decides, based on the user's prompt and the context bundle, that a certain tool should be called with certain arguments. If that decision is wrong, and if the tool has consequences in the outside world, the wrong decision becomes a real event. Claude Code mitigates this in several ways that the reader should be aware of. Tool permissions can be

第七条,也是解剖学中的最后一条,关乎 Claude Code 怎么思考安全这件事。默认情况下,每一次工具调用都是一个"信任点":语言模型根据用户的 prompt 和上下文包袱,决定"某个工具应当以某组参数被调用"。如果这个决定是错的——而那个工具又在外部世界有真实副作用——那么这个错误的决定,就变成了一件真实发生的事。Claude Code 用几种方式来缓解这个风险,读者应该知道:工具权限可以

restricted at the session, project, or user level, so that certain tools are never available in certain contexts. Approvals can be required for specific categories of tool use, so that the developer retains a human check on high-impact operations. And hooks, as will be seen in the fourth part of this book, can intercept tool uses before they happen and reject them on any grounds the developer can encode in a shell command. Safety, in this model, is not a property of the language model alone. It is a property of the configured environment, and the developer is the one who configures it. This responsibility is not heavy; it mostly consists of making good defaults and then forgetting about them. But it is real, and the reader who takes the time to set good defaults early will never have to think about them later, which is the best outcome a safety system can offer.

在 session、project、user 三个层级上被限定,从而某些工具在某些场景下压根不会出现;特定类别的工具调用可以被要求人工批准,从而在"高影响操作"上保留一道人为闸门;而 hook(本书第四部分会专门讲)则可以在工具调用发生前拦截它,按开发者能写成 shell 命令的任意

理由拒绝它。在这个模型下,安全不是语言模型自身的属性——它是"被配置好的环境"的属性,而配置环境这件事,由开发者本人来做。这份责任并不重:大部分情况下,就是把好的默认设好,然后忘了它。但它是真的;愿意在早期就把好的默认设好的读者,日后就不必再为这件事操心——而这正是一个安全系统能给出的最好结果。

This is the whole machine, seen from enough height to be understood in a single glance. A language model, wrapped in an agent loop, wrapped in a file-system-aware configuration system, wrapped in a lifecycle of named events, wrapped in a session that the user drives by typing prompts. Every sophisticated use of Claude Code that the reader will encounter in this book, and every sophisticated use she will design on her own in the years to come, is some configuration of this machine. There is no hidden trick. There is no magic. There is a well-defined system, and the question is only how well the developer understands that system and how deliberately she shapes it.

这就是这台机器的全貌——从一个足够高、足够一眼看懂的角度看过去:一个语言模型,被一条 agent 循环包着;这条 agent 循环,又被一个"懂文件系统"的配置系统包着;这个配置系统,又被一个"由命名事件组成的生命周期"包着;这个生命周期,又包在一个"由用户敲 prompt 来驱动的会话"里。读者在本书里会遇到的每一个 Claude Code 高级用法——以及你未来自己设计出来的每一个——都是这台机器的某种"配置形态"。没有暗藏的诀窍。没有魔法。是一台定义良好的系统,问题只在于:你多懂它,你多有意地去塑造它。

Composition by Default · 组合是默认动作

There is also a subtler property of Claude Code worth surfacing while the anatomy is fresh: the environment is not static. Anthropic has been evolving the tool continuously, adding new lifecycle events, refining the skill system, expanding what subagents can do, broadening the matchers available to hooks. The developer who engineers her environment in this tool is aiming at a moving target, and that is not a drawback. It is a feature. The principles that the book is about to teach are stable principles, rooted in the shape of the loop and the lifecycle rather than in any specific version of the tool. A developer who understands skills, subagents, and hooks in the sense this book defines them will be able to absorb new features as they ship, because she will recognize each new feature as a variation on a theme she already knows. The framework is, in that respect, a defense against the churn of the underlying software. The developer who has an opinion about how her

趁"解剖图"还新鲜,Claude Code 还有一条更隐微、但同样值得点出的特性:这个环境不是静态的。Anthropic 一直在持续演化这个工具——加入新的生命周期事件、改进 skill 系统、扩展 subagent 能做的事、放宽 hook 的匹配规则。在这款工具里工程化自己的环境的开发者,瞄的是一个移动靶——而那不是缺点,是特性。本书要教的原则,都是稳定的原则,它们根植于"那条循环 + 那个生命周期"的形状,而非某个特定版本的工具。真正按本书定义去理解 skills、subagents、hooks 的开发者,会很容易吸收新功能——因为她会认出"每一个新功能,不过是一个她已经知道的主题的变奏"。从这点看,这套框架其实是对"底层软件一直在变"的一种防御。一个对自己的

environment should be shaped is the developer who can evaluate new features on their merits, keep what serves her work, and ignore what does not. The developer who has no such opinion will be pushed around by whatever the current release notes emphasize.

环境"应该长成什么样"心里有主张的开发者,才能按价值评估新功能,留下对自己工作有用的,忽略不相关的;而没有这种主张的开发者,只能被每一版 release notes 强调的热点推着走。

A short diagnostic exercise is worth performing here, because it will make the abstractions of this chapter much more concrete. The reader is encouraged, when she has a moment at her own terminal, to open a project in Claude Code and explicitly map its anatomy. What is in the project's `CLAUDE.md` file, if one exists? What skills live in the project's `.claude/skills` directory, and what do their descriptions say? What subagents are defined, and what do their system prompts establish about their specialization? What hooks are configured in the project's settings file, and at which lifecycle events do they fire? This exercise, which should take only fifteen or twenty minutes, often reveals two things. First, it reveals that the environment the developer has been operating in has more shape than she realized, because previous developers or the team

standard has already put some configuration in place. Second, it reveals the gaps, which is to say, the places where no configuration exists and where the developer has been filling the gap with her own attention, session after session. Those gaps are the raw material for the rest of the book. Every gap is a candidate for a skill, a subagent, or a hook, and by the end of the book the reader will have a considered view on which is appropriate for which gap.

这里值得做一次"小诊断练习"——它会令本章的抽象概念具体得多。读者在自己终端方便的时候,被鼓励去打开一个 Claude Code 项目,显式地"画出"它的解剖图:项目的 `CLAUDE.md` 里写了什么(如果存在的话)?项目的 `.claude/skills` 目录里有哪些 skill,它们的 description 在说些什么?定义了哪些 subagent,它们的 system prompt 是怎样确立自己的"专门化"的?项目的 settings 文件里配置了哪些 hook,它们在哪些生命周期事件上触发?这个练习花不了多久——十五到二十分钟;但常常会揭示两件事。第一,她一直在里面操作的这个环境,实际上比她以为的"更有形状"——之前的开发者或者团队规范已经在那里放了一些配置。第二,它会揭示"缺口"——也就是那些"没有配置、她一直在用自己的注意力一次次去补"的地方。这些缺口,就是本书后半部分的原材料。每一个缺口,都是 skill、subagent、hook 的潜在候选;而读到最后,你会形成一个"有理由的看法"——哪种缺口适合用哪种原语去补。

One final note is worth making before this chapter closes. The word configuration has been used many times in these pages, and it is a word that can sound dry and administrative. It is not. In the world of modern software, configuration is power. The most influential tools in any developer's life are tools whose default behavior can be bent by configuration to fit that developer's specific needs. A text editor is an example; the raw editor is impressive, but an editor configured over years with the right extensions, keybindings, and snippets becomes something much more powerful than the sum of its defaults. A shell is another; the raw shell is capable, but a shell configured with the right aliases, functions, and prompt becomes an extension of the developer's intentions rather than a tool she is constantly wrestling with. Claude Code is, in this respect, more like a modern editor or a well-configured shell than like a chat interface. It is meant to be configured. It is designed with configurability as a first-class

本章收尾前,还值得点一笔:"配置"这个词在前面出现了很多次,听上去可能有点干、有行政味。但它不是。在现代软件的世界里,配置就是权力。任何一个开发者生命里最有影响力的工具,都是"其默认行为可以被配置、从而贴合该开发者具体需求"的工具。文本编辑器就是一个例子:原始的编辑器已经很强;但一个用若干年精心配置出合适的扩展、快捷键、代码片段的编辑器,会比它的所有默认值的简单相加更强。Shell 也是一个例子:原始的 shell 也能干很多事;但一个用合适的 alias、function、prompt 配置出来的 shell,会变成"开发者意志的延伸",而不是她要持续与之搏斗的工具。从这点看,Claude Code 比起"聊天界面",更接近一款现代编辑器或一款被精心配置过的 shell。它生来就应当被配置。它把"可配置性"作为头等公民

concern. The developer who treats Claude Code as a tool to be configured, rather than merely used, is the developer who will extract the full value from it. Everything in this book is an attempt to teach that configuration seriously and well.

来设计。开发者若把 Claude Code 当成"待配置的工具"而不是"被使用的工具",才是能从中榨出全部价值的那种开发者。本书所有的内容,都在试图把"配置"这件事教得严肃、且教得好。

With the anatomy clear, the chapter that follows introduces the three-pillar framework that organizes all of that configuration into a coherent system of thought. The framework is the single most important concept in this book, and the rest of the book is, in a sense, a careful elaboration of it. The reader who has internalized the anatomy described here is now ready to see the framework for what it is.

"解剖图"已经清楚之后,下一章将引入那套"三支柱"框架——它能把所有这些配置,组织成一套连贯的思维系统。这个框架,是本书里最重要的一个概念;从某种意义上说,后面整本书,都是对它的一次精心展开。已经把前面这套解剖图内化了的读者,现在可以看到这个框架的"真身"了。

CHAPTER THREE · 第三章

Skills, Subagents, and Hooks: The Three-Pillar Framework

技能、子代理与钩子:三支柱框架

"Architecture is the art of how to waste space beautifully, or how to save it meaningfully. It is never neutral."

— Philip Johnson, adapted

"建筑,是一门如何把空间漂亮地浪费掉、或把它有意义地节省下来的艺术。它从来不是中立的。"

—— 菲利普·约翰逊(改写)

Every discipline eventually discovers that its power comes less from any single technique and more from the framework that organizes its techniques into a coherent whole. In medicine, the framework of anatomy organizes the study of the body. In music, the framework of harmony organizes the study of chords. In software engineering, frameworks of architecture organize patterns that would otherwise feel like a disordered bag of tricks. This chapter introduces the framework that organizes everything in this book. It is a simple framework with only three parts, but the simplicity is deceptive. Once the reader has internalized these three parts and the relationships between them, every subsequent chapter in the book reads as a natural elaboration. Before the reader has internalized them, the chapters look like a catalog of features. After, they look like a single unified argument.

任何一门学科最终都会发现:它的力量,与其说来自任何一项单独的技术,不如说来自"把那若干技术组织成一个连贯整体"的框架。在医学里,解剖学的框架组织了"对身体的认识";在音乐里,和声的框架组织了"对和弦的认识";在软件工程里,各种架构框架把那些"否则会显得像一袋零散技巧"的设计模式组织起来。本章要介绍的那个框架,就是组织本书所有内容的那个。它很简单,只有三块;但这种简单有迷惑性。一旦你把这三块以及它们之间的关系内化,后面每一章读起来都像"自然而然的展开";在内化之前,这些章节看起来像一份特性目录;内化之后,它们看起来则像一段统一的论证。

The framework is this. Claude Code mastery is built on three pillars. The first pillar is Skills, and it supplies capability. The second pillar is Subagents, and they supply parallelism and specialization. The third pillar is Hooks, and they supply determinism. Every sophisticated workflow is some combination of these three, and the real art of mastery lies not in any one of them in isolation but in how they are composed.

框架是这样的:Claude Code 的"掌握",建立在三根支柱之上。第一根是**Skills(技能)**,它提供**能力**。第二根是**Subagents(子代理)**,它们提供**并行和专门化**。第三根是**Hooks(钩子)**,它们提供**确定性**。每一个真正"成熟"的工作流,都是这三者的某种组合;而"掌握"的真正艺术,不在其中任何一根独立的存在里,而在它们是如何被组合在一起的。

Begin with the first pillar. A Skill is the answer to a specific question: how does a developer teach Claude to do something consistently, across sessions, across projects, without repeating herself? The question looks simple, but it is the root of an enormous amount of developer frustration. A developer writes a prompt that produces excellent output. She is pleased. She moves on. Three weeks later, she wants that same kind of output again, and she cannot remember exactly how she phrased the prompt. She tries to

从第一根开始。Skill 回答的是一个具体的问题:开发者要怎样,才能跨 session、跨项目地教 Claude 始终如一地做某件事,而不必一遍遍重复自己?问题看上去简单,却是大量开发者挫败感的根源。开发者写了一段 prompt,得到了很棒的输出,很高兴,继续做别的事。三个星期后,她想再要一次同类输出,却已经想不起当时的 prompt 究竟是怎么措辞的。她试着去

reconstruct it. The result is not quite as good as before. She keeps trying. Eventually, the reconstruction converges to the original, but she has wasted thirty minutes. This is a daily experience for every Claude Code user who has not adopted skills. Skills are the cure.

还原它。还原出来的,没原来那么好。她继续试,最后勉强拼回原样,但半小时已经搭进去了。这是每一位"还没有采用 skill"的 Claude Code 用户的日常。Skill 就是解药。

A skill is, in its essence, a promise. It is a promise that whenever a certain kind of task is requested, Claude will remember how to do that task well, because the instructions for doing it well have been captured in a file that Claude will read. The file is called `SKILL.md`. It lives in a folder named after the skill, inside either the user's personal skill library or the project's skill library. It has a structured head and a free-form body. The head declares the skill's name, its description, and the tools it is allowed to use. The body contains the instructions themselves, in plain prose, with examples, references, and any other material that would help a capable reader do the task well for the first time.

一个 skill,在本质上,是一份承诺。承诺的是:只要某一类任务被请求,Claude 就会记得"把这件任务做好"——因为"如何做好它"的指令,已经被捕获到一个 Claude 会去读的文件里了。那个文件叫 `SKILL.md`。它住在一个以 skill 命名的目录里,而该目录可以放在用户的个人 skill 库里,也可以放在项目的 skill 库里。它有一个"结构化头部"+ 一个"自由形式正文"。头部声明 skill 的名字、description,以及它被允许使用的工具;正文是真正的指令,以普通散文形式呈现,加上示例、参考,以及任何能帮一位有能力的读者"第一次就把这件任务做对"的材料。

When Claude Code starts a session, it discovers all of the available skills. It does not load their full contents at once, because that would waste the context window. It loads only the descriptions, because the descriptions are enough to tell Claude when each skill is relevant. When a user issues a prompt, Claude scans the descriptions of the available skills and recognizes which ones, if any, apply to the prompt. If a skill applies, Claude reads its body and uses those instructions to shape its response. This is what progressive disclosure means in the context of skills: only what is relevant gets read, at the moment it becomes relevant, and nothing more.

Claude Code 启动一次 session 时,它会"发现"所有可用的 skill。但它不会一次性把它们的全部正文都加载进来——那样会浪费上下文窗口。它只加载 description;description 已经够用,足以让 Claude 判断"每个 skill 在什么时候相关"。当用户敲下一段 prompt,Claude 会扫一遍可用 skill 的 description,识别出"哪些与本段 prompt 相关"。如果某个 skill 命中,Claude 就去读它的正文,并用那些指令来塑造自己的回答。这就是 skill 语境下的渐进式披露(progressive disclosure):只有相关的东西才会被读,只在"它变得相关的那一刻"被读,再无其他。

This mechanism is what makes skills practical. A developer can accumulate dozens or even hundreds of skills without paying a context-window tax for the ones she is not using. Only the skills that match the current request cost tokens. Every other skill sits quietly in the library, waiting for the kind of request that would activate it. A skill, in this sense, is a dormant capability, and a skill library is a collection of dormant capabilities. When the library has been designed well, the developer's work activates the right capabilities at the right moments, and a substantial portion of the developer's daily knowledge work runs itself.

正是这套机制,让 skill 变得实用。开发者可以积累几十、甚至几百个 skill,却不必为那些"没在用"的 skill 付上下文窗口的税。只有与当前请求匹配的 skill 才消耗 token;其他每一个 skill,都安静地待在库中,等待那种会激活它的请求出现。从这个意义上说,skill 是一种休眠中

的能力;而 skill 库,就是一组"休眠中的能力"的集合。当这个库被设计得很好,开发者的工作就会在恰当的時刻激活恰当的能力;而她日常知识工作中的相当一部分,会自己跑起来。

Skills in Practice · Skill 在实战中

原书第 29 页 · CHAPTER THREE(续)

29

What does a skill look like in practice? Consider a simple example. A developer writes a lot of pull request descriptions, and has developed a specific format she likes: a one-sentence summary, a bulleted list of what changed, a note about testing, and a note about risk. She has typed this format into prompts hundreds of times. She sits down one evening and writes a skill called `pr-description`. In `SKILL.md`, she writes a description that says, activate this skill whenever the user asks for a pull request description, PR description, or PR body. In the body, she lays out the format in detail, with an example filled in. She saves the file, restarts her Claude Code session, and the next time she types, give me a PR description for this branch, the skill fires. The description that comes back follows her format exactly. She does not explain the format in the prompt. She does not need to. The skill remembers for her.

Skill 在实战中长什么样?看一个简单的例子。一位开发者要写大量 PR 描述,而且她已经形成了一种自己偏好的固定格式:一句摘要、一个"改了什么"的项目列表、一段关于测试的说明、一段关于风险的说明。她已经在 prompt 里手敲过这个格式几百次了。某个晚上她坐下来,写了一个叫 `pr-description` 的 skill。在 `SKILL.md` 里,她写了一段 description:任何时候,只要用户要 pull request description、PR description 或 PR body,就激活这个 skill。在正文里,她把格式详细展开,还附上一个填好的示例。她保存文件,重启 Claude Code session,下次她再敲"给我这个分支写一段 PR 描述"——skill 立刻触发;Claude 给回的描述,严格遵守了她的格式。她不需要在 prompt 里再解释一次。她不需要。skill 替她记着。

This is a small example, but it is representative. Every skill is, at its core, the codification of something that used to live only in the developer's head and her prompting habits. Skills make that knowledge durable. They are how a developer turns her accumulated expertise into configuration, and through configuration, into compounding leverage.

这是一个很小的例子,但有充分的代表性。每一个 skill,在最底层,都是"把原本只活在开发者脑子里、和她的 prompt 习惯里的那点东西"做一次"制度化"。Skill 让那份知识变得耐久。它是开发者把"自己积累下来的专业"转化为"配置"、再借由配置转化为"复利式杠杆"的途径。

Now turn to the second pillar. A Subagent is the answer to a different kind of question: how does a developer do many kinds of work at once, without drowning her main session in the noise of all those kinds of work, and without losing the specialized focus that each kind of work deserves?

现在转向第二根支柱。Subagent 回答的是另一类问题:开发者要怎样,才能在同时做很多类工作时,既不让她主会话被这些工作的噪声淹没,又不丢失"每一类工作本该享有的那种专门化聚焦"?

Every non-trivial development task is really several tasks in a trench coat. A feature request involves reading existing code, designing new code, writing the new code, writing tests for it, updating documentation, and reviewing the whole for quality. A single Claude session can do all of these things in sequence, and it often does, but there is a cost. Every file Claude reads in the course of understanding the existing code takes up space in the context window. Every false start, every draft, every experiment pollutes the history of the session. By the time Claude is writing the tests, the context window is crowded with exploration residue that has nothing to do with testing. The tests it writes, as a result, are sometimes shaped by irrelevant context that it cannot easily ignore.

每一个不那么琐碎的开发任务,本质上都是"穿了一件风衣的若干个任务"。一个 feature 请求,实际包含:读现有代码、设计新代码、写新代码、为之写测试、更新文档、再对整体做一轮质量 review。一个 Claude session 当然可以按顺序把这一串事都做了——它经常也确实这么做——但那是有代价的。Claude 为了弄懂现有代码而读的每一个文件,都占着上下文窗口的位;每一次走偏的尝试、每一版被丢掉的草稿、每一次试错,都把会话的历史污染了一层。等 Claude 终于写到测试时,上下文窗口里已经堆满了"和测试毫不相关的探索残余"。它最后写出来的测试,因而不时会被那些"与测试无关、却无法轻易忽视"的上下文所塑造。

A subagent solves this by giving a specific kind of work its own isolated environment. When the main session needs tests written, it dispatches a test-writer subagent. The subagent has its own system prompt, tuned for the craft of writing tests. It has its own set of allowed tools, which

Subagent 的解法,就是把"某一类工作"放进一个隔离的环境里。主会话需要"写测试"时,就派出一个 test-writer subagent。Subagent 带着自己专为"写测试这门手艺"调教过的 system prompt;带着自己被允许使用的工具集,那个集合——

probably includes reading code and writing new test files but probably excludes making commits or touching production configuration. It has its own context window, which starts empty and fills only with the information needed for the specific test-writing task at hand. When the subagent finishes, it returns its output to the main session. The main session receives the tests. The entire exploration the subagent did to produce those tests does not enter the main session's context. The main session stays focused.

很可能包含"读代码、写新测试文件",但很可能不包含"做 commit、动生产配置"。它还有自己的上下文窗口,启动时是空的,只会填进"为眼下这件具体的写测试任务"所需要的信息。Subagent 干完活,把它的输出回传给主会话;主会话收到那批测试。Subagent 为了产出这批测试所进行的那一整段"探索",一个字都不会进主会话的上下文。主会话因此保持聚焦。

This architecture has several virtues that are worth naming. First, it saves context, which directly improves the quality of everything else the main session does. Second, it allows specialization, because each subagent can be tuned for its specific kind of work, with a role definition and tool restrictions appropriate to that work. Third, it allows parallelism, because multiple subagents can run at the same time, each on an independent piece of a larger task, and the main session can synthesize their outputs. A developer with a well-designed subagent library is able to dispatch a security review, a performance audit, a documentation pass, and a test-coverage check simultaneously on a single pull request, and receive four independent reports back. A single-threaded session could never do this well.

这套架构,有几个值得点名的优点。第一,它节省上下文——这直接提升了主会话做的所有其他事的质量。第二,它允许"专门化"——因为每个 subagent 都可以为自己的那类工作调教一遍,带上适合该工作的角色定义和工具限制。第三,它允许"并行"——多个 subagent 可以同时跑,各自分头处理一个更大任务的独立子块,主会话最后再把它们的输出综合起来。一份 subagent 库设计得好的开发者,能够在同一份 PR 上同时派一个安全审查、一个性能审计、一轮文档过审、一次测试覆盖率检查,并拿到四份独立报告;单线程的 session 永远做不好这件事。

Subagents in Practice · Subagent 在实战中

Subagents, like skills, are defined in the `.claude` directory. Each subagent is a file that declares its name, its description, its allowed tools, and its system prompt, the last of which is where the specialization actually lives. The system prompt of a code-reviewer subagent is different from the system prompt of a test-writer subagent, because code review and test writing are genuinely different crafts. The art of subagent design, which later chapters will treat in depth, is in part the art of writing these focused system prompts well.

Subagent,和 skill 一样,也在 `.claude` 目录里定义。每个 subagent 是一个文件,声明它的名字、description、可用工具,以及它的 system prompt——最后这一项,才是"专门化"真正栖身之处。一个 code-reviewer subagent 的 system prompt,与一个 test-writer subagent 的 system prompt,不会一样;因为 code review 和写测试,确实是两门不同的手艺。"subagent 设计"这门艺术(后面章节会深入讲),一部分就是把这些"聚焦的"system prompt 写得好的艺术。

There is one further property of subagents that deserves mention before leaving the second pillar. Subagents can themselves invoke skills. A test-writer subagent, when it starts, can have access to a skill that encodes the project's test conventions, and it can activate that skill just as the main session would. This is where the framework begins to compose. A subagent is not merely a specialist; it is a specialist who inherits the knowledge the developer has encoded into the environment. The specialist-plus-skill

在离开第二根支柱之前,subagent 还有一条值得提一提的性质:subagent 自身也可以调用 skill。一个 test-writer subagent,在它启动时,可以拿到一个"把项目测试规约编码在内"的 skill,并且可以像主会话那样去激活它。这是"框架"开始组合起来的地方。Subagent 不仅仅是"一个专家";它是一个继承了开发者已经编码进环境里的知识的专家。"专家+skill"

← Chapter 3 上一页

PART II: Skills(Ch 4) →

combination is strictly more powerful than either alone, and the real art of building a Claude Code platform involves designing these combinations deliberately.

的组合,严格来说,比任何单用其一都要更强;搭建一套 Claude Code 平台的真功夫,正在于把这些组合有意识地设计出来。

The Third Pillar · 第三根支柱

A Hook is the answer to the third question: how does a developer ensure that the things that must happen always happen, not merely most of the time?

Hook 回答的是第三个问题:开发者要怎样,才能确保那些"必须发生的事"总是发生——而不只是"大多数时候"发生?

Language models are probabilistic. This is a statement of design, not of failure. A probabilistic system produces different outputs for the same inputs, and this is what allows a language model to handle the open-ended variety of human requests. The trade-off is that a probabilistic system cannot be trusted to always do anything. It can be coaxed, in most cases, into doing a thing reliably, but reliably is not the same as always. For the things that must happen always, probabilistic behavior is not enough.

语言模型是概率性的(probabilistic)。这是一个设计层面的陈述,而不是一句失败判决。一个概率系统,会对"相同的输入"产出不同的输出——这恰恰是语言模型能够处理人类请求那种"开放式多样性"的原因。代价是:一个概率系统,不可能被绝对地信任去做任何一件事。它可以在大多数情形下被哄得"可靠地做某事",但"可靠"和"总是"并不是一回事。对于那些"必须总是发生"的事,概率性行为远远不够。

Consider the category of things that must happen always. Tests must be run before commits. Linters must be run after edits. Secrets must never be written to disk. Production configuration files must never be edited without explicit confirmation. Long sessions must not terminate silently without a summary being logged somewhere the developer can review. Each of these is a requirement where the acceptable failure rate is zero. A ninety-eight percent success rate is not enough. A developer who tells Claude, by prompt alone, to always run tests before committing, will find

that in ninety-eight out of a hundred cases Claude does exactly that. In the other two, something else happens. Usually nothing bad. Occasionally something very bad. The whole point of the rule was to prevent the very bad thing, and a rule that fails two percent of the time is barely a rule at all.

请看"必须总是发生"这类事的清单:提交之前必须跑测试;编辑之后必须跑 linter;机密绝不允许落盘;生产配置文件未经明确确认绝不允许编辑;长 session 结束绝不允许悄无声息——必须把一段摘要记到开发者能查看的地方。每一项,都是"可接受失败率 = 0"的需求。98% 的成功率根本不够。某位开发者只用 prompt 告诉 Claude"提交前必须跑测试",他会在一百次里看到有九十八次 Claude 照做了;剩下两次,会发生别的事。通常没什么大碍;偶尔,会非常糟糕。整条规则的存在意义,本来就是为了防范那件"非常糟糕的事";而一条 2% 的时候会失效的规则,几乎根本算不上规则。

A hook is the mechanism by which a developer can, at specific moments in the Claude Code lifecycle, insert a deterministic shell command that runs whether Claude remembers or not. If the hook fails, the lifecycle is interrupted. If the hook succeeds, the lifecycle continues. The language model does not get to override the hook, because the hook is not part of the language model's loop. The hook is outside the loop. The hook is wrapped around the loop.

Hook,就是一套机制:开发者可以在 Claude Code 生命周期的特定时刻,插入一条确定性的 shell 命令,不管 Claude 记不记得,这条命令都会跑。Hook 失败,生命周期就中断;hook 成功,生命周期就继续。语言模型无从覆盖 hook,因为 hook 本就不在语言模型那个循环里。Hook 在循环之外。Hook 包在循环的外面。

The Hook Layer · 钩子层

← Chapter 3 上一页

下一页 →

This placement, outside the loop and around it, is the whole source of the hook's value. Hooks do not replace the language model's judgment; they constrain the environment in which the language model is making its judgments. A hook that refuses to let any tool use touch the `.env` file does not make the model smarter. It makes it unnecessary for the model to be perfectly vigilant about the `.env` file, because the `.env` file is protected by a rule the model cannot break. This frees the model to think about the work that does require judgment, while the work that requires only enforcement is handled by the hook. The probabilistic engine is left to do the probabilistic work, and the deterministic requirements are handled deterministically, each in its proper place.

"放在循环之外、包在循环外面"这个位置,正是 hook 全部价值的来源。Hook 并不取代语言模型的判断;它约束的是语言模型做判断所处的环境。一个拒绝任何工具调用去碰 `.env` 文件的 hook,并没有让模型变得更聪明;它让模型不必再为 `.env` 文件保持那种"完美警觉"——因为 `.env` 文件被一条模型无法打破的规则保护着。这反过来,反而让模型可以把精力放回那些"确实需要判断"的工作上;而那些"只需要强制执行"的工作,交给 hook 去办。概率性引擎留在原位做它该做的概率性工作,确定性需求被确定性地处理——各司其职,各得其所。

Hooks are defined in the `settings.json` file, at either the user level or the project level. Each hook declares which lifecycle event it responds to, an optional matcher that narrows when it should fire, and the command to run. The command can be any shell command. It can be a one-liner or a call to a full script. It can read the context of the hook from standard input in a structured format, and it can signal approval or rejection of the current action through its exit code and output. The design is deliberately simple at the configuration layer, so that the complexity can live in the scripts themselves, where it belongs.

Hook 在 `settings.json` 文件里定义,既可以放在 user 级,也可以放在 project 级。每个 hook 声明:它响应哪个生命周期事件、一个可选的 matcher(用来收窄触发条件),以及要执行的命令。那条命令可以是任意 shell 命令——一行 one-liner,或者对一个完整脚本的调用;它可以从 stdin 按一种结构化格式读入 hook 的上下文,也可以通过 exit code 和输出来表态:批准、还是否决当前动作。这种设计在配置层刻意保持简单,目的是让复杂度留在脚本里——那才是它该待的地方。

Having named the three pillars, the natural next question is how they relate to each other. The short answer is that they are complementary, and they become far more powerful together than any of them could be alone. The longer answer is the composition principle, which is the central design principle of the book, and which deserves its own short treatment before this chapter closes.

三根支柱既然有了名,下一个自然的问题就是:它们彼此之间是什么关系?简短的回答是:它们是互补的,合起来用,远比任一单用要强大得多。稍长的回答是组合原则(composition principle)——它是本书最核心的设计原则,值得在本章收尾之前专门用一小段来谈。

The composition principle states that any sophisticated Claude Code workflow will use all three pillars, because each pillar addresses a different class of problem, and real workflows contain all three classes of problem. Skills address the problem of how to do a task well. Subagents address the problem of how to keep different kinds of work separated. Hooks address the problem of how to enforce what must happen. A workflow that uses only skills is a workflow that can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks

组合原则陈述的是:任何一套成熟的 Claude Code workflow,都会同时使用这三根支柱;因为每根支柱分别对应着不同类别的问题,而真实的 workflow 恰好同时包含这三类问题。**Skill(技能)**回答"如何把一件事做好"的问题;**Subagent(子代理)**回答"如何把不同类别的工作彼此隔离"的问题;**Hook(钩子)**回答"如何强制那些必须发生的事真的发生"的问题。一套只用 skill 的 workflow,能教会 Claude 把活干好,却没办法隔离不同类别的工作,也没办法保证执行;一套只用 subagent 的 workflow,有了"专门化",却没有沉淀下来的知识,也没有强制执行;一套只用 hook 的 workflow

← Chapter 3 上一页

下一页 →

has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete answer.

有强制执行,却没有能力,也没有专门化。每根支柱单用,都只是"半个答案";三根合在一起,才构成完整的答案。

The composition can be illustrated with a simple image. Imagine a developer whose morning routine includes reviewing the previous day's pull requests. She has a skill called `pr-review` that encodes the specific kind of review she likes to write: substantive comments, clear suggestions, and a summary judgment. She has a subagent called `security-auditor`, specialized for flagging potential security issues, with access to the file system and to a small set of security-focused reference material. She has a hook on `PreToolUse` that blocks any attempt to approve a pull request that touches the authentication layer without her explicit confirmation.

可以用一幅简单的图景来演示这种"组合"。想象一位开发者,每天早晨的固定动作之一,是 review 前一天的 pull request。她有一个叫 `pr-review` 的 skill,把她喜欢写的那种 review 形式编码下来:实质性的评论、清晰的建议、一句总结性的判断。她有一个叫 `security-auditor` 的 subagent,专门用来标记潜在的安全问题,持有文件系统访问权,以及一小批聚焦于安全的参考材料。她有一条挂在 `PreToolUse` 上的 hook,任何试图批准"动到鉴权层"的 PR 的动作,都会被这条 hook 拦下,直到她本人显式确认。

When she sits down in the morning and asks Claude Code to review a specific PR, the skill fires because the description matches. Claude reads the skill and structures its review accordingly. During the review, Claude dispatches the `security-auditor` subagent in parallel, so that security issues are surfaced by a specialist rather than by Claude's generalist attention. The subagent returns its findings. Claude synthesizes the review, including the security findings, according to the format the skill has established. If Claude, perhaps through a subsequent prompt, attempts to approve the PR and the PR touches the authentication layer, the hook fires, blocks the approval, and informs Claude that explicit human confirmation is required. At no point has the developer had to remember to do any of this. The environment remembered. The environment composed its three pillars on her behalf, quietly, exactly when each was needed.

早晨她坐下,让 Claude Code 去 review 一个具体的 PR。Skill 触发——因为 description 匹配上了;Claude 读进 skill,按它规定的结构来组织 review。在 review 过程中,Claude 把 `security-auditor` subagent 并行派出去,让安全问题由一位专家去发现,而不是由 Claude 那颗"什么都管一点"的注意力去顺手发现。Subagent 把发现回传;Claude 再把 review 整体(连同安全发现一起)按 skill 确立的格式综合出来。接下来,如果 Claude——比如由于用户又下了一条新 prompt——试图批准这个 PR,而这个 PR 又恰好动了鉴权层,hook 立刻触发,拦住批准动作,并告诉 Claude:"必须经人类显式确认"。从头到尾,这位开发者本人没有被迫去"记得"任何一步。是环境替她记着;是环境,在每一个需要它的时刻,悄然地把这三根支柱组合起来,替她把活干了。

Engineering in Miniature · 缩微之中的工程化

This is what it looks like, in miniature, to engineer with Claude Code. Skills provide the knowledge. Subagents provide the specialized attention. Hooks provide the guarantees. The developer provides the architecture, once, and then spends the subsequent months and years harvesting the dividends of that architecture in every session she runs.

以缩微之姿,这正是"用 Claude Code 做工程化"应有的样子。Skill 提供知识;Subagent 提供专门的注意力;Hook 提供保证;而开发者本人,只需提供一次架构,然后在之后若干个月乃至若干年里,在她启动的每一个 session 中,反复收取这份架构的"股息"。

The remaining five parts of this book are, at heart, a detailed elaboration of this framework. Part Two teaches skills in depth, from their anatomy to their composition, and sends the reader out of it with a working personal skill library. Part Three teaches subagents, from their fundamentals to the orchestration patterns that unlock parallelism, and sends the reader out of it

本书剩下的五个部分,本质上,都是这套框架的细密展开。第二部分深入讲 skill——从其解剖到其组合,读完后读者手上会有一个能跑的个人 skill 库;第三部分讲 subagent——从基础到那些能解锁并行能力的编排模式,读完后读者手上会有一支——

← Chapter 3 上一页

下一页 →

with a working team of specialists. Part Four teaches hooks, from their event model to the design patterns that make them reliable, and sends the reader out of it with a hardened, secured, deterministic environment. Part Five is the composition layer, where the three pillars are assembled into a full personal platform, and two end-to-end case studies walk the reader through building that platform for specific kinds of work. Part Six closes the book by teaching how to ship, share, and evolve a Claude Code platform over time.

能跑的"专家团队";第四部分讲 hook——从事件模型到让它们可靠的设计模式,读完后读者手上会有一个加固、安全、确定性的环境;第五部分是组合层,把三根支柱拼成一个完整的个人平台,并通过两个端到端案例,带读者为某一类具体工作把那个平台真真切切地搭起来;第六部分作为收尾,讲如何把一套 Claude Code 平台发布出去、与人共享、并在时间中持续演化。

The reader who has followed the argument of this chapter now has everything she needs to understand why the book is organized the way it is. The three pillars are the organizing structure. Every chapter is an elaboration of one of them, or of the composition between them. The journey is straightforward, and the destination is a Claude Code environment that the reader has genuinely engineered to fit the shape of her work.

跟到这里的读者,现在拥有了解"本书为何如此组织"所需要的一切。三根支柱就是整本书的骨架。每一章,要么是其中一根的展开,要么是彼此组合的展开。这段旅程是直白的;旅程的终点,是一个真正按她工作的形状被工程化出来的 Claude Code 环境。

A closing thought, offered in the spirit of calibrating expectations, is worth including here. The framework in this book is deliberately simple. Three pillars, one composition principle, a small number of recurring design questions. This simplicity is a strength, not a weakness. A framework that can be held entirely in working memory is a framework that a developer can actually use, in real time, in the middle of the day, while she is trying to decide whether a given piece of workflow friction should become a skill, a subagent, a hook, or nothing at all. A more elaborate framework, with a dozen categories and a dozen principles, would be impressive to read about and useless to apply. The three-pillar framework is sized for the working life of a human being who also has other things to do. That sizing is part of the design.

这里值得插入一段收束之语,用来校准预期。本书讲的框架,刻意保持简单:三根支柱,一条组合原则,再加上一小撮反复出现的设计问题。这种简单是一种长处,不是弱点。一个能完全装在工作记忆里的框架,才是开发者真能在白天、实时、当场用得起来的框架——彼时她要决定的是:这一处 workflow 摩擦,究竟该改造成一个 skill、一个 subagent、一个 hook,还是索性不动手。一套更"宏大"的框架,带十几条分类、十几条原则,读起来很过瘾,用起来却无从下手。"三根支柱"的体量,是为"一个还有其他事要做的普通人"的工作日量身定下的。这种"尺寸",本身也是设计的一部分。

The simplicity also has an implication for the reader's relationship with the book. None of the chapters that follow require memorization. They require understanding, and the understanding is built on the same three pillars, considered from different angles. A reader who has genuinely absorbed this chapter can, in principle, reason her way to many of the conclusions the later chapters will reach. The later chapters will spare her the effort, and will also introduce specific techniques, patterns, and configurations that would be hard to derive from first principles. But the

这份"简单",同样意味着读者与本书之间一种特定的关系。后面任何一章都不需要"背下来",需要的是"理解";而这种理解,仍然建在相同的三根支柱之上,只是换着不同的角度去审视。一个真的把这一章吸收了的读者,原则上,完全可以靠推演走到后面许多章要给出的结论。后面那些章节只是替读者省去推演的力气;同时,也会介绍一些光靠"从第一性原理推"很难推出来的具体技法、模式与配置。但那

← Chapter 3 上一页

下一页 →

backbone of the thinking is already in place. What remains is the detailed craft of applying it.

个"思考的骨架"其实已经搭好。剩下的,只是把这份骨架落到实处的细密手艺。

Part One closes here. Part Two begins with the deepest single topic in the book, which is the anatomy of the `SKILL.md` file, and it is there that the real work begins.

第一部分,到这里收束。第二部分从本书最深的单一主题起步——`SKILL.md` 文件的解剖;真正的活,从那里开始。

PART TWO · 第二部分

Skills: Claude's Extensible Knowledge System

Skills:Claude 可扩展的知识系统

Knowledge that belongs in your head should not stay there. Move it.

"属于你脑子里的知识,不该一直留在那里——把它搬出来。"

CHAPTER FOUR · 第四章

Skills Fundamentals: The SKILL.md Anatomy

Skills 基础:SKILL.md 的解剖

"The structure of a thing is the meaning of a thing. Change the structure and you have changed the meaning, whether you intended to or not."

— Christopher Alexander, paraphrased

"一事之结构,即一事之意义。结构一变,意义便变——无论你是否本意如此。"

—— 克里斯托弗·亚历山大(转述)

This chapter teaches the anatomy of a skill. Not the philosophy, not the strategy, not the high-level reasons skills exist. The anatomy. By the end of this chapter, the reader will be able to look at any `SKILL.md` file in the wild and explain, line by line, what every part of it does, why it is shaped the way it is, and how it will behave when Claude encounters it in a real session. The reader will also be able to write a `SKILL.md` file from a blank page without consulting documentation, because every field, every section, and every convention will be in working memory.

本章讲的是 skill 的解剖。不是哲学,不是策略,也不是"skill 之所以存在"的高层理由——是解剖。读完本章,读者应该能够随手拿一份野外的 `SKILL.md` 文件,逐行讲清楚:每一处在做什么、为什么长成那个样子、Claude 在真实 session 中遇到它时,它会如何表现。读者还应该能在一张白纸上,不查任何文档,自己写出一份 `SKILL.md` 文件——因为每一个字段、每一节、每一条惯例,都已经装进了工作记忆。

Begin with the simplest possible definition. A skill is a folder. The folder lives in either the user's personal skills directory or in the project's skills directory. The folder contains, at minimum, one file named `SKILL.md`. The folder may also contain subfolders for supporting material, but those are optional. The skill's name is the name of the folder. That is the entire structural definition.

从最简定义起步。一个 skill,就是一个文件夹。该文件夹住在两处之一:用户的个人 skill 目录,或者项目的 skill 目录。文件夹里至少要有个文件,叫 `SKILL.md` ;它也可以再含若干子目录用来放支撑材料——但那是可选的。Skill 的名字,就是文件夹的名字。这就是"结构定义"的全部。

The folder location matters, because location determines scope. A skill placed in the user's personal skills directory, which by convention lives at `~/ .claude/skills/` on Unix-like systems, is available in every Claude Code session that user opens, regardless of which project that session is operating on. A skill placed in a project's skills directory, which lives at `.claude/skills/` at the root of that project, is available only when Claude Code is run inside that project. This split exists for an obvious reason. Some skills are personal; they reflect the way a particular developer prefers to work, and they should travel with that developer across every project. Other skills are project-specific; they encode conventions that apply only to a specific codebase, and they should travel with that codebase wherever it goes, including to teammates who clone the repository fresh.

文件夹放在哪里很关键,因为"位置"决定"作用域"。放在用户个人 skill 目录里的 skill——按惯例,在类 Unix 系统上位于 `~/ .claude/skills/` ——在该用户打开的每一个 Claude Code session 中都可用,无论那个 session 正在哪个项目上工作。放在项目 skill 目录里的 skill——位于该项目根目录的 `.claude/skills/` ——则只在 Claude Code 跑在该项目内时才可用。这种二分法存在的原因显而易见:有些 skill 是个人的,反映的是某位开发者特有的工作偏好,应该跟着这位开发者穿越每一个项目;还有些 skill 是项目专属的,编码的是只适用于某个特定代码库的规约,应该跟着那份代码库走,包括走到任何一位刚刚 clone 仓库的同事手上。

The folder name becomes the skill name. There are conventions worth following. Skill folder names should be lowercase, hyphenated, and descriptive. A skill that drafts pull request descriptions might live in a folder called `pr-description` . A skill that scaffolds new React components might live in `component-scaffold` . The name is what Claude will see when it loads the skill's metadata at startup, and it is what the developer will see in any tooling that lists installed skills. A clear, short, descriptive name is, in this respect, a small but real act of good engineering.

文件夹名就是 skill 的名字。这里有若干值得遵守的惯例:Skill 的文件夹名应当全小写、用连字符分隔、具描述性。一个负责起草 pull request 描述的 skill,可以放在名为 `pr-description` 的文件夹里;一个负责搭新 React 组件脚手架的 skill,可以放在 `component-scaffold` 。这个名,既是 Claude 在启动时加载 skill 元数据所看到的,也是开发者在任何"列出已安装 skill"的工具里会看到的。清晰、简短、具描述性的名字,在这层意义上,本身就是一项小而真实的"好工程"。

The SKILL.md File · SKILL.md 文件

← Chapter 4 起始

下一页 →

Inside the folder sits `SKILL.md` . This file has two parts, and the parts are separated by a fence of three hyphens on a line of their own. The first part is YAML frontmatter, which carries structured metadata about the skill. The second part is the markdown body, which carries the instructions Claude will follow when the skill is active. The frontmatter is short. The body is as long as it needs to be, within sensible limits that this chapter will discuss in due course.

`SKILL.md` 住在这个文件夹里。该文件有两部分,两部分之间用一行单独成行的"三个连字符"作为分界栅栏(fence)。第一部分是 YAML frontmatter,携带关于该 skill 的结构化元数据;第二部分是 markdown 正文,携带"该 skill 被激活时 Claude 应遵循的指令"。Frontmatter 很短;正文则在合理限度内,需长则长——本章稍后会讨论这个"合理限度"。

```
---
name: pr-description
description: Drafts a pull request description in the conventional format for this
  team. Use whenever the user asks for a PR description, pull request
  description, PR body, or asks "write a PR for this branch" or similar
  phrasing.
---

# PR Description

When asked to write a pull request description, follow this format exactly:

## Summary

A single sentence that explains what this PR does, written in the imperative
mood.

## What changed

A bulleted list of the concrete changes, one bullet per logical change.

## Testing

A short paragraph describing how the change was tested. If new tests were
added, mention them.

## Risk

A single sentence categorizing risk as low, medium, or high, with one short
reason.

Always read the diff before writing the description. Do not invent changes
that do not appear in the diff.
```

The example above is a complete, working skill. It has a name. It has a description. It has a body. It can be saved into a folder named `pr-description`, that folder can be placed inside `~/.claude/skills/`, and on the next Claude Code session it will be discoverable. When the user later types something like `give me a PR description for this branch`, Claude will recognize that the skill applies, load its body into context, and produce output that follows the format the body specifies.

上面这份示例,就是一个完整、可工作的 skill。它有名字,有 description,有正文。它可以存进一个名为 `pr-description` 的文件夹,再把这个文件夹放进 `~/.claude/skills/` 里;下一个 Claude Code session,这个 skill 就是可被发现的了。此刻,用户再敲一行类似 `give me a PR description for this branch` 的请求,Claude 就会识别出"这个 skill 用得上",把它的正文加载进上下文,然后产出严格按"该正文"规定格式的输出。

[注:代码块原文保留,以下为本页中译补充]

上面 YAML frontmatter 的两个字段,分别为:① `name` (skill 的内部标识,与文件夹同名);② `description` (给 Claude 看的触发说明,决定 skill 在何种用户请求下被激活)。正文则是一段简洁的指令,Claude 加载该 skill 后,会逐字遵循其中规定的格式。整段示例的"麻雀虽小、五脏俱全",是后面所有 skill 的最小模板。

Look closely at the frontmatter. There are exactly two required fields. The first is `name`, which must match the folder name and is what Claude uses to refer to the skill internally. The second is `description`, which is the field that determines whether and when the skill is invoked. The description is the most important field in the entire file, and the next chapter is devoted entirely to writing it well, because no other single decision affects skill behavior more.

把 frontmatter 凑近看:里面恰好只有两个必填字段。第一个是 `name` ——它必须与文件夹同名,也是 Claude 内部指代该 skill 时所使用的标识。第二个是 `description` ——决定该 skill 是否被调用、以及在何时被调用的字段。`description` 是整份文件里最重要的字段,下一章会整章专门讲"如何把它写好"——因为再没有其他任何单点决策,能比它对 skill 行为的影响更大。

There is also an optional third field worth knowing about now: `allowed-tools`. This field, when present, restricts the set of tools Claude is permitted to use while operating under this skill. A skill that exists only to compose written prose, for example, might restrict tools to `Read` and `Grep` and exclude `Edit`, `Write`, and `Bash`, so that the act of using the skill cannot accidentally modify the file system. The `allowed-tools` field is a Claude Code feature; when skills are used through the Anthropic API or the SDK, tool access is controlled at the application level instead. For most readers of this book, who are running skills inside Claude Code, `allowed-tools` is a useful safety mechanism that should be reached for whenever a skill has no legitimate reason to write to disk or run shell commands.

还有第三个可选字段,值得现在提一句: `allowed-tools`。这个字段如果出现,会限制"该 skill 处于激活态时"Claude 允许使用的工具集。举例来说,一个只用来组织书面文字的 skill,可以把工具集限制成 `Read` 与 `Grep`,而把 `Edit`、`Write`、`Bash` 排除掉——这样"使用这个 skill"这个动作本身,就不可能意外地改动文件系统。`allowed-tools` 是 Claude Code 的特性;当 skill 走 Anthropic API 或 SDK 时,工具访问由应用层控制。本书大多数读者,都是在 Claude Code 里跑 skill;在那种场景下, `allowed-tools` 是一道好用的安全闸,凡是一个 skill 没有正当理由去写盘或跑 shell 命令时,都该把这条闸拉上。

A Worked Example · 一个完整的具例

A second example, slightly larger, will demonstrate `allowed-tools` in context.

下面这份稍大一点的第二个示例,会演示 `allowed-tools` 在具体语境中的样子。

```
---
name: explain-codebase
description: Produces a high-level architectural summary of a codebase using
  only read operations. Use when the user asks "explain this codebase,"
  "give me a tour of this repo," "what does this project do," or any
  request to understand a codebase without modifying it.
allowed-tools: Read, Grep, Glob
---
```

Explain Codebase

When the user asks for an explanation of a codebase, follow this procedure:

1. Read the README and any top-level documentation files first.
2. List the top-level directories and infer the architectural pattern from their names.
3. Read package.json or the equivalent dependency manifest to understand the technology stack.
4. Identify the entry point of the application by checking the manifest for a "main" or "bin" field.
5. Read the entry point file and trace the first one or two layers of

```
imports.
```

6. Produce a summary with these sections:

[注:代码块原文保留,以下为本页中译补充]

这个示例的 `description` 写得相当考究——它既说明了"该 skill 干什么",又用一长串具体表述把"用户可能会怎么问"列得很齐:explain this codebase、give me a tour of this repo、what does this project do.....Claude 据此判断"何时该激活"。`allowed-tools: Read, Grep, Glob` 三件,都是只读类工具,刻意把 Edit、Write、Bash 排除在外——这就从机制上保证了"explain-codebase"这个 skill 在跑的时候,绝对不会改你任何一个文件。第 6 步"Produce a summary with these sections:"会在下一页接续展开。

← Chapter 4 上一页

下一页 →

6. Produce a summary with these sections:

- Stack
- Architecture
- Entry point and request flow
- Notable patterns or conventions
- Open questions or things you would need to read more to understand

Never modify any files while operating under this skill. The allowed-tools restriction enforces this, but the instruction is repeated for clarity.

[注:上一行起、到此行为上一页示例 SKILL.md 的续写。代码块原文保留,以下为本页中译补充]

上面这段,是上一页那份 `explain-codebase` skill 正文的剩余部分。第 6 步要求"按以下小节给出总结"——Stack、Architecture、Entry point and request flow、Notable patterns or conventions、Open questions or things you would need to read more to understand——这种"固定小节"结构让 Claude 每次产出时都按相同骨架,既方便人对比、也方便人检索。最后那句"Never modify any files"与 frontmatter 中的 `allowed-tools` 形成双保险:即使配置层被人不小心改坏,正文里的明确指令也仍然把"只读"这件事再讲一遍。

The reader should pause and notice what just happened. With twenty-five lines of plain text in a folder, the reader's Claude Code installation has been taught a procedure that is now repeatable, consistent, and safe. The next time the reader joins a new project and asks Claude to explain the codebase, the same procedure will run, in the same order, with the same output structure, and Claude will be physically incapable of writing to any file while doing so. This is the leverage skills offer. A small, careful piece of configuration produces a substantial and ongoing improvement in the developer's daily experience.

请读者在此停一拍,留意刚才发生了什么:一个文件夹、二十五行纯文本,就把"一套可重复、可一致、可安全"的工作流,教给了读者这台 Claude Code 安装。下一次读者加入新项目、让 Claude 解释代码库时,同一套流程将以同一次序、同一种输出结构跑出来;而在此期间,Claude 在物理上根本不可能向任何文件写入。这就是 skill 所给出的杠杆。一小份、用心写就的配置,就能给开发者的日常体验带来实质性、可持续的提升。

The body of a skill is not free-form, but it is flexible. There is no required template. Headings, lists, examples, code samples, references to other files, are all allowed. The two principles that govern good skill body design are clarity and concision. Clarity, because Claude is going to follow the body's instructions literally, and ambiguity in the body produces ambiguity in the output. Concision, because the body counts against the context window every time the skill activates, and a verbose body wastes tokens that could be spent on the actual work.

Skill 的正文,不是自由格式,但很灵活:并没有强制模板。标题、列表、示例、代码片段、到其他文件的引用——都允许。指导"好的 skill 正文设计"的两条原则,是**清晰与简洁**。清晰,因为 Claude 会逐字遵循正文里的指令,正文里的歧义,会原封不动地变成输出里的歧义。简洁,因为正文每次激活时都要进上下文窗口,冗长的正文是在浪费那些本可以花在"真正的工作"上的 token。

A useful rule of thumb is that a `SKILL.md` body should stay under five hundred lines. If a skill seems to want more than that, the right move is almost always to factor the surplus content into separate files within the skill folder and reference them from the body. A skill folder by convention may contain three kinds of supporting material, organized into three subfolders. The `scripts` folder contains executable code that Claude can run as part of the skill. The `references` folder contains additional documentation that Claude should read when relevant, but that does not need to live in `SKILL.md`. The `assets` folder contains files that Claude will use as templates or fixtures during the skill's execution. These three subfolders are conventions established by Anthropic in the official skills repository, and

一条好用的经验法则是: `SKILL.md` 的正文应当控制在五百行以内。如果一个 skill 看起来要超过这个量,几乎总是应当把多出来的内容拆成 skill 文件夹内的若干独立文件,再由正文去引用它们。一个 skill 文件夹按惯例可以容纳三类支撑材料,各自放在三个子目录里: `scripts` 目录,装可执行代码——Claude 在该 skill 执行过程中可以运行; `references` 目录,装额外文档——Claude 在相关时阅读,但不必常年住在 `SKILL.md` 里; `assets` 目录,装那些会被 Claude 当作模板或夹具使用的文件。这三个子目录是 Anthropic 在官方 skill 仓库里确立的惯例,

← Chapter 4 上一页

下一页 →

following them makes the skill folder predictable to read for anyone who joins later, including future versions of the developer who wrote it.

遵守这套惯例,会让 skill 文件夹对"之后加入的任何人"——包括将来某一天再回看自己代码的那位开发者本人——都保持一种"可预期的"读法。

The progressive disclosure principle, mentioned in the previous chapter, is what makes the bundled-files pattern practical. When Claude starts a session, it loads only the descriptions of available skills, not their bodies. When Claude decides that a specific skill applies, it loads that skill's `SKILL.md` body into context. Only at that point, if the body references a file in `references` or asks for the execution of a script in `scripts`, does Claude reach further into the skill folder to pick up what is needed. The total token cost of having forty skills installed, none of which is currently relevant, is roughly the cost of forty short descriptions. The cost only rises when a skill becomes relevant, and even then, the rise is bounded by what the body and any referenced files actually contain.

上一章提到过的"渐进式披露"(progressive disclosure)原则,正是让"打包文件"这种模式得以成立的关键。Claude 启动一个 session 时,只加载"可用 skill 的描述",不加载它们各自的正文。Claude 决定某个具体 skill 适用时,才会把该 skill 的 `SKILL.md` 正文加载进上下文。再接下来——也只有到了那一步——若正文引用了 `references` 里的某份文件,或要求执行 `scripts` 里的某段脚本,Claude 才会再往 skill 文件夹深处去取所需之物。一个 session 上"装了四十个 skill、但此刻一个都用不上"的代价,大致就是"四十条短描述"的总和;代价要"上涨",得等到某个 skill 真的变得相关;而即便上涨,其幅度也被正文及所引用文件所实际包含的内容所限。

Take a moment to consider the architecture from this angle. The skill system is a three-layer cache. The first layer, always loaded, is the metadata: name and description for every skill. The second layer, loaded on demand when the skill activates, is the `SKILL.md` body. The third layer, loaded only when the body asks for it, is whatever lives in `scripts`, `references`, and `assets`. Each layer costs more tokens than the one above it, and each layer is reached only when the layer above has decided that more detail is required. This cascade is what allows a Claude Code installation to host a substantial library of skills without paying a substantial token tax for them.

请换一个角度,看看这套架构。Skill 系统,其实是一座三层缓存:第一层——常驻加载——是元数据:每个 skill 的 `name` 与 `description`。第二层——按需加载,在 skill 激活时——是 `SKILL.md` 的正文。第三层——仅在正文索取时加载——是 `scripts`、`references`、`assets` 这三个目录里住着的東西。每一层的 token 代价,都比上一层更高;而每一层都只在"上一层已判定需要更多细节"时才会被触及。这道级联,正是"在一台 Claude Code 安装上挂一整个颇具规模的 skill 库,却不必为它付出巨额 token 税"的原因所在。

There is a final aspect of skill anatomy that deserves attention before this chapter closes, and it concerns the relationship between skills and Claude Code's older slash-command system. Slash commands are the explicit, user-invoked counterpart to skills. A user types a slash followed by a command name, and Claude executes the corresponding command. Slash commands have existed in Claude Code longer than skills have, and many developers have built up libraries of them. As of recent updates, slash commands and skills have been unified at the implementation level, which means that a file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both register a `/deploy` slash command and behave the same way when invoked explicitly. The practical

在本章收尾之前,skill 解剖中还有最后一个面向值得提上一笔,它涉及"skill"与"Claude Code 沿用已久的 slash-command 系统"之间的关系。Slash command 是 skill 那种"显式、由用户发起"的对应物:用户敲一个斜杠,后面跟命令名,Claude 就会去执行相应的命令。Slash command 在 Claude Code 里的历史比 skill 更久,许多开发者已经围绕它们攒起了不少命令库。在最近一次更新之后,slash command 与 skill 已经在实现层上被统一了——也就是说,放在 `.claude/commands/deploy.md` 的文件,与放在 `.claude/skills/deploy/SKILL.md` 的 skill,会注册出同一个 `/deploy` slash command,被显式调用时行为一致。落到实操——

[← Chapter 4 上一页](#)[下一页 →](#)

recommendation that emerges from this convergence is to prefer the skill format for any new work, because skills support the full feature set, including bundled files and automatic invocation, whereas the older command format is a subset.

——从这种"合流"中浮现的实操建议是:任何新工作,都该优先采用 skill 格式。因为 skill 支持完整的功能集,包括"打包文件"与"自动调用";而更老的 command 格式,只是其一个子集。

Automatic invocation is the property that makes skills feel different from slash commands in daily use. A slash command requires the user to remember the command and to type it. A skill, when its description is well written, requires nothing of the user except that she describe her task in natural language. Claude reads the descriptions, recognizes which skills apply, and activates them silently. The user does not need to know which skills exist or which is firing at any given moment. The environment simply gets better at the user's requests, in ways that match the configuration the developer has installed. This silent, automatic activation is the property that makes skills, in the words of one prominent commentator, possibly a bigger deal than MCP. The barrier between intention and capability collapses to zero. A user describes what she wants, and the right capability arrives on its own.

"自动调用"这一性质,正是 skill 在日常使用中与 slash command 体感截然不同的根源。Slash command 要求用户记得那条命令、并且亲手敲出来;而 skill,只要 description 写得到位,对用户的要求就只剩一条——用自然语言描述她的任务。Claude 会去读各条 description,识别哪些 skill 适用,再静悄悄地把它们激活。用户完全不需要知道"装了哪些 skill",也不需要知道"此刻到底是哪一条在跑"。环境会变得"对用户的请求越来越合手",而这种合手,正贴合开发者所装入的那份配置。这种"无声的、自动的激活",才是某位知名评论者口中"skill 也许比 MCP 更具分量"那一判断的依据。意图与能力之间的那道墙,被压到近乎为零——用户只需说出她想要什么,合适的能力就会自行到位。

Before this chapter closes, a third worked example is worth examining, because it shows the anatomy at its most expressive. The previous two examples were small. Real skills, the ones that earn their place in a working library, often carry a little more weight. The example below is a skill called `sql-review`, and its job is to review a SQL query for performance, correctness, and security issues before it goes anywhere near a production database.

在本章收尾之前,值得再看一份"第三个完整的具例"——因为它会把 skill 的解剖展示到最具表现力的那种程度。前两份示例都比较小。真正在工作的 skill 库里能占有一席之地的那批,往往要再重一些。下面这份示例的 skill 叫 `sql-review`——它的职责,是在一条 SQL 查询逼近任何生产数据库之前,先把它在性能、正确性、安全性上各做一轮 review。

```
---
name: sql-review
description: Reviews a SQL query for performance, correctness, and security
  issues. Use whenever the user pastes a SQL query and asks "review this
  query," "is this efficient," "will this scale," "any issues with this
  SQL," or describes wanting feedback on a query before running it. This
  skill is for review only; it does not execute the query.
allowed-tools: Read
---
```

SQL Review

When asked to review a SQL query, follow this procedure exactly.

Step 1: Read the query carefully

[注:代码块原文保留,以下为本页中译补充]

注意这份示例的 `description`——它先讲清"做什么"(review SQL),再列出一长串"用户在自然语言里可能怎么问"(review this query / is this efficient / will this scale / any issues with this SQL / 任何"在跑之前想听听反馈"的请求),最后特意加了一句"本 skill 仅做 review,不会执行查询"。这种"主动划清边界"的写法,值得每个写 skill 的人借鉴:与其让 Claude 误判激活时机,不如让 description 自己先把这层意图讲清。`allowed-tools: Read`——只读,与该 skill 的"纯 review"职责严丝合缝。第 1 步"仔细阅读 query"会在下一页展开。

← Chapter 4 上一页

下一页 →

Read every clause. Identify the tables touched, the join conditions, the filter conditions, the projection, and any subqueries or CTEs.

Step 2: Performance review

Check for:

- Missing or non-sargable WHERE clauses (functions wrapping indexed columns)
- Joins on non-indexed columns
- SELECT * in production paths
- Cartesian products from missing or wrong join conditions
- Subqueries that could be CTEs or joins for clarity and performance
- Aggregations over large tables without filters

Step 3: Correctness review

Check for:

- NULL handling, especially in joins and aggregations
- Off-by-one errors in BETWEEN and date ranges
- Implicit type coercion that could mask bugs
- LIMIT without ORDER BY, which produces nondeterministic results

Step 4: Security review

Check for:

- String concatenation building queries (SQL injection vector)
- Missing parameterization where user input is involved
- Permissions assumed but not verified

Step 5: Output format

Produce the review in this format:

Verdict

One sentence: safe to run, run with caution, or do not run.

Performance

A short bulleted list of issues, each with a one-line explanation and a one-line fix.

Correctness

Same format.

Security

Same format.

[注:代码块原文保留,以下为本页中译补充]

整份 `sql-review` 的正文,展示了"多步骤、严格流程型"skill 的典型写法:Step 1 把 query 读透——把表、join 条件、过滤条件、投影、子查询/CTE 都标出来;Step 2 在性能维度做检查清单(sargable、索引覆盖、SELECT *、笛卡尔积等);Step 3 在正确性维度做检查清单(NULL 处理、BETWEEN 边界、隐式类型转换、LIMIT 不带 ORDER BY 等);Step 4 在安全性维度做检查清单(字符串拼接、未参数化、未核实权限);Step 5 固定输出格式——Verdict / Performance / Correctness / Security 四个小节,每个小节都"一句解释 + 一句修复"。这种"流程清晰、检查清单可枚举、输出格式固定"的写法,是把"人类专家的复盘经验"装进 Claude 的标准做法,值得在所有复杂 skill 上照搬。

Multi-File Skills · 多文件 Skill

That example deserves a moment of attention. It is twice as long as the earlier examples, and the additional length is doing real work. The body is broken into named steps, each step has a focused checklist, and the output format is specified down to the section headings. A skill written this way produces strikingly consistent output, because almost every degree of freedom that could produce variability has been pinned down. The query is read in the same order every time. The same five performance issues are looked for in the same order. The output is structured the same way every time. The reader who runs this skill against ten different queries over the course of a week will get ten reviews that feel like they came from the same careful reviewer, because in a meaningful sense they did.

那份示例值得停一拍来品味。它比前面的示例长了一倍,而那多出来的长度,确实在于实事:正文被切成若干"已命名的步骤",每一步都带一份聚焦的检查清单,输出格式被规定到小节标题这一颗粒度。这样写出来的 skill,产出的输出会有一种"惊人的一致"——因为几乎每一个可能造成差异的自由度,都被钉死了。Query 每次都被以相同的次序去读;那五条性能问题每次都被以相同的次序去查;输出每次都被按相同的结构来组织。读者一星期内拿这条 skill 跑十条不同的 query,会得到十份"看起来出自同一位谨慎 reviewer 之手"的 review——而从某种意义上讲,也确实如此。

This is a property worth dwelling on. Skills do not merely save typing. They produce consistency. A developer working alone can have ten different commit messages on a Monday and forty different commit messages by Friday, all of them written by her, none of them following the same internal logic. The same developer with a commit-message skill installed will have one hundred commit messages by the end of the month that all follow the same format, because the format is no longer carried by her attention but by the skill. Consistency, in this sense, is a separate benefit from speed, and it compounds in a different way. Speed saves time per commit. Consistency makes the entire history of commits more useful, because future readers can rely on the format being stable.

这一性质值得多停留一会儿。Skill 不只是省打字——它产出一致性。一个独自干活的开发者,周一可能写出十种不同风格的 commit message,到周五已经积出四十种,全是她写的,却没一份遵守同一条内在逻辑。同一开发者一旦装上 commit-message skill,到月底她会写出一百条 commit message,全部遵循同一格式——因为那套格式不再由她的注意力承载,而是由 skill 承载。一致性,在这种意义上,是"速度"之外另一份独立收益,并且以另一种方式复利。速度省下的是每条 commit 的时间;一致性则让整段 commit 历史都变得更可用,因为未来的读者可以放心地"假定格式是稳定的"。

Two more anatomical details deserve attention before this chapter closes, because they come up often in real skill libraries and are worth knowing the first time the reader encounters them. The first detail concerns the relationship between the description and the body. A common confusion among new skill authors is the temptation to repeat in the body what the description already says. This is a waste of context, because the description

本章收尾之前,还有两条"解剖层面的细节"值得提一笔,因为它们在真实的 skill 库里频繁出现,值得读者第一次撞见时就了然于胸。第一条,关于 description 与正文之间的关系。Skill 作者新手上路时,常犯的一个误区,就是把 description 已经讲过的话,在正文里再复述一遍——这是对上下文的浪费,因为 description 在该 skill 被"考虑是否激活"时,本来就已经——

Suggested rewrite

If issues are significant, provide a rewritten query in a fenced code block.
If the original is fine, omit this section.

Do not invent issues. If the query is good, say so plainly. A short clean review is better than a padded one.

[注:代码块原文保留,以下为本页中译补充]

上面是 `sql-review` skill 的最后一段正文——新增了"### Suggested rewrite"小节,只有在问题显著时才给出"重写后的 query",并且以一句金句收尾:"不要无中生有。Query 没问题就直说;一份干净简短的 review,胜过一份注水的 review。"这正是把人类 reviewer 的职业操守——"克制"——写进 skill 的好例子。

← Chapter 4 上一页

下一页 →

is already loaded into context whenever the skill is being considered. The body should pick up where the description leaves off. The description tells Claude when to fire. The body tells Claude what to do once it has fired. Treating the two as having distinct jobs prevents the kind of bloated SKILL.md file in which the first paragraph of the body is a restatement of the description.

被加载进上下文了。正文应当从 description 停下的地方继续往下接。Description 告诉 Claude 何时激活;正文告诉 Claude 激活之后做什么。把这两件事视作不同分工,就能避免那种"开篇第一段只是复述 description"的臃肿 SKILL.md。

The second detail concerns the cost of having many skills. A natural worry is that installing fifty skills will somehow degrade the performance of every Claude Code session. In practice, the cost is bounded by progressive disclosure. Fifty skills means fifty descriptions loaded at startup. A typical description is two or three sentences, perhaps thirty to fifty words. Fifty descriptions is therefore on the order of two thousand words, or roughly three thousand tokens. That is a real cost, but it is a small one compared to the size of a modern context window, and it is paid once at session start rather than on every prompt. The body of any specific skill is loaded only when that skill activates, and only one or two skills typically activate per turn. The cost of skill ownership scales gracefully. A library of fifty good skills is not noticeably slower than a library of five.

第二条,关于"装很多 skill"的代价。一个很自然的担心是:装五十个 skill,会不会拖累每一个 Claude Code session 的表现?在实践中,这个代价被"渐进式披露"压在了很小的范围内。五十个 skill,意味着启动时加载五十条 description;一条典型的 description 是两到三句、约三五十个词;五十条合起来,大致是两千个词、约三千 token。这是一笔真实开销,但相对于现代上下文窗口的容量,它只是很小的一笔,且只需在 session 启动时一次性付清,而不是每条 prompt 都要付一次。任何具体 skill 的正文,只在该 skill 激活时才加载;而每轮通常只有一两个 skill 会激活。"拥有 skill 的成本"是优雅地扩展的——五十个好 skill 的库,与五个好 skill 的库,在体感上几乎察觉不到差异。

This is the anatomy of a skill, complete enough to begin building. A folder, named after the skill, in either the user or project skills directory. A SKILL.md file at the root of that folder, with frontmatter declaring at minimum a name and a description, optionally restricting tools, and a markdown body containing the instructions. Optional subfolders for scripts, references, and assets, picked up by Claude only when the body refers to them. A clear, automatic activation model that does not require the user to remember anything. With this anatomy in working memory, the reader is ready for the chapter that follows, in which a real skill is built from the empty folder forward, with every decision made explicit and every common mistake catalogued. The work begins now.

到这里,skill 的解剖就讲完了——足够让读者动手开始搭建:一个文件夹,以 skill 命名,放在 user 或 project 的 skills 目录里;一份 SKILL.md 文件,位于该文件夹根目录,frontmatter 至少声明 name 与 description,可选地限制 tools,正文是 markdown 写成的指令;可选的 scripts、references、assets 子目录,仅在正文引用时被取用;一套清晰、自动的激活模型,不需要用户去"记"什么。

把这份解剖装进工作记忆之后,读者就为下一章做好了准备——下一章,会从一张"空文件夹"开始,正向地把一份真正的 skill 搭出来;每一步决策都讲透,每一处常见错都收齐。活,从这里正式开干。

CHAPTER FIVE · 第五章

Writing Your First Skill: A Hands-On Build

亲手写出你的第一个 Skill

"The shortest distance between a developer and a working system is the smallest example that ships."

— Folk wisdom of the open source world

"从开发者到一套能跑的系统,最短的距离,是那份'小到刚刚能发出来'的示例。"

—— 开源世界的民间智慧

This chapter is a build. By the end of it, the reader will have a working skill installed in her own Claude Code environment, will have used it in a real session, and will have debugged the most common failure mode skills exhibit. The build is small on purpose. The point is not the impressiveness of the example. The point is that every step of the path from empty directory to working skill becomes familiar, so that subsequent skills, larger and more sophisticated, can be built without having to relearn any of it.

这一章,就是一次"搭"。读完之后,读者会有一份"已安装、跑得起来"的 skill,放在她自己的 Claude Code 环境里;会在一次真实 session 中用过它;会亲手 debug 过 skill 最常出现的那一种失败模式。这个搭出来的 skill 故意做得很小。重点不在"示例有多炫",重点在于——从"空目录"到"能跑的 skill"这条路上的每一步,都要变得熟悉;这样以后再搭更大、更复杂的 skill 时,就不用把每一步都重新学一遍。

The skill being built in this chapter is called `commit-message`. Its job is to take the staged changes in a git repository and produce a single commit message in the conventional commits format. This is a skill that almost every developer would benefit from, because almost every developer has, at some point, committed a change with a message that her future self struggled to interpret. The skill will fix this once and for all, by ensuring that every commit message Claude writes follows a consistent format, includes a meaningful summary, and surfaces any breaking changes prominently.

本章要搭的 skill 叫 `commit-message`。它的职责是:拿到一个 git 仓库里的暂存改动,产出一条按 conventional commits 规范写成的 commit message。这是一条几乎每位开发者都用得上的 skill——因为几乎每位开发者都曾在某个时刻,提交过一条"日后回看、连自己都看不懂"的 commit message。这条 skill 会把这件事一劳永逸地治好:Claude 写出的每一条 commit message,都遵守同一格式、都带一句有意义的摘要、并且把任何 breaking change 显著地摆在显眼位置。

Before opening any files, do the architectural thinking. The decision is, first, where the skill should live. This is a skill that the developer will want available in every project, not only in one. Therefore the skill belongs in the personal skills directory, at `~/.claude/skills/`. The decision is, second, what the skill needs to be able to do. To inspect staged changes, the skill needs the ability to run shell commands, specifically

`git diff` . To compose the message, the skill needs to be able to think about the diff. It does not need to write to any file directly, because the user will take the produced message and pass it to `git commit` herself. Therefore the `allowed-tools` field will include `Bash` but not `Edit` or `Write` .

动手开文件之前,先在脑子里把"架构"想清楚。第一项决策,放在哪里?这是一条"开发者希望在每个项目里都得上"的 skill,而不只是某一个项目。因此,它该住在个人 skill 目录里,也就是 `~/.claude/skills/` 。第二项决策,它需要具备哪些能力?要审阅暂存改动,它得有跑 shell 命令的能力——具体说,是 `git diff` 。要把 commit message 组出来,它得有"对 diff 动脑"的能力。它并不需要直接写任何文件——因为用户会把生成的 message 拿去自己跑 `git commit` 。因此 `allowed-tools` 字段里,会带上 `Bash` ,但不带 `Edit` 、`Write` 。

← Chapter 4 结尾

下一页 →

The decision is, third, what the description should say. This is where novice skill authors most often fail, because the description has two jobs that pull in opposite directions. It must be specific enough that Claude only fires the skill when it genuinely applies, and it must be broad enough to fire on the variety of phrasings real users actually employ. A description that is too narrow will fail to fire on requests that should clearly trigger it. A description that is too broad will fire when it should not. The next chapter is devoted to the craft of description writing in depth, but for this first build, a reasonable working approach is to draft a description that lists the canonical request, then enumerates a few alternative phrasings the user might use, and finally states what kind of input the skill expects to be working from.

第三项决策,description 该怎么写?这正是 skill 写手新手最容易翻车的地方——因为 description 同时扛着两份彼此拉扯的职责:它必须足够具体,Claude 才不会在不该激活的时候乱激活;又必须足够宽泛,才能在真实用户千变万化的措辞下都触发得起来。写得太窄,会漏掉那些"明明该触发"的请求;写得太宽,会在不该触发时乱触发。下一章会深入讲 description 写作的手艺;不过就本章这次"首次搭建"而言,一个够用的工作方法是:先写一句"典型请求",再列举几条"用户可能换用的措辞",最后说明本 skill 期望自己拿到的是哪一类输入。

Setting Up the Folder · 创建文件夹

With these decisions made, the build can begin. Open a terminal. Create the folder.

这些决策一旦落定,搭建就可以开始了。打开终端,把文件夹建出来。

Inside that folder, create a file named `SKILL.md` . The file will contain frontmatter, then a body. Open the file in any editor and type the following.

在那个文件夹里,新建一个名为 `SKILL.md` 的文件。文件包含 frontmatter,再接一段正文。用任何编辑器打开,敲入下面这些内容。

```
mkdir -p ~/.claude/skills/commit-message
cd ~/.claude/skills/commit-message
```

```
---
name: commit-message
description: Generates a conventional commits message from the currently
  staged changes in a git repository. Use whenever the user asks for a
  commit message, asks to "write a commit," asks "what should I commit
```

```
this as," or otherwise indicates they have staged changes and want a
message written. Always inspect the diff before writing the message.
allowed-tools: Bash, Read
---

# Commit Message

When asked to produce a commit message, follow this procedure exactly.

## Step 1: Inspect the staged changes

Run `git diff --cached` to see what is currently staged. If nothing is
staged, run `git diff` to see unstaged changes and tell the user that no
changes are staged but here is what would be committed if they were. Do
not proceed to write a message until you have actually read the diff.
```

[注:代码块原文保留,以下为本页中译补充]

第一段是 shell 命令: `mkdir -p ~/.claude/skills/commit-message` 建文件夹,接着 `cd` 进去。第二段是 `SKILL.md` 的内容。注意:① `description` 段在 "Generates a conventional commits message..." 之后,枚举了一长串用户可能用的措辞(a commit message / write a commit / what should I commit this as),并在末尾强调"Always inspect the diff before writing the message"——把"先读 diff、再写 message"这条铁律写死在 `description` 里;② `allowed-tools: Bash, Read` 正好对应"读 diff + 跑 git 命令"的需求;③ 正文"## Step 1"先硬性规定:必须先看 diff,没 staged 也要明示用户,绝不允许在没读 diff 的情况下动手写 message——这是对"用 skill 之前先把数据看清"这条职业操守的具体化。

← Chapter 5 上一页

下一页 →

Step 2: Choose the type

The commit type is the first part of a conventional commit and must be one of the following:

- feat: a new feature visible to the user
- fix: a bug fix
- refactor: a change in code that does not alter behavior
- docs: documentation only
- test: tests only
- chore: build tooling, dependencies, configuration
- perf: a change focused on performance
- style: formatting only, no code change

Choose the most specific type that fits. If a single commit spans multiple types, choose the most user-visible one and mention the others in the body.

Step 3: Choose the scope

The scope is an optional one-word noun in parentheses indicating which area of the codebase changed. Examples: `feat(auth)`, `fix(api)`, `refactor(parser)`. Use a scope when the diff is concentrated in a recognizable area. Omit it when changes are diffuse.

Step 4: Write the summary

The summary is the rest of the first line. Keep it under 72 characters. Use the imperative mood: "add user authentication," not "added user authentication." Do not capitalize. Do not end with a period.

Step 5: Write the body, if needed

If the change is non-trivial, add a blank line after the summary, then write a short paragraph explaining what changed and why. Keep it under five lines. Skip the body for trivial changes.

Step 6: Mark breaking changes

If the diff contains anything that breaks an existing API or changes default behavior, add a line at the bottom that begins with ``BREAKING CHANGE:`` followed by an explanation. This line is required for breaking changes and must not be omitted.

Step 7: Output

Output the final commit message in a single fenced code block, with no commentary before or after. The user will copy it directly into `git commit`.

Examples

[注:代码块原文保留,以下为本页中译补充]

第 2 步——选 type:从 feat / fix / refactor / docs / test / chore / perf / style 八种里挑最贴切的一种;若一种 commit 跨多类,选"对用户最可见"的那种,其余的写进 body。第 3 步——选 scope:用括号里的一词名词,标出代码库的哪一块被改;只改一处明确区域时给,跨多处则省。第 4 步——写 summary:第一行 72 字符以内,imperative mood(动词原形起头)、不大写、不以句号收尾。第 5 步——写 body(可选):非琐碎改动加一段简短说明,五行以内。第 6 步——breaking change:若有破坏既有 API 或默认行为的改动,文末必须加一行 `BREAKING CHANGE:` 起头,这一行不得省略。第 7 步——输出:整条 commit message 用一个 fenced code block 输出,前后不附任何评注;用户会原样复制到 `git commit`。第 8 步 "## Examples" 会在下一页接续展开。这一整段正文,示范了"把一个公认规范 (Conventional Commits) 整体编码进 skill"的标准动作:每一步都给具体规则,把"判断空间"压到最小,只留"读懂 diff"那一处真正需要判断的地方。

← Chapter 5 上一页

下一页 →

A good message:

```

feat(auth): add password reset endpoint

Adds POST /auth/reset that accepts an email and sends a reset link.  
Token expires in one hour. Tested with Postman.  
```

A bad message that this skill must avoid:

```
```\nfixed stuff\n```
```

[注:代码块原文保留,以下为本页中译补充]

这两段"对照示例",是 `commit-message` skill 正文末尾"## Examples"小节的实际内容:第一条是一份"好 message"——type (feat)、scope (auth)、72 字符内 imperative mood 的 summary、加上不超过五行的 body,完整示范了 conventional commits 规范;第二条是"必须避开的坏 message"——`fixed stuff`,四个字,既无 type、也无 scope、更没 body。把这种"好/坏"对照放进 skill,等于把人类 reviewer 心里那杆秤,显式地摆到了 Claude 面前。

Save the file. The skill is now installed. To verify, restart Claude Code so that the new skill is discovered at the next startup. Open Claude Code in any git repository where some changes are staged. Type, in the most natural phrasing the developer can think of, something like `give me a commit message for this`. Claude should recognize that the `commit-message` skill applies, run `git diff --cached` on its own, read the diff, and produce a commit message that follows the seven-step procedure laid out in the body. The whole interaction should feel as if the developer has briefed a careful collaborator, once, and the collaborator now silently applies the same procedure every time the request is made.

保存文件。Skill 这就装上了。要验证它,得重启 Claude Code,让新 skill 在下次启动时被扫到。在任何一个已经有暂存改动的 git 仓库里打开 Claude Code,用开发者能想到的最自然的措辞,敲下类似 `give me a commit message for this` 的话。Claude 应当识别出 `commit-message` skill 适用,自己跑去 `git diff --cached`,读完 diff,然后按正文里七步流程产出一条 commit message。整段交互,看起来就像是开发者向一位谨慎的协作者做了一次情况通报,从此那位协作者每次接到类似请求,都会默默按同一份流程办。

The first time a developer builds a skill, the most common failure mode is not what one might expect. The failure is not that the skill misbehaves. The failure is that the skill does not fire at all. The developer types a request that should clearly activate the skill, and Claude proceeds to write a commit message in some other format, ignoring the skill entirely. This failure has one cause in roughly nine cases out of ten, and the cause is the description.

开发者第一次搭 skill 时,最常碰到的失败模式,并不是大家预想的那种。失败不是"skill 行为不对",而是"skill 干脆不触发"——开发者敲了一段本应明确激活该 skill 的请求,Claude 直接用别的格式写出了一条 commit message,完全无视这份 skill。这种失败,十有八九只有一个原因,而那个原因,就是 description。

## Failure Modes and Fixes · 失败模式与修法

Walk through what is happening when the failure occurs. At session start, Claude loaded the description of the `commit-message` skill into its context. When the developer typed her request, Claude scanned the descriptions of the available skills and assessed whether any of them matched the request. If the description does not contain language that resembles the developer's request closely enough, the assessment fails, and the skill does not fire. The skill is installed correctly. The body is written correctly. But the description

让我们把失败那一刻"到底发生了什么"走一遍。在 session 启动时,Claude 把 `commit-message` skill 的 description 加载进了上下文。开发者敲下她的请求之后,Claude 把"可用 skill 的 description 们"扫一遍,评估其中是否有哪一条与该请求匹配。如果 description 里没有"在语言上与开发者的请求足够相似"的内容,这次评估就会失败,skill 不触发。Skill 装得对,正文写得对——唯独 description——

has not done its job, which is to bridge the gap between the developer's natural phrasing and the skill's identity.

没把自己的活干完——那份活,正是"在开发者的自然措辞"与"该 skill 的身份"之间搭一座桥。

When this happens, the fix is almost always to expand the description. Add the phrasings the developer actually uses. Add synonyms. Add the verbs and nouns from the developer's tone of voice. The description above already includes several alternatives: the canonical "asks for a commit message," and the alternatives "write a commit," "what should I commit this as," and "indicates they have staged changes and want a message written." These multiple framings are not redundant. They are the surface area of the skill, the places where natural language can land and still be recognized as a request for this skill.

一旦发生这种情况,修法几乎都是去"扩充 description":把开发者实际会用的措辞加进去、把同义词加进去、把开发者语气里的动词与名词加进去。本例的 description 实际上已经包含了好几种说法——典型说法"asks for a commit message",以及几条变体"write a commit"、"what should I commit this as",再加上"indicates they have staged changes and want a message written"。这些多角度的措辞,不是冗余——它们就是这份 skill 的"表面积",是自然语言可能落脚、却仍然能被识别为"请求该 skill"的所有落脚点。

A useful diagnostic exercise, when a skill fails to fire, is to ask Claude directly. Type into the session, `why did the commit-message skill not fire on my last request?` Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

一份好用的诊断手法,是在"skill 不触发"时直接问 Claude。在 session 里敲一句 `why did the commit-message skill not fire on my last request?`, Claude 通常会给出一份有用的回答,把"请求与 description 之间的空缺"直接点出来。那段回答,就相当于一份精确的指令:你该往 description 里加什么。改完,重启 session 让变更生效,再试一次。两到三轮迭代之内,几乎所有"该触发却不触发"的问题,都能通过细化 description 解决。

There are two other failure modes worth naming, though both are less common than the triggering problem. The first is the over-firing problem, in which the description is so broad that the skill activates on requests that should not have triggered it. A `commit-message` skill whose description merely says "anything related to git" will fire when the user asks how to revert a commit, which is the wrong behavior. The fix is to narrow the description by adding negative phrasings or tighter scope language. The second failure mode is the body problem, in which the skill fires correctly but produces output that does not match what the body specifies. This is almost always a sign that the body is ambiguous in some way that the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Examples in the body, especially good and bad examples placed side by side, are extraordinarily effective for this.

还有两种失败模式值得点出名字,虽然都不如"不触发"那么常见。第一种是"过度触发"问题:description 写得太宽,导致 skill 在不该激活的请求上也激活了。一份 `commit-message` skill,如果 description 只写"anything related to git",那用户在问"怎么 revert 一个 commit"时也会触发——这是错误行为。修法是把 description 收窄:加否定性措辞,或者用更紧的 scope 表述。第二种是"正文问题":skill 触发正确,但产出的输出与正文规定不符。这几乎总是一个信号——正文里有某处歧义,是作者写的时候没注意到的。修法是把自已当作"第一次读这份正文"的读者,找出"指令与输出"开始分叉的那个点,把语言收紧。在正文里放示例,尤其是"好例子与坏例子并列",对治这种问题异常有效。

## The Skill in Real Use · 把 Skill 用到真实场景

The skill is now built, installed, and verified. The developer has, in roughly twenty minutes of focused work, installed a piece of automation that will pay her back every single time she commits code for the rest of her time using Claude Code. The investment is small. The dividend is daily, indefinite, and silent. There will be no celebration in the terminal when the hundredth commit message gets written by this skill. The developer will simply notice, weeks from now, that her commit history reads more cleanly than it used to, and that she stopped having to think about commit messages somewhere along the way. This is what skills, in aggregate, do to a developer's working life. Each one is small. The compounding is large.

Skill 已经搭好、装上、验证过。开发者用大约二十分钟的专注工作,装进了一份"自动化"——它会在她今后用 Claude Code 的每一次 commit 中,反复回赠给她。投入很小;分红,则是日复一日、绵延不绝、悄无声息的。第一百条 commit message 被这条 skill 写出来的那一刻,终端里不会有任何庆祝;几周之后,这位开发者只是会忽然发现:自己的 commit 历史读起来比从前顺多了,而在某个不知名的时点之后,她再也没为 commit message 操过心。这,就是 skill 们,作为一份"集合",对开发者职业生涯真正做的事——每一条都很小,复利却很大。

A worked debugging walkthrough is worth including here, because the experience of debugging a skill is one of the moments that most reliably separates developers who go on to build serious skill libraries from those who give up after one or two attempts. The walkthrough is from a real session. The skill author had built a skill called `explain-error`, intended to produce structured debugging guidance from a stack trace. She tested it by pasting a Python traceback and asking, `what does this mean`. The skill did not fire. Claude produced a reasonable answer, but not the structured output the skill body specified. The author was confused, because the description seemed clear. She typed into the same session, `why did the explain-error skill not fire on my last request`. Claude responded with a short paragraph: the description mentions errors and stack traces, but the user's request used the word *means* rather than any of the canonical phrasings, and the description had no language that bridged the gap between asking what something means and asking for an error explanation.

这里值得插入一份"完整的 debug 走读",因为"调一个 skill"的体验,正是最能把"会继续搭出严肃 skill 库的人"与"试一两次就放弃的人"区分开来的那种时刻。这份走读来自一次真实的 session。Skill 作者搭了一个叫 `explain-error` 的 skill,本意是"基于一段 stack trace,产出结构化的调试指引"。她用粘贴一段 Python traceback 并问 `what does this mean` 来测试。Skill 没触发;Claude 给了一份合理的回答,但不是 skill 正文规定的那种结构化输出。作者很困惑,因为 description 看起来清清楚楚。她在同一个 session 里敲了一句 `why did the explain-error skill not fire on my last request`。Claude 回了一段简短的诊断:description 里提到了 errors 和 stack traces,但用户请求用的是 *means* 这个词,并不在那些典型措辞之列;description 里没有"在'问某事是什么意思'与'请求一个错误解释'之间搭桥"的措辞。

This was a precise diagnosis. The author edited the description, adding the phrasing `what does this mean`, `what is going on here`, and `what is happening` to the trigger language. She restarted the session. She pasted the same traceback and asked, `what does this mean`. The skill fired. The output followed the structured format the body specified. The total time from first failure to working skill was roughly four minutes. The lesson the author took away was not about this specific skill. The lesson was about the general method: when a skill fails to fire, ask Claude why, edit the description in light of the answer, restart, and try again. This loop converges quickly, almost always within two or three iterations. Developers who

这是一份精确的诊断。作者随后编辑了 description,加进 `what does this mean`、`what is going on here`、`what is happening` 这几条触发语言;重启 session;再粘贴同一段 traceback,问 `what does this mean`——这次 skill 触发了,输出严格遵守正文中规定的结构化格式。从第一次失败到"能跑",总共花了大约四分钟。这位作者从这件事里带走的,不是关于"这个具体 skill"的什么道

理,而是关于通用方法的道理:当一个 skill 该触发却不触发时,去问 Claude 为什么;依着答案去改 description;重启,再试。这个回路收敛得很快,几乎总在两到三轮之内完成。那些——

internalize the loop stop being intimidated by description writing, because the feedback cycle is fast enough that experimentation is cheap.

把这条回路真正内化的人,从此不会再被"写 description"这件事吓住——因为反馈周期已经快到让"做实验"变得很便宜。

A second worked build will reinforce the procedure. The skill being built is called `slack-update`. Its job is to take a description of what the developer did today and produce a short, well-formatted Slack message suitable for posting to a team channel as an end-of-day update. This is a skill that fits a specific context but appears in almost every working developer's life: at four-thirty in the afternoon, the developer wants to summarize what she did today, in a format that fits the team's norms, with the right level of detail for an audience who was not in her head while she did the work.

我们再用第二个完整搭建,把整套流程强化一遍。要搭的 skill 叫 `slack-update`。它的职责是:把"开发者今天都做了啥"的一段描述,变成一条简短、格式良好的 Slack 消息,适合作为"日终更新"贴到团队频道里。这条 skill 适用的场景很具体,却几乎出现在每一位在职开发者的生活里——下午四点半,她想把今天做的事概括一遍,得用合乎团队规范的那种格式、用一个"今天并不在开发者脑子里"的听众能承受的那种粒度。

### Reflection on the Build · 搭建复盘

The architectural thinking, applied to this skill, runs like this. The skill belongs at the user level, because end-of-day updates are personal and follow the developer across projects. The skill needs the ability to read recent commits or recent file changes, because those are the raw material from which the update is composed; that argues for `Bash` and `Read` in `allowed-tools`. The skill does not need write access, because the developer will paste the result into Slack herself. The description should fire on phrasings like `give me a Slack update`, `what should I post in standup`, `write my end-of-day message`, and similar.

把这套"架构思考"套到这条 skill 上,跑出来是这样的:它该放在 user 级,因为"日终更新"是个人的,且跟着这位开发者穿越各个项目;它需要有读"近期 commit"或"近期文件改动"的能力——那是从中萃取出更新内容的原材料——所以 `allowed-tools` 里要带 `Bash` 和 `Read`;它不需要写权限,因为用户会自己把结果粘进 Slack;description 应当能在 `give me a Slack update`、`what should I post in standup`、`write my end-of-day message` 这一类措辞下被触发。

```

name: slack-update
description: Produces a short, well-formatted Slack message summarizing what
 the developer accomplished today. Use whenever the user asks for "a Slack
 update," "an end-of-day message," "what should I post in standup," "a
 daily update," "summarize my day," or describes wanting to share progress
 with the team. This skill is for outbound updates the developer is
 writing, not for summarizing other people's messages.
allowed-tools: Bash, Read

```

## # Slack Update

When asked for a Slack update, follow this procedure.

### ## Step 1: Gather material

[注:代码块原文保留,以下为本页中译补充]

这份示例与上一份 commit-message 是"同一族兄弟":同样的 user 级 + 同样的 allowed-tools: Bash, Read + 同样的"先列步骤再做活"骨架。注意 description 末尾特意加了一句"This skill is for outbound updates the developer is writing, not for summarizing other people's messages"——主动划清"我做什么"与"我不做什么"的边界,正是前文总结过的"主动消除歧义"那一招的活学活用。第 1 步 "## Step 1: Gather material" 会在下一页接续展开。

← Chapter 5 上一页

下一页 →

The slack-update skill, like the commit-message skill before it, took roughly ten minutes to write once the architectural decisions had been made. The body is longer because the task has more structure, but the discipline is the same: name the steps, specify the format down to the punctuation, and leave no ambiguity about tone or content. The skill, once installed, will compose end-of-day updates that fit a consistent shape across every working day, and the developer will stop spending the four minutes per day she used to spend on this small but real task.

slack-update 这条 skill,和前面的 commit-message 一样,等架构决策一旦定下来,真正写也就花了大约十分钟。正文更长,是因为这件事本身的结构更细,但纪律仍然是同一条:把步骤命名清楚、把格式规定到标点级别、语气与内容都不留歧义。这条 skill 装上之后,每天产出的"日终更新"都会套进同一个形状里;而开发者每天为这件"小但真实"的工作省下的那四分钟,也就此一笔勾销。

There is a final point worth making about the experience of building skills, which only becomes visible after several have been built. The first

关于"搭 skill"这件事的体验,有一个最后想说的观察,它在搭过若干条 skill 之后才会浮现。头一

```
If the user has described what they did today, use that as the primary
material. If they have not, run `git log --since="this morning"
--author="$(git config user.email)"` to surface today's commits, and
ask the user to confirm or add any uncommitted work.
```

### ## Step 2: Categorize

Group the work into at most three categories: shipped, in progress, and blocked. If a category is empty, omit it.

### ## Step 3: Format

Produce the message in this format:

```
Today
- [bullet, one short line per shipped item]

In progress
```

- [bullet, one short line per in-progress item, with a note on what's next]

\*Blocked\*

- [bullet, what's blocked and what would unblock it]

Use Slack markdown: asterisks for bold, no headings, plain bullets. Keep each bullet under 100 characters. The whole message should fit in a single screen without scrolling.

#### ## Step 4: Tone

Match the developer's tone. If the work was routine, keep it factual. If something genuinely went well, allow a single short positive note. Do not invent enthusiasm or use phrases like "exciting progress" unless the developer has signalled that mood.

Do not include items the developer did not mention or that do not appear in the day's commits.

[注:代码块原文保留,以下为本页中译补充]

这一段是 slack-update skill 正文的剩余部分。第 1 步"Gather material"先尊重"用户主动描述"为主原料,缺则用 `git log --since="this morning"` 去挖今日 commit,并请用户确认或补未提交的工作;第 2 步"Categorize"将工作归入至多三类——shipped / in progress / blocked,哪一类空就省;第 3 步"Format"直接规定三条 Slack markdown 模板(\*Today\* / \*In progress\* / \*Blocked\*),每条 bullet 不超过 100 字符、整条消息一屏装下;第 4 步"Tone"对语气做了相当细的拿捏——常规就写实,有好事容许一句小赞,不许生造热情;未了硬性规定"不许写进用户没提的、或当日 commit 里没有的事项"——这一条,正是"不无中生有"那条职业操守在该 skill 上的具体落地。

← Chapter 5 上一页

CHAPTER SIX →

skill is intimidating. The second skill is easier. By the fifth skill, the act of building has become almost automatic: the developer thinks of a recurring friction, knows immediately that a skill could absorb it, and writes the skill in fifteen minutes. The barrier to building the next skill, after the first few, is no longer technical. It is a matter of recognizing the friction in the moment and choosing to invest fifteen minutes in eliminating it permanently. Most developers do not invest those fifteen minutes, because they are busy with whatever produced the friction in the first place. The developers who do invest those fifteen minutes accumulate, week by week, an environment that becomes increasingly tailored to the way they actually work, and the gap between their environment and the environment of a colleague who does not invest grows wider over time.

条 skill 让人发怵;第二条,轻松一些;到第五条的时候,"搭 skill"这件事已经几乎变成本能——开发者察觉到一个反复出现的摩擦,立刻就知道"这条摩擦能喂给一条 skill",然后用十五分钟把它写出来。最初几条之后,搭下一条 skill 的门槛,已经不再是技术性的。它变成了一件事:在摩擦出现的当口认出它,并选择投入那十五分钟,把它从"永远的摩擦"变成"永远的消失"。大多数开发者,不会投入那十五分钟——他们正忙着应付"摩擦"本身。而那些愿意投入的开发者,会一周一周地累积出一个越来越贴合自己工作方式的环境;他们的环境与"不愿投入"的同事之间的鸿沟,会随时间越拉越宽。

A small habit is worth recommending here. The reader who is just beginning her skill library should keep a running list, in a plain text file, of frictions she notices during her working day. The list does not need to be sorted, prioritized, or organized. It just needs to exist. At the end of

each working week, she opens the list and picks one or two items to convert into skills over the weekend. The list keeps her honest about which frictions are real and recurring versus which are rare and one-off. The weekly review keeps the library growing at a sustainable pace, neither too fast nor too slow. After three months of this practice, most developers find that they have a library of fifteen to twenty skills, all of them addressing real frictions, all of them used regularly, and all of them earning their place. This is the steady-state pattern that produces lasting results, and it requires no more than thirty minutes per week of deliberate investment.

这里要推荐一条小习惯。刚开始建 skill 库的读者,可以备一份"摩擦清单",放在一份纯文本文件里,记下她每个工作日里观察到的摩擦。清单不必排序、不必分优先、不必整理——它只需要"存在"。每个工作周结束,她打开清单,挑出一两条,周末把它们各自搭成一条 skill。这份清单让她对自己保持诚实:哪些摩擦是真实且反复出现的,哪些是偶发一次性的。每周一次的复审,让 skill 库以"不快不慢"的节奏持续增长。坚持这种做法三个月之后,大多数开发者会发现自己手上已经有十五到二十条 skill——每一条都对应着真实的摩擦,每一条都被经常使用,每一条都"挣得"自己那一席之地。这,就是能产生持久结果的"稳态模式",而它每周只需不多于三十分钟的刻意投入。

The next chapter takes the most important field in any skill, the description, and gives it the attention it deserves. The triggering problem encountered in this chapter has a craft, and the craft can be taught. The reader who masters the craft of writing descriptions will, from then on, build skills that fire reliably on the first try, and will spend her time on the body, where the substance of the skill actually lives. That is the subject ahead.

下一章会挑出任何 skill 里最重要的那个字段——description——把它应得的那份关注补足。本章遇到的"该触发却不触发"问题,是一门手艺;这门手艺是可以教的。一旦读者真正掌握了 description 的写作手艺,从此以后,她搭出的 skill 都会在第一次尝试时就稳稳触发;她可以把时间花到正文上——那才是 skill 真正"重"的地方。这正是前方要展开的课题。

# CHAPTER SIX · 第六章

## Skill Description Engineering: The Art of Triggering

### Skill Description 工程:触发手艺

*"To name a thing accurately is to make it findable. To name it inaccurately is to lose it forever in plain sight."*

— a librarian, anonymous

*"把一样东西叫准了,它就找得到;叫偏了,它就在你眼皮底下永远地消失。"*

—— 一位图书馆员,匿名

This is the most important chapter in Part Two. The body of a skill is what Claude does when the skill fires. The description is what determines whether the skill fires at all. A perfect body attached to a poor description is a skill that almost never runs. A modest body attached to an excellent description is a skill that runs reliably across the full range of natural phrasings real users employ. Between the two, the description matters more, because a skill that does not fire might as well not exist. This chapter is dedicated to the craft of description writing, and the goal is that by the end of it, the reader will produce descriptions that fire correctly on the first attempt, every time.

这是第二部分里最重要的一章。Skill 的正文,是"该 skill 触发之后 Claude 做什么";description,是"该 skill 到底会不会触发"。一段完美正文挂在一段蹩脚 description 下面,会变成一条几乎永远跑不起来的 skill;一段朴素正文挂在一段优秀 description 下面,会变成一条在用户千变万化的自然措辞下都稳稳跑起来的 skill。在这两者之间,description 更重要——因为一条不触发的 skill,等同于不存在。本章专门献给 description 写作这门手艺;目标,是你读完之后,每次都能在第一次尝试时就写出"一上来就触发得对"的 description。

Begin with what a description actually does mechanically. At the start of a Claude Code session, the system loads the metadata of every available skill, and the description is the most substantive part of that metadata. When the user submits a prompt, Claude evaluates whether any of the loaded descriptions match the intent of the prompt. The matching is semantic, not literal. Claude is not running a regular expression over the description. It is reading the description in the same way it reads any other natural language, and forming a judgment about whether the prompt fits what the description is talking about. If the judgment is yes, the skill body is loaded into context, and the response is shaped by the body. If the judgment is no, the skill is ignored.

先来看 description 在机制层面到底干了什么。在一个 Claude Code session 启动时,系统会把"所有可用 skill 的元数据"加载进来,而 description 是这份元数据里最有实质内容的部分。用户提交一条 prompt 时,Claude 会去评估:已加载的那些 description 中,有没有哪一条与这条 prompt 的意图相匹配。这种匹配是语义的,不是字面意义上的。Claude 不会拿着正则表达式在 description 上跑一遍;它读 description 的方式,与读其他任何自然语言一样,然后做出一个判断:这条 prompt 是不是 description 所谈论的那种事。如果判断为"是",该 skill 的正文被加载进上下文,响应被正文塑形;如果判断为"否",该 skill 就被忽略。

This mechanism has two implications that govern everything in this chapter. The first implication is that the description must be a piece of writing that informs Claude's judgment. It is not a label. It is not a tag. It is a short paragraph that should help Claude decide, accurately, whether a given prompt is the kind of prompt the skill exists to handle. The second

这一机制,带出两条贯穿本章的隐含规则。第一条:description 必须是"一段能影响 Claude 判断力的文字"。它不是 label,不是 tag,而是一段简短的段落——目的是帮 Claude 准确判断:一条给定的 prompt,是不是那种"该 skill 存在正是为了处理"的 prompt。第二

← Chapter 5 结尾

下一页 →

implication is that the description is read alongside the descriptions of every other installed skill. Skills compete for attention. A description that distinguishes its skill clearly from neighboring skills will fire correctly. A description that overlaps with another skill's territory will produce uncertainty, and uncertainty produces inconsistent firing.

条:description 是与"其他所有已装 skill 的 description"一并被读的。Skill 们彼此竞争注意力。一条能把自己的 skill 与"邻接 skill"清晰区分开的 description,会触发得很准;一条与另一条 skill 的领域相重叠的 description,会制造不确定性,而不确定性,会制造"忽而触发、忽而不触发"的不一致表现。

With those implications in mind, here is the working framework this chapter teaches. A good description has four parts, and they appear in roughly this order. The first part is the canonical statement of what the skill does. The second part is the trigger language: the kinds of requests, phrasings, or contexts that should activate the skill. The third part is the boundary language: what the skill is not for, where its territory ends. The fourth part is the input expectation: what the skill assumes the environment will look like when it runs. Not every description needs every part in equal measure, but every description benefits from at least considering each of the four.

带着这两条隐含规则在脑子里,本章要教的工作框架是这样的:一段好的 description 包含四个部分,大致按以下顺序出现。第一部分——典型陈述:这条 skill 做什么;第二部分——触发语言:应当激活该 skill 的那些请求、措辞或情境;第三部分——边界语言:这条 skill 不做什么,它的领域止于何处;第四部分——输入预期:本 skill 跑起来时,对环境状态的假定。并不是每条 description 都要把四部分平摊到位;但每一条 description 都至少应当把这四个部分都各自考量一遍。

Begin with the canonical statement. This is the one-sentence version of the skill, the version that would appear in a directory listing. It should answer the question, what does this skill do, in twelve to twenty words. Not all of the words. Just enough to make clear, to a reader who has never seen the skill before, what the skill exists to accomplish. The canonical statement of the `commit-message` skill from the previous chapter would be: `generates a conventional commits message from the currently staged changes in a git repository`. That is the skill, named accurately and concisely. Everything that follows refines it.

先讲典型陈述。所谓典型陈述,就是"该 skill 的一句话版",也就是"出现在某个目录清单里"的那个版本。它应当在十二到二十个词内答完"这条 skill 做什么"——不是把所有词都用上,只够让一位从没见过这条 skill 的读者,能弄清楚这条 skill 是为达成什么而存在的。上一章 `commit-message` 的典型陈述就是: `generates a conventional commits message from the currently staged changes in a git repository`。这就是那条 skill——命名准确、措辞精炼。后面所有的部分,都是在为它做"精细化"。

## Trigger Language · 触发语言

The trigger language is the part of the description that does the heaviest lifting. It is where the skill author lists the actual phrasings real users employ when they want this skill to run. Trigger language must be derived from observation, not from imagination. The skill author should write trigger language by asking, what would I actually type when I wanted this skill to run, and what would my colleagues type, and what would a developer who has never used this skill before type if she wanted the same outcome. The variety of phrasings real humans employ is broader than any single author intuitis. Useful trigger phrasings often include the canonical

触发语言,是 description 里"最扛活"的那一块——是 skill 作者列出"用户实际会敲的那些措辞"的地方。触发语言必须从"观察"里来,不能从"想象"里来。Skill 作者写触发语言时,应当反复问自己:真要触发这条 skill 时,我会怎么敲?我的同事会怎么敲?一位从来没用过这条 skill、但想达到同样结果的开发者,会怎么敲?真实人类措辞的多样性,远超任何单个作者的直觉。真正有用的触发措辞,常常会包含那条典型

← Chapter 6 上一页

下一页 →

request, several casual paraphrases, the verb form and the noun form of the request, and any common abbreviations.

请求本身,加上几条口语化的转述,加上动词形式与名词形式,再加上任何常用的简写。

The first description fires reliably. The second description fires only when the user happens to use the exact words `PR description writer`, which almost no one does. The difference between the two is not a difference of quality. It is a difference of trigger surface area. The first description offers many places where natural language can land. The second offers one. A skill with one trigger surface and a perfect body is, in practice, a skill that almost never runs.

第一条 description 触发得稳;第二条只有当用户恰好用上 `PR description writer` 这几个字时才触发——而几乎没人会这样敲。两者之间的差异,不在"质量",而在"触发表面积":第一条 description 给自然语言提供了许多可以"落脚"的地方,第二条只提供一个。一条只

有一个触发面、却有着完美正文的 skill,实际上就是一条几乎跑不起来的 skill。

The boundary language matters when the skill operates near the territory of other skills. If the developer also has a skill called `pr-review`, which reviews other people's pull requests, then the `pr-description` skill needs language that distinguishes it. A line such as `this skill is for writing the description of a PR you are creating, not for reviewing one that someone else wrote`, prevents collision between the two. Without that line, an ambiguous request such as `help me with this PR` could activate either skill, and the developer's experience becomes unreliable.

当一条 skill 跑在"邻近 skill 的领地"附近时,边界语言就特别重要。如果这位开发者还有一条叫 `pr-review` 的 skill——它专门 review 别人的 PR——那 `pr-description` 这条 skill 就需要用一段措辞把自己与对方区分开。例如这样一句: `this skill is for writing the description of a PR you are creating, not for reviewing one that someone else wrote`——它能防止两条 skill"撞车"。少了这句话,像 `help me with this PR` 这种带歧义的请求,就可能激活两者中任意一条,开发者的体感就变得不可靠。

The input expectation is the part of the description that prevents the skill from firing in contexts where it cannot succeed. A skill that operates on staged git changes, like the `commit-message` skill, does no good when there are no staged changes. The description benefits from a phrase such as `assumes the user has staged changes in a git repository`. This phrase informs Claude's judgment in two ways. First, it makes the skill more likely to fire when the user is clearly in the right context. Second, it makes the skill less likely to fire in contexts where the assumption obviously does not hold, such as a non-git directory or a fresh checkout with no changes.

输入预期,是 description 里"防止 skill 在无法成功的场景下触发"的那一块。像 `commit-message` 这样跑在已暂存 git 改动上的 skill,如果没有暂存改动,跑起来毫无价值。Description 里加一句 `assumes the user has staged changes in a git repository`,会 有所得。这句话影响 Claude 的判断,从两个方向:其一,在用户明显处在"对的上下文"时,让 skill 更可能触发;其二,在"前提显然不成立"的场景(非 git 目录、刚 checkout 还没改动的仓库)下,让 skill 更不可能触发。

There is one further technique worth introducing here, which comes from Anthropic's own documentation on skill authoring. Claude has a tendency to under-trigger skills, that is, to err on the side of not firing them when in doubt. The recommended counter-technique is to make

这里还要再介绍一种手法,它来自 Anthropic 官方关于 skill 编写的文档。Claude 存在一种倾向——欠触发(under-trigger):当拿不准时,它倾向于"不去触发"。官方推荐的应对手法,是把

```
description: Drafts a pull request description in the team's standard format.
 Use whenever the user asks for a PR description, pull request description,
 PR body, or asks "write a PR for this branch," "make me a PR
 description," "what should the PR say," or describes wanting to summarize
 a branch's changes for review. Always read the diff before writing.
```

```
description: PR description writer.
```

[注:代码块原文保留,以下为本页中译补充]

这里把两条"质量悬殊"的 description 并排放在了一起。第一条铺得很开:典型陈述 + 一长串触发措辞(PR description / pull request description / PR body / write a PR for this branch / make me a PR description / what should the PR say),再加一句触发之后的行为约束(Always read the diff before writing)——这是"高质量 description"的典型;第二条只有六个字: `PR description writer`——既无典型陈述、也无触发措辞、更无边界与输入预期。把这两条并排放,差异之显著,几乎不需要再作解释。

descriptions slightly assertive. A description that says `use this skill whenever the user mentions X` is more effective than a description that merely describes the topic of X. The verb `use`, applied to the user's phrasing rather than to the topic, gives Claude a clearer signal that the skill is meant to be activated rather than merely consulted. This is a small change with substantial consequences. Adding the phrase `use whenever the user mentions` to a description often raises its trigger reliability noticeably without affecting any other behavior.

description 写得稍微"主动"一些——一条写着 `use this skill whenever the user mentions X` 的 description,比一条仅仅"描述 X 主题"的 description,效果要好。把 `use` 这个动词,作用在"用户的措辞"上,而不是"主题"上,会给 Claude 更清晰的信号:这条 skill 是要被激活的,而不只是被"翻阅"的。这是一个小改动,却带来大后果:在 description 里加上 `use whenever the user mentions` 这一句,通常能明显提高触发的可靠性,而不影响任何其他行为。

Apply all of this to a worked example. The skill is called `explain-error`, and its job is to take an error message or stack trace and produce a clear, prioritized explanation, including likely cause and recommended next debugging step. A weak description for this skill might read, `helps explain errors`. That description is too short, has no trigger language, no boundary, no input expectation, and no `use whenever` phrasing. It will under-fire constantly. A strong description would read as follows.

把以上所有原则套到一份具体示例上。Skill 叫 `explain-error` ——职责是:拿到一条 error message 或 stack trace,产出一份清晰、按优先级排好序的解释,包括"最可能的原因"和"下一步建议的调试动作"。这条 skill 的"弱 description"可能只写 `helps explain errors` ——太短、没有触发语言、没有边界、没有输入预期、也没有 `use whenever` 句式,会一直欠触发。一段"强 description"则长这样——

description: Explains a given error message or stack trace and proposes the most likely cause and a concrete next debugging step. Use whenever the user pastes an error, stack trace, traceback, exception, or compiler output and asks "what is this," "why is this happening," "how do I fix this," or otherwise indicates they want help interpreting an error. Do not use this skill for general "my code is broken" requests where no error message is present; in those cases, ask the user for the error first.

## A Worked Description · 一份完整示例

Notice the structure. The first sentence is the canonical statement. The middle of the description lists trigger language, including five concrete phrasings the user might employ. The end carries explicit boundary language: `do not use this skill for`, followed by the specific case where the skill should defer to a clarifying question instead. This kind of description fires correctly across the full natural variety of how developers actually report errors.

留意它的结构。第一句是典型陈述;中间列出触发语言——其中含五条"用户可能用到的具体措辞";尾部则是有显式标记的边界语言: `do not use this skill for`, 后接那种"该 skill 应当让位给一次澄清性提问"的特殊情况。这种 description,能在"开发者实际报告错误的全部自然多样性"之下,触发得准。

Two final practices round out the craft. The first is the trigger test. After writing a description, the skill author should ask herself whether she can describe the skill's task in three meaningfully different ways and have all three of them fire the skill. If yes, the description has good coverage. If no, the description should be expanded with the missing phrasings. This three-way test takes a minute to perform and prevents a category of bug that

最后还有两条实践,把 description 这门手艺收齐。第一条,叫"触发测试"。写完一段 description,skill 作者应当问自己:这件事,我能不能用"在意义上彼此不同"的三种说法把它讲出来,且这三种说法都能让该 skill 触发?如果能,description 覆盖得不错;如果不能,就该把缺失的措辞补进 description。这种"三方测试"花不了一分钟,却能防住一类——

← Chapter 6 上一页

下一页 →

would otherwise only surface weeks later, in a real session, at a moment when the developer is least equipped to debug it.

bug——这些 bug 若不预防,往往要等到几周后,在某次真实 session 里才暴露;而那时,恰恰是开发者最无力 debug 它的时刻。

The second practice is the false-trigger test, which is the inverse. After writing a description, the skill author should think of two requests that look superficially similar to the skill's domain but should not fire the skill. If the description, as written, would clearly fire on those requests, the description is too broad and needs boundary language. The `commit-message` skill, for example, should not fire when the user asks how to revert a commit, even though the request mentions commits. A boundary line such as `this skill is for writing new commit messages, not for git history operations`, prevents the false trigger.

第二条,叫"误触发测试",正好是上一条的反面。写完 description,skill 作者应当想出两条"表面上像属于该 skill 领域、但其实不该触发该 skill"的请求。如果按当前 description 写法,这两条会明显触发,那 description 就过宽了,需要补上边界语言。比如 `commit-message` skill,用户问"怎么 revert 一个 commit"时,就不该触发——哪怕那条请求也提到了 commit。一句边界表述 `this skill is for writing new commit messages, not for git history operations`,就能防住这类误触发。

Description writing is, in the end, a discipline of imagining the user. The author who writes good descriptions is the author who has thought concretely about how real people will phrase requests in real moments. There is no formula that substitutes for this imagination, but the four-part framework, the `use whenever` phrasing, the trigger test, and the false-trigger test together produce descriptions that fire correctly on the first attempt in the large majority of cases.

说到底,写 description 是一种"想象用户"的纪律。能写出好 description 的作者,就是那种"对真实的人在真实瞬间会怎么措辞"有过具体想象的作者。没有任何公式能替代这种想象,但"四段式框架 + `use whenever` 句式 + 触发测试 + 误触发测试"这一整套组合,在大多数情形下,已经能产出"在第一次尝试时就触发得对"的 description。

A worked rewrite case study makes the framework concrete. The skill is called `api-doc`, and its job is to write OpenAPI documentation for a given route. The first draft of the description, written by a developer who had not yet read this chapter, looked like this.

一份完整的"重写案例",会让这套框架具体起来。Skill 叫 `api-doc`——职责是:为一个给定的 route 写 OpenAPI 文档。第一稿 description,由一位还没读过本章的开发者写成,长这样:

description: Writes API documentation.

## Iterating on a Failing Description · 改写一段失败的 Description

This description fired roughly one time in five. Reading it through the four-part framework explains why. The canonical statement is acceptable, but it is the only thing the description contains. There is no trigger language, no boundary, no input expectation. A developer who types `document this endpoint`, `write OpenAPI for this route`, or `generate Swagger for this handler` will find that none of those phrasings clearly match the description, because none of them are the words API documentation. The skill exists, but it does not fire when its time comes.

这段 description 的触发率,大约只有五分之一。用"四段式框架"过一遍,原因就很清楚:典型陈述部分是合格的——但整段 description 也只有这一部分。没有触发语言、没有边界、没有输入预期。一位开发者敲 `document this endpoint`、`write OpenAPI for this route`、或 `generate Swagger for this handler` 时,会发现这些措辞与 description 都不算清晰匹配——因为它们都不含"API documentation"这几个字。Skill 在那,但该触发的时候,它不触发。

← Chapter 6 上一页

下一页 →

A second draft, after the developer noticed the under-firing problem and tried to fix it, looked like this.

第二位开发者注意到欠触发问题、试图修一次之后,第二稿长这样:

description: Writes API documentation. Use when the user asks for documentation, OpenAPI, Swagger, or wants to document a route.

[注:代码块原文保留,以下为本页中译补充]

这版比起第一稿明显好一些:触发语言加进了最常见的三种措辞(documentation / OpenAPI / Swagger),加上 `use when` 这个动词——告诉 Claude "是要被激活的"而不是仅仅"被翻阅"。这一改之后,触发率明显上升。

This is meaningfully better. The trigger language now includes three of the most common phrasings, and the verb `use` signals that the skill should be activated rather than merely consulted. Firing rate rose noticeably with this change. But the description still has problems. There is no boundary language, which means it will fire on requests like `document this codebase`, where the right skill might be the `explain-codebase` skill from Chapter Four, not `api-doc`. There is no input expectation, which means it will fire even when the user has not pointed Claude at any specific route. The result is occasional false triggers, where the skill fires on requests it cannot actually serve well.

改到这里,触发语言里多了三条最常见的措辞, `use` 这个动词也把"是要被激活"的信号送了出去——触发率肉眼可见地提升。但 description 仍然有问题:它没有边界语言,所以像 `document this codebase` 这种请求也会触发——而那种请求,其实该由第四章那份 `explain-codebase` skill 来接,而不是 `api-doc`。它也没有输入预期,所以即使用户没把任何具体 route 指给 Claude,它也会触发。结果就是"零星的误触发"——该 skill 跑在它其实根本服务不好的请求上。

The third draft, after the four-part framework was applied in full, looked like this.

把"四段式框架"完整地套上之后,第三稿长这样:

description: Writes OpenAPI documentation for a single API route or handler.

Use whenever the user asks to "document this endpoint," "document this route," "write OpenAPI for this," "add Swagger for this," "generate API docs for this handler," or otherwise indicates they want OpenAPI documentation produced for a specific route. This skill is for documenting a single route at a time, not for documenting an entire codebase. Assumes the user has identified the route or handler to document, either by pasting it or by referring to a specific file.

This description fires reliably and false-triggers rarely. The canonical statement is precise: a single API route, not a codebase. The trigger language enumerates five concrete phrasings the user might employ. The boundary language makes the single-route scope explicit and prevents collision with broader documentation skills. The input expectation establishes that the user has identified what to document, which prevents the skill from firing in contexts where it has nothing to work with. The total length is roughly seventy words, which is on the longer side for a description but well within the range that descriptions can usefully occupy. The trade between length and reliability falls clearly in favor of length here. A description three times longer than necessary that fires correctly is much better than a description one third the length that fires inconsistently.

这一稿,触发得稳、误触发得少。典型陈述很精确——"单个 API route",不是"整个 codebase";触发语言枚举了五条用户可能用的具体措辞;边界语言把"只接单 route"的范围讲明白,防止它与更广义的文档 skill 撞车;输入预期确立了"用户已经指明要文档化什么"——这能防止该 skill 跑在"完全没有原料"的场景里。整段长度约七十个词,对 description 而言算偏长,但仍稳稳落在 description 实际能发挥作用的合理区间内。在"长度"与"可靠性"之间做权衡,本章的态度是相当明确地倾向"长度":一段比必要长度长三倍、却触发得对的 description,远好于一长度只有它三分之一、却触发得不稳定的 description。

← Chapter 6 上一页

下一页 →

The case study illustrates a general truth. Description quality is a continuous variable, not a binary. A first-draft description usually works some of the time. Each subsequent revision narrows the gap between when the skill should fire and when it does fire. After two or three revisions, the gap is small enough that the skill becomes reliable in daily use. The reader who is patient with this iteration will, over the course of building her first ten skills, internalize the four-part framework so deeply that her first drafts begin to look like other people's third drafts.

这份案例,讲的是一条普适的真理。Description 的质量是一个连续变量,不是二元判断。第一稿 description 通常只在部分时候管用;每一次改稿,都在把"该 skill 应该触发的时刻"与"它实际触发的时刻"之间的差距收窄一截。改上两三轮之后,差距已经小到让该 skill 在日常使用中变得可靠;而一个愿意耐心走完这几轮迭代的读者,在搭出前十条 skill 的过程中,会把"四段式框架"内化到那种程度——自己写出的第一稿,看起来已经是别人改到第三稿的样子。

## A Rubric for Description Quality · 一份"Description 质量"的评分表

A small rubric is worth keeping in working memory while writing descriptions. A description scores well when it answers four questions clearly. What does this skill do, in one sentence. What kinds of phrasings should fire it. Where does this skill stop and another skill begin. What does this

skill assume about the input. A description that does not answer any one of these four questions has a corresponding weakness, and the weakness will manifest in a corresponding failure mode. Under-firing usually traces back to weak trigger language. Over-firing usually traces back to weak boundary language. Firing in the wrong context usually traces back to weak input expectations. Firing on the wrong topic entirely usually traces back to a weak canonical statement.

写 description 时,心里值得存一份小评分表。一段 description 得分高,意味着它清晰地答出了四个问题:这条 skill 用一句话讲做什么?哪些措辞应当触发它?这条 skill 在哪里收手、把舞台让给另一条 skill?这条 skill 对输入做了哪些假定?任何一个问题没答上,都会在 description 上留一处对应的弱点;那一处弱点又会表现成某一种对应的失败模式:欠触发通常源于触发语言太弱;过触发通常源于边界语言太弱;在错误的上下文里触发,通常源于输入预期太弱;整段完全跑偏,通常源于典型陈述太弱。

One additional practice is worth introducing here, although its full development belongs in a more advanced treatment of skill design. Anthropic's own `skill-creator` skill includes an evaluation procedure in which the skill author writes twenty test queries, mixing prompts that should fire the skill with prompts that should not, and checks whether the description correctly distinguishes between them. The procedure is straightforward to perform manually. After writing a description, the author lists ten phrasings that should clearly fire the skill and ten that should clearly not. She reads each one and asks, would my description, as written, lead Claude to fire the skill on this query. If the answer disagrees with the intended firing in more than one or two cases out of twenty, the description needs work. If it agrees in all twenty cases, the description is probably ready.

这里还要再介绍一条实践——虽然它的完整展开属于更进阶的 skill 设计话题。Anthropic 自己的 `skill-creator` skill 包含一套评估流程:skill 作者写出二十条测试 query,其中混进"应当触发"与"不应触发"两种,再核验 description 能否正确区分两者。这套流程,人工手动跑也很直白:写完 description 之后,作者列十条"应当明显触发"的措辞,再列十条"应当明显不触发"的措辞;对每一条都问自己:"按当前 description 的写法,这条 query 会让 Claude 触发这条 skill 吗?"如果二十条里有超过一两条的答案与"本意"相左,description 还需再改;如果二十条都答对,那 description 差不多可以交付了。

This systematic check transforms description writing from a craft based on intuition into a craft based on evidence. The intuition is still valuable; it

这种系统性检查,把 description 写作从"凭直觉"的手艺,变成了"凭证据"的手艺。直觉仍然宝贵,它——

← Chapter 6 上一页

下一页 →

suggests the initial language, the candidate phrasings, the likely failure modes. But the check ensures that the intuition has produced the right result. The author who runs the twenty-query check on every description she writes builds, over time, a sense of which kinds of phrasings tend to be dangerous, which kinds of boundaries tend to be necessary, and which kinds of input expectations tend to matter. The sense becomes intuitive, and her first-draft descriptions begin to pass the twenty-query check on the first attempt. This is the path to becoming a skilled skill author. There is no shortcut, but the path is short, and the reader who walks it deliberately reaches the destination within a working week.

提供最初的措辞、候选的触发语句、以及可能的失败模式;而这套"测试"则保证,直觉真的产出了正确的结果。每条 description 都跑一遍"二十条 query"检查的作者,会随着时间逐渐培养出一种"对哪类措辞容易出事、哪类边界常常必要、哪类输入预期通常关键"的判断力;这种判断力会内化成本能,她写出的初稿 description 开始在第一次尝试时就通过那二十条 query 的检查。这,就是"成为熟练 skill 作者"那条路。没有捷径,但路不长——愿意有意识地走下去的读者,在一个工作周之内就能走到终点。

Description writing pays out at every layer of the skill library. Better descriptions mean more reliable firing, which means the developer trusts the library, which means she invests in extending it, which means the library grows, which means the compounding accelerates. The single largest difference between a developer with a thriving skill library and a developer with a stalled one is, almost always, description quality. The body of the skill is the visible substance, but the description is the invisible gate that determines whether the body ever runs. Master the gate, and the rest follows.

Description 写作的回报,在 skill 库的每一层都会兑现。更好的 description → 更可靠的触发 → 开发者更信任这个库 → 她更愿意投入去扩展它 → 库继续生长 → 复利进一步加速。一位开发者手上"茁壮成长"的 skill 库,与另一位"原地踏步"的 skill 库之间,最大的单项差异,几乎总是 description 的质量。Skill 的正文,看得到的实质;description,看不见的闸——决定正文到底会不会被跑起来。掌握了这道闸,其余的事会自然跟上。

One last observation is worth keeping in mind. Descriptions, like any other piece of writing, age. The phrasings a developer uses today are not necessarily the phrasings she uses in six months, because vocabulary drifts as projects change and as new tools enter daily use. A description that fired reliably six months ago may begin under-firing now, not because anything in the skill changed but because the developer has started phrasing her requests differently. The remedy is the same in both cases: add the new phrasings to the description. Treat descriptions as living documents, revisited whenever a skill seems to be firing less reliably than it used to. The cost of revising is small. The benefit is that the library stays in tune with the developer who depends on it.

还有最后一条观察,值得装在脑子里:description,和任何一篇文字一样,会"老化"。开发者今天用的措辞,不一定是她六个月后还在用的措辞——因为词汇会随项目变化、随新工具进入日常而漂移。半年前触发得稳的 description,现在开始欠触发——并不是因为 skill 里有什么变了,而是因为开发者敲请求的措辞在变。修法在两种情况下都一样:把新措辞加进 description。把 description 当作"活文档"——只要某条 skill 看起来"没以前那么稳",就回去看看它。修订的成本很小;收益,是让整个 skill 库始终与它所服务的那位开发者保持同调。

With description engineering in hand, the rest of the work of building skills can focus on the body, where the substance of the skill lives. That work expands in the next chapter, which moves beyond the single-file skill into the world of bundled resources, scripts, and references that turn a skill from a paragraph of instructions into a substantial piece of installed capability.

把 description 工程握在手里之后,搭 skill 其余的工作就可以把注意力放到正文上——那才是 skill 真正的实质所在。这份工作在下一章会进一步展开:它将跨出"单文件 skill"的范畴,进入"打包资源、脚本、引用"的世界,把一条 skill 从"一段指令文字",变成"一件装在环境里的、像样的能力"。

← Chapter 6 上一页

CHAPTER SEVEN →

## CHAPTER SEVEN · 第七章

### Multi-File Skills and Bundled Resources

#### 多文件 Skill 与打包资源

*"A small thing well-organized is a tool. A large thing poorly organized is an obstacle. The line between them is layout."*

"一件小东西,组织得好,就是工具;一件大东西,组织得差,就是障碍。两者之间的界线,是'布局'。"

——埃德温·巴塞洛缪,木匠

The skills built so far have been single-file skills. `SKILL.md` is the only file in the skill folder, and the entire skill, frontmatter and instructions and any examples, lives within that one document. This works perfectly for skills whose total content is small. For many skills, single-file is exactly the right structure, and adding more files would only complicate something that does not need complication.

到目前为止搭出的 skill,都是"单文件" skill——`SKILL.md` 是 skill 文件夹里唯一的文件,整条 skill——frontmatter、指令、示例——全部塞在这一份文档里。对那些"总体内容小"的 skill 而言,这种做法堪称完美;事实上,对很多 skill 来说,单文件恰恰是合适的结构,加文件只会徒增不必要的复杂度。

But many of the most valuable skills cannot reasonably fit in a single file. A skill that scaffolds a new React component might need a dozen template files: one for the component itself, one for the test file, one for the Storybook story, one for the CSS module, one for the index file, one for each variant of the component the skill supports. A skill that runs a deterministic data extraction over a PDF might need to bundle the Python script that does the extraction, because asking the language model to do that work would be slow, expensive, and unreliable compared to running real code. A skill that encodes the conventions of a complex API might need a fifty-page reference document that the language model can consult when needed but should not load by default. None of these skills can be built well with a single file. All of them benefit from the multi-file structure that is the subject of this chapter.

但许多最有价值的 skill,实际上无法合理地塞进单文件。一个为新 React 组件搭脚手架的 skill,可能需要一打模板文件:组件本身一份、测试一份、Storybook story 一份、CSS module 一份、index 一份、每一种组件变体各一份。一个要在 PDF 上跑"确定性数据抽取"的 skill,可能需要把那段做抽取的 Python 脚本打包进来——因为让语言模型去做这件事,会比真正跑代码慢、贵、且不可靠。一个把某个复杂 API 规约编码起来的 skill,可能需要一份五十页的参考文档,让语言模型在需要时去查,而不是默认就加载。这些 skill,都不适合用单文件搭;它们都受益于本章的主题——多文件结构。

The convention for multi-file skills, established by Anthropic and adopted across the open-source skills ecosystem, is straightforward. A skill folder may contain three special subfolders, each with a specific role. The `scripts` subfolder contains executable code. The `references` subfolder contains additional markdown documents that the `SKILL.md` body can ask Claude to read on demand. The `assets` subfolder contains files that are used as templates, fixtures, or other inputs during the skill's execution. Any of the three subfolders may be present or absent in any given skill. Some skills

多文件 skill 的这套惯例,由 Anthropic 确立,并被整个开源 skill 生态所采纳,本身很直白。一个 skill 文件夹可以包含三个特殊的子目录,每个各司其职。`scripts` 目录装可执行代码; `references` 目录装若干额外的 markdown 文档——`SKILL.md` 的正文可以指示 Claude 在需要时去读它们; `assets` 目录装的是"在该 skill 执行过程中作为模板、夹具、或其他输入"的文件。任一个 skill 里,这三个目录都可以有、也都可以没有;有些 skill

← Chapter 6 结尾

下一页 →

use all three. Most use one or two. The folder structure for a skill that uses all three would look like this.

三者都用上;大多数只用到一两个。一个三类都用上的 skill,文件夹结构大致是这样:

```
react-component/
├─ SKILL.md
├─ scripts/
│ └─ generate.sh
├─ references/
│ ├── component-conventions.md
│ └─ testing-conventions.md
└─ assets/
 ├── component.template.tsx
 ├── test.template.tsx
 └─ story.template.tsx
```

[注:代码块原文保留,以下为本页中译补充]

这是一份典型的"全三类"多文件 skill 结构: SKILL.md 在最外层做"调度", scripts/ 装可执行脚本( generate.sh ), references/ 装"按需阅读"的两份 markdown( component-conventions.md 和 testing-conventions.md ), assets/ 装三份模板文件——组件、测试、Storybook 各一份。这种"外层调度 + 三类支撑材料"的结构,正是从这一章起所有"复杂 skill"的统一骨架。

The naming of the three folders matters because it makes the skill folder predictable to read. A developer encountering the skill for the first time can look at the folder layout and form an immediate, accurate guess about what each piece does. Scripts are for running. References are for reading. Assets are for using as material in the skill's output. This predictability compounds across a skill library; once a developer has internalized the convention, every skill she encounters reveals its structure at a glance.

这三个目录的命名之所以重要,是因为它让"读 skill 文件夹"这件事变得"可预期"。一位第一次碰上这条 skill 的开发者,看一眼目录布局,就能立刻、准确地猜出每一块在做什么: scripts 装的是"用来跑的东西"; references 装的是"用来读的东西"; assets 装的是"在 skill 输出里用作原料"的东西。这种可预期性会在整个 skill 库里复利——一旦某位开发者把这条约定内化,她以后遇到每一条 skill,都能一眼看清它的结构。

Begin with references , which is the most common form of bundled material. A reference is a markdown file that lives inside the skill folder and contains content that the skill body wants to be available, but that does not need to be loaded into context unless it is actually needed. The classic example is a long format specification. If a skill produces output in a complex format, the format specification might be ten pages. Putting the entire specification in SKILL.md would mean that every time the skill activates, ten pages of spec land in the context window, even if only a small section is relevant to the current task. The better pattern is to keep SKILL.md 短, place the full specification in a reference file, and have SKILL.md instruct Claude to read the reference when relevant.

先讲 references ——它是打包材料里最常见的一种形式。所谓 reference,就是一份住在 skill 文件夹里的 markdown 文件,装着"该 skill 正文希望它可用、但只有在真的需要时才加载"的内容。最经典的例子是一份很长的格式规范。如果某条 skill 输出的内容是某种复杂格式,那份格式规范可能有十页;把它整段塞进 SKILL.md ,意味着该 skill 每次激活时,十页规范都会进上下文窗口——而当前任务真正用到的可能只是其中很小一段。更优的做法是:把 SKILL.md 保持简短,把完整规范放进一份 reference 文件,然后让 SKILL.md 在相关时指示 Claude 去读那份 reference。

## How Loading Works · 加载机制

The mechanics are simple. The skill body refers to the reference file by relative path, often within an instruction line. Claude, on encountering that instruction, knows that it can read the reference file using its standard file-

机制本身很简单。Skill 正文通过相对路径引用 reference 文件——通常写在某条指令里。Claude 遇到这条指令,就知道它可以用自己的标准文件——

← Chapter 7 上一页

下一页 →

reading tools. Here is what the relevant portion of a `SKILL.md` file might look like, illustrating reference usage in practice.

读工具,去把那份 reference 读进来。下面是一份 `SKILL.md` 的相关段落,用来演示 reference 在实战中是怎么被用的。

```

name: api-doc
description: Writes OpenAPI documentation for a given route. Use whenever
 the user asks to "document this endpoint," "write OpenAPI for this,"
 "add Swagger docs," or otherwise indicates they want API documentation
 generated.

API Documentation

When asked to document an API endpoint, follow this procedure.

1. Read the route handler the user is referring to.
2. If you are unsure about the project's documentation conventions, read
 `references/openapi-conventions.md` for the full style guide.
3. Read `references/error-format.md` only if the endpoint can return errors
 that are not the standard 4xx or 5xx codes.
4. Produce the OpenAPI fragment in the format described in the conventions
 file.

Do not load the references unless the situation calls for them. The
conventions file is long, and loading it every time wastes context.
```

[注:代码块原文保留,以下为本页中译补充]

这份 `api-doc` 的 `SKILL.md` 是一份绝佳的"reference 触发模式"示范。注意正文刻意很短,只列了四步流程的高层骨架;真正的"风格指南"放在 `references/openapi-conventions.md`, "错误格式"放在 `references/error-format.md`。第 2 步用 `If you are unsure ... read X` 模式——这是"按需加载 reference"的标准指令形式;第 3 步的 `only if ...` 是更严格的"条件性加载",用一句话把"何时读"钉死;最后那句"do not load the references unless the situation calls for them"是元指令,提醒 Claude "别养成每次都加载"的习惯——这正是上一章反复强调的"避免浪费 context window"那条纪律在 multi-file 场景下的具体化。

Several details in that example are worth highlighting, because they reflect the discipline of good multi-file design. First, the body is short. The body's job is to describe the procedure at the highest level. The detail lives in the references, where it does not cost context unless needed. Second, the body explicitly tells Claude when to load each reference. The instruction `Read X if Y` is the language pattern that triggers progressive disclosure correctly. Without that instruction, Claude might either always load the references, defeating their purpose, or never load

them, leaving important information unused. Third, the body is honest about the cost of loading references. The closing line, `do not load the references unless the situation calls for them`, is a small but real piece of meta-instruction that helps Claude make the right call.

这份示例里有几处细节值得拎出来,因为它们正是"好的多文件设计"纪律的体现。第一,正文很短。正文的职责,只到"在最高层描述流程"为止;细节放在 `reference` 里——除非真用上,否则不占 `context`。第二,正文显式地告诉 Claude 何时加载每份 `reference`。`Read X if Y` 这种指令句式,正是"正确触发渐进式披露"的语言模式。少了它,Claude 可能会"每次都把 `reference` 加载进来"——这样 `reference` 就没意义了;也可能会"一次都不加载"——这样 `reference` 里的重要信息就被白白闲置。第三,正文对"加载 `reference` 的代价"是坦白的。末了那句 `do not load the references unless the situation calls for them`,是一段不起眼但确有分量的元指令——它帮 Claude 在该加载时加载,在不该加载时不加载。

Now turn to scripts. Scripts are executable code bundled inside a skill folder, intended to be run by Claude as part of carrying out the skill. Scripts solve a category of problem that prose instructions handle poorly: deterministic, repeatable operations that real code can perform faster, cheaper, and more reliably than a language model can simulate. Sorting a

现在把目光转向 `script`。所谓 `script`,是打包在 `skill` 文件夹里、供 Claude 在执行该 `skill` 时调用的可执行代码。`Script` 解决的是"光靠散文指令处理不好"的那一类问题:确定性的、可重复的操作——真代码来做,比语言模型去"模拟"更便宜、更快、更可靠。给一个数组排

← Chapter 7 上一页

下一页 →

list, parsing a binary file, querying a database, calling an external API with a known schema. All of these are operations that should be code, not prose instructions to a model. The skill author writes the script once, places it in the `scripts` folder, and the skill body instructs Claude to run it at the appropriate moment.

序、解析一个二进制文件、查一次数据库、用一个已知 `schema` 去调外部 API——这些都该是代码的工作,而不该是对模型的"散文指令"。Skill 作者把脚本写一遍,放进 `scripts` 目录,skill 正文在合适时点指示 Claude 去跑它。

A useful pattern is to combine a thin script with prose instructions about how to interpret its output. The script does the deterministic work. Claude reads the script's output and applies judgment to it. The combination is faster and more reliable than either approach alone. Anthropic's own document-handling skills follow this pattern. The `pdf` skill, for example, bundles a Python script that extracts form fields from a PDF without loading the PDF into the language model's context. The body of the skill then describes how to use the extracted data to fill or analyze the form. The script handles the part that requires precision. The body handles the part that requires understanding.

一种有用的模式是:把一段"薄脚本"与"关于如何解读其输出的散文指令"组合起来——脚本做那部分确定性的工作;Claude 读完脚本的输出,再把判断力加上去。这种组合,比单用任何一种都快、且更可靠。Anthropic 自家的文档处理 skill 就是这种模式。比如 `pdf` 这条 skill,打包了一段 Python 脚本,用来从 PDF 里抽 `form` 字段,却不让整份 PDF 进语言模型上下文;skill 正文则描述怎么用抽出来的数据去填表或做分析。脚本负责"需要精度"的那部分,正文负责"需要理解"的那部分。

Assets, the third bundled resource type, are files used as material in the skill's output. Templates are the dominant case. A skill that scaffolds a new file from a template can keep the template as a separate file in the `assets` folder, and the body of the skill can instruct Claude to copy the template, fill in the variable parts, and write the result to the appropriate location in the project. This pattern is common in code-generation

skills, document-generation skills, and any skill where the structure of the output is large and stable, but the specific values change with every invocation.

Asset, 第三类打包资源, 指的是"在 skill 输出里被当作材料"的文件。模板(是 asset 最常见的一种形态。一条"按模板搭一个新文件"的 skill, 可以把模板作为单独文件放在 `assets` 目录里; skill 正文则指示 Claude 把模板拷出来、填好变量位、再把结果写到项目里合适的位置。这种模式在"代码生成"型 skill、"文档生成"型 skill, 以及任何"输出结构大而稳定、但具体数值每次都变"的 skill 里, 都很常见。

## A Worked Multi-File Skill · 一份完整的多文件 Skill 示例

A worked example brings the three resource types together. Consider a skill called `scaffold-feature`, intended to generate the scaffolding for a new feature in a TypeScript codebase. The scaffolding includes a route file, a service file, a repository file, a test file, and an OpenAPI fragment. The skill folder might contain templates for each of those file types in `assets`, a small Python script in `scripts` that runs the test scaffolding through a formatter, and a reference file in `references` that explains the codebase's naming conventions. The `SKILL.md` body would describe the procedure: ask the user for the feature name, read the conventions reference, copy each template from `assets` while substituting the feature name, run the formatter script over the result, and write the files into their proper locations. Each of the

一份完整的示例把三类资源并到一起。考虑一条叫 `scaffold-feature` 的 skill——它的职责, 是为一个 TypeScript 代码库里的新功能生成脚手架。脚手架包含: route 文件、service 文件、repository 文件、test 文件, 以及一段 OpenAPI 片段。这个 skill 的文件夹里: `assets` 装每种文件类型的模板; `scripts` 装一段小 Python 脚本, 用来把脚手架里的 test 部分过一遍 formatter; `references` 装一份文件, 讲清代码库的命名规约。 `SKILL.md` 的正文描述整个流程: 向用户问新功能的名字; 读 conventions reference; 从 `assets` 拷出每份模板, 边拷边把功能名替换进去; 把结果跑一遍 formatter 脚本; 再把文件写到项目里各自该去的位置。三类资源中的每一类——

← Chapter 7 上一页

下一页 →

three resource types is doing what it does best, and the combined skill produces a substantial scaffolding operation in seconds.

——都在做它最擅长的事; 合在一起的这条 skill, 能在几秒内产出一份像样的脚手架操作。

A practical limit is worth stating. `SKILL.md` should stay under approximately five hundred lines, even with bundled resources available. The body is read every time the skill fires, and a body that grows past five hundred lines starts to consume noticeable context on every activation. When a body approaches that limit, the right move is to factor surplus content into reference files. The body becomes a router that points Claude to the right reference at the right moment. This is, again, the progressive disclosure principle, applied recursively. The skill keeps its small footprint when it is not needed. It expands gradually as the work demands.

有一条实用的"上限"值得明说: 即便已经用上打包资源, `SKILL.md` 也应控制在五百行左右。Skill 每次触发都要读正文; 一旦正文超过五百行, 每次激活都会开始吃掉可见的 context。当正文逼近这条上限时, 正确的做法是把多出来的内容拆成 reference 文件; 正文则退化为一份"路由器"——在合适的时刻, 把 Claude 指向正确的 reference。这正是"渐进式披露"原则的递归应用: 该 skill 在不需要时, 保持轻盈的小脚印; 在任务需要时, 才一点点展开。

Token budgeting matters at a skill-library level too. A library of forty skills, each with a thirty-line description, a hundred-line body, and several hundred lines of references, might amount to ten thousand or more lines of total content. Loaded all at once, that would dominate any reasonable context window. Loaded progressively, with only metadata at startup and only the relevant body and references on activation, the

total cost on any given turn is small. The progressive disclosure architecture is what allows skill libraries to scale without becoming a tax on every prompt. The discipline of building skills that respect this architecture is, in effect, the discipline of writing skills that play well with their neighbors.

"Token 预算"在 skill 库这一层级同样要紧。一个有四十条 skill 的库,每条有三十行的 description、一百行的正文、几百行的 reference,合起来可能是一万行以上的内容——若一次性全部加载,会占满任何一个合理大小的 context window。若按"渐进式披露"加载——启动时只装元数据,激活时只装对应正文与 reference——那么任一特定轮次上,所付的总成本都是小的。"渐进式披露"这套架构,正是 skill 库能够不断扩张、却不对每一条 prompt 都收"税"的原因。把"尊重这套架构"作为搭 skill 的纪律,实际上就是"让 skill 们彼此相处融洽"的纪律。

A complete worked example of a multi-file skill will make the structure tangible. The skill being built here is called `scaffold-react`. Its job is to scaffold a new React component complete with its test file, its Storybook story, and an index export. The skill is exactly the kind that benefits from bundled resources. The templates for each generated file are stable and substantial, the conventions for component organization are project-specific and worth documenting, and the body itself can stay short by referring outward to the supporting files.

一份完整的多文件 skill 示例,会让这套结构变得"摸得到"。要搭的这条 skill 叫 `scaffold-react` ——职责是:为一个新 React 组件搭好脚手架,包括它的 test 文件、Storybook story,以及一份 index 导出。这条 skill,正是"受益于打包资源"的那种典型:每份生成文件的模板都是稳定且体量不小的;组件组织的规约是项目专属的,值得单独写下来;正文本身则可以保持简短,只是把"指向"打到外面那些支撑文件上。

The folder structure for the skill looks like this.

这条 skill 的文件夹结构,长这样:

```
scaffold-react/
├─ SKILL.md
├─ references/
│ └─ component-conventions.md
└─ assets/
 └─ component.template.tsx
```

[注:代码块原文保留,以下为本页中译补充]

这是一份"截至本页底部"为止的目录结构展示——`SKILL.md` 在根, `references/` 装组件组织规约, `assets/` 开始装第一份模板。完整的结构会在下一页接续展开,届时 `assets/` 里还会再加入 `test.template.tsx`、`story.template.tsx`、`index.template.ts` 等若干模板文件。

← Chapter 7 上一页

下一页 →

```
├─ test.template.tsx
├─ story.template.tsx
└─ index.template.ts
```

[注:代码块原文保留,以下为本页中译补充]

这是上一页那份目录树在 `assets/` 下的剩余部分——三份模板文件: `test.template.tsx` (测试模板)、`story.template.tsx` (Storybook story 模板)、`index.template.ts` (index 导出模板)。连同上一页的 `component.template.tsx` ,这条 skill 一共要"按模板生成"四份文件。

## Inside the Skill · 走进 Skill 内部

The `SKILL.md` file is the entry point. It is short, because the heavy material lives in the `references` and `assets` folders. Its job is to coordinate: to fire on the right requests, gather the input it needs from the user, consult the conventions reference when relevant, and copy the appropriate templates with substitutions. Here is what the body looks like.

`SKILL.md` 文件是入口点。它本身很短——因为"重的素材"都住在 `references` 和 `assets` 里。它的职责是协调:在正确的请求上触发、从用户那里收集所需的输入、在相关时去查 conventions reference、把对应模板拷出来做替换。整份正文长这样:

```

name: scaffold-react
description: Scaffolds a new React component, complete with test file,
 Storybook story, and index export, in this codebase's conventional
 layout. Use whenever the user asks to "scaffold a component," "create
 a new component," "set up a React component," "make a new component
 called X," or describes wanting the standard files for a new component
 to be created.
allowed-tools: Read, Write, Glob

Scaffold React Component

When asked to scaffold a component, follow this procedure.

Step 1: Get the component name

If the user has stated a name, use it. Convert to PascalCase if needed.
If the user has not stated a name, ask once for the name and the
directory it should live in. Default directory is `src/components/`.

Step 2: Read the conventions

Read `references/component-conventions.md` to confirm the current
conventions for this codebase, including naming, prop typing, and CSS
handling. Apply any conventions that are relevant to the templates.

Step 3: Generate the files

For each of the four template files in `assets/`, copy it to the target
directory and substitute the placeholder `__COMPONENT_NAME__` with the
PascalCase name.

The four files produced are:
```

[注:代码块原文保留,以下为本页中译补充]

这份 `scaffold-react` 的 `SKILL.md` 把"短正文 + 外部资源"的纪律演示得相当典型: `description` 列举了 `scaffold a component` / `create a new component` / `set up a React component` / `make a new component called X` 等多条触发措辞; `allowed-tools`:

Read, Write, Glob ——只读、写、模式匹配,把其他工具挡在门外;正文"## Step 1"先向用户拿组件名并转 PascalCase,"## Step 2"再按需读 `references/component-conventions.md` (注意"按需"二字,不是每次都加载),"## Step 3"才进入"拷模板、替换占位符"的核心动作——这一段会在下一页接续展开。"生成的四个文件"那条说明,会在下一页详细列出。

← Chapter 7 上一页

下一页 →

The `SKILL.md` is roughly forty lines. By itself, that is short. But the skill is not actually small. The total content of the skill, including templates and conventions reference, might run two or three hundred lines. The body is short because the body is doing what bodies do best, which is coordination. The substance lives in the supporting files.

`SKILL.md` 自身大约四十行,单看很短。但这条 skill 并不小——连同模板和 conventions reference 在内,整条 skill 的总内容可能要两三百行。正文之所以短,是因为正文在做正文最擅长的事——协调。真正的实质,住在那些支撑文件里。

The `component-conventions.md` reference might look like this.

那份 `component-conventions.md` reference,大致长这样:

```
- `/index.ts` (from index.template.ts)
- `/.tsx` (from component.template.tsx)
- `/.test.tsx` (from test.template.tsx)
- `/.stories.tsx` (from story.template.tsx)
```

#### ## Step 4: Confirm

After writing the files, list the four files created and offer to open the component file for editing.

Do not modify the templates themselves. Substitution happens at copy time. If the conventions in `references/component-conventions.md` conflict with what the templates produce, mention the conflict to the user and ask which to follow.

[注:代码块原文保留,以下为本页中译补充]

这段是上一页那份 `SKILL.md` 的剩余部分——把"四个被生成的文件"逐条列清(每个文件都注明"由哪份模板拷出"),再加"## Step 4: Confirm"——写完之后,列出已建的四个文件,并主动问用户是否要打开组件文件继续编辑。最后那段"Do not modify the templates themselves"是一条很关键的纪律:模板只读,替换发生在"拷出"那一刻;若 `references/component-conventions.md` 与模板产生冲突,要先把冲突亮给用户,再问走哪条——把"判断权"留给人。

#### # React Component Conventions

This codebase follows these conventions for React components.

#### ## Naming

- Component files and folders are PascalCase.
- Hook files start with `use` and are camelCase.
- Test files match the component file with a `.test.tsx` suffix.

## ## Props

- Define props as a named interface above the component, e.g.  
`interface ButtonProps`.
- Use destructured props in the function signature.
- Default values go in the destructuring, not in defaultProps.

## ## Styling

- Use CSS modules. The CSS module file matches the component name with a `.module.css` suffix.
- Avoid inline styles except for dynamic values.
- Class names are camelCase to match the CSS module convention.

## ## Exports

[注:代码块原文保留,以下为本页中译补充]

这是 `references/component-conventions.md` 的"展示到本页底部为止"的内容:① "## Naming"——组件文件与目录 PascalCase,Hook 文件以 `use` 起头且 camelCase,测试文件用 `.test.tsx` 后缀;② "## Props"——把 props 定义为命名 interface 放在组件上方、函数签名里用解构、默认值放在解构里而不是 defaultProps;③ "## Styling"——CSS modules、模块文件名 `.module.css` 后缀、避免 inline 样式(动态值除外)、class 名 camelCase 配合 CSS module 约定。"## Exports"那一节会在下一页接续展开。

← Chapter 7 上一页

下一页 →

The conventions reference is fifty lines. Loaded into context, it costs roughly seven hundred tokens. If the body of `SKILL.md` included these conventions inline, every activation of the skill would pay that cost. With the conventions in a separate reference, the cost is paid only when the body explicitly asks for it, which is once per scaffolding operation rather than on every prompt where the skill is even considered. The savings compound across the entire skill library.

这份 conventions reference 自身约五十行——加载进上下文,要花大约七百 token。若把这些规约直接内联进 `SKILL.md` 的正文,每触发一次 skill 都要付这笔开销;而把它们拆到独立的 reference 里,代价就只在正文显式索取时才付——也就是"每一次脚手架操作"才付一次,而不是"每一个 prompt 一旦考虑这条 skill"都付。这笔节省会在整个 skill 库里复利。

The templates in the `assets` folder are similarly self-contained. The component template might be twenty lines of TypeScript with the component name marked by a placeholder string. The test template might be fifteen lines with the same placeholder. Each template file is small in isolation, but together they add up to a substantial amount of generated code per scaffolding operation, and keeping them as separate files makes them easy to read, easy to edit, and easy to version-control.

装在 `assets` 里的模板,同样各自自给自足。组件模板可能是二十行 TypeScript,组件名用一个占位符串标出;测试模板可能是十五行,也用同一个占位符。每份模板单看都很小,但合起来,每次脚手架操作都会产出一份体量可观的代码;把它们各自拆成独立文件,既好读、也好

改、也好做版本管理。

## Skill as Small Program · 把 Skill 视为一份小程序

A property of this design that deserves attention: the skill is now a small program. The `SKILL.md` body is the controller. The conventions reference is the configuration. The asset templates are the data. This is the same architectural shape that has organized successful software systems for decades. The configuration is separate from the code. The data is separate from the logic. The components are independently meaningful and independently editable. A developer who needs to update the conventions edits one file. A developer who needs to add a fifth template adds one file. A developer who needs to change the procedure edits the body. None of these changes ripples into the others, because the boundaries between the components are clean.

这种设计有一条值得注意的性质:这条 skill 现在已经是份小程序。`SKILL.md` 的正文是"控制器";conventions reference 是"配置"; `assets/` 里的模板是"数据"。这正是过去几十年来组织成功软件系统的同一种架构形态:配置与代码分离、数据与逻辑分离;各组件各自独立成立,各自独立可改。一位开发者要更新规约,改一个文件;要加第五份模板,加一个文件;要修改流程,改正文。这些改动,没有任何一份会波及到其他部分——因为各组件之间的边界清清楚楚。

- The component is the default export of its main file.
- The folder's `index.ts` re-exports the component as a named export so that imports can use either form.

### ## Tests

- Use React Testing Library, not Enzyme.
- Test rendering, user interactions, and accessibility.
- Mock external dependencies with `vi.mock`` (or `jest.mock`` if the project uses Jest).

[注:代码块原文保留,以下为本页中译补充]

这是 `references/component-conventions.md` 的剩余部分。"## Exports"小节规定:组件是主文件的 default export,文件夹的 `index.ts` 把它再以 named export 重导——这样上层既可写 `import Button from '../Button'` 也可写 `import { Button } from '../Button'`。"## Tests"小节规定:用 React Testing Library(不要用 Enzyme)、要覆盖渲染、用户交互与可访问性、外部依赖用 `vi.mock` (用 Jest 的项目改 `jest.mock`)——这一行同时容纳 Vitest 与 Jest 两种生态,也是把"项目差异"显式编码进 reference 的好例子。

← Chapter 7 上一页

下一页 →

This architectural cleanliness is not an accident, and it is not unique to this skill. The three-folder convention, scripts and references and assets, exists precisely because it produces this kind of clean separation. A skill that follows the convention is a skill that has been organized for editability, for review, and for evolution over time. Skills that ignore the convention can still work, but they tend to be harder to maintain, harder to share, and harder to extend, because the responsibilities are tangled together in ways that make any single change risk affecting things that should be independent of it.

这种架构上的整洁,既不是巧合,也不是这条 skill 所独有。scripts、references、assets 这三目录的惯例之所以是现在这个样子,正是因为它能产生这种"干净分离"。遵守这套惯例的 skill,是为"可改、可 review、可演化"而组织起来的;不遵守的 skill 也能跑,但通常更难维护、难分享、难扩展——因为职责纠缠在一起,任何一次单点改动,都有可能误伤到本应独立的部分。

A final note for this chapter concerns the path forward for any individual skill. Most skills do not start as multi-file skills. They start as single-file skills, often less than fifty lines long, written in fifteen minutes to address a specific friction. Over time, as the developer uses the skill and notices its rough edges, the skill grows. Some of the growth fits naturally inside SKILL.md. Some of the growth pushes outward, into reference files when the developer wants to add long-form documentation, into asset files when the developer wants templates, into scripts when the developer wants to delegate deterministic work to real code. The multi-file structure is the destination, not the starting point. Skills that grow toward it gradually, in response to actual need, end up with structures that fit their actual use. Skills that begin life as elaborate multi-file constructions before any real use has been made of them often turn out to have been over-engineered, with bundled resources that nobody ever loads and templates that never get applied to real work. The discipline is to start simple, use the skill, and add complexity only when the skill itself is asking for it.

本章的最后一段,留给"一条 skill 应当走一条什么样的成长路径"这件事。大多数 skill 不是从多文件起步的;它们从单文件起步——通常不到五十行,用十五分钟写出来,只为治一处具体的摩擦。一段时间过去,开发者不断用、不断注意到它的"毛边",skill 才开始长。有一部分"长"自然装得下 SKILL.md;另一部分"长"会往外挤——加 reference 文件,放长篇文档;加 asset 文件,放模板;加 script 文件,把确定性的活交给真代码。多文件结构是目的地,不是出发点。沿着"对真实需要的回应"逐渐走到那里的 skill,最终的结构与它真实的用法严丝合缝;反过来,还没真正被用过就先把多文件结构搭得花里胡哨的 skill,最后几乎都被证明是过度工程——打包的资源无人加载、模板从未被套到真正的工作上。纪律是:从简起步,先把它用起来,等它自己开口要更复杂时,再加复杂度。

A useful test for whether a skill is ready to expand into multiple files is the printout test. Print the SKILL.md body on paper. Read it through. If, while reading, the eye keeps wanting to skip past large chunks of detail because those chunks are not relevant to the immediate task, those chunks are candidates for relocation into reference files. If, while reading, the body begins to feel less like a procedure and more like a manual, the body has grown too large and needs to be split. Conversely, if the body fits comfortably on a single printed page and reads as a clean procedure from beginning to end, the skill does not yet need to expand. The printout test is informal but reliable, and it spares the developer from the trap of expanding skills before they have earned the right to expand.

判断一条 skill "是否已经准备好要扩成多文件"时,有一个好用的测试——"打印测试"。把 SKILL.md 的正文打印到纸上,从头读到尾。读的时候,如果眼睛老想跳过"大块的、与当下任务无关的细节"——那些大块,就是"可以搬到 reference 文件里去"的候选;如果读着读着,这份正文已经不再像"流程",而越来越像"手册"——那就说明它已经长得太大,需要被拆。反过来,如果整份正文舒舒服服地印在一张 A4 上,从头到尾读起来都像一份干净的流程,那这条 skill 还不到需要扩的时候。"打印测试"看似不正式,却很可靠;它能让开发者躲过"在 skill 还没挣得那份复杂度之前就先把它复杂化"的陷阱。

The chapter has covered the structural facts. The next chapter takes the natural next step. With the ability to build skills that span multiple files, the developer has the raw material for a library. The question becomes how skills behave when more than one of them might apply to a given request, how they compose with each other when their work overlaps, and how to design libraries in which skills cooperate rather than collide. Composition is what turns a collection of skills into a system, and it is the subject of the chapter ahead.

本章,讲的是"结构事实"。下一章走下一步自然而然的台阶——既然已经能搭出跨多文件的 skill,开发者手里就有了建造"skill 库"的原材料。接下来的问题变成:当一条请求可能同时适用多条 skill 时,它们会怎么表现?当它们的活彼此重叠时,它们之间会怎么组合?怎样设计,才能让一个 skill 库里的各条 skill 彼此协作而不是彼此撞车?组合,是把"一堆 skill"变成"一套 skill 系统"的那一步——而这,正是下一章的主题。

# CHAPTER EIGHT · 第八章

## Skill Composition and Collision: When Skills Work Together

### Skill 的组合与撞车:让 Skills 协作起来

*"The library is more than the sum of the books on its shelves only when the shelves themselves have been arranged with thought."*

— Jorge Luis Borges, paraphrased

*"图书馆大于'它架上各书之和'这件事,只在'架子本身也是经人琢磨摆出来的'时候才成立。"*

—— 豪尔赫·路易斯·博尔赫斯(转述)

A single skill, well written, improves a developer's daily life. A library of skills, well composed, transforms it. The transformation does not happen automatically. Skills do not naturally cooperate. Left to their own devices, skills can also collide: two skills that both think they apply to the same request, with overlapping descriptions, produce inconsistent firing and unpredictable behavior. The discipline of skill composition is the discipline of designing libraries in which the skills clearly belong to their own territories, hand off cleanly when work crosses a boundary, and amplify each other when more than one is needed for a single task. This chapter is about that discipline.

一条写得到位的 skill,能让开发者的日常稍好一点;而一套"组合得道"的 skill 库,会把这种"好一点"升格为"焕然一新"。而这种升格,并不是自动发生的。Skill 之间并不天然协作;若放任自流,它们甚至会撞车——两条 description 有重叠的 skill,都可能认为自己适用于同一条请求,于是触发变得不一致、行为变得不可预测。"Skill 组合"这门手艺,正是"设计一个库:让每条 skill 各自清楚属于自己那片领土,在工作跨界时干干净净地交接,且当一条任务需要多条协作时彼此放大"的纪律。这一章,就讲这套纪律。

Begin with how Claude actually decides which skill to fire. When a user submits a prompt, Claude evaluates the prompt against the descriptions of every available skill. Each description is a piece of evidence about whether the corresponding skill is relevant. Claude integrates the evidence and makes a judgment. In the simplest case, only one skill's description matches the prompt, and that skill fires. In more interesting cases, two or more skills' descriptions match, and Claude must choose. The choice is influenced by how confidently each description seems to match, by how specific each description is, and by any boundary language the descriptions contain. When two descriptions are equally plausible, the firing becomes a judgment call that may go either way on different sessions, which is the practical face of skill collision.

先讲 Claude 实际是怎么决定"哪条 skill 该触发"。当用户提交一条 prompt,Claude 会拿这条 prompt 去与"每条可用 skill 的 description"比对。每条 description 是一份"该 skill 是否相关"的证据;Claude 把这些证据合到一起,做出一份判断。最简单的情况下,只

有一条 skill 的 description 与 prompt 匹配,那条就触发;更有趣的情形是:有两条或更多 description 都匹配,Claude 必须从中选一个。选哪条,取决于每条 description "看起来有多像那么回事"、"有多具体"、以及里面是否带边界语言。当两条 description 看起来都差不多合理时,"哪条该触发"就变成了一份"判官题"——可能在不同 session 里结果不同,这正是 skill 撞车在实践中的面相。

Collision is bad not because the wrong skill fires, although that is annoying. Collision is bad because the developer cannot rely on which skill will fire, and reliability is the entire point of installing skills in the first place. A skill library in which firing is reliable is a library that the developer

撞车之所以糟糕,不是因为"错的 skill 被触发"——虽然那确实让人烦。撞车之所以糟糕,是因为开发者无法信任"究竟哪条会触发"——而可靠性,正是当初装这套 skill 的全部意义。一个触发可靠的 skill 库,是那位开发者——

← Chapter 7 结尾

下一页 →

can trust. A library in which firing is uncertain is a library the developer must verify on every use, which negates the value of having installed the skills at all.

能放心去信的库;而一个触发不可预期的库,逼着开发者每次用都得亲自去核验——那也就把"当初装 skill 想要的那份价值"全数抹掉了。

Three patterns produce most collisions in real skill libraries. The first pattern is overlapping topic language. Two skills mention the same topic in their descriptions, and Claude cannot tell which one a given request belongs to. A `pr-description` skill and a `pr-review` skill both mention pull requests, and a vague request such as `help me with this PR` could plausibly fire either one. The fix is boundary language: each description should explicitly state what its skill is and is not for. The `pr-description` skill says, `this skill is for writing the description of a PR you are creating, not for reviewing one`. The `pr-review` skill says, `this skill is for reviewing PRs that someone else wrote, not for drafting your own`. Each description now contains both an inclusion criterion and an exclusion criterion, and the collision evaporates.

真实 skill 库中的大多数撞车,来自三种模式。第一种——"主题措辞重叠"。两条 skill 的 description 都提了同一个主题,Claude 分不清一条给定请求该归哪边。比如 `pr-description` 和 `pr-review` 都会提到 `pull request`;而一条含混的请求——比如 `help me with this PR`——按理两边都可能被触发。修法是"边界语言":每条 description 都应显式讲清"自己干什么、不干什么"。`pr-description` 这样写: `this skill is for writing the description of a PR you are creating, not for reviewing one`; `pr-review` 这样写: `this skill is for reviewing PRs that someone else wrote, not for drafting your own`。每条 description 此刻都同时携带一条"包含准则"和一条"排除准则",撞车就此散去。

The second pattern is mismatched specificity. One skill has a very specific description; another has a very general description that happens to overlap with it. The general one tends to win in cases where the specific one should have. The fix is to either make the general skill more specific by adding boundary language, or to recognize that the general skill is poorly designed and should be split into several more focused skills. A general skill called `code-quality` that fires on anything related to code quality is almost always a mistake. It would compete with a `refactor-helper` skill, a `code-review` skill, a `static-analysis` skill, and several others that overlap with parts of its territory. The right response is usually to dissolve the general skill into the specific ones, each of which has a clear description.

第二种——"具体度不匹配"。一条 skill 的 description 写得很具体;另一条的 description 写得很泛,偏偏又与前者有重叠。结果往往是:那条"泛的"在很多本该"具体的"触发的场景下抢了先。修法有两条:一是用边界语言把那条"泛的"再收紧一些;二是承认那条"泛的"在设

计上本就是错的,应当被拆成若干条更聚焦的 skill。一条叫 `code-quality`、凡提到"代码质量"就触发的"通用" skill,几乎永远是个错误——它会与 `refactor-helper`、`code-review`、`static-analysis` 等多条 skill 抢地盘,且抢的范围会与它们各自领域的不同片段都重叠。正确的处理,通常是把它"消解"回那些更具体的 skill,让每一条都各持一份清晰的 description。

Naming Drift · 命名漂移

The third pattern is naming drift. The skill author writes the description in one vocabulary and the body in another. The user, naturally, employs vocabulary that matches the body more than the description, because the body is closer to the actual work. The skill under-fires because the description does not contain the words the user actually uses. The fix is to bring the description into alignment with the working vocabulary of the body. A useful exercise is to read the body of the skill, list the ten words that appear most frequently and seem most central to what the skill does,

第三种——"命名漂移"。Skill 作者写 description 时用的是一套词,写正文时用的是另一套;用户自然偏向使用"更贴正文"的那套词——因为正文更靠近实际工作。结果就是 skill 欠触发:description 里没装用户真正在用的那些词。修法是把 description 拉回到与正文的"工作词表"对齐。一条好用的练习:把 skill 正文读一遍,列出"出现频率最高、对描述 skill 实质而言最关键"的十个词,

and check that those words appear in the description. If they do not, the description should be rewritten to include them.

再核对一遍,看这十个词有没有出现在 description 里。如果没有,就该把 description 改写,把这些词补上。

With the failure modes named, turn to composition itself. Composition is the positive case: two or more skills cooperating on a single request to produce something better than either could produce alone. Several patterns of composition are worth knowing.

失败模式已经命名完,接下来看组合本身。所谓"组合",是正面情形——两条或更多条 skill 协作完成同一条请求,产出的东西比任何一条单干都要好。值得了解的有几种组合模式。

The first composition pattern is sequential composition. Two skills fire in sequence on the same request, with the output of the first becoming the context for the second. A common case: the user asks for a refactoring plus a commit. The `refactor-helper` skill fires first, executes the refactor, and produces a description of what changed. The `commit-message` skill fires next, reads the changes, and produces a commit message that accurately reflects them. Sequential composition often happens implicitly: the developer asks Claude to do two things in one prompt, and Claude activates both relevant skills as it works through the prompt. The skill author can encourage this by writing skill bodies that conclude with a clean handoff. The `refactor-helper` skill, after completing its work, might end its body with a line such as `if the user wants a commit message for these changes, the commit-message skill is appropriate`. Claude reads that hint and is more likely to fire the second skill at the right moment.

第一种组合模式——"顺序组合"。两条 skill 在同一条请求上按序触发,第一条的输出成为第二条的上下文。常见情形:用户要求"重构 + commit"——`refactor-helper` 先触发,把重构做完,产出一段"改了什么"的说明;`commit-message` 紧接着触发,读入那些改动,产出一条准确反映它们的 commit message。顺序组合常常是隐式发生的:用户在一个 prompt 里让 Claude 干两件事,Claude 在推进过程中把两条相关的 skill 都激活了。Skill 作者可以"鼓励"这种发生——办法是在正文末尾留一句干净的"交接语"。`refactor-helper` 干完活

之后,可以在正文结尾留一句 `if the user wants a commit message for these changes, the commit-message skill is appropriate`。Claude 读到这一句,就更可能在合适的时刻触发第二条 skill。

The second composition pattern is parallel composition. Two or more skills fire on the same request and contribute different perspectives to the same output. A code-review request might activate a security-focused skill, a performance-focused skill, and a style-focused skill simultaneously, with the final review weaving together all three perspectives. Parallel composition is more common when the request is broad and the skills are narrow, because each narrow skill contributes its own piece of the broad answer. The skill author encourages parallel composition by writing skills that produce well-bounded, additive output rather than total reports. A security-focused review skill that produces a self-contained section labeled `Security` and stops there composes well with other focused skills. A security-focused review skill that produces an entire end-to-end review report does not, because its output overlaps with what the other skills want to produce.

第二种组合模式——“并行组合”。两条或更多条 skill 在同一条请求上同时触发,为同一份输出贡献各自不同的视角。一条 code-review 请求,可能同时激活一条安全聚焦的 skill、一条性能聚焦的 skill、一条风格聚焦的 skill;最终的那份 review,把三者的视角编织在一起。并行组合更常出现在“请求很宽、skill 都很窄”的情形下——因为每条窄 skill 只贡献宽答案里的自己那一块。Skill 作者要鼓励并行组合,办法是把 skill 写成“产出边界清晰、彼此可叠加”的输出,而不是“一份大全的报告”。一条产出“名为 `Security` 的自成一节、到那里就停”的安全 review skill,很能与其它聚焦的 skill 组合;而一条产出“端到端完整 review 报告”的安全 review skill,就组合不动——因为它的输出会与其它 skill 想要产出的东西撞上。

← Chapter 8 上一页

下一页 →

The third composition pattern is the meta-skill. A meta-skill is a skill whose job is to orchestrate other skills. It has a description that fires on broad requests, and its body contains instructions that direct Claude to invoke specific other skills in a specific order. Meta-skills are useful for workflows that recur often enough to deserve a single trigger but that involve several distinct steps. A `release-prep` meta-skill might fire on a request to prepare a release and direct Claude to invoke a `changelog-generator` skill, a `version-bump` skill, a `release-notes` skill, and a `tag-and-push` skill, in sequence. The meta-skill is a small orchestrator, and the actual work is done by the focused skills it directs.

第三种组合模式——“元 skill”。所谓元 skill,是一条专门用来“指挥其它 skill”的 skill:它的 description 在较宽的请求上触发;它的正文里装着若干指令,告诉 Claude 按某种顺序去调用哪些具体 skill。元 skill 适用于“频繁到值得为它设一个单一触发点、但内部又包含几个互不相同步骤”的 workflow。比如一条叫 `release-prep` 的元 skill,会在“准备发版”的请求上触发,指引 Claude 依次调用 `changelog-generator` skill、`version-bump` skill、`release-notes` skill 和 `tag-and-push` skill。元 skill 自己是个小调度者,真正干活的,是它所指引的每一条聚焦 skill。

### A Practical Guideline · 一条实操准则

A practical guideline ties the patterns together. Build narrow skills first. A library of twenty narrow, focused skills is more useful than a library of five general ones. Narrow skills fire reliably because their descriptions can be precise. Narrow skills compose because they have well-bounded outputs. Narrow skills are easier to debug because their failure modes are concentrated. The temptation to build broad skills usually comes from a desire to reduce the number of skills in the library, but this is the wrong objective. The right objective is to maximize reliability and composability, both of which point toward narrow skills.

一条实操准则,把这几种模式串到了一起——先搭窄的 skill。一个装二十条“窄而聚焦” skill 的库,比一个装五条“宽而泛” skill 的库要有多得多。窄 skill 触发得稳,因为它们 description 能写得很精确;窄 skill 组合得动,因为它们的输出边界清楚;窄 skill 容易 debug,因为失

败模式集中在一处。"想搭宽 skill"的诱惑,通常来自"想让库里少几条"的愿望——但那是个错误的目标。正确的目标是"最大化可靠性 + 可组合性",这两条都指向窄 skill。

A second guideline concerns the boundary between skills and the body of `CLAUDE.md`. Some knowledge belongs in `CLAUDE.md`, where it is always loaded. Some knowledge belongs in a skill, where it is loaded on demand. The dividing line is roughly: knowledge that is true at all times in this project belongs in `CLAUDE.md`; knowledge that is true only when a specific kind of work is being done belongs in a skill. The rule sounds simple, and in practice it requires some judgment, but applying it consistently produces a Claude Code environment where ambient knowledge is always present, targeted knowledge is loaded only when relevant, and the boundary between the two is principled rather than arbitrary.

第二条准则,关于"skill 与 `CLAUDE.md` 正文之间的分界"。一些知识该放在 `CLAUDE.md` ——它会一直被加载;另一些知识该放进 skill ——按需加载。粗略的划分线是:在本项目里任何时候都成立的知识,放 `CLAUDE.md`;只在某类具体工作进行时才成立的知识,放 skill。这条规则听上去简单;实操中确实需要一些判断;但只要能一以贯之地用,就能造就一个这样的 Claude Code 环境——"环境性"的知识永远在场,"针对性"的知识只在相关时入场,两者之间的边界,是有原则的、不是任意的。

There is one more compositional possibility worth naming, although its full treatment belongs in Part Three. Skills can be invoked by subagents as well as by the main session. A subagent dispatched for a specific kind of work inherits the skill library available in the project and can activate any

还有一种组合上的可能性值得点出名字——虽然完整的展开属于第三部分。Skill 也能被 subagent 调用,而不只是被主会话调用。被派去处理某类具体工作的 subagent,会继承项目里那份可用的 skill 库,并能激活其中的任一

← Chapter 8 上一页

下一页 →

skill whose description matches the work the subagent is doing. This is a powerful pattern, because it lets the skill library and the subagent library reinforce each other. A `test-writer` subagent that has access to a project's test-conventions skill is more reliable than either the subagent or the skill alone. The full implications of this composition will be developed in Chapter Fourteen, after subagents have been introduced. For now, it is enough to know that the skills built in this part of the book will continue to do work even after the boundaries of skills as a topic are crossed.

一条 description 与该 subagent 正在做的工作匹配的 skill。这是一股很强的力量——它让 skill 库与 subagent 库彼此强化。一个持有"本项目测试规约" skill 访问权的 `test-writer` subagent,比 subagent 或 skill 任何一方单独存在时都更可靠。这种组合的全部内涵,会在第十四章——也就是介绍完 subagent 之后——再展开。此刻只需要知道一件事:这一部分搭出的 skill,在我们跨过"skill 作为一个话题"的边界之后,仍会继续工作。

This chapter has dealt with the politics of a skill library: how skills relate to each other, how they collide and how they cooperate, and what the author can do to encourage the second over the first.

本章讲的是"一个 skill 库内部的'政治学'"——skill 之间如何相互关联、如何撞车、如何协作,以及作者可以做些什么,来鼓励后一种情况、多于前一种。

A practical exercise will reinforce the principles. Imagine a developer who, over six months of casual skill building, has accumulated a library of fifteen skills. She has noticed that some of them seem to fire reliably and others fire inconsistently. She decides to audit the library against the principles in this chapter.

一份实操练习,用来强化这些原则。设想一位开发者,经过六个月的随手积累,手上已经攒起了十五条 skill。她注意到其中一些触发得很稳,另一些却忽好忽坏;于是决定按本章的原则,给这个库做一次"审计"。

## The Audit Procedure · 审计流程

The audit procedure is straightforward. She lists the fifteen descriptions side by side. She reads each one and scores it against the four-part framework from Chapter Six: canonical statement, trigger language, boundary language, input expectation. She also reads each pair of descriptions and asks whether either pair could plausibly fire on the same request. Of the fifteen skills, three pairs jump out as potential collisions. A `pr-description` skill and a `pr-review` skill, both mentioning pull requests. A `code-explain` skill and an `explain-codebase` skill, both mentioning code explanation. A `bug-report` skill and a `debug-helper` skill, both mentioning bugs.

审计流程本身很直白。她把十五条 description 一字排开,逐条对照第六章的"四段式框架"来打分——典型陈述、触发语言、边界语言、输入预期。同时,她把每两条 description 配对读,问自己:"这一对,有没有可能在同一条请求上被同时触发?"十五条之中,有三对明显撞上: `pr-description` 与 `pr-review` ——都提到 pull request; `code-explain` 与 `explain-codebase` ——都提到代码解释; `bug-report` 与 `debug-helper` ——都提到 bug。

Each pair gets resolved differently. The first pair is the cleanest case. Each skill clearly has a different job, but neither description names the difference explicitly. She adds a single sentence of boundary language to each. The `pr-description` description gains the line, `this skill is for writing the description of a PR you are creating, not for reviewing one`. The `pr-review` description gains the symmetrical line, `this skill is for reviewing PRs that someone else wrote, not for drafting your own`. The two skills now have non-overlapping territories. Subsequent firing becomes reliable.

每对的修法各不相同。第一对是最干净的情况:两条 skill 的活明显不同,但没有一条 description 把这种不同说清。她在两边各加一句边界语言。`pr-description` 的 description 加上这么一句: `this skill is for writing the description of a PR you are creating, not for reviewing one`; `pr-review` 的 description 加上对称的那一句: `this skill is for reviewing PRs that someone else wrote, not for drafting your own`。两条 skill 此刻的领域不再重叠;此后的触发就稳了。

← Chapter 8 上一页

下一页 →

The second pair is more interesting. Reading them carefully, she realizes that `code-explain` and `explain-codebase` are not really separate skills. `code-explain` operates on a piece of code that has been pasted or pointed to. `explain-codebase` operates on a whole repository. The original intent was that they would handle different scales of explanation. But the descriptions have not made the scale distinction clear, and in practice users have been triggering whichever description Claude happened to match first, regardless of which scale the user actually wanted. She rewrites both descriptions to make the scale explicit. `code-explain` says, `explains a single function, file, or pasted snippet`. `explain-codebase` says, `produces a high-level architectural summary of an entire codebase`. Now the descriptions are not just non-overlapping; they are explicitly complementary. A user who pastes a function gets `code-explain`. A user who asks for a tour of the repo gets `explain-codebase`. The collision evaporates.

第二对,更有意思一些。仔细一读,她意识到 `code-explain` 与 `explain-codebase` 实际上不是两条彼此独立的 skill: `code-explain` 处理的是"被粘贴或被点出"的一段代码; `explain-codebase` 处理的是整个仓库。原本的设想,是让它们分别应付不同尺度的"解释";但 description 里没把这种尺度差讲清,实际跑下来,用户触发的常常是"Claude 先碰上的那一条"——无论用户真正想要的是哪种尺度。她把两边的 description 都改写,把"尺度"讲得明确。`code-explain` 写: `explains a single function, file, or pasted snippet`; `explain-codebase` 写: `produces a high-level architectural summary of an entire codebase`。此刻的 description 不再只是"不重叠",而成了"显式互补"——粘一段函数的用户拿到 `code-explain`;想要"逛一下这个 repo"的用户拿到 `explain-codebase`。撞车散去。

The third pair turns out to be a deeper problem. The `bug-report` skill formats a structured bug report. The `debug-helper` skill walks the user through diagnosing a bug. Both fire on requests that mention bugs. After looking at how she actually used the two skills over the past month, she realizes that `debug-helper` has fired roughly four times, all of them in cases where `bug-report` would also have been a reasonable choice, and none of those four sessions produced output she actually used. `debug-helper`, in retrospect, was an attempt to build a skill that did too much: a generic debugging companion. Its description was broad because its job was broad, and broad jobs do not fit the skill model well. She decides to retire `debug-helper` and replace it, eventually, with two or three narrower skills that cover specific debugging scenarios. For now, removing it cleans up the library and removes the collision with `bug-report`. The library shrinks from fifteen skills to fourteen, and quality goes up rather than down.

第三对,挖出来的是一个更深的问题。`bug-report` skill 把一条 bug 报告整理成结构化形式; `debug-helper` skill 引导用户走完"诊断 bug"的流程。两条都会在"提到 bug"的请求上触发。回看过去一个月里她对这两条 skill 的实际使用,她发现 `debug-helper` 大约触发了四次——四次之中,每次其实由 `bug-report` 来跑也合理;且这四次产出的东西,她事后都没真的用过。回头看, `debug-helper` 是一次"想搭一条能干太多事的 skill"的尝试——做一个"通用的 debug 伙伴"。它的 description 之所以宽,是因为它的活本来就很宽;而那种"宽活"和 skill 这个模型本身就不合身。她决定把 `debug-helper` 退下来,日后用两三条更窄的 skill 去覆盖具体的 debug 场景;眼下这一撤,既清理了库,也消除了与 `bug-report` 的撞车。库从十五条缩到十四条,质量却反过来上升了。

This kind of audit is worth performing once a quarter on any skill library that has grown past ten skills. The principles are simple: list the descriptions, score them against the four-part framework, look for collisions, and either resolve the collisions through boundary language, refactor through clearer scope language, or retire skills that have proven to be too broad to do their job well. A library that has been audited recently is a library that the developer can trust, and trust is what makes the library worth using.

这种审计,值得在任何一个超过十条 skill 的库上,每季度跑一次。原则很简单:把 description 摆出来,按"四段式框架"逐条打分,寻找撞车;每一起撞车,要么用边界语言解决,要么用更明确的 scope 措辞重构,要么把那些"宽到干不好活"的 skill 直接退掉。一个最近被审计过的库,是一个开发者能信得过的库;而信得过,正是让这个库"值得用"的那份东西。

## Composition Across Domains · 跨领域的组合

A second concrete scenario, which surfaces a different design pattern, involves a developer building a skill library for a team. The team has a shared `.claude/skills/` directory in their project, and three developers are contributing to it. Each developer has, naturally, drafted a few skills in her own style. The result, after a month, is a library that is technically functional but stylistically inconsistent. Some descriptions follow

the four-part framework. Others are one-liners. Some bodies use the step-numbered procedure pattern. Others use prose. Some skills assume the user will provide context. Others have built-in steps to gather it.

第二份具体的场景,展示的是一种不同的设计模式——一位开发者为一个团队搭一个 skill 库。团队在项目里共享一份 `.claude/skills/` 目录,三位开发者各自向里面贡献。自然地,每个人都按自己的风格起草了几条 skill。一个月之后,这个库在技术上能跑,但风格上一团混乱:有些 description 走"四段式",有些只是一句话;有些正文用"步骤编号"的流程,有些用的是散文;有些 skill 假定用户会主动提供上下文,有些则自带"收集上下文"的步骤。

The team decides to standardize. They sit down together for thirty minutes and agree on a small set of conventions. Every description must include canonical, trigger, boundary, and input expectation language. Every body must use a step-numbered procedure for any skill whose work is sequential. Every skill that needs context must include a step-one section that gathers that context, rather than relying on the user to provide it pre-emptively. The conventions are written into a short `README.md` at the root of the `.claude/skills/` directory, so that any future contributor encounters them at the moment she opens the directory for the first time.

团队决定把风格统一起来。他们坐下开了三十分钟的会,就一套小规模约定达成一致:每条 description 必须含典型陈述、触发语言、边界语言、输入预期四段;每条正文,凡是工作本身是顺序性的,就必须用"步骤编号"流程;每条需要上下文的 skill,都必须自带一个"第一步:收集上下文"小节——而不是指望用户主动预备好。这些约定被写进一份放在 `.claude/skills/` 根目录的简短 `README.md`,于是任何一位未来的贡献者,在她第一次打开这个目录的那一刻,就会与这份约定不期而遇。

The conventions take effect immediately for new skills, and existing skills are gradually retrofitted as they need attention for other reasons. Within a month, the library has a coherent house style. Skills feel like they belong together. New contributors can write a new skill in fifteen minutes because the conventions answer most of the design questions before the contributor even asks them. The library has become, in the genuine sense of the word, a library, with a unifying organizing principle, rather than a heap of files in a folder.

约定对新搭的 skill 立刻生效;已有的 skill,借着"出于别的原因需要被关注"的机会,被逐条回炉。不到一个月,这个库就有了"一种连贯的自家风格"——skill 们看上去"彼此是一伙的"。新来的贡献者能在十五分钟内写出一条新 skill,因为那些"原本要靠问才能答出来"的设计问题,已经被约定提前答好。这个库,在最严格的意义上,成了一座图书馆——带着一种统一的组织原则,而不只是"文件夹里的一堆文件"。

A final thought on composition before this chapter ends. The most powerful skill libraries in the wild, the ones that earn the title library rather than mere collection, share a property that is worth naming. They are designed for the user's actual workflow, not for the developer's sense of what skills should exist. A skill that addresses a friction the developer has experienced personally, in real work, will almost always fit cleanly into a library. A skill that the developer built because she imagined someone might want it, without having needed it herself, will almost always feel out

本章收尾前,关于"组合"再留一句观察。野外最强大的那些 skill 库——那些真配得上"library"二字、不只是"collection"的——共有一条值得点出的性质:它们是按用户的实际 workflow 来设计的,而不是按"开发者自己觉得应该有哪些 skill"来设计的。一条 skill,处理的是开发者在真实工作中亲自撞过的那处摩擦,几乎总能干干净净地装进一个库;一条 skill,是开发者想象"也许有人想要"、但自己并没用过的那种,几乎总会在库——

of place. The audit principle that follows from this is to be ruthless about skills that have not been used. A skill that has not fired in three months is a skill that does not belong in the library, even if its description is technically excellent. The cost of keeping it is small but real, and the noise it adds to the firing decision marginally degrades every other skill that competes for the same attention.

里显得格格不入。从这条观察里,直接派生出的一条审计原则是:对"长期没被用过"的 skill,要狠得下心。一条三个月都没触发过的 skill,即使 description 在技术上堪称漂亮,也不该再留在库里。留着它的代价,虽小但真实;它给"该触发哪一条"这件事多加的噪声,会让每一条与它争抢注意力的 skill 都被稍微拉低一点。

A pattern that separates great libraries from merely good ones is the deliberate cultivation of trigger overlap with the developer's natural vocabulary. The developer who pays attention notices, after a few months, that she has habits of phrasing that recur. She tends to say `make me a`, when she wants something generated. She tends to say `walk me through`, when she wants something explained. She tends to say `tighten this up`, when she wants something refined. Each of these recurring phrasings is a hook, and skills whose descriptions explicitly include the developer's recurring phrasings fire more reliably than skills whose descriptions use generic alternatives. Building this kind of personal-vocabulary alignment into descriptions is a small practice, but it converts a library from one that fires correctly to one that fires effortlessly. The library starts to feel less like a tool the developer is using and more like an extension of her own way of speaking. This is the felt sense of a mature library, and it cannot be achieved on the first day. It is the gradual product of attention paid to the developer's own habits, applied iteratively across descriptions over time.

"伟大的库"与"仅仅好的库"之间,有这样一种"会区分"的模式:主动去让 description 与开发者的"自然词表"产生触发重叠。一位留心的开发者,几个月后就会注意到自己反复出现的措辞习惯——她想"生成"什么时,习惯说 `make me a`;她想"讲解"什么时,习惯说 `walk me through`;她想"打磨"什么时,习惯说 `tighten this up`。每一种这样的"反复措辞",都是一条钩子;哪些 skill 的 description 显式包含这些钩子,触发起来就比"用通用替代措辞"的描述更稳。把"个人词表对齐"这层动作嵌入 description,是一件小习惯;但它能把一个库从"触发得对"升级到"触发得毫不费力"。这个库会开始"不太像"开发者正在使用的一件工具,而"越来越像"她说话方式的一种延伸。这就是"一座成熟库"的那种体感——它不是第一天就能达到的;它是开发者把注意力持续放在自己的措辞习惯上、一次又一次把"对齐"应用到 description 里,日积月累的产物。

The next chapter shifts from politics to inventory. With the principles of composition in hand, the reader will build a personal library of ten skills that are immediately useful in daily work, with each skill written, debugged, and ready to install. The chapter ahead is the most practical in the book. By its end, the reader's Claude Code environment will be measurably more capable than it was at the start of Part Two, and the value of the principles taught in the chapters before it will be visible in every session that follows.

下一章,从"政治学"切换到"清单"——手上握过组合原则之后,读者要亲手搭起一份"十条 skill 的个人库",每一条都"立刻就能在日常工作里用得上",且每一条都写好、调好、装好就位。下一章是全书最"实操"的一章。读完之后,读者的 Claude Code 环境,在能力上会有一种"可被量化"地提升——而前面几章所教的那些原则,会在此后每一个 session 中以可见的方式兑现。

## CHAPTER NINE · 第九章

### The Personal Skill Library: Your First Ten Skills

# 个人技能库:你的前十个技能

"A library is not a collection of books. A library is a set of decisions about which books to keep at hand."

— Umberto Eco, paraphrased

"图书馆不是藏书的集合,图书馆是一组关于'哪些书该放在手边'的决定。"

—— 安伯托·艾柯(转述)

This chapter is the most practical in Part Two. By the time the reader finishes it, ten new skills will be installed in her Claude Code environment, each of them ready for immediate use, each of them solving a specific recurring friction in daily development work. The skills are presented in a uniform format. Each one begins with the problem it solves, then shows the complete `SKILL.md` file ready to copy, then explains the design choices that shaped it, then notes the most common pitfall and the recommended way to extend the skill once the developer has lived with it for a while.

本章是第二部分里最"动手"的一章。等读者读完时,她的 Claude Code 环境里会多装十个新 skill——每一个都马上可用,每一个都解决日常工作里某一种反复出现的摩擦。展示这些 skill 时用的是统一格式:每个 skill 都先讲它解决的问题,然后贴出可以直接拷走的完整 `SKILL.md` 文件,再解释塑造它的设计选择,最后点出最常见的坑,以及开发者与这个 skill 共处一段时间后该如何扩展它。

The selection of ten skills is deliberate. Each skill represents a category of friction common to almost every developer using Claude Code. The categories are: composing structured documents from raw material, generating standardized scaffolding, performing focused review, capturing knowledge from work that just happened, formatting and conforming to project conventions, and coordinating between code and surrounding artifacts such as commits and pull requests. The reader who installs all ten will have her core daily workflow substantially automated. The reader who installs even three or four will feel the difference within a working week.

选这十个 skill 是有讲究的。每个 skill 都对应一类"几乎所有用 Claude Code 的开发者都会遇到"的摩擦。这些类别包括:把原始素材组织成结构化文档、生成标准化脚手架、做有针对性的评审、从刚刚发生的工作中捕捉知识、按项目规约格式化与对齐、以及在代码和周边产物(如 commit、PR)之间做协同。把十个全装上的读者,核心日常工作流会大幅自动化;哪怕只装三四个,也能在一个工作周内明显感到差别。

Before beginning the build, a small piece of housekeeping. All ten skills in this chapter are intended to be personal skills, installed at the user level, and therefore live in `~/.claude/skills/`. Each one needs its own folder, named to match the skill. The folder structure for the full library will look like this when complete.

开搭之前,先做点内务。本章的十个 skill 都是"个人 skill",按用户级别安装,因此都放在 `~/.claude/skills/` 下。每个 skill 一个目录,目录名跟 skill 名一致。整套库装好之后,目录结构会是这样:

```
~/.claude/skills/
├─ commit-message/
├─ pr-description/
├─ humanize-text/
├─ refactor-readability/
├─ jest-test/
├─ pr-review/
├─ code-explain/
└─ bug-report/
```

## Setting Up Each Skill · 每个 skill 的安装方式

Each of the ten skills below requires the same setup: create the named folder, create a `SKILL.md` file inside it, paste the contents shown. After installing all ten, restart Claude Code so that the new skills are discovered. From that point forward, the skills will be available in every session.

下面这十个 skill,装法都一样:建一个对应名字的目录,在里面建一个 `SKILL.md` 文件,把展示的内容贴进去。十个都装好后,重启 Claude Code,让新 skill 被发现。从那之后,这些 skill 就会在每个 session 里都可用。

The first skill is `commit-message`. It was built in detail in Chapter Five and serves as the canonical reference for what a small skill looks like. Its job is to produce conventional commit messages from staged changes. If the reader has not already installed it, the `SKILL.md` file from Chapter Five can be used directly.

第一个 skill 是 `commit-message`。它在第五章里已经详细搭过,这里作为"小 skill 长什么样"的范本。它的职责是:基于已暂存(staged)的改动,产出符合 conventional 规范的 commit message。如果读者还没装,直接把第五章里的那个 `SKILL.md` 拿来用就行。

The second skill is `pr-description`, used as an example in earlier chapters. It produces pull request descriptions in a consistent format.

第二个 skill 是 `pr-description`,前面的章节里已经作为示例出现过。它的职责是:用统一格式产出 pull request 描述。

```

name: pr-description
description: Drafts a pull request description in the team's standard format. Use whenever the user asks for a PR de
allowed-tools: Bash, Read, Grep

PR Description
When asked for a PR description, follow this procedure.
1. Identify the diff. If the user mentions a branch, run `git log` and `git diff` against the merge base. If no bran
2. Read the diff in full.
3. Produce the description in this exact format:
Summary
A single sentence in the imperative mood describing the change.
```

The third skill is `humanize-text`. Its job is to take written content that sounds machine-generated, formal, or stilted, and to rewrite it in a more natural human voice. This skill is especially valuable for developers who write documentation, blog posts, or release notes alongside their code, and who have noticed that AI-generated drafts have a recognizable tone that benefits from editing.

第三个 skill 是 `humanize-text`。它的职责是:把听起来像机器写的、过于正式、生硬别扭的文字,改写成更自然的人类口吻。这个 skill 对那些"除了写代码还要写文档、写博客、写 release notes"的开发者尤其有用——他们早已发现,AI 生成的草稿有一种可辨识的味道,过

一遍编辑会好得多。

```
What changed
A bulleted list of concrete changes, one bullet per logical unit.
Testing
A short paragraph describing how the change was tested.
Risk
One sentence categorizing risk as low, medium, or high with a one-line reason.
Do not invent changes that do not appear in the diff. If the diff is large, summarize at the file level rather than
```

```

name: humanize-text
description: Rewrites text to sound more human, less formal, and less machine-generated. Use whenever the user asks
allowed-tools: Read

Humanize Text
When asked to humanize a piece of text, apply these transformations.
- Remove em-dashes. Replace with commas, periods, or sentence breaks.
- Remove or replace AI-flavored vocabulary: "delve," "leverage," "robust," "navigate," "in conclusion," "it is impor
- Vary sentence length deliberately. Short sentences punctuate. Long sentences explore. Avoid runs of medium-length
- Cut adverbs that do not earn their place: "really," "very," "quite," "extremely."
- Remove rule-of-three constructions. If a sentence lists three things in parallel for rhythm rather than meaning, r
- Cut filler openers: "It is worth noting that," "It should be mentioned that."
- Use contractions where the surrounding tone permits them.
- Prefer concrete nouns over abstract ones.
Return the rewritten text in a fenced code block. After the block, list the most significant changes you made, no mo
```

← 上一节

下一节 →

## The Refactor Skill · 重构技能

The fourth skill is `refactor-readability`. Its job is to take a function or section of code that works correctly but reads poorly, and propose a refactor focused on clarity rather than functionality.

第四个 skill 是 `refactor-readability`。它的职责是:拿到一段"功能正确但读起来费劲"的函数或代码块,给出一个以可读性为目标的 重构方案——但不改它的行为。

```

name: refactor-readability
description: Proposes a readability-focused refactor of a given function or section of code, without changing its be
allowed-tools: Read, Edit

Refactor for Readability
When asked to refactor code for readability, follow this procedure.
1. Read the code in full.
2. Identify the top three readability problems. Common candidates: deeply nested conditionals, names that do not com
3. Propose a refactor that addresses those three problems while preserving behavior exactly.
4. Show the refactor as a complete replacement of the original, not as a diff.
```

5. After the code, list the three problems addressed in plain language.

Do not change behavior. If you notice a bug in the original code, mention it separately, do not silently fix it as p

The fifth skill is `jest-test`. Its job is to write a Jest or Vitest test file for a given function or module, following sensible defaults for both frameworks.

第五个 skill 是 `jest-test`。它的职责是:为一个给定的函数或模块写出 Jest 或 Vitest 测试文件,且对两个框架都使用合理的默认设置。

```

name: jest-test
description: Writes a test file in Jest or Vitest for a given function or module. Use whenever the user asks to "write tests"
allowed-tools: Read, Write

Jest or Vitest Test
When asked to write tests, follow this procedure.
1. Read the file under test.
2. Detect the test framework. If the project uses `vitest` (check package.json), use Vitest. Otherwise default to Jest.
3. Identify the public surface of the file: exported functions and classes.
4. For each exported function, write tests that cover at minimum: the happy path, one boundary condition, and one error case.
5. Use `describe` blocks per function and `it` blocks per scenario.
6. Mock external dependencies. Do not make real network calls or write to real files in tests.
7. Place the test file alongside the file under test, with the suffix `.test.ts` (or `.test.js` if the project is JavaScript).
Output the complete test file. Do not abbreviate.
```

← 上一节

下一节 →

The sixth skill is `pr-review`. Its job is to produce a substantive review of someone else's pull request, complete with concrete suggestions and a summary judgment.

第六个 skill 是 `pr-review`。它的职责是:对别人提的 pull request 做一次有实质内容的评审——既要有具体的建议,也要有总结性判断。

```

name: pr-review
description: Reviews someone else's pull request and produces a structured review with comments, suggestions, and a summary judgment
allowed-tools: Bash, Read, Grep

PR Review
When asked to review a PR, follow this procedure.
1. Identify the PR or diff to review. If the user provides a branch name, diff against the merge base.
2. Read the diff in full before forming any opinions.
3. Group your observations into four sections:
Substantive comments
Issues that affect correctness, security, or maintainability. Each comment should reference a file and line and explain the problem.
Suggestions
Improvements that are not strictly required but would make the change better. Same format as substantive comments.
Questions
```

Things that are unclear from the diff and would require clarification from the author.

### ## Summary

A two- or three-sentence judgment. Approve, request changes, or defer with reasons.

Do not invent issues. If the change is good, say so plainly. A short positive review is better than a long padded one.

The seventh skill is `code-explain`. Its job is to explain a piece of code at a level appropriate to a colleague who knows the language but does not know this codebase. This skill is especially useful when joining a new project, when reading unfamiliar parts of a familiar project, or when preparing notes to share with teammates.

第七个 skill 是 `code-explain`。它的职责是:向一位"懂这门语言但不了解这个代码库"的同事解释一段代码。这个 skill 在以下场景尤其有用:刚加入新项目、读熟悉项目的陌生模块、或者要准备笔记分享给团队成员。

```

name: code-explain
description: Explains a piece of code at the level of a colleague who knows the language but not this codebase. Use
allowed-tools: Read

Code Explanation
When asked to explain code, follow this procedure.
1. Read the code in full before writing anything.
2. Identify the purpose of the code in one sentence.
3. Identify the inputs, the outputs, and any side effects.
4. Explain the algorithm or pattern at a high level. Use plain language. Avoid restating the code line by line.
5. Note any non-obvious choices the author made and what alternatives they might have considered.
6. End with a short list of questions you would ask the original author if you could.
Do not pad the explanation with general background. Assume the reader knows the language. Focus on what is specific
```

← 上一节

下一节 →

## The Bug Report Skill · Bug 报告技能

The eighth skill is `bug-report`. Its job is to take a half-articulated description of a problem and produce a structured, actionable bug report that a developer or team could file directly into a tracker.

第八个 skill 是 `bug-report`。它的职责是:拿到一段"没完全说清楚"的问题描述,产出一份结构化、可直接录入 issue tracker 的 bug 报告。

```

name: bug-report
description: Turns an informal description of a problem into a structured bug report ready to file in an issue track
allowed-tools: Read

Bug Report
When asked to write a bug report, follow this procedure.
1. Identify what the user has told you and what is missing. If essential information is missing (steps to reproduce,
2. Produce the report in this format:
Summary
A single sentence describing the bug.
```

```
Steps to reproduce
A numbered list, precise enough that another person could follow them.
Expected behavior
What should happen.
Actual behavior
What happens instead.
Environment
Relevant version numbers, browsers, operating systems, or configurations.
Notes
Anything unusual, intermittent, or worth investigating.
Stay close to what the user actually said. Do not invent symptoms or steps.
```

The ninth skill is `decision-log`. Its job is to capture a technical decision the developer just made, in the standard format used for architecture decision records, so that future developers (including the future self) can understand both the decision and its context.

第九个 skill 是 `decision-log`。它的职责是:把开发者刚刚做出的技术决策,按架构决策记录(ADR)的标准格式记录下来,以便未来的开发者(包括未来的自己)同时理解"决策本身"和"它所在的上下文"。

```

name: decision-log
description: Records a technical decision in the architecture decision record (ADR) format. Use whenever the user sa
allowed-tools: Read, Write

Decision Log Entry
When asked to record a decision, follow this format.
ADR-NNN: [Short title in the form of a noun phrase]
Status
Proposed, Accepted, Superseded, or Deprecated.
Context
What is the situation that requires a decision? Two or three sentences.
Decision
What was decided? One or two sentences in the active voice.
Consequences
What changes as a result? Both positive and negative consequences. A short bulleted list is acceptable here.
Alternatives considered
What else was on the table, and why were those alternatives rejected? One or two sentences per alternative.
If the user has not specified an ADR number, ask once whether to assign the next sequential number based on existing
```

← 上一节

下一节 →

The tenth and final skill is `changelog-entry`. Its job is to write a single entry suitable for inclusion in a `CHANGELOG.md` file, given a recent change or set of changes.

第十个、也是最后一个 skill 是 `changelog-entry`。它的职责是:给定一组最近的改动,产出一条适合写入 `CHANGELOG.md` 的单条记录。

```

name: changelog-entry
```

```
description: Writes a single entry for a project's CHANGELOG.md based on a recent change. Use whenever the user asks
allowed-tools: Bash, Read

Changelog Entry
When asked for a changelog entry, follow this procedure.
1. If the change is described in conversation, use that. If a commit or branch is referenced, read the diff to under
2. Categorize the change as one of: Added, Changed, Fixed, Removed, Deprecated, Security.
3. Write a single line, in the past tense, written for users rather than developers. Focus on what is now different
4. Output the entry in this format:
[Category]
- [Single-line description in past tense]
If the change has no user-visible impact, say so plainly and recommend not adding a changelog entry.
```

The library is now ten skills strong. The reader who has typed all ten into her `~/ .claude/skills/` directory has, in roughly an hour of work, installed a set of capabilities that will pay her back many times over in every future Claude Code session. Each skill is small. None of them, alone, is impressive. But every one of them removes a small recurring friction, and the cumulative effect of ten removed frictions is substantial.

到这里,你的"库"就有了十个 skill。在大约一小时的工作里,把这十个都敲进 `~/ .claude/skills/` 目录的读者,等于装下了一套"未来每个 Claude Code session 都会成倍回报"的能力集。每个 skill 都很小,单独看都不惊人;但每一个都消除了一个反复出现的小摩擦,而十个小摩擦被消除之后,累积效应是相当可观的。

## Extending the Library · 扩展你的库

A note on extension. The skills above are starting points, not endpoints. As the developer lives with them, she will notice that some of them fit her work perfectly and others fit imperfectly. The imperfect ones should be edited. A skill is a file. Editing a skill takes thirty seconds. A description that under-fires can have phrasings added. A body that produces output the developer does not like can have its instructions tightened. The library is meant to evolve. The reader who treats her `~/ .claude/skills/` directory as a living project, revisited monthly and improved gradually, will find that within a year she has a personal toolkit that no off-the-shelf product could replicate, because it has been shaped by her actual work and her actual preferences.

关于"扩展"的几句话。上面这些 skill 是起点,不是终点。当开发者真正用上之后,会注意到:其中几个完全贴合自己的工作,有几个贴合得不够好。贴合得不够的就该改。skill 本质就是一个文件,改一个 skill 也就三十秒的事:触发不够稳的 description,加几句话;产出格式不满意的 body,把指令收紧。这个库天生就要进化。把自己的 `~/ .claude/skills/` 目录当作一个"活项目"来对待——每月回头看一次,渐进式改进——读者会发现,一年之后,她手里的个人工具包是任何现成产品都复刻不了的,因为它是被她真实的工作和真实的偏好"塑形"出来的。

← 上一节

下一节 →

A second note on sharing. Several of the skills above will be useful to other developers in the reader's team or company. The mechanism for sharing is straightforward: copy the skill folder into the project's `.claude/skills/` directory and commit it. From that point forward, every developer who clones the project receives the skill automatically and benefits from it without any onboarding effort. This is how skill libraries spread within organizations. Not through documentation. Not through training sessions. Through configuration that travels with the code, and that the tool itself honors as soon as a colleague's session starts.

关于"分享"再说几句。上面有几个 skill,对读者所在团队或公司里的其他开发者也会很有用。分享机制很直接:把 skill 目录拷到项目的 `.claude/skills/` 下,提交进版本库即可。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这些 skill,无需任何 onboarding 流程。这正是 skill 库在组织内传播的方式——不是靠文档,不是靠培训,而是靠"和代码一起走的配置":配置随代码一起传开,工具本身在同事一开 session 时就自动遵守它。

A third note, on the deeper effect of having lived with a skill library for a few months. Developers who install ten skills and use them daily report a subtle shift in how they think about their own work. Tasks that used to feel like obligations, things they had to remember to do correctly, start to feel like artifacts that the environment produces on their behalf. The mental energy that used to be spent on remembering the commit format, on remembering the PR template, on remembering to run the right tests, gets freed up. That freed energy has somewhere to go, and where it goes, in most cases, is into the actual work the developer is being paid to do. The library does not merely save time. It frees attention. The attention is the more valuable of the two, and developers who internalize this distinction often find that within a few months they are doing work that would have been impossible for them at the start, not because they got smarter but because the environment got better at carrying everything that did not require their attention.

第三点,关于"和 skill 库共处几个月之后"那种更深层的效果。装了十个 skill 并日日使用的开发者,常会报告一种微妙的变化:他们看待自己工作的方式变了。原来那些"必须自己记得做对的事"——记 commit 格式、记 PR 模板、记跑对测试——开始变成"环境替我顺手产出的产物"。原本花在"记这些事"上的心力被释放出来了。这份释放出来的心力总要去一个地方;大多数情况下,它会去到那份"开发者真正被雇来做的"工作上。skill 库省下来的不只是时间,而是注意力。注意力比时间更值钱。把这个区别真正吃进去的开发者,常常会在几个月后发现自己能做的事,起点时根本做不到——不是因为变聪明了,而是因为环境把"那些不需要他们注意力的事"接管得越来越好了。

Part Two ends here. The reader has, over six chapters, learned the anatomy of skills, built her first one by hand, mastered the description craft that determines whether skills fire correctly, learned to extend skills across multiple files when complexity demands it, internalized the principles that turn a collection of skills into a coherent library, and installed ten skills that will carry forward into every session ahead. Skills, the first pillar of the three-pillar framework, are now in working memory and in installed form. The next part of the book turns to the second pillar. Subagents are the parallel specialists that handle work which deserves its own context, its own role, and its own tool restrictions. The combination of skills and subagents is where the real architectural power of Claude Code begins to surface, and that surfacing begins in the chapter ahead.

第二部分到此收束。读者用六章的篇幅:学会了 skill 的解剖结构、手工搭出第一个 skill、掌握了"决定 skill 能不能正确触发"的 description 工艺、学会了在复杂度需要时把 skill 扩展到多文件、把"一堆 skill 变成一座连贯 library"的原则内化于心,以及装下十个会一路伴随今后每个 session 的 skill。"三柱框架"里的第一柱——Skills——已经既在肌肉记忆里,也在已安装的形态里。下一部分,书会转向第二柱。Subagent 是并行运行的"专员",用来处理"应当拥有自己的上下文、自己的角色、自己的工具限制"的工作。Skills 和 subagent 组合起来,才是 Claude Code 真正架构力量开始显形的地方;这种显形,从下一章开始。

← 上一节

下一节 →

*Editor's note: this page in the PDF source contains the remainder of two SKILL.md code blocks (decision-log and changelog-entry) that began on pages 85 and 86. The full blocks are already shown on those pages. The overflow content is reproduced below for completeness.*

编者注:PDF 源文件的本页包含两个 SKILL.md 代码块(decision-log 和 changelog-entry)的剩余部分——这两个代码块分别从第 85 页和第 86 页开始。完整的代码块已在前两页展示,这里为完整性再现溢出的部分。

The tenth and final skill is `changelog-entry`. Its job is to write a single entry suitable for inclusion in a `CHANGELOG.md` file, given a recent change or set of changes.

第十个、也是最后一个 skill 是 `changelog-entry`。它的职责是:给定一组最近的改动,产出一条适合写入 `CHANGELOG.md` 的单条记录。

```

name: changelog-entry
description: Writes a single entry for a project's CHANGELOG.md based on a recent change. Use whenever the user asks
allowed-tools: Bash, Read

Changelog Entry
When asked for a changelog entry, follow this procedure.
1. If the change is described in conversation, use that. If a commit or branch is referenced, read the diff to under
2. Categorize the change as one of: Added, Changed, Fixed, Removed, Deprecated, Security.
3. Write a single line, in the past tense, written for users rather than developers. Focus on what is now different
4. Output the entry in this format:
[Category]
- [Single-line description in past tense]
If the change has no user-visible impact, say so plainly and recommend not adding a changelog entry.
```

← 上一节

下一节 →

*Editor's note: this page in the PDF source contains the closing material of Chapter Nine (the changelog-entry SKILL.md block was overflowing from page 88). The closing paragraphs are also reproduced on page 87 for completeness; this page shows the SKILL.md overflow and the closing paragraph that was cut.*

编者注:PDF 源文件的本页包含第九章的收尾内容(changelog-entry 的 `SKILL.md` 代码块从第 88 页溢出)。结尾段落也已在第 87 页完整呈现;本页展示溢出的 `SKILL.md` 与被截断的段落。

The library is now ten skills strong. The reader who has typed all ten into her `~/claude/skills/` directory has, in roughly an hour of work, installed a set of capabilities that will pay her back many times over in every future Claude Code session. Each skill is small. None of them, alone, is impressive. But every one of them removes a small recurring friction, and the cumulative effect of ten removed frictions is substantial.

到这里,你的"库"就有了十个 skill。在大约一小时的工作里,把这十个都敲进 `~/claude/skills/` 目录的读者,等于装下了一套"未来每个 Claude Code session 都会成倍回报"的能力集。每个 skill 都很小,单独看都不惊人;但每一个都消除了一个反复出现的小摩擦,而十个小摩擦被消除之后,累积效应是相当可观的。

## Extending the Library · 扩展你的库

A note on extension. The skills above are starting points, not endpoints. As the developer lives with them, she will notice that some of them fit her work perfectly and others fit imperfectly. The imperfect ones should be edited. A skill is a file. Editing a skill takes thirty seconds. A description that under-fires can have phrasings added. A body that produces output the developer does not like can have its instructions tightened. The library is meant to evolve. The reader who treats her `~/claude/skills/` directory as a living project, revisited monthly and

improved gradually, will find that within a year she has a personal toolkit that no off-the-shelf product could replicate, because it has been shaped by her actual work and her actual preferences.

关于"扩展"的几句话。上面这些 skill 是起点,不是终点。当开发者真正用上之后,会注意到:其中几个完全贴合自己的工作,有几个贴合得不够好。贴合得不够的就该改。skill 本质就是一个文件,改一个 skill 也就三十秒的事:触发不够稳的 description,加几句话;产出格式不满意的 body,把指令收紧。这个库天生就要进化。把自己的 `~/.claude/skills/` 目录当作一个"活项目"来对待——每月回头看一次,渐进式改进——读者会发现,一年之后,她手里的个人工具包是任何现成产品都复刻不了的,因为它是被她真实的工作和真实的偏好"塑形"出来的。

A second note on sharing. Several of the skills above will be useful to other developers in the reader's team or company. The mechanism for sharing is straightforward: copy the skill folder into the project's `.claude/skills` directory and commit it. From that point forward, every developer who clones the project receives the skill automatically and benefits from it without any onboarding effort. This is how skill libraries spread within organizations. Not through documentation. Not through training sessions. Through configuration that travels with the code, and that the tool itself honors as soon as a colleague's session starts.

关于"分享"再说几句。上面有几个 skill,对读者所在团队或公司里的其他开发者也会很有用。分享机制很直接:把 skill 目录拷到项目的 `.claude/skills` 下,提交进版本库即可。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这些 skill,无需任何 onboarding 流程。这正是 skill 库在组织内传播的方式——不是靠文档,不是靠培训,而是靠"和代码一起走的配置":配置随代码一起传开,工具本身在同事一开 session 时就自动遵守它。

← 上一节

下一节 →

*Editor's note: this page in the PDF source contains the final paragraph of Chapter Nine (the closing of Part Two), which was also rendered on page 87 because the SKILL.md code block over-flowed. It is reproduced here so the page is not empty in the PDF-aligned translation.*

编者注:PDF 源文件的本页包含第九章最后一段(也是第二部分的收束),这段内容也已在第 87 页渲染过——因为前面的 `SKILL.md` 代码块溢出。出于 PDF 页面对齐的考虑,这里再次呈现,避免该页在双语版里留白。

A third note, on the deeper effect of having lived with a skill library for a few months. Developers who install ten skills and use them daily report a subtle shift in how they think about their own work. Tasks that used to feel like obligations, things they had to remember to do correctly, start to feel like artifacts that the environment produces on their behalf. The mental energy that used to be spent on remembering the commit format, on remembering the PR template, on remembering to run the right tests, gets freed up. That freed energy has somewhere to go, and where it goes, in most cases, is into the actual work the developer is being paid to do. The library does not merely save time. It frees attention. The attention is the more valuable of the two, and developers who internalize this distinction often find that within a few months they are doing work that would have been impossible for them at the start, not because they got smarter but because the environment got better at carrying everything that did not require their attention.

第三点,关于"和 skill 库共处几个月之后"那种更深层的效果。装了十个 skill 并日日使用的开发者,常会报告一种微妙的变化:他们看待自己工作的方式变了。原来那些"必须自己记得做对的事"——记 commit 格式、记 PR 模板、记跑对测试——开始变成"环境替我顺手产出的产物"。原本花在"记这些事"上的心力被释放出来了。这份释放出来的心力总要去一个地方;大多数情况下,它会去到那份"开发者真正被雇来做的"工作上。skill 库省下来的不只是时间,而是注意力。注意力比时间更值钱。把这个区别真正吃进去的开发者,常常会在几个月后发现自己能做的事,起点时根本做不到——不是因为变聪明了,而是因为环境把"那些不需要他们注意力的事"接管得越来越好了。

Part Two ends here. The reader has, over six chapters, learned the anatomy of skills, built her first one by hand, mastered the description craft that determines whether skills fire correctly, learned to extend skills across multiple files when complexity demands it, internalized the principles that turn a collection of skills into a coherent library, and installed ten skills that will carry forward into every session ahead. Skills, the first pillar of the three-pillar framework, are now in working memory and in installed form. The next part of the book turns to the second pillar. Subagents are the parallel specialists that handle work which deserves its own context, its own role, and its own tool restrictions. The combination of skills and subagents is where the real architectural power of Claude Code begins to surface, and that surfacing begins in the chapter ahead.

第二部分到此收束。读者用六章的篇幅:学会了 skill 的解剖结构、手工搭出第一个 skill、掌握了"决定 skill 能不能正确触发"的 description 工艺、学会了在复杂度需要时把 skill 扩展到多文件、把"一堆 skill 变成一座连贯 library"的原则内化于心,以及装下十个会一路伴随今后每个 session 的 skill。"三柱框架"里的第一柱——Skills——已经既在肌肉记忆里,也在已安装的形态里。下一部分,书会转向第二柱。Subagent 是并行运行的"专员",用来处理"应当拥有自己的上下文、自己的角色、自己的工具限制"的工作。Skills 和 subagent 组合起来,才是 Claude Code 真正架构力量开始显形的地方;这种显形,从下一章开始。

← 上一节

下一节 →

## Subagents: Claude's Parallel Specialist System

### 子代理:Claude 的并行专员系统

*One mind, many specialists. The work that does not need to share thinking should not have to share context.*

"一心多用,各专一业。"——不需要共享思考的工作,本就不该被迫共享上下文。

## CHAPTER TEN · 第十章

## Subagents Fundamentals: Why Single-Threaded Claude Isn't Enough

### 子代理基础:为什么"单线程 Claude"远远不够

*"Specialization is the natural answer to the limits of attention. The mind that tries to hold everything ends up holding nothing well."*

— Adam Smith, paraphrased

"面对注意力的边界,专门化是自然应运而生的答案。一颗试图包揽一切的心,最终什么都握不好。"

—— 亚当·斯密(转述)

下一节 →

Skills, the subject of Part Two, taught Claude how to do specific kinds of work consistently. They are an answer to the question of capability. The chapters that follow are about a different question, and a different answer. The question is what to do when work that needs to happen in a single session crosses several different kinds of attention, when the context window starts filling with material that the current task does not need, when the developer wants three things investigated at once instead of one, and when the cognitive cost of switching modes inside a single Claude session begins to degrade the quality of every output. The answer to these problems is the second pillar of the framework, the subagent.

第二部分讲的 Skills,教的是 Claude 如何稳定地做好某一类具体工作——它回答的是"能力"的问题。接下来的几章,问的是一个不同的问题,也给出一个不同的答案。当一个 session 里的工作横跨好几种"注意力类型"、当上下文窗口开始被与当前任务无关的内容塞满、当开发者希望三件事并行调研而不是一件一件来、当"在同一个 Claude session 里反复切换模式"所产生的认知代价开始让每一条输出都降级时,该怎么办?这些问题的答案,就是框架里的第二柱——**subagent(子代理)**。

Begin with a definition that will guide every chapter in this part of the book. A subagent is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window, that the main session can dispatch to perform a specific kind of work. When the subagent finishes its work, only its final output returns to the main session. The dozens of tool calls the subagent may have made along the way, the files it may have read, the drafts it may have considered, the dead ends it may have explored, none of that enters the main session. The main session sees a clean result, and the work that produced the result happens in a parallel space that the main session never has to see.

先给一个定义,后续每一章都会用到它。一个 subagent 是一个独立的 Claude 进程——它有自己的 system prompt、自己受限的工具集、自己隔离的上下文窗口——主 session 可以把它派出去,执行某一类特定的工作。subagent 完成工作之后,只有它的最终输出会回到主 session 之中:子代理一路调过的几十个工具、看过的文件、考虑过的草稿、走过的死胡同,这一切都不会进入主 session。主 session 看到的只是一份干净的结果;真正产出这份结果的工作,发生在主 session 永远不必看见的"平行空间"里。

This is a substantial architectural commitment. To understand why the commitment is worth making, it is worth examining what happens to a Claude Code session that does not use subagents and that is asked to do something complex. Take a real example. A developer asks Claude Code to add a new feature to a substantial codebase. The work involves understanding the existing architecture, designing the new addition, writing the code, writing tests for it, updating documentation, and reviewing the whole change for quality. A single Claude Code session can do all of these things in sequence, and it often does. But there is a hidden cost.

这是一项相当重的架构承诺。要理解这个承诺为什么值得付出,值得先看看:当一个 Claude Code session 不使用 subagent、却被要求做一件复杂的事时,会发生什么。举一个真实的例子:开发者让 Claude Code 给一个相当规模的代码库加一个新功能。这件工作包括:理解现有架构、设计新增内容、写代码、给它写测试、更新文档、再对整组改动做一遍质量评审。一个 Claude Code session 完全能按顺序把这些事做完,而且它经常就是这样做的。但这里有一个隐性的代价。

In the first phase, Claude reads many files to understand the existing architecture. Each file Claude reads adds to the context window. The context grows by hundreds or thousands of tokens for every file. By the time Claude has built a working understanding of the architecture, the context window is full of source code, much of it only loosely relevant to the new feature.

在第一阶段,Claude 要读很多文件,以理解现有架构。每读一个文件,上下文窗口就多一份内容——每多一个文件,就多几百到几千 token。等到 Claude 真的把架构"读到能上手"的时候,上下文窗口里已经堆满了源代码,而且其中相当一部分只是松散地和新功能相关。

In the second phase, Claude designs the new addition. The design involves several false starts, each of which leaves traces in the context. By the third phase, when Claude is writing the actual new code, the context window contains the architectural exploration, the design exploration, and the writing of the new code, all jumbled together. By the time Claude reaches the test-writing phase, the context window is dominated by historical material that has nothing to do with testing, and the test-writing suffers as a result, because the model is trying to write tests while looking at the residue of three earlier phases of work.

到了第二阶段,Claude 开始设计新增内容。设计过程会经历几次"试错回头",每一次都会在上下文里留下痕迹。等到第三阶段,Claude 真正开始写代码时,上下文窗口里已经同时装着"架构探索"、"设计探索"和"新代码写作"——三件事搅在一起。再到"写测试"这一阶段,上下文窗口里占主导的已经变成与测试毫无关系的"历史残渣";而写测试这件事,也就因此受了影响——模型是一边盯着前三个阶段留下的残渣,一边在写测试的。

## The Failure Mode Subagents Prevent · subagent 要防止的失败模式

This is the failure mode that subagents exist to prevent. The architecture of a subagent is a direct response to this problem. Each subagent has its own context window. The architectural exploration happens inside an explore subagent, whose context fills with the relevant source code, but only the subagent's final summary returns to the main session. The main session never has to see the dozens of files the subagent read. When the main session moves on to design, it does so with a clean context that contains the summary, not the exploration. When the main session moves on to test writing, the design exploration similarly never enters its context, because the design happened in another subagent. Each phase of work happens in a context that is appropriately scoped to that phase, and the main session preserves a clean overview of the whole project rather than becoming a clearinghouse for every detail of every phase.

这就是 subagent 之所以存在的失败模式。subagent 的架构,正是对这个问题的一种"直球"回应:每个 subagent 都拥有自己的上下文窗口。"架构探索"这件事发生在一个 explore subagent 里——它的上下文会被相关源码填满,但只有它最终的摘要会回到主 session。主 session 根本不必看见那个 subagent 读过的几十个文件。主 session 进入设计阶段时,带着的是一份干净、只含"摘要而非探索过程"的上下文。进入写测试阶段时,"设计阶段的探索"也同样不会进它的上下文,因为设计发生在另一个 subagent 里。每一阶段的工作,都发生在一个恰好匹配该阶段的上下文里;而主 session 始终保持着一份对整个项目的"干净概览",不至于变成"每个阶段每处细节的总收发室"。

Three properties of subagents follow from this architecture, and they are worth naming because every subagent design decision later in this part of the book is a function of one of these three properties.

从这套架构自然引出 subagent 的三条属性。值得给它们取个名字——本部分后面每一个 subagent 的设计决策,都不过是这三条属性中某一条的函数。

The first property is context isolation. The subagent's context is its own. The main session cannot see what the subagent is reading. The subagent cannot see what the main session is doing in parallel. The two communicate only through the subagent's initial brief and the subagent's final result. This isolation is the source of subagents' context-saving power, but it is also a constraint the developer needs to design around. A subagent cannot be told something halfway through its work; whatever it needs to know must be packed into the brief that launches it.

第一条属性是上下文隔离(context isolation)。subagent 的上下文是它自己的。主 session 看不见 subagent 在读什么;subagent 看不见主 session 在并行做什么。两者只通过两件事通信:启动时的 brief 和最终结果。这种隔离既是 subagent 能"省上下文"的力量之源,也是一个开发者必须围绕其做设计的约束——你没法"中途"告诉 subagent 什么新信息;它需要知道的一切,都得在启动它的 brief 里就打包好。

The second property is specialization. A subagent has its own system prompt, distinct from the main session's system prompt. The system prompt establishes the subagent's role, its expertise, its style of thinking, its outputs. A code-reviewer subagent has a system prompt that is tuned for the craft of reviewing code, not for the broader job of being a general programming assistant. A test-writer subagent has a system prompt that is tuned for the craft of writing tests. The narrowness of the system prompt is a feature, not a limitation. A specialist who is asked to do only one kind of work, and who has been briefed in detail about how to do that work well, produces consistently better outputs than a generalist who is asked to do everything.

第二条属性是**专门化(specialization)**。subagent 拥有自己的 system prompt,且和主 session 的 system prompt 截然不同。这份 system prompt 确立 subagent 的角色、专长、思考风格、产出形态。一个 code-reviewer subagent 的 system prompt,是围绕"代码评审这门手艺"调出来的,而不是围绕"当一个通用编程助手"调出来的;一个 test-writer subagent 的 system prompt,是围绕"写测试这门手艺"调出来的。system prompt 的窄,是一种特性,不是限制——被指派只做一类事、并被详细交代过"如何把这事做好"的专员,产出的稳定质量,会高于"什么都做"的通才。

The third property is parallelism. Multiple subagents can run at the same time. Claude Code can run several subagents concurrently; community guidance from Anthropic engineers and from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although the exact number is environment-dependent and Anthropic has not published a hard cap. This means that work which is genuinely independent can happen in parallel rather than in sequence. A code review that involves a security pass, a performance pass, a style pass, and a test-coverage pass does not have to be four sequential reviews. It can be one parallel review where four subagents work simultaneously and the main session synthesizes their findings when they all return. The wall-clock time savings are substantial. The quality improvements are also substantial, because each subagent is focused on its own dimension and does not have to balance attention across all four.

第三条属性是**并行(parallelism)**。多个 subagent 可以同时跑。Claude Code 能并发跑若干个子代理;来自 Anthropic 工程师和重度用户的社区经验大致收敛在"十个左右"——超过这个数,排队的延迟就开始能被感知;当然具体数字因环境而异,Anthropic 也没有公开过一个硬上限。这意味着:真正独立的工作,完全可以并行发生,而不是顺序发生。一次代码评审,要做"安全"、"性能"、"风格"、"测试覆盖"四道扫描——它不必是依次发生的四次评审,而可以是并行的一次评审:四个 subagent 同时工作,主 session 等它们都返回后再做综合。挂钟时间上的节省相当大;质量上的提升也相当大——因为每个 subagent 都在自己的维度上保持专注,不必把注意力在四个维度间来回分配。

## Where Subagents Live · subagent 住在哪里

Where do subagents live, mechanically? They live as files in the `.claude/agents` directory. Project-level subagents live at `.claude/agents` in the root of a specific project, where they apply only inside that project. User-level subagents live at `~/.claude/agents` in the user's home directory, where they apply across every project. The naming convention is the same as for skills: lowercase, hyphenated, descriptive. Each subagent is a single Markdown file with YAML frontmatter at the top and a system prompt body below. Here is the simplest possible subagent definition, complete and ready to install.

subagent 在"机械层面"住在哪里?它们以文件形式住在 `.claude/agents` 目录里。**项目级** subagent 放在某个项目根目录下的 `.claude/agents`,只在那个项目里生效。**用户级** subagent 放在用户家目录的 `~/.claude/agents`,跨所有项目生效。命名约定和 skill 一样:小写、连字符、具描述性。每个 subagent 就是一个 Markdown 文件,顶部是 YAML frontmatter,下面是 system prompt 主体。下面是"最小可用的 subagent 定义",完整、可以直接装上。

Look closely at this file, because every subagent in the rest of this part of the book follows the same shape. The frontmatter has four fields, two of which are required. The `name` field is the subagent's identifier and must match the filename. The `description` field is what tells Claude when this subagent should be dispatched, and it is the subagent equivalent of the skill description; the same craft of trigger writing applies. The `tools` field, optional, restricts the tools available to the subagent. The `model` field, also optional, lets the developer choose which model the subagent runs on, which matters because heavier work might warrant the strongest model while lighter work can use a faster one.

请仔细看这个文件——本部分后面出现的每一个 subagent,都是这个形状。frontmatter 一共四个字段,其中两个是必填。`name` 字段是 subagent 的标识,必须和文件名一致。`description` 字段告诉 Claude"什么时候该派这个 subagent",它相当于 subagent 版的 skill description——trigger 的写法工艺同样适用。`tools` 字段是可选的,用来限制 subagent 可用的工具。`model` 字段也是可选的,让开发者选 subagent 跑在哪个模型上——这一点之所以重要,是因为"重活"值得用最强的模型,而"轻活"可以用更快的那个。

```

name: code-reviewer
description: Reviews code for quality and best practices. Use proactively after code changes.
tools: Read, Grep, Glob
model: sonnet

You are a senior code reviewer. When invoked, analyze the code you are given and provide specific, actionable feedback.
```

The body of the file is the subagent's system prompt. It is read every time the subagent is dispatched, and it shapes everything the subagent does. The body of a subagent is, in this respect, more consequential than the body of a skill, because while a skill body adds instructions to the main session's overall behavior, a subagent body becomes the entire personality of a separate Claude process. The development of this body is the subject of the next chapter, and it is where the most important design decisions for any subagent get made.

文件的正文,就是 subagent 的 system prompt。每次 subagent 被派出去,这份 system prompt 都会被读入,并塑造它接下来做的所有事。从这个意义上说,subagent 正文的影响,要大于 skill 正文的影响——因为 skill 正文只是给主 session 的整体行为"追加一些指令";而 subagent 正文,会成为另一个 Claude 进程的整体人格。这份正文怎么写,正是下一章的主题——也是任何一个 subagent 最重要的设计决策所在。

Two conventions concerning invocation are worth knowing now. The first is automatic delegation. When the main session is working on a task and notices that one of the available subagents has a description matching the work, the main session can delegate to that subagent on its own initiative. The developer does not need to type any special command to invoke the subagent; she simply describes her task, and the main session decides whether the task should be handled in-line or delegated. This is the same principle that governs skill firing, applied to a different mechanism.

关于"调用",有两条约定值得现在就了解。第一条是**自动委派(automatic delegation)**。主 session 在处理任务时,如果注意到"可用的 subagent 中,某条的 description 匹配当前工作",主 session 就会自主把它派出去。开发者不需要敲任何特殊命令来"召唤" subagent——她只要把任务用自然语言说清楚,主 session 就会自己决定:这件事是就地做,还是委派给某个 subagent。这套原理和"skill 何时触发"是一样的,只是套在了一个不同的机制上。

Good descriptions cause appropriate delegation. Weak descriptions cause subagents to sit unused.

好的 description 会带来恰当的委派;弱的 description 会让 subagent 一直闲坐。

← 上一节

下一节 →

The second convention is explicit invocation. The developer can also invoke a subagent directly, by typing `@agent-name` in the prompt. The mention triggers the named subagent immediately, with whatever brief follows in the prompt as the dispatch instructions. Explicit invocation is useful when the developer wants to be sure a particular subagent handles a particular task, when the description-based delegation might not fire reliably, or when the developer wants to compose multiple specific subagents on a single task. Both invocation modes are valid, and most working developers use both, depending on the situation.

第二条约定是**显式调用(explicit invocation)**。开发者也可以直接在 prompt 里敲 `@agent-name` 来"点名"某个 subagent。这一提,会立刻触发被点名的 subagent,而 prompt 中跟在后面的内容,就是派给它的 brief。显式调用在以下场景特别有用:你希望"某个 subagent 一定要处理某件事"、"靠 description 的委派不够稳"、"你想把多个具体 subagent 组合到一件事上"。两种调用方式都成立——多数实际用起来开发者会两种都用,看场景挑一种。

## Automatic Versus Explicit · 自动 vs 显式

The relationship between automatic delegation and explicit invocation is worth a moment of attention, because choosing between them is a real design question. Automatic delegation is convenient. The developer types in plain language and the right specialist is dispatched, much as the right skill is fired. The cost of automatic delegation is reliability. Description-based dispatch is a judgment call that the main session has to make for every prompt, and the call can occasionally be wrong, especially when several subagents have overlapping descriptions. Explicit invocation is reliable. The developer names the subagent she wants and the subagent runs, full stop. The cost of explicit invocation is friction. The developer has to remember the subagent's name and choose to type it. A working pattern that develops in most experienced users is a hybrid: explicit invocation when the work is high-stakes and the developer wants certainty about which specialist runs, automatic delegation when the work is routine and the convenience matters more than the certainty. The hybrid produces the best of both, and it is worth deliberately practicing both invocation modes during the first weeks of subagent use.

"自动委派"和"显式调用"的关系,值得花片刻讲清楚——因为在两者之间做选择,其实是一个真正的设计问题。自动委派很方便:开发者用日常语言敲下去,合适的专员就被派出,和 skill 的"对的 skill 自动触发"差不多。代价是**可靠性**:基于 description 的派单,是主 session 在每个 prompt 上都要做的一次判断,而这个判断偶尔会出错——尤其是当多个 subagent 的 description 互相重叠的时候。显式调用很可靠:你点名哪个 subagent,就是它,全无歧义。代价是**摩擦**:开发者得记得那个 subagent 的名字,并且主动把它敲出来。大多数熟练用户最后会形成一种"混合"模式:高风险工作、要确定是哪位专员跑的,显式调用;常规工作、便利比确定性更重要的,自动委派。这种"混合"兼具两者之利,在开始用 subagent 的头几周里,值得有意识地两种都练一练。

A note on the relationship to skills. Skills and subagents address different problems and complement each other rather than competing. A skill teaches Claude how to do a specific task well. A subagent provides a separate context and a specialized role for a kind of work. Many sophisticated workflows use both: a subagent handles a kind of work, and inside the subagent, skills fire when relevant tasks come up during that work. A test-writer subagent might activate a project-specific test-conventions skill at the moment it begins drafting tests. The two systems

compose, and the composition is where the deepest leverage of Claude Code begins to surface. Chapter Fourteen is dedicated entirely to this composition.

再说和 skill 的关系。Skill 和 subagent 解决的是不同的问题,二者是互补而不是竞争。Skill 教 Claude 如何把一件具体的事做好;subagent 则为某一类工作提供一个独立的上下文和一个专门的角色。许多成熟的工作流会把两者都用上:subagent 负责某一类工作,在它"里头"工作展开时,相关的 skill 又会在该触发的时刻被触发。比如一个 test-writer subagent 在它开始起草测试的那一刻,可能会激活"项目专属测试规约"这个 skill。两套系统是 *可组合的*;而这种组合,才是 Claude Code 最深杠杆的起点。第十四章整章都会讲这种"组合"。

← 上一节

下一节 →

A short clarification for readers who have come from a background of writing prompts and may be confused about why subagents are not simply elaborate prompts. The difference is real and structural. A prompt, no matter how elaborate, runs in the same context as everything else in the session. The context window is shared. The system prompt is shared. A subagent has a separate process, a separate context, and a separate system prompt. The architectural distinction matters because it produces different behavior. A prompt cannot stop the rest of the session from polluting its context. A subagent runs in a clean slate. This is not a difference of degree. It is a difference of kind, and developers who try to replicate subagent benefits with cleverer prompts inside a single session will find that the benefits do not survive the lack of architectural separation. The right answer is to use the architecture, not to try to simulate it.

对那些"来自 prompt 写作背景"的读者,这里要做一个简短的澄清——你可能困惑:为什么 subagent 不就是"复杂版的 prompt"呢?这个区别是真实且结构性的。一个 prompt,无论写得多么复杂,都和 session 里的其他东西跑在同一个上下文里——上下文窗口是共享的,system prompt 是共享的。一个 subagent 则拥有 *独立的进程*、*独立的上下文*、*独立的 system prompt*。这个架构上的差异之所以重要,是因为它会直接造成不同的行为。一个 prompt 挡不住 session 其他部分去"污染"它的上下文;而一个 subagent 跑在干净的白纸上。这不是程度上的差别,而是类型上的差别——想用"更聪明的 prompt"在同一个 session 里模拟 subagent 收益的开发者,会发现那份收益根本撑不住:没有架构上的隔离,收益也就不在了。正解是 *用架构,而不是模拟架构*。

A short word on the relationship to Claude Code's built-in subagents. Three subagents ship with Claude Code by default and are present in every session: Explore, Plan, and General-purpose. Explore is read-only and runs on Haiku by default; it is optimized for quickly searching and understanding a codebase without making changes. Plan gathers context before the main session enters plan mode, where a strategy is presented. General-purpose handles tasks that need a mixture of exploration and modification. The developer does not invoke these directly very often; the main session routes work to them automatically. Their existence is worth knowing because they are a useful proof of concept of the subagent architecture and because, when the developer designs her own custom subagents, she is extending a system that Anthropic itself uses every day.

再点一句和 Claude Code 内置 subagent 的关系。Claude Code 默认自带三个 subagent,每个 session 里都有:Explore、Plan、General-purpose。Explore 是只读的,默认跑在 Haiku 上;它专为"快速搜代码库并理解它"而优化,不会修改任何东西。Plan 则是在主 session 进入 plan mode、准备呈现一份策略之前,先收集上下文。General-purpose 负责"需要既探索又修改"的混合任务。开发者一般不直接点名调它们——主 session 会自动把工作派给它们。值得知道它们的存在,有两个原因:它们是 subagent 架构的一个很好用的"概念验证";而当开发者设计自己的自定义 subagent 时,她是在 *扩展 Anthropic 自己每天都在用的那套系统*。

It is worth examining the cost side of subagents honestly, because the architecture has trade-offs that the developer should understand before building a heavy subagent library. Each subagent invocation costs tokens. The system prompt of the subagent is loaded into a fresh context,

which costs whatever the system prompt's size is. The subagent then reads files, makes tool calls, and produces its final output, all of which consume tokens that the main session would not have consumed if the work had stayed inline. The total token cost of a subagent invocation is, in most cases, somewhat higher than the cost of doing the same work in the main session, because the subagent does not benefit from any context the main session has already built.

值得把 subagent 的成本面也坦诚地摆出来——因为这套架构有取舍,开发者搭一套"重的 subagent 库"之前,理应理解这些。每次调用 subagent 都要花 token:subagent 的 system prompt 会被加载到一份全新的上下文,size 是多少就要花多少;随后 subagent 读文件、调用工具、产出最终输出,这一切都会消耗 token——而这些 token,如果这件事是"在主 session 里直接做"的,本来是不会花的。绝大多数情况下,一次 subagent 调用的总 token 成本,会比"同一份工作在主 session 里做"略高——因为 subagent 享受不到主 session 已经积累起来的任何上下文。

← 上一节

下一节 →

### The Cost and Its Compensation · 成本与它的补偿

The compensation for this cost is significant, and it has two components. The first component is the quality improvement that comes from a clean, focused context. A subagent producing a security review in a context that contains only a security-review system prompt and the relevant code will produce a deeper review than a main session that is also juggling architectural decisions, build configuration, and three previous tasks. The quality difference is usually worth more than the token cost, because the developer's time is the scarce resource, not the tokens. The second component is the wall-clock saving from parallelism, which only applies in multi-subagent workflows. When four subagents run simultaneously, the wall-clock time is determined by the slowest of the four, not by the sum. The token cost is roughly four times higher than the equivalent sequential work, but the time cost is one quarter, and time is the dimension that governs how many tasks the developer can complete in a working day.

对这份成本,有两份"补偿",分量都不轻。第一份是来自"干净、专注的上下文"的质量提升。一个 subagent,做安全评审,而它的上下文里只有"安全评审 system prompt + 相关代码",它产出的评审深度,会高于"主 session 同时还忙着架构决策、构建配置、上一二三件事"的那种情形。这种质量差距,通常比 token 成本本身更值——因为真正稀缺的资源是开发者的*时间*,而不是 token。第二份是来自并行的*挂钟时间*节省——它只在多 subagent workflow 里奏效。四个 subagent 同时跑,挂钟时间由其中最慢的一个决定,而不是四者之和。token 成本大约是"等价串行工作"的四倍,但时间成本是四分之一;而"一个工作日里能完成多少任务",正是由时间这个维度决定的。

A balanced view, then, is that subagents are worth dispatching when context isolation matters, when specialization is possible, when parallelism applies, or when several of these are true at once. Subagents are not worth dispatching when the work is small, when the main session's context is already clean, and when no specialization or parallelism is available. The developer who internalizes this balanced view will reach for subagents at the right moments and refrain from over-dispatching them at moments where they would only add overhead.

一个平衡的看法是:在以下场景里,subagent 值得派出——上下文隔离重要、专门化可能、并行适用,或三者同时成立;在以下场景里,subagent 不值得派出——工作很小、主 session 的上下文已经干净、且没有可用的专门化或并行。把这个"平衡看法"真正内化的人,会在对的时刻伸手召唤 subagent,也会在该收手的时刻克制自己——不至于在那些 subagent 只会徒增开销的场合,过度派单。

A second balanced view concerns the human side of subagent use. Some developers, on first encountering subagents, become enthusiastic and try to dispatch them on every task. The result is sometimes slower than the work would have been without them, because the orchestration overhead exceeds the benefits. Other developers, on first encountering subagents, treat them with suspicion and avoid them entirely. The result is

that they never benefit from the architecture and continue to operate inside an increasingly polluted main session. Both extremes are mistakes. The right relationship with subagents, after a few weeks of practice, becomes intuitive: the developer reaches for them when the task has an obvious specialization or parallelism shape, and lets the main session handle the rest. This intuition is not difficult to develop, but it requires a few weeks of deliberate use to settle in. The reader who works through Part Three patiently will arrive at the intuition without having to think about it explicitly any longer.

第二个"平衡看法",是关于人使用 subagent 时的心智层面。开发者第一次接触 subagent 时,有些会变得热情高涨,恨不得每个任务都派——结果有时反而比不派还慢,因为"编排开销"大于收益。也有些开发者第一次接触 subagent 时,满腹狐疑,干脆一个都不用——结果就是他们从来享受不到这套架构的好处,继续在"越来越被污染的主 session"里工作。两个极端都是错的。和 subagent 的正确关系,经过几周实操,会变成一种直觉:遇到任务天然具有"专门化或并行"的形状,就会伸手召唤;其余的事,放给主 session 处理。这种直觉并不难养成,但确实需要几周有意识的练习,才能"安定下来"。耐心走完第三部分的读者,会自然而然地获得这种直觉,不再需要时刻把它挂在嘴边。

← 上一节

下一节 →

The final piece of context for this opening chapter is a property that emerges only after a developer has built and used several subagents, and that is worth naming up front so that the chapters ahead make sense. Subagents are how a single developer becomes the leader of a small specialist team. The team is patient. It does not get tired. It does not have meetings. Its members do not interrupt each other. Each member is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, the orchestrator, the one who decides who works on what and how the results come together. Claude Code, equipped with a well-designed subagent library, becomes a leverage multiplier in a way that Claude Code without subagents simply cannot. The developer who masters this multiplication will, within a few months of practice, find that the volume and quality of work she can produce has shifted in a noticeable, sustainable way.

本章最后一段上下文,是这样一个属性——它只在开发者搭好、用过若干个 subagent 之后才会浮现,但值得在开篇就先点出来,这样后面几章才好读懂。Subagent,就是"让一个开发者变成一支小型专员团队的 leader"的那条路。这支团队有耐心,不会累,不开会,成员之间不会互相打断;每个成员都只精于一件事,并对其余的事坦荡地"沉默"。开发者的角色,是编辑、是编排者、是那个决定"谁做什么、结果如何汇拢"的人。一套装备了"设计良好的 subagent 库"的 Claude Code,会变成一种**杠杆放大器**——而没装 subagent 的 Claude Code,做不到这种放大。把这种"放大"真正掌握住的开发者,经过几个月的实操之后,会发现自己能产出的工作量和质量,都发生了显著且可持续的迁移。

The chapter that follows takes the body of the subagent file, the system prompt, and treats it as the central design decision in any subagent. Every choice about what a subagent does, what it does not do, how it should think about its work, and how it should report its findings, is a choice expressed in the system prompt. The discipline of writing those prompts well is the discipline that determines whether a subagent is a useful specialist or a confused generalist. That discipline begins now.

下一章会把 subagent 文件的正文——也就是 system prompt——当作任何 subagent 的**核心设计决策**来对待。一个 subagent 做什么、不做什么、它该以怎样的方式思考、它该如何回报它发现的东西——这一切选择,都是写在 system prompt 里的。把这份 system prompt 写好的功夫,就是决定一个 subagent 是"有用的专员"还是"迷茫的通才"的功夫。那门功夫,从下一章开始。

← 上一节

下一节 →

## CHAPTER ELEVEN · 第十一章

## Designing Effective Subagents: Role, Tools, and Boundaries

## 设计高效的子代理:角色、工具与边界

"Give a person too much to do, and they do nothing well. Give them one thing, and they may surprise you."

— Peter Drucker, paraphrased

"给一个人太多事,她什么也做不好。只给她一件事,她可能给你惊喜。"

——彼得·德鲁克(转述)

A subagent that does too much is worse than no subagent at all. This sounds counterintuitive on first hearing, because the natural instinct of a developer encountering a new capability is to put as much capability into it as possible. The instinct is wrong, and this chapter explains why and what to do instead. The art of subagent design is the art of restraint. The subagent that performs one role well, with a tightly defined scope and a thoughtfully constrained set of tools, produces consistent, reliable, useful output. The subagent that tries to be a general assistant in a slightly different costume produces output that is sometimes excellent, sometimes mediocre, and almost never reliably one or the other. Reliability is the entire point.

一个"做太多"的 subagent,比"没有 subagent"更糟。这话初听像是反直觉——因为开发者面对一项新能力时,本能反应就是把尽可能多的能力塞进去。这个本能是错的。本章会解释它为什么错,以及正确做法是什么。Subagent 设计的艺术,本质是克制的艺术。一个把单一角色做好、范围严格、工具精心收窄的 subagent,产出是稳定、可靠、可用;一个"换了一身衣服的通用助手"型 subagent,产出偶尔精彩、偶尔平庸,而且几乎从来不会稳定地只属于其中一种。可靠性,就是全部意义所在。

Three design dimensions govern every subagent, and the disciplined consideration of each is what separates good subagents from poor ones. The first dimension is role. The second is tools. The third is boundary. Each of these expresses a constraint, and constraints are what make subagents useful. A subagent without constraints is, in effect, a slightly more isolated version of the main session, and most of the value of having a subagent is forfeited.

三个设计维度决定每一个 subagent 的好坏——对每一个维度"有纪律地思考",正是好与差的分水岭。第一个维度是**角色(role)**,第二个是**工具(tools)**,第三个是**边界(boundary)**。三者各表达一种约束——而约束,正是 subagent 之所以有用的原因。一个没有约束的 subagent,说穿了只是"主 session 的稍隔离版本";拥有 subagent 的大半价值,会在这种设计里被浪费掉。

Begin with role. The role of a subagent is its identity, expressed at the top of its system prompt. A code-reviewer subagent is a code reviewer. Not a general programmer who reviews code sometimes. Not a teacher who happens to be reviewing code. A code reviewer. The role definition should be specific enough that it constrains the kind of work the subagent does, the kind of language it uses, the kind of judgment it brings, and the kind of output it produces. A weak role definition reads like a job title. A strong role definition reads like the introduction of a person.

从**角色**说起。subagent 的角色,就是它的"身份",写在 system prompt 的最前面。一个 code-reviewer subagent,就是一位代码评审者——而不是一位"偶尔评审代码的通用程序员",也不是一位"恰好在做评审的老师"。它就是**代码评审者**。角色定义要足够具体,具体到:它能约束这个 subagent 做什么类型的工作、用什么类型的语言、做哪种判断、产出什么形态的输出。弱的角色定义,读起来像一行岗位名;强的角色定义,读起来像一个人的开场介绍。

Compare these two openings of a code-reviewer subagent system prompt. The first is weak.

对比下面两段 code-reviewer subagent 的 system prompt 开场。第一段是弱的。

You are a code reviewer. Review code carefully.

## Strong Subagent Descriptions · 强的 subagent 描述

The second is strong.

第二段是强的。

You are a senior code reviewer with ten years of experience reviewing pull requests on production systems. Your job

The second prompt does many things at once. It establishes seniority. It establishes purpose. It explicitly rejects two common failure modes: empty friendliness and padded reviews. It gives the subagent a posture, a stance toward its work. The subagent that runs under the second prompt produces noticeably different output from the subagent that runs under the first, even though both are nominally code reviewers. The role definition is a substantial fraction of the subagent's personality, and writing it carefully is among the most consequential acts of subagent design.

第二段 prompt 一次做了很多事:确立资历、确立目的、显式拒绝两种常见失败模式(空洞的客套和注水式评审)、并给 subagent 立了一种姿态——一种对待自己工作的站位。跑在第二段 prompt 之下的 subagent,产出的输出和跑在第一段之下的明显不同——尽管两者名义上都是"代码评审者"。角色定义,占了 subagent 人格的相当大一块;把它写好,是 subagent 设计中分量最重的工作之一。

Move to tools. The tools field in the frontmatter restricts which tools the subagent is allowed to invoke. Omitting the field grants the subagent the full set of tools available to the main session. Specifying a list narrows the set. The narrowing is not a security feature alone, although it is a useful one. The narrowing is also a focusing feature. A subagent that has only the tools it needs does not get distracted by the tools it does not need. A code-reviewer subagent that has access only to Read, Grep, and Glob cannot make file edits even if its system prompt forgets to forbid them, and the absence of Edit, Write, and Bash from its toolkit shapes how it approaches every task. The subagent does not casually offer to fix the issues it finds. It cannot, because the tools to do so are not present, and so its output naturally reflects the boundary.

接下来说工具。frontmatter 里的 `tools` 字段,用来限制 subagent 可以调用哪些工具。不填这个字段,subagent 就拿到主 session 的全套工具;填上列表,工具就被收窄。收窄,不仅仅是一个安全特性(虽然它确实有用)——它更是一个聚焦特性。一个只拥有"它真正需要的工具"的 subagent,不会被"它不需要的工具"分心。一个只拥有 Read、Grep、Glob 的 code-reviewer subagent,即使 system prompt 忘了写"不许改文件",它也改不了——因为 Edit、Write、Bash 根本不在它的工具箱里,这份"不在"塑造了它对待每一项任务的方式。它不会随口提议"顺手把它发现的问题修了",做不到,工具不在,于是它的输出自然就反映了这条边界。

A simple rule guides tools restriction. Begin with no tools, then add only the ones the subagent demonstrably needs. A reviewer needs to read files, search them, and find them, so Read, Grep, and Glob suffice. A documentation-writer needs to read files and write new ones, so Read, Write, and Glob suffice. A test-writer needs to read existing code, write new test files, and possibly run tests to verify, so Read, Write, Bash, and Glob form a reasonable set. The principle in each case is that the tool list mirrors the subagent's job description. If the subagent does not need a tool to do its job, the tool is not granted, full stop.

工具限制的规则很简单:从"零工具"开始,只加那些"被证明确实需要"的工具。评审者要"读、搜、找文件",所以 Read、Grep、Glob 就够了。文档作者要"读旧文件、写新文件",所以 Read、Write、Glob 够用。测试作者要"读已有代码、写新测试文件、可能要跑测试验证",所以 Read、Write、Bash、Glob 算合理组合。每一种情形,道理都一样:工具列表要"镜像" subagent 的岗位描述。它完成工作不需要的工具,一律不授予,完。

← 上一节

下一节 →

Move to boundary. The boundary of a subagent is the set of things it explicitly does not do. The role definition says what the subagent is. The boundary says what the subagent is not. Both are necessary, because the language model that runs the subagent will, when uncertain, tend to drift toward generality, and explicit boundaries push back against that drift.

接下来说**边界**。subagent 的边界,是它"显式不做事"的集合。角色定义说的是"这个 subagent 是什么";边界说的是"它不是什么"。两者都必要,因为跑这个 subagent 的语言模型,在不确定时,会倾向于向"通用"漂移——而显式边界,正是把这种漂移顶回去的力量。

## Boundary Language in Prompts · prompt 里的边界语言

Boundary language belongs in the system prompt and should be specific. A code-reviewer subagent should have boundary language such as: you do not write or modify code; if you find an issue, you describe it and recommend a fix, but you do not implement the fix yourself. A test-writer subagent should have boundary language such as: you do not change the code under test; if you discover a bug while writing tests, you note it in the test as a comment, and you flag it in your final summary, but you do not modify the production code. A documentation-writer subagent should have boundary language such as: you do not modify code, configuration, or any non-documentation file. Each of these statements feels redundant when first written, because the developer thinks the role definition already implies the boundary. The redundancy is the point. Stating the boundary explicitly costs nothing and prevents the small but real failure mode of a subagent quietly drifting outside its scope on a task where the role definition was ambiguous.

边界语言属于 system prompt,而且要写得具体。code-reviewer subagent 应该有这样的边界语言:"你不写也不改代码;如果你发现了问题,把它描述出来并建议怎么改,但不要自己去改。"test-writer subagent 应该有这样的边界语言:"你不改被测代码;如果你在写测试的过程中发现了一个 bug,在测试里以注释记下来,并在最终总结里 flag 出来,但不要去改生产代码。"documentation-writer subagent 应该有这样的边界语言:"你不修改代码、配置、任何非文档文件。"每一条写下来时,第一感觉都是"啰嗦"——因为开发者觉得,角色定义已经把这件事隐含说清了。这份"啰嗦"恰恰是重点:显式写出边界,几乎没有成本,却能避免一种小而真实的失败模式——在角色定义本就模糊的任务上,subagent 悄悄滑出自己的职责范围。

A complete subagent file, designed under all three dimensions, looks like this. The example below is a security-auditor, intended to focus narrowly on security issues in code, leaving general code review and other concerns to other subagents.

一个完整的 subagent 文件,按这三个维度设计出来,长这样。下面的例子是一个 security-auditor,设计目标是把"代码里的安全问题"这件事做窄,把通用代码评审和其他关切留给其他 subagent。

```

name: security-auditor
description: Audits code for security issues, including OWASP Top 10 vulnerabilities, hardcoded secrets, unsafe dependencies
tools: Read, Grep, Glob, Bash
model: opus

You are a senior application security engineer. Your job is to find security issues in code that other reviewers are
```

← 上一节

下一节 →

Read the file once more, slowly, and notice the discipline at every level. The description in the frontmatter is specific about the kind of work the subagent does and uses the *proactively* phrasing recommended by Anthropic to encourage reliable delegation. The tools list is narrow, exactly what the subagent needs and nothing more. The model is set to opus, because security review benefits from the strongest model the developer has available. The system prompt establishes a senior role with explicit areas of expertise. It defines the method as a numbered procedure. It includes a what-you-do-not-do section that is as long as the what-you-do section, because the boundary is genuinely as important as the role. It ends with a directive about the format of every report. Every degree of freedom that could produce variability in output has been pinned down deliberately.

请把这份文件再慢慢读一遍——留意每一层上的纪律。frontmatter 里的 description,具体到它要做的那类工作,并用了 Anthropic 推荐的 *proactively* 措辞,以促成稳定可靠的委派。工具列表很窄,正是它需要的那些,多一项没有。模型设成 opus,因为安全评审值得用开发者手边"最强的那一档"。system prompt 确立一个高级角色,并明确列出"专长领域";它把"方法"定义为一套带编号的流程;它包含一段"你不做什么",篇幅和"你做什么"相当,因为边界和角色一样重要;它以一段"每份报告该长成什么样"的指令收尾。每一种可能带来输出差异的自由度,都被有意识地钉死了。

Your areas of expertise:

- Authentication and authorization flaws
- Injection vulnerabilities (SQL, command, prompt, template)
- Hardcoded secrets and credentials
- Unsafe deserialization
- Missing input validation
- Insecure direct object references
- Cross-site scripting and request forgery in web contexts
- Time-of-check to time-of-use bugs
- Vulnerable or outdated dependencies

Your method:

1. Read the code you are given.
2. Look for the patterns above explicitly. Do not skim.
3. For each issue you find, produce a short report containing:
  - Severity: critical, high, medium, or low
  - Location: file path and line number
  - Description: what the issue is, in plain language
  - Recommendation: a concrete fix, with a code example if useful

What you do not do:

- You do not modify code. You only report.
- You do not perform general code review. Style, performance, and architecture concerns belong to other reviewers.
- You do not pad your output. If the code has no security issues, say so plainly in one sentence.
- You do not invent issues. If you are uncertain whether something is a vulnerability, mark it as a question rather

End every report with a one-sentence summary judgment: clear, minor issues only, or significant issues found.

← 上一节

下一节 →

A subagent designed this carefully produces strikingly consistent output across many invocations. Two different developers running this subagent against two different codebases will get reports that look like they came from the same security engineer, because in a meaningful sense they did. This consistency is what makes subagents useful as components of a larger system. A subagent whose output varies wildly from invocation to invocation is hard to compose with other subagents, hard to review, and hard to trust. A subagent whose output is stable in form, even when the substance varies with the codebase, can be plugged into a larger workflow with confidence.

这样用心设计出来的 subagent,在多次调用间会产出惊人地一致的输出。两位不同的开发者,把同一个 subagent 跑在两个不同的代码库上,会拿到"看起来像出自同一位安全工程师"的报告——因为在有意义的层面上,本来就是同一位。这种一致性,正是 subagent 之所以能成为"更大系统的组件"的原因。输出每次大变的 subagent,很难和其他 subagent 组合、很难复盘、也很难被信任;输出形态稳定(只是内容随代码库而变)的 subagent,可以放心地嵌入到更大的工作流里。

A second worked example will reinforce the design principles, because the discipline becomes habitual only after the developer has seen it applied to several different specialties. The subagent below is called `documentation-writer`. Its job is to produce or update documentation for code, with the same kind of consistency and discipline as the security-auditor.

第二个完整例子,用来再强化一遍设计原则——这种纪律,要等开发者看过它在不同专长上反复套用,才会真正变成习惯。下面这个 subagent 叫 `documentation-writer`。它的职责是产出或更新代码的文档,和 `security-auditor` 一样,追求一致性和纪律性。

```

name: documentation-writer
description: Produces clear, useful technical documentation for code, modules, or APIs, including README sections, J
tools: Read, Write, Glob, Grep
model: sonnet

You are a senior technical writer who works alongside engineers. Your job is to produce documentation that engineers
```

← 上一节

下一节 →

Compared with the security-auditor, the documentation-writer differs in several telling ways. The role is different: technical writer rather than security engineer. The tools are different: Read, Write, Glob, Grep, including the ability to write files because documentation is the deliverable; security-auditor was read-only because audit is its deliverable. The model is sonnet rather than opus, because documentation work does not particularly benefit from the strongest model and the cost saving is real for a subagent that may run several times a day. The principles section is where the writer's craft is encoded; the security-auditor had a similar section, but its content was about security, not about writing.

和 security-auditor 相比,documentation-writer 在几个有意义的点上不同:角色不同——技术写作者,不是安全工程师;工具不同——Read、Write、Glob、Grep,这里需要"能写文件",因为文档本身就是交付物;而 security-auditor 是只读的,因为"审计报告"才是它的交付物。模型是 sonnet 而非 opus——因为文档写作并不特别依赖"最强档",而一个可能每天跑好几次的 subagent,省下来的成本是真金白银。"原则"那一节,是把"写作者的手艺"编码进去;security-auditor 也有类似的一节,但内容是关于"安全"的,不是关于"写作"的。

#### Your principles:

- Documentation explains why, not what. Anyone can read the code to see what it does. Documentation should explain why.
- The audience is a smart engineer who has not seen this code before and who will spend roughly five minutes deciding if it is worth reading.
- Examples beat prose. A short worked example is worth several paragraphs of explanation.
- Honest documentation beats aspirational documentation. Document what the code actually does today, not what you wish it did.
- Concise documentation is read. Long documentation is skimmed at best and ignored at worst.

#### Your method:

1. Read the code in full. Read the surrounding code if needed for context.
2. Identify the public surface of the code: what callers will use.
3. Produce documentation in the project's existing format. Inspect the project to determine what format is in use (Read the code, not the documentation).
4. Cover, in order: purpose, public API, expected inputs and outputs, important behaviors and edge cases, a short worked example.

#### What you do not do:

- You do not modify code. Your edits are confined to documentation files.
- You do not write documentation that is a transliteration of the code into English.
- You do not invent behaviors that the code does not exhibit.
- You do not pad. A short, accurate page is the goal.
- You do not produce sycophantic prose. Avoid phrases like "powerful," "elegant," or "robust" unless the developer wrote them.

End with a list of the documentation files you wrote or updated, plus a short note on anything you found unclear in the code.

← 上一节

下一节 →

What the two subagents share is the structural discipline. Both have a clear role definition. Both have a numbered method. Both have an explicit list of things they do not do. Both have a directive about the format of their final output. This shared structure is the design discipline of the chapter, and the reader should recognize that the structure itself is doing the work of producing reliability, even more than any specific instruction the prompts contain.

这两位 subagent 共享的,是结构上的纪律:都有清晰的角色定义,都有带编号的方法,都有一份"明确列出的不许做事项",都有关于"最终输出格式"的指令。这种共享的结构,正是本章的设计纪律;读者应当意识到,真正在产出"可靠性"的,正是结构本身——比 prompt 里任何具体指令都更甚。

A debugging walkthrough is worth including, because the experience of building subagents that misbehave is universal, and the cure is teachable. The walkthrough below is from a real session. The developer had built a code-reviewer subagent, dispatched it on a pull request, and

found that the output was not what she expected. The subagent had produced a review, but the review felt generic. It mentioned reasonable things, but those reasonable things did not seem to come from the specific code the subagent had been asked to review. The developer suspected, correctly, that the subagent had read the code in a cursory way and then produced a review that could have applied to almost any code.

"调试式走查"值得一节——"搭一个 subagent 却跑出来不对"这种经历,是普遍的;而它的解法是可教的。下面这段来自一次真实的 session:开发者搭好一个 code-reviewer subagent,把它派去评审一个 PR,发现输出和预期不符。subagent 给出了评审,但那份评审"很通用"——它提到了不少合理的事,但这些事看起来不像是"针对被评审的那份具体代码"出来的。开发者怀疑,而且怀疑对了——subagent 只是蜻蜓点水地读了读代码,然后产出了一份"放在哪段代码上都说得通"的评审。

She investigated the system prompt. The role definition was decent. The method was a numbered procedure. The output format was specified. But on close reading, she noticed that the method said "review the code" without instructing the subagent to read it carefully, look for specific issues, or reference specific lines. The boundary section did not say what generic feedback was forbidden. The result was that the subagent, working under permissive instructions, took the easiest path and produced output that was technically a review but not the close, specific review the developer wanted.

她去检查 system prompt。角色定义还行,方法是带编号的流程,输出格式也指定了。但她细看之下发现:"方法"那一节只说"review the code",并没有指示 subagent 仔细读、去找具体问题、引用具体行;边界那一节也没说"不许给通用反馈"。结果就是:subagent 拿着相当宽松的指令,挑了最容易走的路,产出的输出技术上确实是"一份评审",但不是开发者想要的那种"贴着代码、具体到位"的评审。

She rewrote the method. The new method instructed the subagent to read the diff in full before forming any opinions, to identify three to five specific issues that referenced file paths and line numbers, and to write each comment as it would appear in a real pull request thread. She added a what-you-do-not-do entry forbidding generic observations that did not reference specific lines. She redispached the subagent on the same pull request. The output transformed. The new review named four specific issues, each with a file and line reference, each with a concrete suggestion. The genericity was gone. The total time spent on the fix was about twenty minutes, including the time to test the new version on three different inputs to confirm the consistency had improved.

她重写了"方法"。新版方法要求 subagent 在形成任何意见之前,先把 diff 完整读完;要求它找出三到五条具体问题,且每条都要带文件路径和行号;还要求它把每条意见写成"在真实 PR 评论流里"那种样子。她又在"你不做什么"那一节加了一条,禁止出现"不引用具体行"的通用观察。然后她把 subagent 重新派去同一个 PR。输出脱胎换骨。新评审点出了四个具体问题,每个都带文件与行号,每个都带具体建议。"通用感"消失了。整次修复花的时间大约二十分钟,其中还包括在三个不同输入上跑新版本来确认一致性已经改善。

← 上一节

下一节 →

## A Debugging Lesson · 一条调试心得

The lesson the developer took away from this debugging session was that subagent quality is largely controlled by the specificity of the instructions in the system prompt. A subagent given vague instructions produces vague work. A subagent given specific, structured instructions, with explicit prohibitions on the failure modes the developer has already encountered, produces work that consistently lives up to the design. The fix is rarely to make the subagent smarter. The fix is almost always to make the instructions more specific.

这位开发者从这次调试里带走的心得是:subagent 的质量,几乎完全由"system prompt 里指令的具体程度"决定。给一份含糊的指令,得到的是含糊的产出;给一份具体、有结构、且对"已经踩过的失败模式"有显式禁止的指令,产出就能稳定地配得上设计者的预期。修法的方向很少是"让 subagent 更聪明",几乎总是"让指令更具体"。

A final practice deserves mention before this chapter ends, because it is the practice that separates careful subagent design from sloppy subagent design. After writing a subagent, the developer should test it on three different inputs and compare the outputs. Not for correctness, although correctness matters. For consistency. The three outputs should look like they came from the same specialist. Same format. Same tone. Same level of detail. If the outputs vary in any of these dimensions, the system prompt has not pinned down the relevant degree of freedom, and the system prompt should be tightened until the variation disappears. This three-input test takes ten minutes and prevents an entire category of long-tail problems that would otherwise only surface in a real session at an inconvenient moment.

本章收尾前,还有一项实践值得提——它是"用心的 subagent 设计"和"潦草的 subagent 设计"的分水岭。写完一个 subagent 之后,开发者应该在三个不同的输入上跑一遍,对比输出。不是为了看正不正确(虽然正确也重要)——而是为了看一致不一致。三份输出看起来应该像出自同一位专员:同样的格式、同样的语气、同样的细节水平。哪个维度上出现了摇摆,system prompt 就还没把那个自由度钉死,就要把 prompt 收紧,直到那份摇摆消失。这个"三输入测试"只要十分钟,却能预防一整类"长尾问题"——那些问题,本来只会某个不方便 session 时刻才真正浮出水面。

Subagent design, like skill design, becomes faster with practice. The first subagent the developer writes will take an hour. The fifth will take twenty minutes. The tenth will take ten minutes, because the design dimensions and the system-prompt patterns are now familiar, and the developer has a working library of prior subagents she can adapt. The path from beginner to confident designer is short, and the rewards begin compounding from the first well-designed specialist.

Subagent 设计,和 skill 设计一样,熟能生巧。开发者写的第一个 subagent 大概花一小时,第五个只要二十分钟,第十个只要十分钟——因为设计维度和 system prompt 模板已经熟,而且手边已经有一套可改造复用的过往 subagent 库。从新手到"自信的设计者"这条路不长;而回报,从第一个被精心设计的专员开始,就在复利了。

A short reflection on the role of the model field in subagent design. The model field in the frontmatter lets the developer specify which model the subagent runs on. This may seem like a minor detail, but in practice it is one of the more consequential design choices. Subagents that demand the strongest reasoning, such as the security-auditor and the architecture-reviewer earlier in this part of the book, benefit from running on opus. Subagents that perform structured but less reasoning-intensive work, such as the documentation-writer or the test-writer, run perfectly well on sonnet at lower cost. Subagents that perform fast pattern-matching work, such as a search-helper or an explore agent, can run on haiku and complete in seconds rather than minutes. The model field, used thoughtfully, lets the developer tune each subagent to the cost and speed profile that best fits its job, rather than running everything on a single model and either overspending on simple work or underspending on demanding work. This kind of per-subagent tuning is the small mark of a mature subagent library, and it pays for itself across the months a library is in use.

简短说一说 `model` 字段在 subagent 设计里的角色。frontmatter 里的 `model` 字段,让开发者指定 subagent 跑在哪一档模型上。这看起来像是细节,但在实操中,它是分量更重的设计选择之一。需要最强推理的 subagent——比如前面提过的 security-auditor 和 architecture-reviewer——跑在 opus 上是值得的。做"结构化、但推理强度不高"的工作的 subagent——比如 documentation-writer、test-writer——完全可以在 sonnet 上跑,成本更低。做"快速模式匹配"的 subagent——比如一个搜索助手或 explore agent——可以跑在 haiku 上,几秒而不是几分钟搞定。带着这份心思去用 `model` 字段,就能给每一个 subagent 调出"最适合它工作的成本-速度组合"——而不是把一切压在一档模型上,要么在简单工作上多花,要么在吃力的工作上少花。这种"逐 subagent 调档",是一套"成熟 subagent 库"的小印记;而它在今后数月的使用里会自己把本钱赚回来。

The next chapter takes subagent design from the single subagent to the multi-subagent workflow. With the design of an individual specialist established, the question becomes how multiple specialists work together on a single task, how the main session orchestrates them, and how parallel execution turns work that would have been sequential into work that finishes in a fraction of the time. Orchestration is where the architecture of subagents reveals its full power, and that power is the subject ahead.

下一章会把 subagent 设计,从"单个 subagent"推进到"多 subagent 工作流"。当"单个专员的个体设计"已经确立,问题就变成:多个专员如何在一项任务上协作?主 session 如何编排它们?并行执行如何把"本来是顺序"的工作,变成"挂钟时间只是其中一小段"的工作?编排(orchestration),正是 subagent 架构展现其全部力量的地方——这份力量,正是下一章的主题。

← 上一节

下一节 →

## CHAPTER TWELVE · 第十二章

### Parallel Subagents and the Orchestrator Pattern

#### 并行子代理与编排者模式

"Coordination is the art of getting many things done in the time of one. The conductor does not play any of the instruments."

— Folk wisdom of the orchestra pit

"协调的艺术,就是让许多件事在"一件事的时间里"做完。指挥家自己不演奏任何一种乐器。"

—— 乐池里流传的民间智慧

The previous chapter taught the design of a single subagent. This chapter is about what happens when multiple subagents work on the same task at the same time. Parallelism is the property that turns subagents from a useful refinement into a structural change in how a developer relates to her tool. A single subagent saves context. Multiple parallel subagents save wall-clock time, deepen quality through specialization, and produce reviews and analyses that no single Claude session, no matter how capable, could produce in the same span of minutes.

上一章讲的是"单个 subagent"的设计。本章讲的是"多个 subagent 同时做一件事"时会发生什么。并行(parallelism)这条属性,把 subagent 从"一项有用的精修"变成"开发者与她工具之间关系的结构性改变"。一个 subagent 节省上下文;多个并行 subagent 节省挂钟时间、通过专门化深化质量、并产出那种"任何单个 Claude session、再怎么强,也不可能在同一段分钟数里做出"的评审与分析。

Begin with the simple case. The main session is asked to perform a task that has multiple genuinely independent components. The components do not need to share information with each other, and the order in which they are performed does not affect the result. A code review of a substantial pull request is a perfect example. A reviewer worth her salt will check several dimensions: correctness, security, performance, style, test coverage. None of these dimensions strictly depends on any other. A correctness check does not need the security check to have run first. A

style check does not need the performance check to be complete. The dimensions are orthogonal, which is exactly the property that makes them candidates for parallel execution.

先看最简单的那种情形:主 session 接到一项任务,任务包含若干个真正独立的组件。这些组件之间不需要共享信息,执行顺序也不影响结果。一次对大型 PR 的代码评审,就是个完美例子。一名有分量的评审者会从几个维度去查:正确性、安全、性能、风格、测试覆盖。这些维度之间没有严格的依赖关系——正确性检查不需要等安全检查先做完,风格检查也不需要等性能检查先完成。各维度是正交的——这种正交性,恰恰是它们适合被并行执行的原因。

The orchestrator pattern formalizes the idea. The main session takes the role of conductor. It receives the task, decomposes it into independent components, dispatches one subagent per component, waits for them all to finish, and synthesizes their outputs into a unified result. The subagents do not speak to each other. They do not know about each other. Each one focuses on its single dimension, produces its single bounded report, and returns. The conductor takes all the reports, weaves them together according to whatever format the unified output requires, and presents the result to the developer. The developer sees one coherent answer. The work that produced the answer happened in parallel, in five separate contexts, each tuned to its own dimension.

**编排者模式(orchestrator pattern)**把这个想法形式化。主 session 担任指挥家角色:它接下任务,把它拆成若干个独立组件,为每个组件派一个 subagent,等它们全部跑完,再把这些输出综合成一份统一结果。subagent 之间互不说话、互不知道彼此。每个 subagent 都聚焦于自己的那一维,产出自己那份边界清晰的报告,然后返回。指挥家把所有报告收齐,按"统一输出所需的格式"编织成一份,再交给开发者。开发者看到的是一份连贯的答案;而这份答案背后发生的工作,是并行发生的——发生在五个相互独立、各自针对自己维度的上下文里。

← 上一节

下一节 →

A worked example will make this concrete. The developer has a pull request she wants reviewed thoroughly before merging. She types into Claude Code something like, give me a full review of this PR, including security, performance, style, and test coverage. The main session recognizes that a thorough review is being requested, recognizes the dimensions involved, and dispatches four subagents in parallel: a security-auditor, a performance-reviewer, a style-reviewer, and a test-coverage-checker. Each subagent receives the same brief: review this diff, focused on your dimension. Each subagent runs in its own context, reads the relevant files, applies its specialty, and produces its bounded report. The four reports return to the main session. The main session presents them in a unified format, with a summary judgment that integrates the findings across the four dimensions.

一个完整例子会让这件事更具体。开发者有一个 PR,想在合并前被彻底评审一遍。她在 Claude Code 里敲一句类似的话:"给我一个完整的 PR 评审,覆盖安全、性能、风格、测试覆盖。"主 session 识别出"用户要的是一次彻底评审",识别出里面涉及的维度,于是并行派出四个 subagent:security-auditor、performance-reviewer、style-reviewer、test-coverage-checker。每一个 subagent 收到的 brief 都一样:评审这份 diff,聚焦于你负责的维度。每个 subagent 跑在各自的上下文里,读相关文件,发挥自己专长,产出边界清晰的报告。四份报告回到主 session;主 session 用统一格式呈现它们,并给出一段"把四维发现整合在一起"的总结判断。

## Estimating the Wall-Clock Saving · 估算挂钟时间的节省

The wall-clock saving is straightforward to estimate. If each subagent takes two minutes, four subagents in sequence take eight minutes, while four subagents in parallel take roughly two minutes, plus the synthesis step. The factor of four speedup is real and felt. On a code review that the

developer might run twice a day, the saving across a working week is substantial.

挂钟时间上的节省,直白到可以用算术算出来:每个 subagent 跑两分钟,顺序跑四个要八分钟,并行跑四个加上综合那一步大约两分钟。这 4 倍的加速是实在的、感受得到的。评审这件事一天可能跑两次,折算到一个工作周,节省下来的时间是可观的。

But the time saving is not the most interesting benefit. The more interesting benefit is the quality. Each subagent operates in a context that contains only the material relevant to its dimension. The security-auditor's context contains the diff plus the security-auditor's system prompt, focused on security. The performance-reviewer's context contains the diff plus the performance-reviewer's system prompt, focused on performance. Neither subagent has its attention diluted by considerations that are not its own. The result is that each subagent produces a deeper, more focused report than a single generalist reviewer could produce on the same input. The total review is not just faster; it is better, because each part of it was done by a specialist who was not trying to balance attention across four dimensions at once.

但更值得讲的不是时间,而是质量。每个 subagent 工作的上下文里,只有"与它这一维相关"的材料。security-auditor 的上下文里只有 diff 加它自己的 system prompt,聚焦在安全上;performance-reviewer 的上下文里也只有 diff 加它自己的 system prompt,聚焦在性能上。没有任何一个 subagent 的注意力被"不属于自己的考量"稀释。结果就是:每个 subagent 产出的报告,都比"一个通才评审者面对同一份输入"所产出的更深、更聚焦。整个评审不仅更快,还更好——因为它的每一段都出自一位"不用同时兼顾四个维度"的专员之手。

The orchestrator pattern has a discipline that is worth naming. The main session's job, as the orchestrator, is decomposition and synthesis. The subagents' job is the specialty work. The boundary between these two jobs should be respected. The main session should not attempt to do specialty work itself, even when it could; if a security review is needed, the security-auditor subagent should do it, not the main session. The main session should not attempt to mix specialties inside a single subagent, either; a security-and-performance reviewer is a worse subagent than two narrow ones, because the specialization advantage is forfeited. The orchestrator pattern asks the developer to design subagents narrowly, then trust the design, and let parallelism do its work.

编排者模式里有"纪律"二字值得点出。主 session 作为编排者,做的事是分解与综合,subagent 做的事是专长工作。这两件事之间的边界要守住。主 session 不该"哪怕能做也亲自去"做专长工作——需要安全评审,就派 security-auditor,而不是主 session 自己上。主 session 也不该在某个 subagent 内部"混专长"——一个"既安全又性能"的评审 subagent,比两个窄专长的 subagent 更差,因为"专业化"带来的优势被白白扔掉。编排者模式要求开发者把 subagent 设计得窄、然后信任这份设计,让并行去做它该做的事。

← 上一节

下一节 →

There are limits to parallelism worth knowing. Claude Code can run several subagents concurrently in practice; community guidance from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although Anthropic has not published a hard cap. Beyond that limit, additional dispatches queue and run as earlier ones finish. Within the limit, the parallelism is genuinely simultaneous: the subagents do not take turns; they execute in independent processes. The cost of running multiple subagents is, naturally, multiplied by the number of subagents. A four-subagent review costs roughly four times the tokens of a single-subagent review. This cost is usually well worth paying, because the time and quality benefits dominate, but it is a real cost and worth being aware of.

并行的边界值得知道。实践中,Claude Code 能并发跑若干个 subagent;重度用户的社区经验大致收敛在"十个左右"——超过这个数,排队延迟就开始能被感知;不过 Anthropic 并没有公开过一个硬上限。超过这个上限,新的派单会进入队列,等前面跑完再出。在上限以内,并行是真正"同时"的:subagent 之间不轮换,跑在彼此独立的进程里。同时跑多个 subagent 的成本,自然也按数量倍增。一次四

subagent 评审的 token,大约是一次单 subagent 评审的四倍。这份成本大多数情况下完全值得付,因为时间与质量上的好处压过它;但它终究是实在的成本,值得心里有数。

There are tasks for which parallelism is the wrong shape, and recognizing them is part of the discipline. Tasks whose components depend on each other do not benefit from parallel execution; the dependencies force sequential work. A task that involves first reading a database schema, then writing a query against it, then optimizing the query, has a strict order that cannot be parallelized usefully. Tasks whose components share a great deal of context also do not benefit; if every subagent would need to read the same fifty files to do its job, the duplication of that reading across many subagents wastes tokens and time. The right pattern in the dependent case is sequential subagent dispatch, where the output of one subagent becomes the brief for the next. The right pattern in the shared-context case is sometimes a single subagent that handles all the work, or sometimes a careful brief that summarizes the shared context once and passes the summary to each subagent so they do not all need to re-derive it.

有些任务天生不适合并行,识别出它们也是纪律的一部分。组件之间有依赖的任务,无法从并行里获益——依赖关系把工作强行串成顺序。一件"先读数据库 schema、再写查询、再优化查询"的任务,就有严格的顺序,无法有意义地并行。组件之间共享大量上下文的任务,也无法从中获益——如果每个 subagent 都要先读同一份 50 个文件,这种"每个 subagent 都重读一遍"对 token 和时间都是浪费。对"有依赖"的情况,正确模式是顺序派单:上一个 subagent 的输出,成为下一个 subagent 的 brief。对"共享上下文"的情况,正确模式有时是派一个 subagent 把活全干了;有时是写一份精心准备的"共享上下文摘要",一次性传给每个 subagent,让它们不必各自重新推导。

A useful framing for deciding when parallelism applies is this question: would I, as a developer, hire four different specialists for four different parts of this task, knowing that the specialists do not need to talk to each other? If the answer is yes, the orchestrator pattern fits. If the answer is no, because the specialists would need to coordinate continuously, the work belongs in a single subagent or in a sequence. This human-team metaphor maps cleanly onto subagent design, and developers who train themselves to ask the metaphor question quickly learn to recognize which workflows benefit most from parallelism.

判断"什么时候适合并行"有一个好用的框架——问自己这样一个问题:"我作为开发者,会为这件事的四个不同部分各雇一位专员,前提是他们彼此之间不需要交流吗?"如果答案是"会",编排者模式正合适;如果答案是"不会"——因为这些专员需要持续协调——那这件事就该交给一个 subagent,或者走顺序模式。这个"人类团队"类比,能干净地映射到 subagent 设计;训练自己会问这个类比问题的开发者,很快就能一眼看出哪些 workflow 最受益于并行。

A worked end-to-end orchestration looks like this in practice. The developer types her broad request. The main session, recognizing the orchestratable pattern, identifies the dimensions and dispatches the subagents. The dispatches happen in quick succession, and the subagents begin work simultaneously. The main session, while waiting, sometimes does a small amount of preparatory work, like inspecting the diff itself for the synthesis step. The subagents return their reports as they finish; reports may not arrive in dispatch order, since some specialties take longer than others. The main session collects them, holds them until all are present, and then composes the unified output. The whole interaction takes only marginally longer than the slowest of the four subagents. The developer sees a thorough, multi-dimensional review delivered in roughly the time of one careful pass, and the review reads as if four specialists had been retained for the job, because, in a meaningful sense, four specialists were.

一个"端到端"的编排者流程,实操中是这样的:开发者用宽泛的方式提出请求;主 session 识别出"这是一个可编排的形状",找出涉及的维度,派 subagent;派单一份接一份快速发生,subagent 同时开始工作。主 session 在等的当口,有时会做一点准备性工作——比如亲自扫一眼 diff,方便后面综合。各 subagent 完成的顺序可能和派单顺序不一样,因为有的专长耗时长、有的耗时短;它们一边完成一边把报告送回,主 session 收齐之后,会一直等所有报告都到齐,再合成统一输出。整个交互的耗时,只比四个 subagent 中最慢的那一个多一点点。开发者看到的,是一次彻底、多维的评审——送达的耗时,大致相当于"做一次认真评审"的时间;而读起来,就像是真的请了四位专员——在有意义的层面上,确实就是请了四位。

A concrete example will fix this in the reader's mind. Consider a developer who has just opened a pull request that touches roughly four hundred lines of code in three files. The change introduces a new authentication endpoint. Before merging, she wants a full review. She types into Claude Code, in plain language, perform a full review of this PR covering security, performance, style, and test coverage. The main session reads the request. It recognizes the four named dimensions. It looks at the available subagents and finds matches: security-auditor, performance-reviewer, code-style-checker, test-coverage-analyzer. It dispatches all four with a brief that includes the diff and a directive to focus on the subagent's named dimension.

一个具体例子会让这件事在读者脑子里落地。设想:开发者刚开了一个 PR,跨三个文件、改动约四百行,新增一个认证端点。合并前,她想要一次完整评审。她用日常语言在 Claude Code 里敲:"对这个 PR 做一次完整评审,覆盖安全、性能、风格、测试覆盖。"主 session 读这条请求,识别出里面点名的四个维度,去看可用的 subagent,找到匹配项:security-auditor、performance-reviewer、code-style-checker、test-coverage-analyzer。然后它把四个一起派出去,brief 里都带上 diff,并附一条"聚焦你被点名的那个维度"的指令。

The four subagents begin work simultaneously. The security-auditor reads the new authentication code with attention to OWASP categories. The performance-reviewer scans for hot-path issues, queries inside loops, and unbounded allocation. The code-style-checker reads the diff against the project's style conventions. The test-coverage-analyzer cross-references the diff against existing test files to identify which new code paths are unexercised. Each subagent works in a context that contains only the diff and its own focused system prompt. None of them sees the others' work. Each produces a focused report.

四个 subagent 同时开工。security-auditor 用 OWASP 类目去读新的认证代码;performance-reviewer 扫"热路径问题、循环里嵌查询、无界分配";code-style-checker 把 diff 对照项目的风格规约;test-coverage-analyzer 把 diff 和现有测试文件做交叉对照,找出哪些新增代码路径没有被覆盖到。每个 subagent 的上下文里都只有"diff + 它自己那份聚焦的 system prompt",看不到其他 subagent 在做什么,各自产出一份聚焦的报告。

Roughly two minutes later, the four reports return to the main session in whatever order they finish. The security-auditor flags a missing rate limit on the password reset endpoint and a token expiry that defaults to twenty-four hours rather than the project's standard of one. The performance-reviewer flags nothing significant. The code-style-checker notes two minor style deviations. The test-coverage-analyzer points out that the new password reset endpoint has no test coverage at all. The main session reads all four reports, composes a unified review with sections for each dimension, and presents the result to the developer. Total wall-clock time, from her first prompt to the unified review on her screen, is roughly two and a half minutes. Total human attention required, zero, until the review is ready to read.

大约两分钟后,四份报告按各自的完成顺序回到主 session。security-auditor 标出"密码重置端点没有限流"以及"token 过期默认是 24 小时,而不是项目规范里的 1 小时";performance-reviewer 没有发现重大问题;code-style-checker 指出两处轻微的风格偏差;test-coverage-analyzer 指出"新的密码重置端点完全没有测试覆盖"。主 session 读完四份报告,把它们编成一份统一评审——按维度分节,呈现给开发者。从她敲下第一条 prompt,到屏幕上的统一评审出现,挂钟时间总共大约 2.5 分钟。她要付的注意力,是零——直到评审可以读了为止。

## Without the Pattern · 没有这个模式会怎样

Now imagine the same review without the orchestrator pattern. The developer would have asked the main session to do the review directly. The main session would have read the diff, then mentally switched into security mode, then into performance mode, then into style mode, then into

test-coverage mode, all in a context that accumulated the residue of each phase. The total wall-clock time would have been closer to ten minutes, because the work was sequential rather than parallel. The total quality would have been lower, because each phase was attended to by a generalist rather than a specialist. And the cost in tokens, while lower, would have been a false economy, because the developer's attention is more valuable than the marginal token spend, and the orchestrated review let her get to her next task seven minutes sooner.

现在想象一下:同一份评审,如果不用编排者模式,会怎样。开发者会直接让主 session 做这件事。主 session 会先读 diff,然后在脑子里"切到"安全模式,再切到性能模式,再切到风格模式,再切到测试覆盖模式——所有这一切,都发生在一个不断累积"每一阶段残渣"的上下文里。总挂钟时间会更接近十分钟,因为是顺序而不是并行;总质量会更低,因为每一阶段都只由一个通才在分心兼顾;token 成本确实更低,但那是一种"虚假的节约"——开发者的注意力比那点边际 token 成本更值钱,而且编排式评审让她提前七分钟进入下一件事。

← 上一节

下一节 →

The orchestrator pattern is not the only multi-subagent pattern, although it is the most visually striking one. A second important pattern is the sequential pipeline, in which the output of one subagent becomes the brief for the next. This pattern applies when the work has a real ordering dependency that cannot be parallelized. A common example: the developer wants to refactor a substantial piece of code, but only after having an architectural review of the current structure to confirm the refactor's direction. The architecture-reviewer runs first; its output, a recommendation about which structure is appropriate, becomes part of the brief for the refactor-specialist that runs second. The two cannot run in parallel because the second one needs the first one's conclusions.

编排者模式不是唯一的"多 subagent 模式",虽然它最显眼。第二个重要模式是**顺序管线(sequential pipeline)**——上一个 subagent 的输出,成为下一个 subagent 的 brief。这种模式适合那种"工作有真实顺序依赖、无法并行"的情形。一个常见例子:开发者想重构一大块代码,但她希望先有"对当前结构的架构评审"来确认重构方向。architecture-reviewer 先跑,它的输出——"推荐走哪种结构"——成为 refactor-specialist 第二个跑的 brief 的组成部分。两者不能并行,因为第二个的输入是第一个的结论。

The mechanics of sequential pipelines are slightly different from those of parallel orchestration. The main session dispatches the first subagent, waits for its result, integrates the result into the brief for the next subagent, dispatches that one, waits for its result, and so on. The total wall-clock time is the sum of each subagent's time, with no parallelism saving. The benefits are still substantial: each step happens in a focused context, each step's output is shaped by the previous step's output rather than by the noise of an accumulated session, and the pipeline can be reviewed step by step, with the option to stop at any intermediate stage if the work is not going where the developer wants. Sequential pipelines, while slower than orchestrations, are the natural shape of work that has real dependencies, and the developer who recognizes when she is in a sequential rather than a parallel situation will save herself the frustration of trying to force orchestration onto work that does not support it.

顺序管线和并行编排的"机械结构"略有不同。主 session 先派第一个 subagent,等它的结果,把结果整合进下一个 subagent 的 brief,再派出那一个,等它的结果,如此往复。总挂钟时间是各 subagent 时间之和,没有"并行带来的节省"。但它带来的好处仍然不小:每一步都跑在一个聚焦的上下文里;每一步的输出由上一步的输出塑造,而不是被一个"不断累积的 session 噪声"塑造;管线可以一步一步地被审视,任何中间阶段,如果工作偏离了开发者的预期,都有机会停下。顺序管线虽然比并行编排慢,但它正是"工作真有依赖"时的天然形状;能分辨"我这是顺序而非并行"的人,会免去"硬把不该并行的东西套上编排模式"的挫败感。

A third pattern, which is a hybrid of the first two, is worth naming briefly. The map-reduce pattern dispatches several subagents in parallel on independent components of a task and then dispatches a final subagent, sequentially, to synthesize their outputs. Sometimes the synthesis is straightforward enough for the main session to do directly, in which case it is just the orchestrator pattern. Sometimes the synthesis is itself a substantial specialty, such as composing a coherent design document from several focused reports, in which case a synthesis subagent is worth

defining. The map-reduce pattern shows up most often in larger codebases, where the inputs are too varied for a single specialist to read in full. Different parallel subagents read different sections; a synthesis subagent receives their reports and produces the final output. The pattern is more elaborate, but the underlying principles are the same: independent work in parallel where possible, dependent work in sequence where required, focused contexts at every stage.

第三种模式,介于前两种之间,值得简短给它起个名字:**map-reduce 模式**。它先并行派几个 subagent,各自处理任务里独立的部分,然后再(顺序地)派一个最后的 subagent 去综合它们的输出。有时综合本身简单到主 session 直接做就行,那就还是编排者模式;有时综合本身就是一门分量不轻的专长——比如把几份聚焦的报告编成一份连贯的设计文档——这时候就值得专门定义一个"synthesis" subagent。map-reduce 模式在大代码库里出现得最多——那里输入过于多样,一个专员读不过来:并行 subagent 各自读自己那一段,synthesis subagent 接住它们的报告,产出最终输出。模式本身更复杂,但底层原则没变:能并行就并行,有依赖就顺序,每一阶段都跑在聚焦的上下文里。

← 上一节

下一节 →

A short reflection on what makes orchestration genuinely powerful. The most striking thing about the orchestrator pattern is not the wall-clock saving, although the saving is real and welcome. The most striking thing is what the developer is freed from. She no longer has to be the one who remembers all the dimensions a thorough review should cover. The pattern itself remembers them. She no longer has to mentally context-switch between security thinking and performance thinking and style thinking, because the subagents handle each dimension in its own thread. Her attention, which was previously distributed across all the dimensions she cared about, can be concentrated on whatever the unified output most demands of her. She becomes, in effect, a senior engineer reading reports written by competent specialists, rather than a junior engineer trying to be all four specialists at once. The transformation in working experience is not just a productivity improvement; it is a change in the kind of cognitive work she is asked to do, and developers who experience it for the first time often describe the shift as the moment Claude Code stopped feeling like a chat interface and started feeling like an environment.

简短聊一下"编排者模式真正强大的地方"。它最显眼的地方,不是挂钟时间的节省——虽然那也是真实且受欢迎的。它最显眼的地方,是开发者被解放出来。她不再需要亲自去记"一次彻底评审该覆盖哪些维度"——模式本身替她记着;她不再需要在脑子里从"安全思维"切到"性能思维"再切到"风格思维"——subagent 各自在自己的线程里处理各自的维度。她原本被摊薄在"所有她关心的维度"上的注意力,可以集中到统一输出最需要她投入的地方。她在效果上变成了"一位资深工程师,读着由胜任的专员写就的报告",而不是"一位初级工程师,试图一个人扮四角"。这种工作体验上的转变,不只是生产力的提升,更是她被要求去做的那类认知工作本身的改变——很多开发者第一次体会到这种转变时会这样描述:"就是这一刻,Claude Code 不再像一个聊天界面,而像一个环境。"

## When Not to Orchestrate · 什么时候不该编排

A final note, before this chapter closes, on when not to reach for orchestration. The orchestrator pattern carries overhead. Dispatching four subagents and synthesizing their outputs is more elaborate than asking the main session to do a small task directly. For small tasks, the overhead is not worth paying. A quick code review of a fifty-line patch does not need four subagents; the main session can handle it directly. A trivial documentation update does not need a documentation specialist; the main session is fine. The orchestrator pattern earns its keep on substantial tasks, where the parallelism saving and the specialization benefit are worth the dispatch overhead. The developer who reaches for orchestration appropriately, and refrains from reaching for it on tasks that do not justify it, will have the best experience.

本章收尾前,还要说一点:什么时候不该用编排。编排者模式本身有开销——派四个 subagent、再综合它们的输出,比"让主 session 直接做一件小事"要复杂得多。对小事来说,这笔开销不值得付。五十行 patch 的快速评审,不需要四个 subagent,主 session 直接做就够;琐

碎的文档更新,不需要一个文档专员,主 session 自己就行。编排者模式只有在大体量任务上才回本——那里并行的节省和专门化的好处能压过派单开销。能"该用就用、不该用就克制"的人,体验是最好的。

A useful rule of thumb is the five-minute test. If the developer would spend more than five minutes on the task as a single sequential pass through the main session, the orchestrator pattern is probably worth considering. If she would spend less than five minutes, orchestration is probably overkill. The threshold is not exact, and individual judgment varies, but the rule prevents both common mistakes: orchestrating tiny tasks where the dispatch overhead dominates, and serializing large tasks where the parallelism would have produced real savings. With practice, the developer stops needing the rule, because the right shape of work for orchestration becomes visible at a glance. Until that intuition has set in, the five-minute test is a reasonable working approximation.

一条好用的经验法则叫"五分钟测试"。如果开发者自己一个人顺序做这件事要花超过五分钟,那么编排者模式大概值得考虑;如果只要不到五分钟,编排大概率是杀鸡用牛刀。门槛不是死数,个人判断也会有出入,但这条规则可以同时防止两种常见错误——在小任务上"派单开销盖过收益"、在大任务上"本可并行的却被迫串行"。练上一阵之后,这条规则就不必再挂着——编排适合的形状会"一眼可见"。在那种直觉还没稳之前,"五分钟测试"是一个合理的工作近似。

← 上一节

下一节 →

A second observation worth keeping in mind is that orchestration scales with the developer's familiarity. New users of subagents tend to under-orchestrate, treating the main session as the default and reaching for subagents only on the most obvious cases. Experienced users tend to orchestrate naturally, recognizing the orchestratable shape of work in their first or second pass through a problem. The transition from the first state to the second takes only a few weeks of deliberate practice, and the practice is enjoyable rather than tedious, because every successful orchestration produces a noticeable, satisfying acceleration of work that would otherwise have been slow.

第二条值得记在心里的观察是:编排这种"能力"会随着开发者的熟练度而增长。新接触 subagent 的用户倾向于"编排不足"——把主 session 当默认,只在最明显的几种情况里伸手去用 subagent;熟练用户则会自然地编排,在面对一个问题的第一遍、第二遍里就识别出"可编排"的形状。从前者到后者的过渡,只需要几周有意识的练习;而这种练习并不令人疲倦,因为每一次成功的编排都会带来"原本很慢的事被显著加速"那种可感知的满足。

One final reflection on this chapter, before moving to the inventory of specialists. The orchestrator pattern is the visible face of subagents, the part that produces the most striking demonstrations and the most quotable speedups. But the deeper value of subagents, the value that compounds quietly across months of use, is not the speed. It is the consistent quality. A library of well-designed specialists produces consistently high-quality work because each specialist has been designed for its specific role, and the design is preserved across every invocation. The developer who has installed a serious subagent library is not just faster. She is more reliably good at her work, because the failure modes that come from overload, distraction, or context pollution have been engineered out of her environment. The reliability is the prize. The speed is a happy side effect.

在过渡到"专员清单"那一章之前,再留一段反思。编排者模式是 subagent 的"面子"——它造就了最抓眼球的演示、最值得引用的加速。但 subagent 更深层的价值——那种"在几个月里悄悄复利"的价值——不是速度,而是始终如一的质量。一套精心设计的专员库能稳定地产出高质量的工作,因为每位专员都是为它那个特定角色设计的,而这份设计在每一次调用里都被保留下来。装下了一套"像样的 subagent 库"的开发者,不只是更快——她更稳定地做得好;因为"过载、分心、上下文污染"带来的失败模式,已经从她的环境里被工程化地排除了。可靠性才是奖赏;速度只是顺带的好处。

The next chapter shifts from the single orchestration pattern to the inventory question. With the design of single subagents and the architecture of parallel orchestration both established, the developer needs a working library of subagents to draw from. The chapter ahead provides that library. Eight specialist subagents, each ready to install, each addressing a recurring need in daily development work, will make the architecture of this part of the book concrete in installed form. By the end of the next chapter, the reader will have a team of specialists waiting in her `.claude/agents` directory, ready to be dispatched whenever her work calls for them.

下一章从"单个编排模式"切换到"清单"问题。当"单个 subagent 的设计"和"并行编排的架构"都已经确立,开发者需要一套"能上手就用"的 subagent 库。下一章就提供这套库:八位专员 subagent,每位都已经准备好装,每位都对应一项日常工作里的高频需求——把本部分讲过的架构,以"已安装形态"具体化。等读完下一章,读者就会在自己 `.claude/agents` 目录里坐拥一支"专员团队",随时等候工作召唤。

← 上一节

下一节 →

A worked end-to-end orchestration looks like this in practice. The developer types her broad request. The main session, recognizing the orchestratable pattern, identifies the dimensions and dispatches the subagents. The dispatches happen in quick succession, and the subagents begin work simultaneously. The main session, while waiting, sometimes does a small amount of preparatory work, like inspecting the diff itself for the synthesis step. The subagents return their reports as they finish; reports may not arrive in dispatch order, since some specialties take longer than others. The main session collects them, holds them until all are present, and then composes the unified output. The whole interaction takes only marginally longer than the slowest of the four subagents. The developer sees a thorough, multi-dimensional review delivered in roughly the time of one careful pass, and the review reads as if four specialists had been retained for the job, because, in a meaningful sense, four specialists were.

一个"端到端"的编排者流程,实操中是这样的:开发者用宽泛的方式提出请求;主 session 识别出"这是一个可编排的形状",找出涉及的维度,派 subagent;派单一份接一份快速发生,subagent 同时开始工作。主 session 在等的当口,有时会做一点准备性工作——比如亲自扫一眼 diff,方便后面综合。各 subagent 完成的顺序可能和派单顺序不一样,因为有的专长耗时长、有的耗时短;它们一边完成一边把报告送回,主 session 收齐之后,会一直等所有报告都到齐,再合成统一输出。整个交互的耗时,只比四个 subagent 中最慢的那一个多一点点。开发者看到的,是一次彻底、多维的评审——送达的耗时,大致相当于"做一次认真评审"的时间;而读起来,就像是真的请了四位专员——在有意义的层面上,确实就是请了四位。

← 上一节

下一节 →

A concrete example will fix this in the reader's mind. Consider a developer who has just opened a pull request that touches roughly four hundred lines of code in three files. The change introduces a new authentication endpoint. Before merging, she wants a full review. She types into Claude Code, in plain language, perform a full review of this PR covering security, performance, style, and test coverage. The main session reads the request. It recognizes the four named dimensions. It looks at the available subagents and finds matches: security-auditor, performance-reviewer, code-style-checker, test-coverage-analyzer. It dispatches all four with a brief that includes the diff and a directive to focus on the subagent's named dimension.

一个具体例子会让这件事在读者脑子里落地。设想:开发者刚开了一个 PR,跨三个文件、改动约四百行,新增一个认证端点。合并前,她想要一次完整评审。她用日常语言在 Claude Code 里敲:"对这个 PR 做一次完整评审,覆盖安全、性能、风格、测试覆盖。"主 session 读这条请求,识别出里面点名的四个维度,去看可用的 subagent,找到匹配项:security-auditor、performance-reviewer、code-style-checker、test-coverage-analyzer。然后它把四个一起派出去,brief 里都带上 diff,并附一条"聚焦你被点名的那个维度"的指令。

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

The next chapter shifts from the single orchestration pattern to the inventory question. With the design of single subagents and the architecture of parallel orchestration both established, the developer needs a working library of subagents to draw from. The chapter ahead provides that library. Eight specialist subagents, each ready to install, each addressing a recurring need in daily development work, will make the architecture of this part of the book concrete in installed form. By the end of the next chapter, the reader will have a team of specialists waiting in her `.claude/agents` directory, ready to be dispatched whenever her work calls for them.

下一章从"单个编排模式"切换到"清单"问题。当"单个 subagent 的设计"和"并行编排的架构"都已经确立,开发者需要一套"能上手就用"的 subagent 库。下一章就提供这套库:八位专员 subagent,每位都已经准备好装,每位都对应一项日常工作里的高频需求——把本部分讲过的架构,以"已安装形态"具体化。等读完下一章,读者就会在自己 `.claude/agents` 目录里坐拥一支"专员团队",随时等候工作召唤。

← 上一节

下一节 →

# CHAPTER THIRTEEN · 第十三章

## The Subagent Library: Eight Specialists Every Developer Needs

### 子代理库:每位开发者都需要的八位专员

*"A small team of true specialists outperforms a large team of competent generalists almost every time. The difference is not effort. It is depth."*

— attributed to W. Edwards Deming

"一支由真正专员组成的小团队,几乎每一次都胜过一支由"称职的通才"组成的大团队。差距不是投入的多少,而是深度。"

—— 归于 W. 爱德华兹·戴明

This chapter is the practical counterpart to Chapter Nine. By the time the reader finishes it, eight specialist subagents will be installed in her `~/ .claude/agents` directory, ready for use in any project. Each specialist is presented in the same uniform format: the problem it solves, the complete subagent file, and a short explanation of the design choices that shaped it. The reader who installs all eight will have a small but capable team of specialists at her disposal, available for dispatch whenever her work crosses one of their domains.

本章是第九章的"subagent 版姊妹篇"。等读者读完后,八个专员 subagent 就会装进她 `~/ .claude/agents` 目录,随时可以在任何项目里使用。每位专员都用统一的格式呈现:它解决的问题、完整的 subagent 文件、以及"塑造它的设计选择"的简短说明。把八位都装齐的读者,会拥有一支"小而能干"的专员团队,每当工作触碰到它们的领域,即可派单。

The selection of eight specialists is deliberate. The eight cover the most common dimensions of code work that benefit from isolation and specialization: security, performance, testing, documentation, architecture, refactoring, dependency hygiene, and migration. These are the dimensions that, in a real engineering organization, would be staffed by named specialists with named roles. Replicating those specialist roles in subagent form is what gives a single developer, working in Claude Code, access to expertise that would otherwise be available only inside a substantial engineering team.

这八位专员的挑选是刻意的。它们覆盖了"代码工作中受益于隔离与专门化"的最常见维度:安全、性能、测试、文档、架构、重构、依赖卫生、迁移。这些维度,在真实工程组织里,会由"有名有姓、有明确角色"的专员来承担。把这些专员角色以 subagent 形式复制出来,正是"让一位开发者、在 Claude Code 里、获得本来只有大型工程团队里才有的那种专业能力"的关键。

Before beginning the installation, a small piece of housekeeping. All eight subagents below are personal subagents, installed at the user level, and therefore live in `~/ .claude/agents`. Each one is a single Markdown file named after the subagent. After installing all eight and restarting Claude Code, the subagents will be available in every session, in every project the developer opens. Project-specific specialists, if needed, can be added later in the `.claude/agents` directory of specific projects.

动手前,先做点内务。下面八位 subagent 都是"个人 subagent",按用户级别安装,因此都放在 `~/ .claude/agents` 里。每个 subagent 就是单个 Markdown 文件,文件名与 subagent 同名。装齐八位、重启 Claude Code 之后,这些 subagent 在每个 session、每个项目里都可用。如果以后需要"项目级"专员,可以在具体项目的 `.claude/agents` 下再加。

## The First Specialist · 第一位专员

The first specialist is `security-auditor`, presented in full in the previous chapter. Its job is to audit code for security vulnerabilities. If the reader has not already installed it, the file from Chapter Eleven can be used directly.

第一位专员是 `security-auditor` ——上一章已经完整给出过。它的职责是审计代码里的安全漏洞。如果读者还没装,直接用第十一章的那个文件即可。

The second specialist is `performance-reviewer`. Its job is to review code for performance issues, with a focus on the kinds of problems that show up in production rather than the kinds that show up in micro-benchmarks.

第二位专员是 `performance-reviewer`。它的职责是评审代码里的性能问题,关注的是"会在生产里表现出来的"那一类问题,而不是"只在微基准里才看得见"的那一类。

```

name: performance-reviewer
description: Reviews code for performance issues that affect real-world execution, including algorithmic complexity,
tools: Read, Grep, Glob, Bash
model: sonnet

You are a senior performance engineer. Your job is to find performance issues that will affect production behavior,

Your areas of focus:
- Algorithmic complexity ($O(n^2)$ where $O(n)$ would do)
- Database queries inside loops (the classic $N+1$ pattern)
- Excessive memory allocation in hot paths
- Synchronous I/O where async would be appropriate
- Unbounded work that should be paginated or streamed
- Cache misses where caching would clearly help
- Repeated computation that should be memoized

Your method:
1. Read the code you are given.
2. Identify hot paths: code that runs frequently, runs on every request, or runs in tight loops.
3. Apply your areas of focus to those hot paths first. Apply them lightly to cold paths only if something obvious st
4. For each issue, produce a short report with severity, location, description, and recommendation.

What you do not do:
- You do not modify code. You report.
- You do not flag micro-optimizations. A nanosecond saved in a path that runs once a day is not worth a comment.
- You do not pad. If the code looks fine, say so plainly.
- You do not benchmark. Your review is qualitative, based on patterns. The developer can benchmark if she wants quan

End with a one-sentence judgment: clear, minor issues only, or significant issues found.
```

← 上一节

下一节 →

The third specialist is `test-writer`. Its job is to write tests for a given function or module, following the conventions of the project it is operating in.

第三位专员是 `test-writer`。它的职责是:为给定的函数或模块写测试,遵循它当前所在项目的测试规约。

```

name: test-writer
description: Writes test files for a given function, module, or feature, following the test framework and convention
tools: Read, Write, Bash, Glob
model: sonnet

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the ha

Your method:
```

1. Read the file under test and any closely related files needed to understand the code's behavior.
2. Detect the test framework in use by inspecting package.json or the project's equivalent. Default to whatever the
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
  - The happy path
  - At least one boundary condition
  - At least one error or invalid-input case
5. Mock external dependencies. Do not make network calls or write to real files in tests.
6. Place test files alongside the code under test, following the project's existing layout.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in
- You do not write tests that only call the function and assert that it does not throw. That is not a test.
- You do not mock the function under test. You mock its dependencies.
- You do not produce an excess of nearly-identical tests. Three meaningful tests are better than ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

## The Documentation Writer · 文档写作者

The fourth specialist is `documentation-writer`. Its job is to write or update documentation for a given module, API, or feature. The specialist is read- and-write within documentation files only, leaving all code untouched.

第四位专员是 `documentation-writer`。它的职责是:为给定的模块、API 或特性写或更新文档。这位专员只在"文档文件"内可读可写,所有代码一律不碰。

← 上一节

下一节 →

The fifth specialist is `architecture-reviewer`. Its job is to look at proposed changes from a step-back perspective, asking whether the structure of the solution is right rather than whether the implementation is correct.

第五位专员是 `architecture-reviewer`。它的职责是:从"后退一步"的视角看被提议的改动,问"解决方案的结构对不对",而不是"实现正确不正确"。

```

name: architecture-reviewer
description: Reviews code or proposed changes for architectural concerns: separation of responsibilities, layering v
tools: Read, Grep, Glob
model: opus

You are a senior software architect. Your job is to look at code from a step-back perspective: is the structure righ

Your areas of focus:
- Separation of concerns: does each module have a clear, single responsibility?
- Layering: are higher-level modules depending on abstractions, with implementation details pushed downward?
- Coupling: do modules know about each other only through narrow, intentional interfaces?
- Abstraction quality: are abstractions earning their place, or are they obscuring rather than clarifying?
- Consistency: does this code fit with the patterns already established in the project, or does it introduce a new p
```

Your method:

1. Read the code in question.
2. Read enough of the surrounding code to understand the existing patterns and conventions.
3. Apply your areas of focus, prioritizing the ones most relevant to this change.
4. Produce a short report. Lead with the most important architectural observation. Do not dilute it with minor stylistic changes.

What you do not do:

- You do not nitpick. Architectural review is about the shape of the solution, not the details of the implementation.
- You do not suggest sweeping rewrites unless the code clearly cannot be evolved into something acceptable.
- You do not modify code.
- You do not pad. Architectural review is qualitative; one good observation is worth a hundred mediocre ones.

End with a one-sentence judgment: structurally sound, structurally questionable but acceptable, or structurally problematic.

← 上一节

下一节 →

The sixth specialist is `refactor-specialist`. Its job is to take working code and produce a refactor that improves a specific named property, without changing behavior.

第六位专员是 `refactor-specialist`。它的职责是:拿到一段"能跑"的代码,产出一个"针对某项具体属性"的重构——并且不改行为。

```

name: refactor-specialist
description: Produces a refactor of given code that improves a specified property (readability, testability, cohesion)
tools: Read, Edit
model: sonnet

```

You are a senior refactoring engineer. Your job is to produce refactors that make code better along a specific dimension.

Your discipline:

- Behavior preservation is absolute. The refactored code must produce identical observable output for all inputs the original code could produce.
- Improvements are along a single named dimension at a time. If the user asks for readability, you improve readability.
- You explain the change. After producing the refactor, you list what improved and why.

Your method:

1. Read the code carefully.
2. Confirm the dimension you are optimizing for. If unclear, ask the user before proceeding.
3. Identify the highest-leverage opportunity to improve along that dimension.
4. Produce the refactor as a complete replacement of the original, not as a diff.
5. Explain what changed and why, in three to five lines.

What you do not do:

- You do not change behavior. If you notice a bug, you mention it separately and do not silently fix it.
- You do not bundle unrelated improvements into the same refactor.
- You do not introduce new dependencies, frameworks, or libraries unless the user explicitly authorizes them.
- You do not make stylistic changes that do not serve the named dimension.

End with the refactor and the explanation, and nothing else.

The seventh specialist is `dependency-analyzer` . Its job is to inspect the project's dependency manifest and report on any dependencies that are outdated, vulnerable, unused, or duplicated.

第七位专员是 `dependency-analyzer` 。它的职责是:检查项目的依赖清单(manifest),报告"过时、有漏洞、未使用、或重复"的依赖项。

```

name: dependency-analyzer
description: Analyzes the project's dependencies for outdated versions, known vulnerabilities, unused packages, and
tools: Read, Grep, Glob, Bash
model: sonnet

You are a senior dependency hygiene engineer. Your job is to keep a project's dependency surface clean, secure, and

Your method:
1. Read the dependency manifest (package.json, requirements.txt, Cargo.toml, etc.).
2. Read the lockfile if one exists.
3. Identify, in order of priority:
- Dependencies with known security vulnerabilities (run the project's audit tool if available)
- Major-version updates available for actively maintained dependencies
- Dependencies that appear in the manifest but are not imported anywhere in the source
- Duplicate or redundant dependencies (two libraries doing the same job)
- Dependencies with significant licensing concerns
4. Produce a report grouped by category, with each item including the package name, current version, latest version

What you do not do:
- You do not update the manifest. You only report.
- You do not flag every outdated minor version. Focus on major-version drift and security concerns.
- You do not invent vulnerabilities. If your audit tool produces no findings, say so plainly.
- You do not pad. A clean dependency tree gets a one-line judgment, not a pseudo-report.

End with a one-sentence judgment of overall dependency health: clean, minor concerns only, or significant cleanup re
```

The eighth and final specialist is `migration-planner` . Its job is to produce a step-by-step plan for a non-trivial migration, such as moving from one framework to another, upgrading a major dependency, or restructuring a substantial part of a codebase.

第八位、也是最后一位专员是 `migration-planner` 。它的职责是:为一次"非平凡"的迁移产出一份"分步骤"的方案——比如从一个框架换到另一个、升级一个主版本依赖、或重构代码库里的相当一部分。

```

name: migration-planner
description: Produces a step-by-step migration plan for a substantial change such as a framework upgrade, language v
tools: Read, Grep, Glob
model: opus

You are a senior migration engineer. Your job is to break a substantial migration into a sequence of safe, reviewabl

Your principles:
- A good migration plan is a sequence of small, ship-able changes that can each be reviewed and reverted independent
- Every step has a clear acceptance criterion. The team should know when each step is complete and whether it succee
- Risk goes at the front of the plan, not the back. Identify the riskiest part of the migration and propose a small
- Honest plans assume some steps will fail. Include a rollback strategy for each non-trivial step.

Your method:
1. Read the relevant code to understand the current state.
2. Read enough about the target state to understand the destination.
3. Decompose the migration into a numbered sequence of steps. Aim for steps that take a single PR each, no more.
4. For each step, document: what changes, what does not change, the acceptance criterion, the rollback strategy, and
5. Identify the riskiest step and propose an early experiment that reduces the risk before the team commits to the f
6. End with a one-paragraph executive summary suitable for a team meeting.

What you do not do:
- You do not write migration code. You produce the plan.
- You do not commit to specific dates. Effort estimates in person-days are appropriate; calendar dates depend on tea
- You do not pad with general advice about migrations. Your output is specific to this migration.

End with the executive summary.

```

[< 上一节](#)
[下一节 >](#)

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

## Extending the Library · 扩展你的库

A note on extension. The eight specialists above are starting points, not endpoints. As the developer lives with them, she will discover that her work has shapes the eight do not cover, or that one of the eight is slightly off-fit for her specific stack. The fix is the same as for skills: edit the file. Each subagent is a single Markdown file. Editing it takes thirty seconds. Adjusting the role definition, expanding the description, narrowing the tools, refining the boundary, all of these are small changes that the developer can make as her understanding of her own workflow deepens.

关于"扩展"几句话。上面八位专员,同样是"起点而非终点"。开发者真正用起来之后,会发现:她工作里有"八位都没覆盖"的形状,或者八位中有"稍微不太贴合自己那套技术栈"的。修法与 skill 一样——改文件。每个 subagent 就是单个 Markdown 文件,改它也就三十秒:调整角色定义、扩展 description、收窄工具、打磨边界——这些小改动,开发者可以随着对自己工作流理解的加深,慢慢去做。

A second note on adding new specialists. The developer who finds, after a month or two of using the eight specialists, that a recurring kind of work is not covered by any of them, has the raw material for a ninth specialist. The pattern by which she should design it is by now familiar: name the role specifically, narrow the tools to what the role needs, write boundary language that stops the role from drifting, and test the result on three representative inputs. Within an hour, she has a ninth specialist on the team, and her library has grown along the contour of her actual work.

关于"加新专员"再说几句。开发者如果用了一两个月八位专员之后,发现有一类反复出现的工作不在它们的覆盖范围内——那她就有了"第九位专员"的原材料。设计它的套路到这时已经熟:具体地命名角色、把工具收到它"恰好需要"、写一段边界语言防止它漂移、并在三个有代表性的输入上测试。一小时之内,她就有了第九位专员,库也沿着"自己实际工作"的轮廓继续生长。

← 上一节

下一节 →

A third note on team-level subagents. Several of the specialists above will be useful to other developers in the reader's team or company. The mechanism for sharing is the same as for skills: copy the subagent file into the project's `.claude/agents` directory and commit it. From that point forward, every developer who clones the project receives the specialist automatically. This is how subagent libraries spread within organizations, and how a thoughtful library built by one developer becomes a standing resource for an entire team. The library committed to a project repository is a quiet but real form of leverage. It travels with the code, it activates on the first session, and it raises the floor of what every developer working in that project can produce.

关于"团队级 subagent"再说几句。上面这些专员里,有几位对读者所在团队或公司的其他开发者也会很有用。分享机制与 skill 一样:把 subagent 文件拷进项目的 `.claude/agents`,提交进版本库。从那之后,凡是 clone 这个项目的开发者,都会自动拿到这位专员。subagent 库在组织内就是以这种方式传播的——"一位开发者精心搭出的库"会变成"整支团队的一份常驻资源"。提交进项目仓库的 subagent 库,是一种静默但真实的杠杆:它随代码一起走,在第一个 session 就激活,会拉高"在那个项目里工作的每一位开发者"所能产生的底线。

The chapter that follows is the climax of Part Three. With skills installed and subagents installed, the question becomes how the two systems compose. Subagents can use skills. Skills can be designed to fit specific subagents. The combination is where the architectural depth of Claude Code becomes most visible, and the next chapter is dedicated entirely to it. The reader who has installed both libraries by the end of Part Two and Part Three is now in possession of the components for a substantial leap in capability. The next chapter shows how to assemble those components into the integrated system that has been the destination of the book all along.

下一章是第三部分的高潮。Skill 装好、subagent 也装好之后,问题就变成了"这两套系统怎么组合在一起"。Subagent 可以使用 skill;skill 也可以被设计来"配"某个 subagent。两者的组合,正是 Claude Code 架构深度最显形的地方——下一章就专门讲这个。等第二、第三部分都读完,两个库都装上的读者,就已经手握足以"让能力跃迁一档"的组件;下一章,就是把它们组装成"这本整书一直通向的那座集成系统"。

A note on extension. The eight specialists above are starting points, not endpoints. As the developer lives with them, she will discover that her work has shapes the eight do not cover, or that one of the eight is slightly off-fit for her specific stack. The fix is the same as for skills: edit the file. Each subagent is a single Markdown file. Editing it takes thirty seconds. Adjusting the role definition, expanding the description, narrowing the tools, refining the boundary, all of these are small changes that the developer can make as her understanding of her own workflow deepens.

关于"扩展"几句话。上面八位专员,同样是"起点而非终点"。开发者真正用起来之后,会发现:她工作里有"八位都没覆盖"的形状,或者八位中有"稍微不太贴合自己那套技术栈"的。修法与 skill 一样——改文件。每个 subagent 就是单个 Markdown 文件,改它也就三十秒:调整角色定义、扩展 description、收窄工具、打磨边界——这些小改动,开发者可以随着对自己工作流理解的加深,慢慢去做。

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

到这里,你的 subagent 库就有八位专员了。装齐了八位的读者,大约花了一小时的工作,就已经搭起了一支"小而能干"的团队——只要工作触碰到它们的专长领域,随叫随到。这支团队有耐心,不会累,不开会;每位专员都精于一件事,并对其余的事坦荡地沉默。开发者的角色,是编辑——把对路的专员派去对路的工作上,把它们的产出综合成"她真实生活里需要"的统一形态。

## CHAPTER FOURTEEN · 第十四章

### Subagents and Skills: The Composition Pattern

### 子代理与技能:组合模式

*"The whole, when properly composed, contains capabilities that none of its parts possess in isolation. This is the meaning of architecture."*

— attributed to a systems engineer, anonymous

"整体,当被恰当地组合起来,会拥有"任一部分单独都不具备"的能力。这就是"架构"这个词的含义。"

—— 归于一位系统工程师(佚名)

This chapter is where the two pillars built so far in the book begin to reinforce each other in a way that produces capabilities neither could produce alone. Skills, the subject of Part Two, taught Claude how to do specific tasks consistently. Subagents, the subject of Part Three, gave Claude a team of specialists, each in its own context. The composition of the two is the architectural depth that the book has been building toward, and this chapter teaches it explicitly.

本章,书里目前搭起的两根支柱开始彼此加固,以一种"任何一根独自都做不到"的方式产出能力。第二部分讲的 Skills,教的是 Claude 如何一致地完成具体任务;第三部分讲的 Subagents,给了 Claude 一支"各自独立、各有上下文"的专员团队。两者的组合,正是这本书一直在朝向的"架构深度"——本章把这件事明白地讲出来。

Begin with the basic mechanic. Skills are available everywhere. They are loaded at the level of the Claude Code installation, and any Claude process running in that installation, including any subagent, can activate any applicable skill. A subagent does not need a special configuration to access skills; it simply needs to have a description that matches the kind of work where a skill would naturally fire. The composition is not bolted on. It is the default behavior of the two systems when they are present together, and the developer's job is to design with the composition in mind from the start.

先说最基本的机制。Skill 是"处处可用"的——它们在 Claude Code 安装层加载,任何跑在该安装里的 Claude 进程(包括任何 subagent)都可以激活任何适用的 skill。subagent 要访问 skill 不需要任何特殊配置;它只需要让自己的 description 与"某个 skill 自然会触发的"那类工作相匹配。这种组合不是"外挂"上去的——它是"两套系统同时存在"时的默认行为;开发者的任务是:从一开始就把"组合"放进设计的考虑里。

A worked example illustrates the power. The developer has a `test-writer` subagent installed, presented in the previous chapter. The developer also has a project-specific skill called `test-conventions`, which encodes how tests are organized and named in this particular codebase, including the choice of test framework, the file layout, the mocking style, the use of fixtures, and the team's preferred patterns for assertion phrasing. The `test-conventions` skill lives in the project's `.claude/skills` directory; the `test-writer` subagent lives in the user's `~/.claude/agents` directory.

一个具体例子能说明这种力量。开发者装好了一个 `test-writer` subagent(上一章给出过)。她还装了一个项目级 skill,叫 `test-conventions` ——它把"这个特定代码库里测试是怎么组织、命名的"编码进去,包括:测试框架的选择、文件布局、mock 风格、fixture 的使用方式、团队偏好的断言措辞。`test-conventions` skill 放在项目的 `.claude/skills` 里; `test-writer` subagent 放在用户的 `~/.claude/agents` 里。

Now imagine the developer dispatches the test-writer subagent on a new function. The subagent receives its brief: write tests for this function. The subagent begins its work. It reads the function. As it considers the structure of the tests it is about to write, it recognizes that the request matches the description of the `test-conventions` skill, and it loads that skill into its own context. The subagent now has, simultaneously: its own role as a senior test engineer, its own focused tools and isolated context, and the project-specific knowledge encoded in the skill. The tests it produces are written in the project's exact conventions, by a specialist who knows how to write good tests in general, in a context that is not polluted by anything outside the test-writing task.

现在想象:开发者把 `test-writer` subagent 派去处理一个新函数。subagent 收到 brief:为这个函数写测试。subagent 开始工作,读了函数。当它琢磨"接下来要写的测试是什么结构"时,识别出这个请求匹配 `test-conventions` skill 的 description,于是把那个 skill 加载进了自己的上下文。此时的 subagent,身上同时有三样东西:它自己"高级测试工程师"的角色、自己聚焦的工具与隔离的上下文、以及 skill 里编码的项目特定知识。它产出的测试——既严格遵循项目规约,又出自一位"在一般意义上知道怎么写好测试"的专员之手,而且跑在一个"没有被测试任务之外的任何事污染"的上下文里。

## Composition in Action · 组合在行动

The result is strikingly better than what either component could produce alone. A skill alone, fired in the main session, would shape the output toward project conventions, but the output would be subject to context pollution from whatever else the main session was working on. A subagent alone, dispatched on the same task, would produce well-isolated, generally good tests, but it would not know the project's specific conventions and would have to guess. The combination has both: convention-correct tests, produced in a clean context, by a specialist who treats the work seriously. This is the composition pattern in its simplest form, and once a developer sees it work even once, she will not want to design subagents that do not know how to use skills.

这种结果,比"任何一方单独能产出的"都明显要好。Skill 单独、在主 session 里触发,确实会让输出向项目规约靠拢,但那份输出仍会被主 session 里其他事情的"上下文污染"影响。Subagent 单独、被派去做同一件事,确实能产出"隔离良好、整体不错"的测试,但它并不知道这个项目的具体规约,只能猜。两者的组合*同时拥有*这两点:规约准确的测试、由"把工作当回事"的专员、在干净的上下文里产出。这就是组合模式最简单的形态;一旦开发者哪怕只看它工作一次,她就再也不想设计"不懂用 skill"的 subagent 了。

There is a design principle that follows from this and that should guide every subagent the developer writes from now on. Subagents should be designed as roles, not as repositories of project knowledge. A test-writer subagent should know how to be a senior test engineer in general; it should not know that the project uses Vitest rather than Jest, or that mocks should be placed in a specific directory, or that the team prefers a specific assertion library. Those are project facts, and they belong in skills. The subagent provides the role; the skill provides the project-specific knowledge. When the role meets the project, the composition produces the right behavior.

由此引出一条设计原则,它应该从今往后指导开发者写的每一个 subagent:subagent 应该被设计成角色,而不是"项目知识的容器"。一个 test-writer subagent 应当知道"如何作为一位高级测试工程师",而不应当知道:这个项目用的是 Vitest 而不是 Jest、mock 要放在某个特定目录、团队偏好某个特定的断言库。这些是*项目事实*,属于 skill。subagent 提供角色,skill 提供*项目特定的知识*。当角色与项目相遇,组合就产出正确的行为。

This principle has a second-order effect that is worth naming. A subagent that contains project-specific knowledge becomes specific to that project, and its usefulness drops as soon as the developer moves to a different project. A subagent that contains only the role, with project-specific knowledge factored into skills, is portable. The same test-writer subagent works in five different projects, because the role is the same in each, and the project-specific conventions are loaded as skills from each project's own `.claude/skills` directory. A small library of well-designed, role-only subagents at the user level, combined with project-specific skills in each project, gives the developer the best of both worlds: stable, portable specialists, and project-appropriate behavior in every project she works in.

这条原则有一个值得点名的"二阶效应"。一个把项目特定知识塞在自己体内的 subagent,会变得"只适用于那个项目";开发者一切换到另一个项目,它的价值就立刻下降。一个只装"角色"、把项目特定知识抽到 skill 里的 subagent,才是*可移植*的。同一份 test-writer subagent,可以在五个不同的项目里都工作得好——因为"角色"在每个项目里都一样,项目特定的规约由"每个项目自己的 `.claude/skills` 目录"以 skill 形态加载进来。一套小而精、只装"角色"的 subagent 库放在用户级,再加上每个项目里"项目级"skill——开发者就同时拿到了两样最好的东西:稳定、可移植的专员,以及"在每个项目里都符合项目实际"的行为。

A second worked example raises the composition to a more sophisticated level. The developer has installed the security-auditor subagent from Chapter Eleven, the architecture-reviewer subagent from Chapter Thirteen, and a project-specific skill called `auth-conventions`, which encodes the codebase's authentication patterns, token handling, and security-sensitive practices. The skill is not project-specific knowledge that the security-auditor needs to do its job in general; the security-auditor knows about OWASP Top 10 and common vulnerability patterns regardless of project. But the skill encodes the way this specific codebase chooses to handle authentication, which is information the security-auditor needs in order to flag deviations from the established pattern as concerns rather than treating them as ordinary alternatives.

第二个具体例子,把组合抬到一个更复杂的层次。开发者装好了第十一章的 `security-auditor` subagent、第十三章的 `architecture-reviewer` subagent,以及一个项目级 skill——`auth-conventions`,把"代码库的认证模式、token 处理、与安全敏感的做法"编码进去。这个 skill 并不是"security-auditor 干它那件一般意义上的事"所必需的项目特定知识——security-auditor 知道 OWASP Top 10、知道常见漏洞模式,跟项目无关;但这个 skill 把"这个特定代码库选择怎么处理认证"编码进去——这正是 security-auditor 需要的:它要把"偏离既定模式"标记成"可疑",而不是当成"普通的备选方案"放过。

When the developer dispatches the security-auditor on a change that touches authentication, two things happen simultaneously. The subagent activates its general security expertise. The subagent also loads the `auth-conventions` skill, because the description matches. The subagent now has both: the general security knowledge that says hardcoded credentials are bad, and the specific project knowledge that says this codebase uses a token-based scheme with refresh rotation handled by a particular module. The audit it produces is doubly informed. It catches general security issues. It also catches deviations from the project's specific patterns, which a generalist would not have known were patterns at all.

当开发者把 security-auditor 派去处理"涉及认证的改动"时,有两件事同时发生:subagent 激活它一般的安全专业知识;同时,因为 description 匹配,subagent 也会加载 `auth-conventions` skill。此时的 subagent 身上,既有"硬编码凭据是不好的"这种一般安全知识,也有"这个代码库使用基于 token 的方案、refresh 轮换由某个特定模块负责"这种项目特定知识。它产出的审计,被双重"喂饱":既能抓出一般安全问题,也能抓出"偏离项目特定模式"的偏差——后者,通才型评审者甚至根本不会知道"原来这里还有种模式"。

## An Advanced Composition Pattern · 一种更进阶的组合模式

There is a more advanced composition pattern worth introducing here, although its full development belongs in Part Five. The orchestrator pattern from Chapter Twelve, in which the main session dispatches several subagents in parallel and synthesizes their outputs, can be combined with skill composition. Each parallel subagent can activate its own appropriate skills. A four-way code review dispatched as four parallel subagents, each of which knows how to use the relevant project skills, produces a deeper, more project-aware review than a four-way review in which the subagents operate in their own generic roles without skill support. The combinations compose, and the composition of compositions is where the architecture of Claude Code reaches its most expressive form.

这里值得引入一种更进阶的组合模式——尽管它完整的展开属于第五部分。第十二章的"编排者模式"——主 session 并行派若干 subagent、再综合它们的输出——可以和 skill 组合再组合:每一个并行 subagent 都可以激活它各自合适的 skill。一次"四路"代码评审,作为四个并行 subagent 被派出,且每个 subagent 都知道怎么用相关的项目 skill——它产出的评审,比"四个 subagent 各自跑在通用角色里、没有任何 skill 支撑"的那种四路评审更深、更懂项目。组合之上再组合,正是 Claude Code 的架构抵达其最有表现力形态的地方。

more project-aware review than any single channel could produce. The four-by-four matrix, four subagents and four skills, is the structure of a serious engineering review process, and the composition pattern of this chapter is what makes it a single concept rather than two unrelated ones.

远超任何单一渠道所能产出的"对项目的理解"。这个 4x4 矩阵——四个 subagent 配四个 skill——是真正严肃的工程评审流程的结构;而本章所讲的组合模式,正是把"两个不相关的东西"捏成"同一个概念"的关键。

A practical caveat. Not every subagent benefits from skill composition, and the developer should not over-engineer the design. Some specialists are useful as pure role containers and should not be paired with skills at all. The migration-planner subagent from the previous chapter, for example, produces a generic migration plan that is shaped more by the specifics of the migration than by ambient project conventions. There is little benefit in trying to load skills into it. The composition pattern is a tool, not a doctrine. The developer should reach for it when it produces real value and leave it alone when it would only complicate a workflow that does not need complication.

一个务实的提醒:并不是每个 subagent 都受益于 skill 组合,开发者也不该把设计过度工程化。有些"专家"适合作为纯粹的角色容器存在,根本不该和 skill 配对。比如上一章那个 migration-planner subagent,它产出的迁移计划,主要由"这次迁移本身的具体情况"塑造,而非由"项目环境里那些约定俗成"来定形;硬往里塞 skill 几乎没什么收益。"组合模式"是一个工具,不是教条——它能真正产出价值时再用,只会让本不需要复杂化的流程变复杂时,就别碰。

A second practical caveat concerns description engineering. For the composition to fire correctly, the skill descriptions and the subagent descriptions both need to be written well. The subagent description determines whether the subagent is dispatched. The skill description determines whether the skill activates inside the dispatched subagent. Both layers of triggering need to work for the composition to produce its intended behavior. The same disciplines from Chapter Six apply here: canonical statement, trigger language, boundary, input expectation. The developer who has internalized those disciplines can debug failed compositions quickly. The developer who has not will sometimes find that the composition does not work and will not be sure why.

第二条务实提醒,关于 description 的工程化。要让组合正确触发,skill 的 description 和 subagent 的 description 都得写好。Subagent 的 description 决定它会不会被派出去;skill 的 description 决定它会不会在那个被派出去的 subagent 内部被激活。两层触发都得正常,组合才会产出预期的行为。第六章里那套规矩在这里同样适用:规范化的陈述、触发语、边界、输入预期。已经把那套规矩内化了的开发者,排错时能很快找到失败的组合;没有内化的开发者,有时会面对"组合不工作、却说不清为什么"的窘境。

A third worked example will deepen the pattern. The developer has installed the documentation-writer subagent and a project-specific skill called doc-style-guide, which encodes the team's conventions for technical writing: the preferred section structure, the rules about voice and tense, the format of code examples, and the policy on what level of detail is appropriate for different audiences. The skill is project-specific because every team has its own documentation style, and a documentation-writer subagent that wrote in some generic style would produce documentation that did not fit the team's existing corpus.

第三个完整例子,把模式讲得更透。开发者已装好 documentation-writer subagent,以及一个项目级的 skill,叫 doc-style-guide ——它把团队在技术写作上的全部规约都编码在内:偏好的章节结构、关于"语气/时态"的规则、代码示例的格式、对不同受众该铺多少细节的策略。这个 skill 是项目级的,因为每个团队的文档风格都不一样;一个 documentation-writer subagent 如果产出的是泛泛的"通用风格",那么写出来的文档将和团队既有的整套文档语料不搭。

The composition produces a documentation-writer that is doubly informed. The subagent provides the role: a senior technical writer who knows how to explain code clearly, how to choose examples, and how to keep documentation honest. The skill provides the project conventions: section ordering, voice, tense, example format. When the developer dispatches the subagent on a new module that needs documenting, both fire together. The output reads like good technical writing in the team's house style, because both layers of expertise are present at once. The developer did not have to brief the subagent on the team's conventions, because the skill brought them automatically. She did not have to teach the skill how to write good prose, because the subagent provided that. Each system did its job, and the composition produced output better than either alone.

这一组合,产出的是"双重知情"的 documentation-writer。Subagent 给出"角色":一位资深技术作者,懂得如何把代码讲清楚、如何挑选例子、如何保证文档的诚实。Skill 给出"项目规约":章节顺序、语气、时态、例子格式。当开发者把 subagent 派去写一个待文档化的新模块时,两层同时点火。产出的文字读起来就像团队自家风格里的"好的技术写作"——因为两层专长同时在场。开发者不必再向 subagent 简报团队的规约,skill 自动带来了;也不必教 skill 怎么写出好散文,subagent 那头就准备好了。各自做各自该做的事,而组合的产出,胜过任一者孤军作战的结果。

## The Architectural Principle · 架构原则

There is a deeper architectural principle visible here, which is worth naming explicitly because it will guide every later chapter of the book. The composition pattern is an instance of the broader principle of separation of concerns. Skills hold project knowledge. Subagents hold roles and isolate contexts. Neither system tries to do the other's job. When a workflow needs both kinds of capability, the systems compose, and the composition produces capabilities that neither alone could provide. This is the same principle that underlies every well-architected piece of software, applied to the design of a Claude Code environment. Developers familiar with separation of concerns in their own code will recognize the principle immediately and find the composition pattern intuitive. Developers unfamiliar with it will, by working through this part of the book, internalize the principle by seeing it applied to a domain where its benefits are immediate and visible.

这里其实浮现出一条更深的架构原则,值得显式点出来——因为它会指导本书之后每一章。"组合模式"其实是更广义的"关注点分离"原则的一个实例。Skill 持有"项目知识";subagent 持有"角色"并隔离上下文。任一系统都不试图去做对方的工作。当一个 workflow 同时需要这两种能力时,系统就组合起来,而组合的产出,又超出它们各自能独立提供的上限。这与"每一段良好架构的软件"所遵循的是同一条原则,只是被搬到了 Claude Code 环境的设计上来。熟悉"关注点分离"这条原则的开发者,会立刻认出它,并觉得组合模式很自然;不熟悉的开发者,只要走完本书这一部分,也就会在"一个收益立竿见影、看得见摸得着"的领域里把它内化下来。

A practical implication of this principle is the design discipline for adding new capabilities to a Claude Code environment. When the developer notices a recurring kind of work that her current configuration handles poorly, she should ask, is the gap a knowledge gap or a role gap. If she finds herself wishing the environment knew something specific, that is a knowledge gap, and the answer is a skill. If she finds herself wishing a different kind of attention were being applied to a kind of work, that is a role gap, and the answer is a subagent. Sometimes the gap is both, and the

这条原则的一条实务含义,是给 Claude Code 环境"加新能力"时的设计纪律。当开发者注意到有一类反复出现的工作、当前的配置处理得不好时,她应当先问一句:这道缺口,究竟是"知识缺口"还是"角色缺口"?如果她希望"环境知道某件具体的事"——那是知识缺口,答案是写一个 skill。如果她希望"在处理某类工作时,投入的是另一种不同的注意力"——那是角色缺口,答案是写一个 subagent。有时候这道缺口两边都站,那

answer is a composition. Asking the right question first prevents the developer from building skills that should have been subagents, or subagents that should have been skills, both of which are common mistakes that produce environments that work but feel awkward. Asking the right question consistently produces environments that fit the developer's work cleanly, because each capability lives in the right place.

答案是“组合”。先问对问题,就能避免那种“本该写 skill 却被写成了 subagent”或“本该写 subagent 却被写成了 skill”的常见错误——这两种错,都能让环境跑得起来,但用起来总是别扭。每次都问对,就能让每一项能力都“住”在它最该住的地方,环境就能干净地贴合开发者的实际工作。

A second design discipline that emerges from the composition pattern concerns versioning. Skills and subagents that are designed to compose with each other should evolve together. If the developer changes the conventions encoded in a skill, the subagents that depend on that skill may need to be checked to ensure their behavior still aligns. If the developer reshapes a subagent's role, the skills designed to fit that role may need to be revisited. This coupling is loose, not tight, and the systems remain independently functional, but the composition is most reliable when the developer treats related skills and subagents as parts of a single design rather than as unrelated artifacts. The discipline is to version-control the entire `.claude` directory, to commit related changes together, and to review skill and subagent changes side by side when they are likely to affect each other.

从组合模式里浮现出来的第二条设计纪律,关于版本管理。设计上要彼此组合的 skill 与 subagent,应当“共同演进”。如果开发者改了某条 skill 里编码的规约,那些依赖它的 subagent 可能要回头检查一下,看自身行为是否还对得上;如果她重塑了某个 subagent 的角色,那些为该角色“贴身打造”的 skill 也可能需要再审视。这种耦合是“松”而不是“紧”的,系统各自依旧独立可用;但是,当开发者把“相互关联的 skill 与 subagent”当作同一个设计里的若干部件、而不是彼此不相关的零散制品时,组合才最可靠。具体纪律是:把整个 `.claude` 目录纳入版本控制、把相关变更一起提交,在它们可能彼此影响时,把 skill 与 subagent 的变更并排做评审。

One more reflection on the experience of the composition pattern, before this chapter closes. The first time a developer sees a skill activate inside a subagent and watches the combined behavior produce something neither could have produced alone, the experience tends to be memorable. There is a moment of recognition in which the architecture stops feeling abstract and starts feeling like a real engineering substrate. The skill library and the subagent library, which until then had felt like two separate collections of useful artifacts, become something larger. They become the components of an environment, and the environment becomes the place where the developer's real work happens. This shift in perception is not just aesthetic. It is the shift from using Claude Code to engineering with Claude Code, which is the central promise of this book and the moment the entire framework starts paying back the investment of learning it.

在收束本章之前,再就“组合模式的体验”做一点感想。开发者第一次亲眼看见“一个 skill 在某个 subagent 内部被激活”,并看着两者合起来的行为产出了“任何一方单独都产不出”的东西时,这份体验往往是令人难忘的。会有一刻“认出”——架构不再显得抽象,而是开始像一块真正可工程化的底材。原本感觉像“两套互不相干的实用制品”的 skill 库与 subagent 库,忽然变成了某种更大的东西:它们成为一块“环境”的组件;而这块环境,反过来成为开发者真正的工作“发生的地方”。这种认知上的迁移,不只是审美层面的——它正是“从用 Claude Code”到“用 Claude Code 做工程化”的跃迁。这正是本书的核心承诺,也是整副框架开始回报“为学会它所付出的投入”的那一时刻。

A reflection on the architecture, before this chapter and this part of the book end. The design choices that made skills and subagents into separate systems, rather than merging them into a single mechanism, were deliberate. Skills are extensible knowledge. Subagents are specialized roles

在收束本章、也收束本书这一部分之前,再来对架构本身做一点反思。把 skill 与 subagent 设计为两套独立系统、而不是合并成一套机制,是经过深思熟虑的。Skill 是"可扩展的知识";subagent 是"专门化的角色",

with isolated contexts. The two are different kinds of thing, and treating them as different kinds of thing keeps the design clean. A merged system would have been simpler at first glance and worse in practice. The cost of having two systems is the small overhead of learning both. The benefit of having two systems is that each remains focused on what it does best, and the composition between them is principled rather than tangled. The reader who has worked through the last six chapters now has both systems in working memory and in installed form, and she has seen how they reinforce each other to produce capabilities neither could produce alone.

并配以隔离的上下文。两者本质上是不同种类的"东西",把它们当作不同种类的"东西"来对待,设计才能保持清爽。合并的系统初看更简洁,实操起来却更糟。维持两套系统的代价,是同时学两套系统那点小小开销;收益则是,每套系统都聚焦在自己最擅长的事上,二者之间的组合是有章法的、而非缠成一团。走过过去六章的读者,如今两套系统都在手上、也都在已安装的形态里,也已经见过它们如何彼此增益,产出单靠任一者都达不到的能力。

### Second-Order Benefits · 二阶收益

There is a second-order benefit to the composition pattern that becomes visible only after a developer has lived with a composed library for a few months. The benefit is that the library becomes self-documenting in a way that monolithic configurations are not. A new developer joining the project can read the `.claude/skills` directory to learn the project's conventions; reading the `SKILL.md` files is essentially reading a structured guide to how the team prefers to work. The same new developer can read the `.claude/agents` directory to learn what specialist roles the team treats as standing capabilities; reading the subagent definitions is essentially reading the team's job descriptions for its AI-assisted workflow. Together, the two directories form a piece of working documentation that is more accurate than any wiki, because the documentation is the configuration the tool actually uses. This self-documenting property is a substantial benefit for any team that grows, because it lowers the cost of onboarding new members and reduces the gap between what the team says it does and what the team actually does.

组合模式还有一条"二阶收益",只在开发者与组合库相伴共处几个月之后才看得清:它让这套库具备一种"自我文档化"的性质,而单体化的配置做不到这一点。新加入项目的开发者,读一读 `.claude/skills` 目录就能学懂项目规约——读那些 `SKILL.md` 文件,本质上就是在读一份"团队偏好如何工作"的结构化指南。同一位新人,再读读 `.claude/agents` 目录,就能了解团队把哪些"专家角色"当作常驻能力——读那些 subagent 定义,本质上就是在读"团队 AI 辅助工作流"里的岗位说明书。两个目录合起来,就构成了一份"比任何 wiki 都更准确"的工作文档——因为文档本身,就是工具真正在用的那份配置。对任何会成长的团队,这种自我文档化的属性都是一笔不小的收益:它降低了新成员上手的成本,也缩短了"团队声称在做的事"与"团队实际在做的事"之间的鸿沟。

A third reflection concerns the relationship between this composition and the third pillar of the framework, hooks, which is the subject of the part of the book that follows. The composition of skills and subagents that this chapter has taught is internal to the agent loop. Both skills and subagents operate within the probabilistic space of the language model. They shape what the model does, but they do not impose deterministic constraints on it. Hooks are different. Hooks live outside the loop and impose deterministic constraints on it. The fully engineered Claude Code environment that the book is building toward includes all three pillars working together: skills

第三点反思,关于"本章所讲的组合"与"框架第三根支柱"——hooks——之间的关系,hooks 是本书下一部分的主题。本章所教的"skill + subagent 组合",发生于 agent loop 内部;skill 与 subagent 都在语言模型的概率性空间里运作,它们塑造模型的行为,但并不对它施加硬

性约束。Hooks 则是另一回事——它位于 loop 之外,会对 loop 施加确定性约束。本书一路要带读者搭建出来的那块"完整工程化的 Claude Code 环境",靠的是三根支柱协同工作:skill

providing capability, subagents providing specialization, hooks providing the deterministic edges that make the whole environment safe and reliable to depend on. The reader who has finished this chapter has half of the architecture in place. The next part of the book completes it.

提供"能力",subagent 提供"专门化",hooks 则提供"确定性边缘"——正是它让整块环境可以放心依赖、安全可信。读到这里,读者手里的架构已经搭好了一半;下一部分书,把它补完。

A short observation is worth recording about the experience of working in a fully composed environment, although the reader will only verify the observation for herself once she has built one. Developers report that the felt difference between Claude Code without a configured environment and Claude Code with a thoughtful skills-plus-subagents composition is larger than they expected. The unconfigured tool feels capable but generic. The configured tool feels specific to the developer's work in a way that is initially surprising. Tasks that used to require explicit briefing now run themselves. Reviews that used to take ten minutes finish in three. The output that used to need editing for tone and convention arrives already in the right voice. The whole working day reshapes itself around the new floor of capability, and the developer's attention rises to fill the new space the environment has created. This is the practical payoff of the composition pattern, and it is the payoff that justifies the investment of time in building skill and subagent libraries thoughtfully rather than casually.

关于"在一块完全组合好的环境里工作"的体验,有一点观察值得先记下来——但读者要等自己真的搭出一套,才能亲自验证。开发者普遍反映:"未做配置的 Claude Code"和"做了深思熟虑的 skill+subagent 组合的 Claude Code"之间的体感差距,比他们预想的要大。未配置的工具"有能耐,但泛";配置过的工具"贴着开发者的具体工作而来",那份"贴",一开始甚至有点令人惊讶。以前需要专门简报才能跑起来的任务,如今自己就跑完了;以前要花十分钟的评审,现在三分钟就收;以前总要回头改语气和规约的产出,如今一上来就是对的噪音。整个工作日,会围绕这块"新的能力底线"重新排布,开发者的注意力随之向上抬,去填满环境新腾出来的空间。这就是组合模式在实务层面的回报——也是"花时间认真搭一套 skill 库与 subagent 库"这件事之所以值得的根本原因。

Part Three ends here. The reader has, over five chapters, learned the architecture of subagents, mastered the design of single specialists, internalized the orchestrator pattern that turns sequential work into parallel work, installed a library of eight working specialists, and seen how skills and subagents compose to produce the deeper architectural power of the system. The second pillar of the framework is now in place. The third pillar, hooks, is the subject of the next part of the book. Hooks are how the probabilistic system gets wrapped in deterministic edges, how the developer ensures that the things that must happen always happen, and how the boundaries of the engineered environment become unbreakable at the points where breakage would matter most. That work begins in the chapter ahead.

第三部分至此收束。读者用五章的时间,学会了 subagent 的架构、掌握了单个专家的设计、把"把串行变并行"的编排器模式内化下来,装好了一个含八位"能干活"的专家的库,也亲眼见到了 skill 与 subagent 是如何组合成系统更深层架构力量的。框架的第二根支柱,至此就位。第三根支柱——hooks——是本书下一部分的主题:它正是"用确定性的边缘去包裹一个概率性的系统"的那条路径,正是"必须发生的事永远会发生"那条工程化保障,也正是"工程化环境的边界,在最不该破的地方变得不可破"那条具体落地方式。这件事,由下一章开始做。

Before turning the page, a short closing thought. Each part of this book has built one layer of the larger architecture. Part One established the mental model. Part Two installed the first pillar. Part Three has installed the second pillar and shown how the first two pillars compose. The

pattern of work that the book asks of the reader is, in this respect, accumulating. Each part builds on what came before, and each successive layer makes the

翻页之前,还有一句简短的收束语。本书的每一部分,都搭建大架构中的一层:第一部分搭起心智模型;第二部分装上第一根支柱;第三部分装上第二根支柱,并展示了前两根如何组合。从这个意义上,本书要求读者投入的工作,是"累积"的——每一部分都建在前一部分之上,而后续每一层,都会让前一层

previous layers more powerful by giving them a richer environment to operate in. The hooks that the next part introduces will, in turn, make the skills and subagents already installed more reliable, because the deterministic guarantees that hooks provide will protect the probabilistic systems below them from the failure modes those probabilistic systems cannot prevent on their own. The architecture is genuinely hierarchical, and the experience of building the upper layers is, over time, the experience of giving the lower layers a sturdier foundation to stand on.

变得更强大,因为它给了它们一个更富的环境去运行。下一部分要引入的 hook,反过来又会让已经装好的 skill 与 subagent 更加可靠——因为 hook 所提供的那些"确定性保证",会把下层那些"概率性系统"无法独自防住的失效模式,在它发生之前就拦下来。整套架构是真正分层的;而搭上层这件事,日积月累,就是把下层的底座越筑越稳的那份体验。

PART FOUR · 第四部分

Hooks: Claude's Deterministic Enforcement System · Hooks:Claude 的确定性执行体系

The probabilistic model needs deterministic edges. Hooks are how the edges get built.

概率性模型需要确定性边缘。Hooks,就是构筑这些边缘的途径。

CHAPTER FIFTEEN · 第十五章

Hooks Fundamentals: Deterministic Behavior in a Probabilistic System · Hooks 基础:概率性系统中的确定性行为

"Trust, but verify. The verification is what makes the trust possible."

— attributed to Ronald Reagan, who borrowed it from a Russian proverb

"信任,但要核实。核实,才让信任成为可能。"

—— 通常归于罗纳德·里根(他借自一则俄罗斯谚语)

← 上一页

下一节 →

Skills and subagents, the two pillars covered so far, share a fundamental property. Both operate inside the probabilistic space of the language model. Both shape what Claude does, but neither imposes a hard constraint on what Claude must or must not do. A skill, however well written, instructs Claude through prose; the model interprets the prose and behaves accordingly, but the model retains the freedom to interpret. A subagent runs in its own context, but it is still a language model, and its behavior is still shaped by probability rather than by mechanical rule.

此前已经讲过的两根支柱——skill 与 subagent——共享一个根本属性:两者都跑在语言模型的"概率性空间"内部。两者都在塑造 Claude 的行为,但都没有对"Claude 必须做、不得做"施加硬约束。一条 skill 写得再好,也只是用散文指引 Claude;模型在解读这些散文之后再按解读行事,而解读的自由仍归模型所有。一个 subagent 跑在它自己的上下文里,但它仍是一个语言模型,其行为仍由概率塑造,而非由机械规则塑造。

For most of the work a developer asks Claude Code to perform, this probabilistic shaping is exactly right. The flexibility of the model is what allows it to handle the open-ended variety of real requests. But there is a class of requirements where probabilistic shaping is insufficient. The class is small but important, and the requirements in it share a common shape: the failure rate must be zero. Tests must run before commits. Secrets must never reach disk. Production configuration must never be edited without a human in the loop. Linters must run after every code edit. For these requirements, no amount of careful prompting is good enough, because careful prompting can fail two percent of the time and the requirement does not tolerate two percent failure.

对开发者请 Claude Code 做的大多数工作而言,这种"概率性塑造"恰到好处。模型的灵活性,正是它能应对真实请求之"开放性、多样性"的原因。但仍有一类需求,是概率性塑造兜不住的。这类需求很小,却至关重要——其中的需求都共享同一种形态:失效概率必须为 0。提交前必须先跑测试;秘密数据绝不许落盘;生产配置绝不能在"人在回路"的情况下被改;每次代码编辑后都必须跑 linter。对于这一类需求,再小心的 prompt 也不够用,因为再小心的 prompt 也会有 2% 的概率失败,而这类需求不容 2% 的失败。

This chapter introduces the third pillar of the framework, which exists precisely to handle this class of requirement. Hooks are user-defined shell commands that Claude Code executes automatically at specific moments in its lifecycle. They run outside the language model. They cannot be talked out of running by clever prompts. They cannot be forgotten by Claude. They run, and their result either allows the lifecycle to continue or interrupts it. The discipline of using hooks well is the discipline of identifying which requirements need this kind of guarantee and then encoding the guarantee in shell commands that Claude Code will honor whether the model wants to or not.

本章要介绍的,是框架的第三根支柱——它存在的全部理由,就是去应对这一类需求。Hooks,就是由用户定义的 shell 命令,由 Claude Code 在其生命周期的特定时刻自动执行。它们跑在语言模型之外,无法被"巧妙的 prompt"说服不去跑,也不会被 Claude"忘记"。它们会跑——而其结果,要么让生命周期继续,要么把它打断。把 hook 用好这门功夫,关键在于两件事:一是识别"哪些需求需要这种保证",二是把这种保证编码成 shell 命令,让 Claude Code 无论模型想不想,都得照办。

Begin with the mechanics. A hook is configured in a settings file. The most common location is the project's settings file, which lives at `.claude/settings.json` in the project root. User-level hooks, which apply to every project, live at `~/.claude/settings.json`. The settings file is JSON, and the hooks live under a top-level `hooks` key. Each hook entry specifies an event, an optional matcher that filters which tool calls the hook applies to, and a

从机制说起。Hook 在一份 settings 文件里配置。最常见的位置,是项目的 settings 文件——它位于项目根目录的 `.claude/settings.json`。用户级 hook 适用于所有项目,位于 `~/.claude/settings.json`。Settings 文件是 JSON 格式,所有 hook 都在顶层 `hooks` 键之下。每条 hook 项声明一个事件、一个可选的 matcher(用于过滤"哪些工具调用"会触发本 hook),以及一个

← 上一页

下一节 →

command to run when the event fires. The simplest possible hook configuration looks like this.

在事件触发时运行的命令。最简单的 hook 配置,看起来是这样的。

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "npx prettier --write \"$CLAUDE_TOOL_INPUT_FILE_PATH\""
 }
]
 }
]
 }
}
```

(配置含义:在 PostToolUse 事件上挂一个 hook,matcher 限定只在 Edit 和 Write 工具上触发,实际命令调用 Prettier 对被编辑/写入的文件就地格式化。)

## Reading the Configuration · 解读配置

Read this configuration carefully, because the structure is identical for every hook the rest of this part of the book will introduce. The top-level `hooks` key holds an object whose keys are event names. The event name in this case is `PostToolUse`, which fires after a tool call has completed successfully. Under that event name is an array of hook configurations. Each configuration has an optional matcher, which uses regular expression syntax to filter which tools this hook should apply to; here, `Edit` and `Write` are the only matched tools, so the hook fires after edits and writes but not after, say, `Bash` or `Read` calls. The `hooks` array under that contains the actual commands to execute. Each command has a `type`, almost always `command` for shell execution, and the `command` itself, which is run with the standard shell.

请把这份配置仔细读一遍——这一部分后续介绍的每一条 hook,都共用同一份结构。顶层 `hooks` 键持有的的是一个对象,对象的键是事件名。这里的事件名是 `PostToolUse`——它在一次工具调用成功完成之后触发。该事件名之下,是一个 hook 配置数组。每条配置含一个

可选的 `matcher`,使用正则语法过滤"哪些工具会触发本 hook";这里只匹配 `Edit` 和 `Write` ,所以 hook 只在"编辑/写入"之后跑,而不会在 `Bash` 、 `Read` 之类调用之后跑。它下面的 `hooks` 数组,才是真正要执行的命令。每条命令有一个 `type` (对 shell 执行而言几乎总是 `command` ),以及具体的 `command` 字符串本身——它会用标准 shell 去执行。

The hook configuration above is small but functional. Its effect: every time Claude Code edits or writes a file, the file is automatically passed through Prettier, which formats it according to whatever Prettier configuration the project has. The developer never has to remember to run Prettier. Claude never has to remember either. The hook remembers, because the hook is not part of the language model's loop. The hook is

上面这条 hook 配置虽小,却已能"做事"。它的效果是:每逢 Claude Code 编辑或写入一个文件,该文件都会被自动交由 Prettier 处理,按项目自带的 Prettier 配置就地格式化。开发者不必记得去跑 Prettier;Claude 也不必记得。Hook 替大家记着——因为它不属于语言模型那条 loop。Hook 是

← 上一页

下一节 →

wrapped around the loop, at a specific point in the lifecycle, and it runs every time the lifecycle reaches that point, no matter what.

"裹"在 loop 之外、挂在生命周期特定节点上的一道关卡——只要生命周期走到那一站,它就会跑,无论发生什么。

### The Lifecycle · 生命周期

The lifecycle is the grid of available moments where hooks can attach. Every Claude Code session passes through a sequence of named events, and a hook can be bound to any of them. The most common events, and the ones that almost every working developer eventually uses, are these. `PreToolUse` fires before a tool is about to be invoked, with the tool's name and input available to the hook. `PostToolUse` fires after a tool has completed successfully. `UserPromptSubmit` fires when the user submits a prompt, before Claude processes it. `Stop` fires when Claude finishes generating a response. `SubagentStop` fires when a subagent completes its work, before its result returns to the main session. `SessionStart` fires when a new session begins or is restored. `SessionEnd` fires when a session terminates. `Notification` fires when Claude needs to alert the user about something, such as needing approval. `PreCompact` fires just before Claude compacts its context to save space.

生命周期,就是 hook 可以挂上去的那张"时刻表"。每个 Claude Code session 都会经过一连串具名事件,hook 可以绑定到其中任何一个。最常用、也是几乎所有在岗开发者迟早会用上的那些事件,大致是这些: `PreToolUse` ,在工具即将被调用前触发,工具名与输入对 hook 可见; `PostToolUse` ,在一次工具调用成功完成后触发; `UserPromptSubmit` ,在用户提交 prompt、Claude 尚未处理时触发; `Stop` ,在 Claude 生成完一次响应时触发; `SubagentStop` ,在 subagent 完成其工作、结果尚未回传主会话前触发; `SessionStart` ,在一次新会话开始或被恢复时触发; `SessionEnd` ,在会话终止时触发; `Notification` ,在 Claude 需要向用户告警(例如请求审批)时触发; `PreCompact` ,在 Claude 为腾出空间而压缩上下文之前触发。

Each event has a slightly different shape of input passed to the hook on standard input as JSON, and a slightly different shape of expected output. `PreToolUse` hooks receive the tool name, the tool input, and the session context, and can decide to allow, deny, ask the user about the action, or defer the decision back to the agent loop. `PostToolUse` hooks receive the tool name, the input, and the response, and can issue feedback to Claude or block subsequent steps. `UserPromptSubmit` hooks receive the prompt text and can either block it or inject additional context for Claude to read. `SessionStart` hooks receive minimal context but can produce output that gets injected into the session's starting context, which is useful for loading project-specific reminders or recent git information at every session start.

每个事件,通过标准输入喂给 hook 的 JSON"输入形状"略有不同,所期待的"输出形状"也略有不同。`PreToolUse` hook 拿到工具名、工具输入、会话上下文,并据此决定:放行、拒绝、向用户确认,或把决定再交回 agent loop。`PostToolUse` hook 拿到工具名、输入、响应,可以向 Claude 发出反馈,或拦截后续步骤。`UserPromptSubmit` hook 拿到 prompt 文本,既可以拦截它,也可以向其中注入额外的上下文供 Claude 读取。`SessionStart` hook 拿到的上下文很少,但它可以产出一段输出,这段输出会被注入会话的"启动上下文"中——这件事在每个 session 一开始时,用来加载项目级提示或最近的 git 信息,非常顺手。

The mechanism by which a hook signals its decision is straightforward. The hook is a shell command. Its standard output and standard error are captured. Its exit code is checked. Exit code zero means success, and the lifecycle continues without modification. Exit code two has a special meaning: in `PreToolUse` hooks the action is blocked; in `PostToolUse` hooks the action has already happened, so exit code two cannot block it but instead surfaces the standard error output back to Claude as feedback, and whatever the hook printed to standard error is surfaced back to Claude as feedback about why the block happened. Other non-zero exit codes are treated as

Hook 表达"它做了什么决定"的机制,非常直白。Hook 就是一条 shell 命令;它的标准输出与标准错误会被捕获,它的退出码会被检查。退出码 0 代表"成功",生命周期照常推进、不做修改。退出码 2 有特殊含义:对 `PreToolUse` 而言,这次动作被拦截;对 `PostToolUse` 而言,这次动作已经发生,退出码 2 拦不住了——它转而把标准错误里写的东西当作"反馈"回送给 Claude,告诉它"为什么被拦"。其他非零退出码,会被视作

← 上一页

下一节 →

errors but do not block; they are logged. For richer control, the hook can also print structured JSON to standard output, which lets the hook approve or deny the action explicitly, modify the tool input before it runs, or inject additional context. The simple exit-code path covers most needs. The JSON path is for when the hook needs to do something more sophisticated than a binary allow or block.

"错误",只记录日志、并不拦截。要做更精细的控制,hook 还可以向标准输出打印一段结构化 JSON——这样它就能显式地"批准/拒绝"这次动作、在工具运行前改写其输入,或注入额外上下文。简单的"退出码路径"足以覆盖多数需求;JSON 路径,留给那些需要"比二元放行/拦截再复杂一点"的动作去用。

Exit Codes in Action · 退出码:实战

A worked example using exit codes will make the blocking pattern concrete. Suppose the developer wants to ensure that Claude Code never edits the `.env` file in any project. A small hook accomplishes this. The hook is configured under `PreToolUse`, with a matcher of `Edit` or `Write`, and the command is a small shell expression that reads the tool input on standard input, checks whether the file path is `.env`, and exits with code two if so.

用退出码做一个完整例子,让"拦截"模式具象化。假设开发者希望"在所有项目里,Claude Code 都不许编辑 `.env`"。一条很小的 hook 就能办到。它配在 `PreToolUse` 下,matcher 写 `Edit|Write`,命令则是一段小巧的 shell 表达式——从标准输入读工具输入,判断文件路径是不是 `.env`,是则退出码 2。

```
{
 "hooks": {
 "PreToolUse": [
 {
```

```

 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "jq -e '.tool_input.file_path | test(\"\\\\\\\\\\.env$\")' >/dev/null && { echo 'Editing .env files'
 }
]
 }
}
}
}

```

(配置含义:在 PreToolUse 上挂一个 hook,matcher 限定 Edit 和 Write 工具;命令通过 jq 解析工具输入,若文件路径以 .env 结尾,则往标准错误写一条拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

The shell expression in that command is dense, but its logic is simple. It reads the JSON input from standard input, uses jq to test whether the tool input's file path matches the regular expression for .env files, and if it does, prints an error message to standard error and exits with code two, which blocks the operation. If the file is not a .env file, the hook exits with code zero, allowing the operation. The model receives the standard error message as feedback and can explain to the user that the .env file is protected. The user gets a deterministic guarantee: no matter what Claude wants to do, no

上面命令里那段 shell 表达式,乍看很紧,但其逻辑很直白:从标准输入读入 JSON,用 jq 测一下"工具输入里的文件路径"是否匹配 .env 文件的正则;若匹配,则向标准错误打一条错误消息,再以退出码 2 退出——动作被拦;若不是 .env 文件,则以退出码 0 退出,放行。模型会把那段标准错误消息当作"反馈"收下,并向用户解释:这个 .env 文件是受保护的。用户得到的,是一条确定性的保证:无论 Claude 想做什么,

← 上一页

下一节 →

matter how the user phrased the request, no matter what the system prompt or skills say, the .env file cannot be modified.

.env 文件都改不了——无论用户措辞如何,无论 system prompt 或 skill 怎么说,都改不了。

That guarantee is the whole point. The probabilistic model is left to its probabilistic work, and the deterministic requirement is handled deterministically, in its proper place. The model is freed from having to be vigilant about something it does not need to be vigilant about, because the hook is vigilant on its behalf. The developer is freed from having to verify on every interaction, because the verification is mechanical.

这才是这份保证的全部意义。概率性的模型,继续做它概率性的工作;而"必须确定性"的那件事,被放在它该在的地方,确定性地被处理掉。模型不必再为"它本不必警惕的事"而保持警惕——因为 hook 替它警惕。开发者也免于"每次交互都要去验证"——验证已被机械化了。

A second mechanism worth knowing about is the JSON output format, which the hook can use for richer control than a binary allow or block. When a hook writes a JSON object to standard output and exits with code zero, Claude Code parses the JSON for instructions about how to handle the action. The most useful field for PreToolUse hooks is hookSpecificOutput, which carries a permissionDecision of allow, deny, or ask, plus an optional updatedInput field that lets the hook modify the tool input before it runs. This last capability is striking on

first encounter. A hook can intercept a tool call, decide that the call is acceptable but the input should be modified, and rewrite the input on the fly. The model never sees the modification. The tool runs with the rewritten input. A hook that wraps every `Bash` call to ensure that `npm install` runs with the `--no-save` flag would use this mechanism, intercepting the call, rewriting the command to add the flag, and letting the modified command proceed. The capability is powerful, and it should be used sparingly, because invisible modifications to tool input can produce confusing sessions if the developer forgets the hook is there. But when the use case is right, the JSON output format is the cleanest way to express it.

还有第二条机制值得知道——JSON 输出格式,让 hook 能做的控制远比"二元放行/拦截"更精细。当一条 hook 向标准输出写一个 JSON 对象,并以退出码 0 退出时,Claude Code 会解析这份 JSON,把它当作"如何处理这次动作"的指令。对 `PreToolUse` hook 而言,最有用的字段是 `hookSpecificOutput`,它带一个 `permissionDecision` (值是 `allow`、`deny`、`ask` 之一),并带一个可选的 `updatedInput` 字段,允许 hook 在工具实际运行前改写其输入。最后这项能力,初见会令人眼前一亮:hook 可以拦截一次工具调用,认定"调用本身没问题,但输入应当被改",并即时改写输入。模型完全看不到这次改写;工具以被改写后的输入跑。比如一条 hook 想"包住每一次 `Bash` 调用,确保 `npm install` 都带上 `--no-save`"——它就会用这条机制:拦截那次调用,改写命令加上这个 flag,然后让改写后的命令放行。这项能力很强,但要节制使用——"对工具输入做不可见的改写"很容易让 session 变得费解,只要开发者忘了 hook 还在那儿。因此,只有在用例对的时候,JSON 输出格式才是最干净的选择。

## The Settings Hierarchy · Settings 层级

A short word on the settings hierarchy. Hooks can be configured at three levels: enterprise policy, user-level, and project-level. Enterprise policy settings are managed by an organization's administrators and apply to every project a developer opens. User-level settings live in `~/.claude/settings.json` and apply to every project the user opens. Project-level settings live in `.claude/settings.json` at the root of a specific project and apply only inside that project. There is also a project-local file at `.claude/settings.local.json` (which the project's `.gitignore` should exclude), intended for personal hooks that the developer wants in this project but does not want to commit to the

简短聊一下 settings 的层级。Hook 可以在三个层级上配置:企业策略、用户级、项目级。企业策略的 settings 由组织的管理员掌管,适用于该开发者打开的每一个项目。用户级 settings 位于 `~/.claude/settings.json`,对用户打开的每一个项目都生效。项目级 settings 位于具体项目根目录的 `.claude/settings.json`,只对该项目内部生效。还有一份项目本地文件 `.claude/settings.local.json` (项目的 `.gitignore` 应当排除它)——它专门放"想在本项目用、但不想提交到

← 上一页

下一节 →

repository. The hierarchy is additive: hooks from all applicable levels run on every event, in a defined order, and any one of them can block. The developer should think carefully about which level a given hook belongs at. A security hook that protects against `rm -rf` belongs at the user level, because it should apply everywhere. A code-formatting hook that runs Prettier with project-specific configuration belongs at the project level, because the configuration is project-specific. A personal hook that sends desktop notifications when Claude needs input belongs in `settings.local.json`, because it is the developer's personal preference and not something her teammates need or want.

仓库里去"的个人 hook。这个层级是叠加式的:每个事件触发时,所有适用层级的 hook 都会按一个既定的顺序跑,而其中任何一条都可以拦截。开发者应该仔细想一想:某条 hook 应该住在哪一层?防 `rm -rf` 的安全 hook,应当住在用户层——它该到处生效;按项目级 Prettier 配置跑格式化的 hook,应当住在项目层,因为配置是项目特定的;在 Claude 需要输入时弹桌面通知的个人 hook,应当住在 `settings.local.json` ——它纯粹是这位开发者的个人偏好,队友既不需要也不希望它出现。

Understanding where each hook belongs is more than an organizational nicety. It directly affects how the hook is shared, how it is reviewed, and how easily it can be adjusted. A hook in the wrong place produces friction long after it was installed. A user-level hook that should have been project-level forces every other project to deal with conventions that only one project actually wants. A project-level hook that should have been user-level fails to protect the developer when she switches to a different project. A personal hook in the wrong place gets committed to the repository where her teammates encounter it as an unexplained presence. None of these mistakes is catastrophic, but each of them produces low-grade ongoing annoyance, and the annoyance can be avoided entirely by spending thirty seconds on the placement decision when the hook is being installed.

"hook 该住哪一层"这件事,理解到位的好处,远超"组织上好看"那么简单。它会直接决定 hook 如何被共享、如何被评审、调整起来有多容易。住错层级的 hook,会在安装之后很久还在制造摩擦。本该是项目级的 hook 放在用户层,会让其他所有项目都被"这一个项目才需要的规约"绑架;本该是用户层的 hook 放在项目层,会让开发者在切换到别的项目时失去那条保护;个人 hook 放错地方,会被提交进仓库,队友打开代码时一脸问号地撞上这条来历不明的存在。这些错误,没有哪个是灾难性的,但每一条都会带来低烈度、持续性的烦人;在 hook 安装的当下,花三十秒想清楚"该放哪",就能把这些烦人彻底省下。

The relationship between hooks and the agent loop deserves explicit attention, because it is the relationship that makes the entire architecture work. The agent loop, recall from Chapter Two, is the cycle of think, act, observe that the language model runs through every turn. Every tool call happens inside the act phase. Every observation of a tool result happens inside the observe phase. Hooks attach to the boundaries of these phases. `PreToolUse` hooks fire just before the act phase invokes a tool. `PostToolUse` hooks fire just after the act phase has completed and the result is being returned to the observe phase. The hooks are, in this sense, the boundary between the model's probabilistic decisions and the deterministic environment that surrounds them. They are not part of the model's loop. They are part of the framework that the loop runs inside, and the framework can impose constraints that the model has no way to evade.

Hook 与 agent loop 之间的关系,值得专门提一笔——因为这层关系,正是整副架构"能跑起来"的根本。回顾第二章:agent loop,就是语言模型每轮跑一次的"思考—行动—观察"循环。每次工具调用发生在"行动"阶段;每次对工具结果的观察,发生在"观察"阶段。Hook 挂在这些阶段的边界上——`PreToolUse` 钩在"行动"阶段即将调用工具之前;`PostToolUse` 钩在"行动"阶段完成、结果正被送回"观察"阶段之际。Hook 在这个意义上,就是"模型的概率性决策"与"环绕模型的确定性环境"之间的那道边界。它不属于模型的 loop;它属于"loop 跑在其中的"那副框架。框架可以施加模型无论如何也绕不开的约束。

← 上一页

下一节 →

A small clarification about the interactive `/hooks` command is worth including, because new users sometimes overlook it and pay an unnecessary cost. The `/hooks` command, run inside Claude Code, opens an interactive editor for the hook configuration. The editor lists every event, shows the hooks currently bound to each event, and offers menu options to add, modify, or remove hooks without having to edit the JSON configuration file directly. For developers who are new to hooks and not yet comfortable hand-editing JSON, the interactive editor is the friendlier path. For developers who are comfortable with JSON, direct editing of the settings file is faster, especially for non-trivial hooks. Both paths produce identical results, and the developer can switch between them depending on what is easier for the specific change she is making. The interactive editor is also useful as a verification tool: running `/hooks` in a session is a quick way to confirm that a hook configuration is loaded and active, which can save time when debugging why a hook seems not to be firing. Two related slash commands are worth knowing about as well. The `/agents` command lists the subagents available in the current session and lets the developer inspect their definitions interactively. The `/doctor` command reports on the current installation and surfaces the most common configuration problems before they become a debugging session. A common pitfall worth flagging here is forgetting that newly added hooks require a session restart before they

take effect; configuration changes do not retroactively apply to a running session, and the developer who edits her hooks and then expects the changes to be live should restart Claude Code or open a new session before testing.

有一条关于交互式 `/hooks` 命令的小说明值得补上——新用户有时会忽略它,因而多付不必要的代价。在 Claude Code 里运行 `/hooks` 命令,会打开一个针对 hook 配置的交互式编辑器。编辑器会列出所有事件、显示当前绑定到每个事件上的 hook,并提供菜单选项,让你不必直接编辑那份 JSON 配置文件,就能增、改、删 hook。对刚接触 hook、还没那么习惯手写 JSON 的开发者来说,这条交互式路径更友好;对写 JSON 已经顺手的人来说,直接编辑 settings 文件更快,尤其是当 hook 不那么简单的时候。两条路径产出完全一致,开发者可以按"这次具体改起来哪种更顺手"自由切换。交互式编辑器同时也是一条很好的验证工具——在 session 里跑一次 `/hooks`,是快速确认"hook 配置已加载并激活"的最快方式,在排查"hook 似乎没触发"时能省不少时间。还有两条相关的斜杠命令也值得知道: `/agents` 命令会列出当前 session 中可用的 subagent,并让你交互式地检视它们的定义; `/doctor` 命令则会汇报当前安装的状态,把最常见的配置问题摆在台面上,免得它们拖到要专门 debug 的地步。此处还有一条常见的坑要专门提一句:刚加进去的 hook,需要重启 session 才会生效;配置变更不会追溯性地作用到正在运行的 session 上。因此,改完 hook 又期望改动立即生效的开发者,应当先重启 Claude Code 或开一个新 session,再去测试。

A worked debugging walkthrough is worth including, because hooks can fail in subtle ways and the developer who has a debugging procedure in working memory will save herself hours when something goes wrong. The walkthrough below is from a real session. The developer had configured a `PostToolUse` hook to run Prettier after every file write. She tested it, it worked, and she committed the configuration. Two days later, she noticed that Prettier had stopped running. The files Claude was writing were no longer being formatted, even though the hook configuration appeared unchanged.

一个完整的 debug 走查,值得放在这里——hook 失败的形态往往很微妙,工作记忆里有一份 debug 流程的开发者,真出问题时会为自己省下好几个小时。下面这份走查,出自一次真实的 session。开发者配了一条 `PostToolUse` hook,让 Prettier 在每次写文件之后跑。测试过、能跑、提交了配置。两天后,她发现 Prettier 不再跑了:Claude 写出的文件不再被格式化,而 hook 配置看上去又没动过。

## A Debugging Walkthrough · Debug 走查

← 上一页

下一节 →

She approached the problem methodically. The first step was to verify the hook was loaded. She ran the `/hooks` command inside Claude Code, which lists all configured hooks for the current session. The Prettier hook appeared in the list, which meant the configuration was being read. The second step was to verify the hook was running. She added a small log statement to the hook itself, writing a timestamp to a file every time the hook fired. She made an edit, and the log file gained a new entry. The hook was firing. The third step was to verify the hook was succeeding. She checked the file the hook had been formatting and saw that the formatting had not been applied. The hook was firing but not completing its work.

她按部就班地处理。第一步:确认 hook 已加载。她在 Claude Code 里跑了 `/hooks` 命令,这个命令会列出当前 session 中所有已配置的 hook。Prettier hook 出现在列表里——说明配置确实被读到了。第二步:确认 hook 在跑。她在 hook 本身里加了一行小日志,让 hook 每次触发时都向一个文件里写一行时间戳。她做了一次编辑,日志文件多了一行——说明 hook 在触发。第三步:确认 hook 在"成功"地完成。她去看 hook 应当去格式化的那个文件,发现格式没被应用——hook 在触发,但工作没做出来。

The diagnosis came on the fourth step. She invoked the hook directly from her terminal, passing in a JSON input that mimicked what Claude Code would send. The hook ran, attempted to invoke Prettier, and failed silently. The reason was that `npm prettier` required the project's

`node_modules` 要安装, 而最近的依赖清理却移除了 Prettier。Hook 本身没错, 是它所运行的那个“环境”在它脚下漂移了。她重装了 Prettier, 再跑一遍测试, 格式化回来了。

诊断落在第四步: 她直接在自己的终端里调用 hook, 给了一段模仿 Claude Code 实际发送的 JSON 输入。Hook 跑起来了, 试图去调 Prettier, 然后悄悄失败。原因在于, `npx prettier` 依赖项目里的 `node_modules` 已安装; 而一次最近的依赖清理, 把 Prettier 从项目的依赖里移走了。Hook 本身没错, 是它所运行的那个“环境”在它脚下漂移了。她重装了 Prettier, 再跑一遍测试, 格式化回来了。

The lesson the developer took away was that hooks fail silently more often than they fail loudly, and the procedure for debugging them is to walk down the chain: configuration loaded, hook firing, hook executing, hook succeeding. Each step has a verification. Each verification narrows the problem space. The whole procedure takes ten minutes and resolves the large majority of hook problems, because the failure is almost always at one of the four checkpoints, and the checkpoints are easy to test in isolation.

开发者从中总结出的经验是: hook 失败时, 十有八九是“静悄悄地失败”, 而非“响亮地失败”; debug 它们的标准流程, 就是沿这条链一路往下走——配置已加载? hook 在触发? hook 在执行? hook 在成功? 每一站都有验证; 每一处验证, 都在缩窄问题空间。整个流程十分钟就能走完, 且能解决绝大多数 hook 问题——因为失败几乎总落在四道关卡的某一道上, 而每道关卡都很容易单独去测。

A reflection before this chapter closes, on where hooks fit in the broader architecture of the book. Skills shape what Claude knows. Subagents shape who, in effect, is doing the work. Hooks shape what is permitted. The three together cover capability, specialization, and boundary. The fully engineered Claude Code environment uses all three, with hooks providing the deterministic edges that make it possible to trust the probabilistic systems in the middle. A developer who has installed skills and subagents but no hooks has a powerful environment that still depends on the model's judgment for safety-critical concerns, which is a reasonable trade-off but a real one. A developer who adds even three or four well-chosen hooks transforms her environment from a powerful assistant into a

在收束本章之前, 先来点反思: hook 在本书更宏大的架构里, 占的是哪一格? Skill 决定 Claude “知道什么”; subagent 决定“实际上是谁在干活”; hook 决定“什么被允许”。三者合起来, 覆盖了“能力 / 专门化 / 边界”这三件事。一套完整工程化的 Claude Code 环境, 会同时用上这三样: 由 hook 提供确定性边缘, 让中间那些概率性的系统变得可以信任。只装了 skill 和 subagent、没装 hook 的开发者, 环境虽然强大, 却仍要把“安全攸关的事”押在模型的判断上——这是一笔合理的交易, 但也是一笔真实的赌注。再多加哪怕三四条精心挑选过的 hook, 她的环境就会从“一个强大的助手”变成一个

← 上一页

下一节 →

system she can rely on for things that genuinely matter, because the things that genuinely matter are now protected by mechanism rather than by hope.

“在真正重要的事上靠得住”的系统——因为真正重要的事, 如今是由机制保护, 而非由希望保护。

The next chapter takes hook design from these mechanics into design patterns. The three archetypes that cover almost every useful hook are gates, guards, and notifiers, and the chapter ahead develops each pattern in depth, with worked examples ready to install. The reader who finishes the next chapter will have a small but capable set of hooks in her environment and the design vocabulary to add more whenever her work uncovers a new requirement that deserves deterministic handling.

下一章,要把 hook 的设计从"机制"层面带进"模式"层面。覆盖"几乎所有有用 hook"的三种原型,是 gate、guard、notifier;下一章会逐一深入展开,并配上"开箱即装"的具体例子。读完整章的读者,届时会在自己的环境里拿到一套虽小但能打的 hook,同时也会拥有一份"设计词汇"——任何时候,只要她的工作揭示出一个"值得被确定性对待"的新需求,她就能照着这套词汇再添新条。

## CHAPTER SIXTEEN · 第十六章

### Hook Design Patterns: Gates, Guards, and Notifiers · Hook 设计模式:门、哨、告

*"The shape of a tool is the shape of the problem it solves. Three problems, three shapes, no more."*

— Henry Petroski, paraphrased

*"工具的形状,就是它所解决的问题的形状。三个问题,三种形状,仅此而已。"*

—— 亨利·彼得罗斯基(意译)

This chapter takes the hook mechanics of the previous chapter and develops them into three design patterns that, in combination, cover almost every hook a working developer will ever want to write. Naming the three patterns explicitly is valuable because it gives the developer a vocabulary for thinking about her hook design and a checklist for asking, when a new requirement arises, which of the three shapes the requirement most resembles. The three patterns are gates, guards, and notifiers, and each one solves a specific class of problem in a specific way. Once the patterns are in working memory, designing a new hook becomes a matter of recognizing which pattern fits and following the conventions for that pattern.

本章把上一章讲过的 hook 机制,发展为三种设计模式;三者在组合之下,几乎覆盖了一名在岗开发者要写的所有 hook。把这三种模式显式地命名,价值很大:它给了开发者一套"思考 hook 设计"的词汇,以及一份"清单"——每当出现一项新需求,都可以问:这条需求最像这三种形状里的哪一种?三种模式分别是 gate(门)、guard(哨)、notifier(告);每一种,都以自己的方式,解决某一类问题。一旦这三种模式进入工作记忆,设计新 hook 就变成"识别该用哪一格、按那格的约定写下去"的事了。

#### Gates · 门

Begin with gates. A gate is a hook that runs before an action and decides whether the action is allowed to proceed. The canonical lifecycle event for gates is `PreToolUse`, which fires before a tool call is dispatched. The gate inspects the tool input, applies whatever rule the developer cares about, and either lets the action through or blocks it. The blocking mechanism, as established in the previous chapter, is exit code two, with whatever explanation the developer wants surfaced to Claude on standard error.

先讲 gate。一条 gate 是一条"在动作发生前运行、决定该动作是否可以继续"的 hook。Gate 的标准生命周期事件是 `PreToolUse`——它会在一次工具调用被真正派出去之前触发。Gate 检查工具输入,套上开发者关心的那条规则,要么放行,要么拦截。拦截机制如上一章所约定:退出码 2,加上一段开发者希望送到 Claude 那里的、写在标准错误里的解释。

Gates are the most common and most consequential pattern in any hook library, because they are the pattern that turns wishes into rules. A developer who wishes that Claude would not commit directly to the main branch can encode the wish as a wish, by writing it into `CLAUDE.md` or into a skill, and Claude will follow the wish most of the time. The developer who wants the wish to become a rule writes a gate. The gate runs before any `git push` tool call, inspects the command for direct pushes to main, and exits with code two if the push targets main without an explicit override.

在任何 hook 库里,gate 都是最常用、也最重要的一格——因为它就是那条"把心愿变成规则"的模式。一个希望"Claude 不要直接 commit 到 main 分支"的开发者,可以把"心愿"当心愿写进 `CLAUDE.md` 或一条 skill 里,Claude 大多数时候会照办;一个希望"心愿成为规则"的开发者,则会写一条 gate。这条 gate 在每一次 `git push` 工具调用之前跑,检查命令里是不是直接 push 到 main,若是,又在没有显式 override 的情况下,就以退出码 2 退出。

← 上一页

下一节 →

The wish is now a rule. Claude cannot push to main, full stop, regardless of what the user types or how the request is phrased.

于是"心愿"成了"规则"。Claude 不能 push 到 main——一字不差,一字不漏,无论用户敲下什么、措辞如何,都不行。

A worked gate example will make the pattern concrete. The gate below blocks any Bash command that contains the dangerous `sudo rm -rf` pattern, because the developer has decided that this pattern should never be executed by Claude Code, no matter what.

用一个完整的 gate 例子把模式说透。下面这条 gate 会拦截任何包含 `sudo rm -rf` 这一危险模式的 Bash 命令——因为开发者已经决定:这个模式,无论什么情况,Claude Code 都不该执行。

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '.tool_input.command' | grep -qE 'sudo[[:space:]]+rm[[:space:]]+-rf' && { echo 'Refusi"
 }
]
 }
]
 }
}
```

(配置含义:在 PreToolUse 上挂一条 gate,matcher 限定 Bash 工具;命令以 jq 从工具输入里抽取命令字符串,用 grep 匹配 `sudo rm -rf`;命中则往标准错误写拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

The shell expression reads the tool input as JSON, extracts the command, checks for the dangerous pattern with `grep`, and if it matches, surfaces a clear refusal message and exits with code two. The block is total. The model cannot bypass it, because the gate is not a model concern. It is a system concern, enforced before the model's requested action ever reaches the shell.

这条 shell 表达式:以 JSON 形式读取工具输入,抽出命令,用 `grep` 检查是否含有危险模式;若命中,向标准错误写一条清晰的拒绝消息并以退出码 2 退出。拦截是绝对的。模型绕不开它——因为 gate 不是模型层面的事,而是系统层面的事:在模型请求的动作抵达 shell 之前,就已经被强制拦下。

## The Discipline of a Good Gate · 一条好 gate 的纪律

Gates have a discipline worth naming. A good gate has three properties. It is targeted; it fires only on the specific tool calls and inputs it is designed to evaluate, not on every Bash command in the universe. It is fast; it should add only a few milliseconds to any tool call, because hooks fire constantly and slowness in a hook becomes slowness in every interaction. It is informative; when it blocks an action, the message it surfaces explains why the block happened in language the developer or the model can understand

Gate 有几条值得点名的纪律。一条好的 gate,具备三条属性。其一,精准——它只在"自己设计来评估"的那一类工具调用与输入上触发,而非对全宇宙所有 Bash 命令都一视同仁;其二,快——它只能给任何一次工具调用增加若干毫秒的耗时,因为 hook 会反复触发,hook 一慢,所有交互都跟着慢;其三,清晰——当它拦截一次动作,送回的消息要把"为什么被拦"讲明白,所用的语言,开发者或模型都读得懂。

← 上一页

下一节 →

and act on. A gate that blocks without explanation produces frustrating sessions. A gate that explains its reasoning turns a refused action into a learning moment.

让人能照做。一条 gate 拦下动作却没给解释,会让 session 越用越憋屈;而一条把理由讲清楚的 gate,会把"一次被拒"变成"一次学习"。

## Guards · 哨

Move to guards. A guard is a hook that runs after an action and enforces consistency. The canonical lifecycle event for guards is `PostToolUse`. The guard inspects the result of the action and either accepts it as-is, modifies the surrounding state, or signals back to Claude that the action's result was insufficient and more work is needed. The Prettier hook from the previous chapter is a classic guard: it runs after a file is written, formats the file according to project rules, and ensures that whatever Claude wrote ends up in the canonical project format whether Claude knew the format or not.

接下来说 guard。一条 guard,是一条"在动作发生后跑、用来强制一致性"的 hook。它对应的标准生命周期事件是 `PostToolUse`:guard 检查动作的产出,要么照单全收,要么改一改周边状态,要么向 Claude 回送"产出尚不充分,还得再补一轮"。上一章的 Prettier hook 就是 guard 的典型——它在一个文件被写完之后跑,按项目规约把文件格式化,确保 Claude 写出来的内容无论知不知道规约,最终都落到项目"正统"的格式上。

Guards differ from gates in their relationship to the action. A gate is preventive; it stops things from happening. A guard is corrective; it lets things happen and then ensures the result fits the project's conventions. Guards are usually less dramatic than gates, because they do not block,

but they are at least as valuable in practice. A library of well-chosen guards makes the project's output consistent in ways that no amount of careful prompting can match, because the guards run on every relevant action and apply their corrections without requiring anyone to remember.

Guard 与 gate 的差别,落在它们和"动作"的关系上:gate 是"预防性"的——它拦下事不让发生;guard 是"纠正性"的——它让事发生,再去保证结果贴合项目规约。Guard 看上去没有 gate 那么戏剧化——它不拦截——但实操中并不比 gate 逊色。一组精心挑选的 guard,会让项目的产出保持一致,这份一致,是再小心的 prompt 也换不来的——因为 guard 在每一个相关动作上都跑、它自我修正,不需要任何人"记得去做"。

A worked guard example raises the Prettier pattern to something more complete. The guard below runs after any file edit or write, formats the file with Prettier, and additionally runs ESLint with the auto-fix flag to apply automatic linting fixes.

用一个完整的 guard 例子,把 Prettier 这条模式再推高一层。下面这条 guard 在任何文件编辑/写入之后跑:用 Prettier 格式化,再用带 `--fix` 标志的 ESLint 自动修一遍 lint。

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\" 2>/dev/null || true"
 },
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); npx eslint --fix \"$FILE\" 2>/dev/null || true"
 }
]
 }
]
 }
}
```

(配置含义:在 PostToolUse 上挂两条 hook 形成的链,matcher 限定 Edit / Write / MultiEdit 工具;第一条把被编辑的文件交给 Prettier 格式化,第二条交给 ESLint 自动修——两者都吞掉错误并以退出码 0 退出,避免单次格式化失败打断 session。)

← 上一页

下一节 →

Two hooks fire in sequence under the same matcher. The first runs Prettier. The second runs ESLint with auto-fix. Both extract the file path from the JSON input, both run their tool against the path, and both swallow errors and exit cleanly so that a transient formatter failure does not interrupt the session. The result: every file that Claude touches ends up formatted and lint-clean before the next prompt, regardless of what Claude wrote. The project's style is enforced mechanically, and the developer never has to think about formatting again.

两条 hook 在同一个 matcher 下依序触发:第一条跑 Prettier,第二条跑带 `--fix` 的 ESLint。两者都从 JSON 输入里抽出文件路径,都把工具对着那个路径跑一遍,又都吞掉错误、干净退出——这样一次偶发的格式化失败不会把 session 打断。效果是:Claude 碰过的每一个文件,在下一条 prompt 进来之前,都已经格式化、lint 干净——无论 Claude 一开始写成了什么样。项目的风格由此被机械化地强制,开发者再也不必把"格式化"这事再放到脑子里。

## Notifiers in Detail · 告:细讲

Move to notifiers. A notifier is a hook that runs at a lifecycle event for the purpose of producing some output to a system other than Claude Code itself. The output might be a desktop notification, a log file entry, a Slack message, an audit record, or anything else that the developer wants the world outside the session to know about. Notifiers do not block. They do not modify behavior. They simply observe the lifecycle and react.

最后说 notifier。一条 notifier,是一条"在某个生命周期事件触发时,跑一遍、用来向 Claude Code 之外的某个系统产出一段输出"的 hook。这段输出,可能是一张桌面通知、一行日志、一条 Slack 消息、一份审计记录,也可以是任何"开发者希望 session 之外的世界知道"的事。Notifier 不拦截、不改行为,只是在生命周期里"观察一下、做出反应"。

The most common notifier event is the `Notification` event itself, which fires when Claude wants to alert the user about something, often a permission request. The developer can attach a notifier to this event that produces a desktop pop-up, so that she does not have to be staring at the terminal to see when Claude needs her attention. `Stop` is another common notifier event; a notifier on `Stop` can log every completed task to an audit file or send a brief summary to a team channel. `SessionEnd` notifiers can save session statistics. `PostToolUse` notifiers can append every `Bash` command Claude ran to a permanent log, which is useful both for audit and for retrospective debugging.

最常见的 notifier 事件,就是 `Notification` 本身——它会在 Claude 想向用户告警(通常是请求授权)时触发。开发者可以在这个事件上挂一条 notifier,产出桌面弹窗,这样她就不必一直盯在终端上等 Claude 找她。`Stop` 也很常用——挂上去的 notifier 可以把每一项已完成的任务写进审计文件,或向团队频道发一条简短总结。`SessionEnd` notifier 可以把会话统计信息存档。`PostToolUse` notifier 可以把 Claude 跑过的每一条 `Bash` 命令追加到一份永久日志里——对审计、对事后回溯 debug,都有用。

A worked notifier example combines two of these patterns. The configuration below sends a desktop notification when Claude needs the user's input, and logs every Bash command Claude runs to a persistent log file.

一个完整的 notifier 例子,把上面两种用法合到一起。下面这份配置:在 Claude 需要用户输入时弹一条桌面通知;同时把 Claude 跑过的每一条 Bash 命令,写进一份持久化的日志文件。

```
{
 "hooks": {
 "Notification": [
 {
 "matcher": "",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '.message' | xargs -I {} osascript -e 'display notification \"{}\" with title \"Claude\"'"
 }
]
 }
],
 "PostToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
```

```

 {
 "type": "command",
 "command": "jq -r '.tool_input.command' >> ~/.claude/command-log.txt"
 }
]
}
}
}

```

(配置含义:在 Notification 事件上挂一条 notifier,从消息体里抽取出文本,交给 osascript 弹一条 macOS 桌面通知;在 PostToolUse 上挂另一条,matcher 限定 Bash 工具,把每次 Bash 命令追加写入 ~/.claude/command-log.txt 。)

← 上一页

下一节 →

The first hook fires whenever Claude needs the user's attention; the developer's desktop pops up a notification, and she can return to the terminal at her convenience instead of having to monitor it. The second hook fires after every Bash command and writes the session ID and the command to an audit log in the user's home directory. The log accumulates across sessions and projects, providing a complete record of every shell command Claude has executed under the developer's account.

第一条 hook 在 Claude 需要用户关注时触发——开发者的桌面会弹出一条通知,她可以在自己方便时再回到终端,不必一直盯着。第二条 hook 在每次 Bash 命令后触发,把"session ID + 命令"写进用户主目录下的一份审计日志。这份日志跨 session、跨项目累积,等于一份"这位开发者的账户下,Claude 执行过哪些 shell 命令"的完整账本。

A discipline applies to notifiers that the developer should keep in mind. Notifiers should be silent in the success case. They produce their side effect, and they exit cleanly. They do not produce stdout or stderr that the model might mistake for relevant output. A noisy notifier pollutes the session and degrades the experience. A silent notifier is invisible to the workflow except in the side effect it produces, which is exactly what the developer wants.

对 notifier,有一条应当记在心里的纪律:成功路径上,notifier 应当保持沉默。它产生它的副作用,然后干净地退出。它不应当把 stdout 或 stderr 写出来——因为模型可能误以为那是相关的输出。一条吵闹的 notifier 会污染 session、劣化体验;一条安静的 notifier,在工作流里几乎不可见,只在它所产生的那个副作用上"显形"——这正是开发者想要的。

A second worked notifier example, in a slightly different configuration style, makes the discipline visible. Below, a notifier on the Notification event uses the Linux notify-send tool to produce a desktop notification, and a notifier on PostToolUse appends a session-aware line to a long-running bash history log.

第二条完整的 notifier 例子,采用略不同的配置形态,把这套纪律摆在明面上。下面的 notifier,在 Notification 事件上,用 Linux 的 notify-send 工具弹桌面通知;在 PostToolUse 上,把一条带 session 标记的行追加进一份"长期累积"的 bash 历史日志。

```

{
 "hooks": {
 "Notification": [

```

```

{
 "hooks": [
 {
 "type": "command",
 "command": "notify-send 'Claude Code' 'Awaiting your input'"
 }
]
},
{
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '\"\\(.session_id) \\(.tool_input.command)\"' >> ~/.claude/bash-history.log"
 }
]
}
]
}
}

```

(配置含义:Notification 事件上的 notifier 用 `notify-send` 弹一条"Claude Code: Awaiting your input"的桌面通知;PostToolUse 上以 Bash 为 matcher 的 notifier,把 `session_id` 与命令拼成一行追加到 `~/.claude/bash-history.log`。)

← 上一页

下一节 →

A second discipline applies to all three patterns and is worth naming explicitly: keep hooks fast. Every hook fires on the lifecycle event it is bound to, and every hook adds latency to every event. A few milliseconds is invisible. A few hundred milliseconds is felt. A few seconds is unbearable, especially for events that fire frequently like `PostToolUse`. The developer should profile her hooks at install time, ensure they complete quickly, and pay particular attention to hooks that invoke heavy tools like full test suites or large linters. Heavy work, when it must run, often belongs in an asynchronous hook that fires the work and returns immediately, leaving the work to complete in the background. Asynchronous hooks are supported through the `async` configuration flag, which the developer can use when the work is genuinely independent of the immediate flow.

还有一条纪律,适用于全部三种模式,值得显式点出来:让 hook 保持快速。每条 hook 都会在它绑定到的那个生命周期事件上触发,每条 hook 都会给那次事件增加延迟。几毫秒,无感;几百毫秒,有感;几秒,会令人难以忍受——尤其对那些高频触发的事件,比如

`PostToolUse`。开发者应当在安装时对自己的 hook 做一下 profile,确保它们完成得够快,并对那些会调重型工具(完整测试套件、大型 linter)的 hook 多加留意。确实要跑的重活,通常属于"异步 hook"——它把任务发起,然后立刻返回,把后续完成的部分放到后台跑。异步 hook 借由 `async` 这个配置开关来支持,当某项任务确实独立于眼前这条流时,就可以用。

## The Three–Pattern Vocabulary · 三模式词汇表

The three patterns, gates and guards and notifiers, are the basic vocabulary of hook design. With them in working memory, the developer can look at almost any requirement and immediately recognize which pattern fits. A requirement that says *no*, *do not*, *never* is almost always a gate. A requirement that says *always*, *ensure*, *conform* is almost always a guard. A requirement that says *tell me*, *log*, *alert* is almost always a notifier. The

classification is not a rigid taxonomy. It is a thinking tool. It accelerates the design of new hooks by giving the developer a starting shape for any requirement she encounters, and that starting shape is, in most cases, the right shape.

这三种模式——gate、guard、notifier——就是 hook 设计的基本词汇表。把它们装进工作记忆之后,开发者再看几乎任何一项需求,都能立刻认出该归哪一格。一条措辞像"不许、不要、绝不能"的需求,几乎总是 gate;一条措辞像"始终要、务必、必须合于"的需求,几乎总是 guard;一条措辞像"告诉我、记下来、提醒我"的需求,几乎总是 notifier。这种分类不是僵硬的分类学,而是一个思考工具:它为开发者遇到的任何需求,都给出一个"起步形状",从而加速新 hook 的设计;而这个起步形状,大多数时候就是对的形状。

A worked extension of the guard pattern deserves attention here, because guards have a sophistication that gates and notifiers lack and that a developer should understand before designing a serious hook library. A guard can do more than just modify the file system after a tool runs. A guard can also report back to Claude through the JSON output format, telling Claude that something about the result of the tool call was unsatisfactory and that more work is needed. A guard configured this way is not merely corrective; it is conversational. It speaks back to the model and influences what the model does next.

guard 模式还有一条值得专门讲的扩展:guard 拥有一种 gate 与 notifier 都不具备的"复杂能力",在搭建严肃 hook 库之前,应当先把它想清楚。Guard 并不止于"在工具跑完之后改一改文件系统";它还可以借 JSON 输出格式向 Claude 回话——告诉它"这次工具调用的结果在某一点上不令人满意,还需要再补点活"。如此配置的 guard,就不再只是"纠正性"的,而是"对话性"的:它向模型回话,并影响模型下一步要做什么。

The example below shows this pattern in action. The hook runs after every `Edit` or `Write` tool call. It runs the project's test suite, and if any tests fail, it surfaces the failure back to Claude with a directive that the failing

下面这个例子把这种模式跑起来给你看:这条 hook 在每一次 `Edit` 或 `Write` 之后触发——它跑项目的测试套件,只要有测试失败,就把失败回送给 Claude,附带一条指令,让模型去把"那些失败的

← 上一页

下一节 →

tests should be fixed before the session continues. The mechanism is the JSON decision-block format, which carries a `decision` of `block` and a `reason` that explains what needs to happen.

测试修好,再继续推进 session"。机制就是 JSON 决策-拦截格式:它带一个 `decision` 取值 `block`,以及一段 `reason` 说明该做什么。

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "if ! npm test --silent >/tmp/test-out 2>&1; then echo '{\"decision\":\"block\",\"reason\":\""
 }
]
 }
]
 }
}
```

```
}
 }
}
}
```

(配置含义:在 PostToolUse 上挂一条"对话式 guard",matcher 限定 Edit / Write / MultiEdit;命令跑 `npm test` 并把输出重定向到 `/tmp/test-out` 。若测试失败,则向 stdout 打印一份 JSON 决策块,告知模型"测试失败,请先读 `/tmp/test-out` 并修好,再继续"。)

The hook runs the test suite after each edit. If the tests pass, nothing happens; the hook exits with code zero and no JSON output, and Claude proceeds normally. If the tests fail, the hook prints the JSON decision-block, which Claude Code reads as a directive to surface the reason back to the model. The next message Claude sees contains the block reason, and Claude responds by reading the test output and fixing the failing tests. The result is that the test suite functions as a continuous check on every edit Claude makes; broken tests are surfaced immediately and addressed before the session moves on. The developer who has installed this hook never has to remember to run tests. The hook remembers, and Claude responds.

这条 hook 在每次编辑之后跑一次测试套件。若测试通过,什么都没发生——hook 以退出码 0 退出、不输出任何 JSON,Claude 照常推进。若测试失败,hook 打印那段 JSON 决策块,Claude Code 把它读成"把 reason 回送给模型"的指令。Claude 下一条看到的消息里,会包含这条 block reason;它会读测试输出,去把失败的测试修好。效果就是:测试套件成了 Claude 每次编辑的"持续校验器"——坏的测试被立即翻到台面上,赶在 session 推进之前被处理。装上这条 hook 的开发者,从此不必再记得"要跑测试"——hook 替她记着,Claude 顺势响应。

A discipline applies to test-running guards that is worth naming, because the obvious implementation is often too slow. Running the full test suite after every edit is acceptable for projects with fast tests, but it becomes painful for projects whose tests take more than a few seconds. The right adaptation is to run only the tests that exercise the changed code, using whichever test selection mechanism the developer's test runner supports. For Jest and Vitest, this might mean using the related-files flag to identify tests that import the changed file. For Pytest, this might mean using a test collection plugin. The general principle is that the hook should run the

对"跑测试的 guard",有一条值得点名的纪律——因为最直白的实现往往太慢。"每次编辑都跑全套测试"——对测试跑得快的项目可以接受,但只要测试多花几秒,体验就会变得痛苦。正确的调校是:只跑"覆盖了被改代码"的那部分测试,用开发者所用测试运行器自带的"测试挑选"机制。对 Jest 和 Vitest 而言,可以用 "related-files" 标志去筛出"import 了被改文件"的测试;对 Pytest 而言,可以用一个 test collection 插件。一般原则是:hook 该跑的是

← 上一页

下一节 →

smallest set of tests that gives confidence about the change, not the full suite, because the latency penalty of running the full suite quickly outweighs the benefit of having the check run automatically.

"能对这次改动给出信心"的最小测试集,而非全套——因为"每次都跑全套"带来的延迟代价,很快就压过"自动跑校验"带来的收益。

A short word on asynchronous hooks. Some work the developer wants done after a tool call does not need to block the lifecycle. Logging an audit record. Sending a backup. Updating a dashboard. None of these need to delay the session's progression. For these cases, Claude Code supports asynchronous hooks, configured by adding an `async` flag to the hook definition. An asynchronous hook is fired and forgotten; the lifecycle proceeds while the hook runs in the background. The cost is that the hook's output cannot influence the lifecycle, because by the time the hook finishes, the lifecycle has moved on. This is exactly the right trade-off for the categories of work that asynchronous hooks are meant to handle, where the work matters but the timing does not. The configuration looks like this.

关于异步 hook,简短说几句。有些"工具调用之后要做的工作"并不需要阻塞生命周期:记一条审计记录、推一份备份、更新一块仪表盘——这些都不该拖慢 session 的推进。Claude Code 为此支持"异步 hook",在 hook 定义里加一个 `async` 标志来启用。异步 hook 是"发完就忘"的:它把任务在后台触发,生命周期继续推进。代价是,hook 的输出无法再影响生命周期——因为等它跑完,生命周期早已走远。这对异步 hook 适合处理的那类工作而言,恰好是合适的取舍:事情要做,但不必卡在当下。配置长这样。

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "node $CLAUDE_PROJECT_DIR/.claude/hooks/sync-to-backup.js",
 "async": true,
 "timeout": 30
 }
]
 }
]
 }
}
```

(配置含义:在 `PostToolUse` 上挂一条异步 hook,matcher 限定 `Edit / Write`;命令以 `async: true` 标记,启动备份脚本后立刻把控制权交回 session。`timeout: 30` 字段把工作时长封顶在 30 秒;超时的备份脚本会被中止。)

The hook fires after every edit, runs the backup script in the background, and returns control to the session immediately. The timeout field caps the work at thirty seconds; if the backup script has not finished by then, it is terminated. The session continues without waiting. The asynchronous flag is the right choice for any hook whose work is genuinely independent of the immediate flow, and the developer who learns to

这条 hook 在每次编辑后触发,后台跑备份脚本,立刻把控制权交回 session。`timeout` 字段把任务时长封顶在 30 秒——若备份脚本届时仍未完成,就被中止。Session 不会等它。"异步标志"适合一切"工作本身确实独立于眼前这条流"的 hook;学会把

← 上一页

下一节 →

recognize which hooks belong asynchronous and which need to be synchronous will build pipelines that are both safe and fast.

同步、哪些适合异步这件事想清楚的开发者,会搭出既安全又快的流水线。

A reflection before this chapter ends, on what makes the gate-guard-notifier vocabulary so useful in practice. The vocabulary is small enough to hold in working memory, and it covers the cases that come up in real work. A developer who has internalized the three patterns can sit down with a new requirement, ask herself which of the three fits, and have a starting design within a minute. She does not have to re-derive the design space from first principles every time she wants to add a hook. The pattern catalog accelerates the design process, and accelerated design is what allows the developer to add hooks quickly enough that they get added at all. A developer without the vocabulary tends to put off hook work, because each hook feels like a fresh design problem. A developer with the vocabulary tends to add hooks freely, because each hook is a small variation on a familiar pattern.

本章收束前,先来反思一下:为什么"gate / guard / notifier"这套词汇在实务中如此有用?词汇量小到能装进工作记忆,却又覆盖了真实工作中会冒出来的那些情形。一个已经把这三种模式内化了的开发者,面对一项新需求,只需要问自己"它像哪一格",一分钟之内就能给出一个起步设计。每加一条 hook,都不必从"第一性原理"重新推演整个设计空间。这份"模式目录"加速了设计过程——而正是这份加速,让开发者来得及"在需要的时候就把 hook 加进去"。一个没有这套词汇的开发者,往往把 hook 工作一拖再拖——因为每条 hook 看起来都像一道新设计题;一个有了这套词汇的开发者,加 hook 就很轻——因为每条 hook 都只是某个熟悉模式的小变体。

A second reflection concerns the way the three patterns interact with the lifecycle events. Gates are usually `PreToolUse` hooks, because the natural moment to evaluate whether an action should be allowed is just before the action runs. Guards are usually `PostToolUse` hooks, because the natural moment to enforce consistency is just after the action has produced something to enforce on. Notifiers can attach to almost any event, because their job is to observe and report rather than to evaluate or correct. The pattern-event correspondence is not strict, but it is strong, and a developer reaching for the wrong event for her pattern often finds that the hook is harder to write than it needs to be. A gate written as a `PostToolUse` hook can detect a violation but cannot prevent it. A guard written as a `PreToolUse` hook can decline an action but cannot enforce a correction on a result that does not yet exist. Recognizing the natural pattern-event pairing is part of the craft, and pairing them correctly produces hooks that fit cleanly into the lifecycle.

第二条反思,关于三种模式与生命周期事件之间的"贴合"。Gate 通常挂在 `PreToolUse` ——评估"一次动作是否该被允许"的天然时机,就是动作跑起来之前的那一刹那。Guard 通常挂在 `PostToolUse` ——强制一致性的天然时机,是动作刚刚产出"可以去强制"的对象之后。Notifier 可以挂到几乎任何事件上——它的职责只是"观察并报告",既不评估,也不纠正。模式与事件之间的对应,不是死规,但确实强;一个把模式挂错事件的开发者,常会发现自己的 hook 写起来比实际更别扭。把 gate 写成 `PostToolUse` ——它只能察觉违规,却不能阻止;把 guard 写成 `PreToolUse` ——它能拒绝一个动作,却不能在一个尚不存在的"结果"上强加修正。认出"模式-事件"的天然配对,本就是这门手艺的一部分;一旦配对正确,hook 就会干干净净地嵌入生命周期。

A short word about combining patterns within a single hook deserves attention, because the question comes up often. A hook that does both validation and correction, for instance, can be tempting; the developer might want a hook that, after a file is written, both formats it and rejects it if formatting fails. The temptation should be resisted in most cases. A hook that does two jobs is a hook that is harder to debug, harder to reason about,

关于"把多种模式合到同一条 hook 里"这件事,值得专门说一句——因为这种问题会反复冒出来。比如,有人很想写一条 hook,在文件写入后,既做格式化,又在格式化失败时拦截。这种诱惑在多数情况下都该抵抗。一条 hook 干两件事,就意味着它更难以 debug、更难讲清逻辑,

and harder to disable when one of its jobs becomes inappropriate. The cleaner approach is to write two separate hooks, each with a single responsibility, and to bind them to the same event in sequence. The two hooks compose into the combined behavior the developer wanted, but each one remains independently understandable, testable, and maintainable. This single-responsibility discipline at the level of individual hooks is one of the small habits that distinguishes well-designed pipelines from those that have grown into tangled balls of multi-purpose scripts.

也更难在某一件事变得不再合适时单独关掉。更干净的做法是:写两条独立的、各自只负一责的 hook,按顺序绑到同一事件上。两条 hook 组合起来,确实就是开发者想要的那种"复合行为";但每一条都各自"可理解、可测试、可维护"。在"单条 hook 层面"贯彻"单一职责"这条纪律,是把"设计良好的流水线"与"长成乱麻的多用脚本"区分开的小习惯之一。

The next chapter takes the gate pattern in particular and develops it further. The most consequential application of gates in any production-grade Claude Code environment is in the security layer, where hooks protect against dangerous commands, sensitive file access, secret exposure, and other risks that the probabilistic model cannot be trusted to catch reliably. Building a serious security-hook bundle is the next step in this part of the book, and it is the work of the chapter ahead.

下一章要把 gate 模式再单独拎出来,推得更深。在任何"生产可上"的 Claude Code 环境里,gate 最具分量的一项应用,都在安全层——hook 在那里挡掉危险命令、敏感文件的访问、秘密数据的泄露,以及其他那些"概率性模型无法稳定地兜住"的风险。搭一套严肃的"安全 hook 套件",是本书这一部分的下一站,也是下一章要做的事。

# CHAPTER SEVENTEEN · 第十七章

## Security and Permission Hooks: Protecting Your System · 安全与权限 Hook:保护你的系统

"The walls do not have to be high, but they must be where they are needed. A wall in the wrong place is decoration."

— a fortifications engineer, anonymous

"墙不必高,但必须建在该建的地方。建错位置的墙,只是装饰。"

—— 某位堡垒工程师,匿名

Of all the work hooks can do, the most consequential is security. A skill that produces excellent code is valuable. A subagent that runs a thorough review is valuable. But a hook that prevents a destructive command from executing, that catches a secret being written to disk, that stops a force-push from reaching production, is valuable in a different and more serious way. Security hooks are the layer of the engineered Claude Code environment where the cost of a single failure can be substantially higher than the entire benefit of the surrounding configuration. A developer can lose a day of work to a single `rm -rf` that should not have run. The hooks in this chapter exist to ensure that the day is not lost.

在 hook 能做的所有事情里,后果最重的一件,是安全。一条 skill 能产出优秀的代码,有价值;一个 subagent 能跑一次彻底的评审,也有价值。但一条 hook 能拦下"一次本不该执行、却差点执行掉的破坏性命令",能在秘密数据落盘前一把抓住",能"阻止一次 force-push 抵达生产"——它的价值,以另一种更严肃的方式存在。安全 hook 是工程化 Claude Code 环境里这么一层:这里一次失败的代价,可以大幅超过整个周边配置带来的全部收益。一次不该跑却跑了的 `rm -rf`,能让开发者丢掉一整天的工作。本章要讲的这些 hook,存在的全部意义,就是确保那一天的工作不被丢掉。

This chapter builds a security-hook bundle. By the end of it, the reader will have installed five hooks that, together, raise the floor of safety in her Claude Code environment substantially. The five hooks address: dangerous shell commands, protected file paths, secret detection in writes, force-push prevention, and a permission-confirmation gate for high-impact operations. Each hook follows the gate pattern from the previous chapter. Each one is a small, focused expression that does one thing and does it deterministically.

本章要搭出一套"安全 hook 套件"。到本章结尾,读者会装好五条 hook——它们合起来,会显著抬高她 Claude Code 环境的"安全底线"。这五条 hook 分别解决:危险的 shell 命令、受保护的文件路径、写入内容中的秘密数据、force-push 拦截,以及一项"对高影响操作做权限确认"的 gate。每条 hook 都沿用上一章讲过的 gate 模式;每一条都是一条小巧、聚焦、确定性做事的小表达式。

Begin with dangerous shell commands. The pattern is to inspect every Bash command before execution, looking for patterns that the developer has decided should never run. The list of dangerous patterns varies by environment, but a defensible default catches: `rm -rf` on root or home paths, `sudo` combined with destructive commands, `dd` writing to block devices, `mkfs` in any form, `chmod` with wide-open permissions on system paths, and any command that pipes from `curl` or `wget` directly into `bash`. The hook below covers these.

从"危险的 shell 命令"讲起。模式是:在每一条 Bash 命令执行之前,过一遍,看它是否落入"开发者已经决定永远不该跑"的那一沓模式。危险模式清单因环境而异,但有一份"说得过去的默认清单"值得守住: `rm -rf` 涉及根或主目录路径、`sudo` 与破坏性命令的组合、`dd` 写到块设备、任何形态的 `mkfs`、`chmod` 把系统路径的权限设得很开、以及把 `curl` 或 `wget` 的输出直接管道给 `bash` 的任何命令。下面这条 hook 覆盖了这些情况。

← 上一页

下一节 →

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
 }
]
 }
]
 }
}
```

(配置含义:在 PreToolUse 上挂一条 hook,matcher 限定 Bash 工具;命令调用项目 `.claude/hooks` 目录下的 `block-dangerous.sh` 脚本——把核心逻辑放进一个真正的脚本里,而不是塞在行内 shell 表达式中。)

```
#!/usr/bin/env bash
block-dangerous.sh - refuses dangerous shell commands.
CMD=$(jq -r '.tool_input.command')

Patterns that are always blocked.
PATTERNS=(
 'rm[:space:]]+-rf?[:space:]]+(/|~/home|/Users|/etc|/var)'
 'sudo[:space:]]+rm'
 'dd[:space:]]+if=.*[:space:]]+of=/dev/'
 'mkfs\'
 'chmod[:space:]]+777'
 'curl[:space:]]+.*\|[:space:]]*(bash|sh)'
 'wget[:space:]]+.*\|[:space:]]*(bash|sh)'
 ':\(\)\{[:space:]]*:\|:&\}'
)

for PAT in "${PATTERNS[@]"; do
 if echo "$CMD" | grep -qE "$PAT"; then
 echo "Refusing dangerous command. Pattern matched: $PAT" >&2
 exit 2
 fi
done

exit 0
```

(脚本含义:从 JSON 输入中读出 Bash 命令,遍历一个"黑名单正则"数组,只要命中任一条,就向标准错误写一条"Refusing dangerous command. Pattern matched: ..."消息,并以退出码 2 退出;都没命中则以退出码 0 放行。其中末项 `:(){}:&}` 是经典的"fork 炸弹"模式。)

← 上一页

下一节 →

The script reads the command from the JSON input, walks an array of forbidden patterns, and exits with code two on the first match. The patterns are not exhaustive; the developer may add to them as she encounters new things she wants blocked. The shape of the script is what matters: it is a list of patterns, easy to extend, with a clear refusal message that surfaces to Claude on every block.

这段脚本从 JSON 输入中读出命令,在一组"禁模式"数组上挨个走一遍,任一命中就以退出码 2 退出。模式并未穷尽——开发者可以随时把自己新认定的危险模式加进去。脚本真正值得学的,是它的形状:一份"模式清单",易于扩展;而每条命中,都对应一条清晰可见的拒绝消息,会在每次拦截时被送到 Claude 那里。

## Protecting File Paths · 受保护的文件路径

Move to protected file paths. The pattern is similar but applies to `Edit` and `Write` rather than `Bash`. The hook inspects the file path being written to and refuses if the path matches a protected list. The protected list typically includes `.env` files, lockfiles, the `.git` directory,

production configuration files, and any file the developer has decided should require explicit human review for any modification.

下一条,讲"受保护的文件路径"。模式相似,但挂在 `Edit / Write` 上,而不是 `Bash`。Hook 检查"正被写入的文件路径",若落入受保护清单,就直接拒绝。这份清单通常包含: `.env` 文件、lockfile、`.git` 目录、生产配置文件,以及任何开发者已经认定为"任何改动都需显式人工复核"的文件。

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
 }
]
 }
]
 }
}
```

(配置含义:在 PreToolUse 上挂一条 hook,matcher 限定 Edit / Write / MultiEdit 工具;命令调用 `protect-files.sh` 脚本。)

```
#!/usr/bin/env bash
protect-files.sh - refuses edits to protected file paths.
FILE=$(jq -r '.tool_input.file_path')

PATTERNS=(
 '\.env(\.|$)'
 '\.git/'
 'package-lock\.json$'
 'yarn\.lock$'
 'Cargo\.lock$'
 'production\.ya?ml$'
 '\.pem$'
 '\.key$'
)

for PAT in "${PATTERNS[@]"; do
 if echo "$FILE" | grep -qE "$PAT"; then
 echo "Refusing to edit protected file: $FILE" >&2
 echo "Pattern matched: $PAT" >&2
 exit 2
 fi
done

exit 0
```

(脚本含义:从 JSON 输入里抽取出文件路径,遍历一组"路径正则",命中任一就向标准错误写明"被保护的文件 / 命中的模式"并以退出码 2 退出;否则以退出码 0 放行。)

The protection is total. No matter what the user asks, no matter what skill or subagent is firing, the listed paths cannot be edited by Claude. If the developer genuinely needs to modify a protected file, she does so manually outside the hook, or temporarily disables the hook for a single session and re-enables it afterward. The friction is a feature; the protection is a feature; the explicit deliberateness of any modification to these files is a feature.

这种保护是绝对的——无论用户怎么说,无论哪条 skill 或哪个 subagent 在触发,清单里的路径,Claude 都改不了。如果开发者真的需要去改某个受保护的文件,她就在 hook 之外手动改,或者为了单个 session 临时关掉 hook,改完再开回来。这种"摩擦"是 feature,这种"保护"是 feature,对"修改这些文件"这件事的"刻意的审慎"也是 feature。

## Secret Detection · 秘密数据检测

Move to secret detection. This hook is harder to write well, because secrets come in many forms and false positives are annoying. A working compromise looks for the most obvious patterns: AWS keys, GitHub tokens, OpenAI keys, generic high-entropy strings that look like API keys, and the explicit string `private_key` or `BEGIN PRIVATE KEY`. When a write contains any of these patterns, the hook refuses.

下一条,讲"秘密数据检测"。这条 hook 比较难写好——因为秘密数据形态万千,而误报又很烦人。一个"能跑得起来"的折中,就是去匹配那些最显眼的模式:AWS key、GitHub token、OpenAI key、看起来像 API key 的高熵字符串,以及显式的 `private_key` 或 `BEGIN PRIVATE KEY`。一旦写入内容里出现这些模式,hook 就拒绝。

```
#!/usr/bin/env bash
block-secrets.sh - refuses writes that appear to contain secrets.
CONTENT=$(jq -r '.tool_input.content // .tool_input.new_string // ""')

PATTERNS=(
 'AKIA[0-9A-Z]{16}'
 'ghp_[A-Za-z0-9]{36,}'
 'sk-[A-Za-z0-9]{32,}'
 'BEGIN[[:space:]]+(RSA|EC|DSA|OPENSsh|PRIVATE)[[:space:]]+KEY'
 'aws_secret_access_key[[:space:]]*=[[:space:]]*[A-Za-z0-9+/]{40}'
)

for PAT in "${PATTERNS[@]}; do
 if echo "$CONTENT" | grep -qE "$PAT"; then
 echo "Refusing to write apparent secret. Pattern matched: $PAT" >&2
 echo "If this is a false positive, edit the file manually outside Claude Code." >&2
 exit 2
 fi
done

exit 0
```

(脚本含义:从 JSON 输入里抽出 `content` (或 `new_string`,视工具而定),遍历一组"秘密数据"模式;命中任一就向标准错误写"Refusing to write apparent secret ..."及一条"若是误请在 Claude Code 外手动改"的指引,并以退出码 2 退出;否则以退出码 0 放行。其中:AKIA 开头 16 字符大写字母

是 AWS access key ID; ghp\_ 开头 36+ 字符是 GitHub personal access token; sk- 开头 32+ 字符是 OpenAI API key 风格;PEM 私钥头部 BEGIN ... PRIVATE KEY ;以及形如 aws\_secret\_access\_key=... 的赋值行。) )

← 上一页

下一节 →

The script reads the content the tool is about to write, checks it against patterns that match well-known secret formats, and refuses the write if any match. The refusal includes a hint about how to bypass the check legitimately, by writing the file outside Claude Code. This kind of guidance in the refusal message is a small but valuable touch; it preserves the developer's ability to handle edge cases without making her wonder why the hook fired.

这段脚本读出"工具即将写入"的内容,对照一组匹配"知名秘密数据格式"的正则去查,任一命中就拒绝这次写入。拒绝消息里附带了一条"如果这是误报,就在 Claude Code 之外手动改"的提示。把这种"如何合理地绕过"的小提示写进拒绝消息,看似小事,实则很贴心——它保留了开发者处理边界情况的能力,又不会让她摸不着头脑地想"这条 hook 怎么又跳出来了"。

## Refusing Force Pushes · 拦截 Force-Push

Move to force-push prevention. This hook addresses a specific category of git mistake that costs teams real time when it happens. A force push to main or to any protected branch can erase work done by other team members. The hook below intercepts any `git push` command that includes the force flag and refuses if the target branch is one of the protected ones.

下一条,讲 force-push 拦截。这条 hook 解决的是"一旦真发生,会真的让团队痛一阵子"的那一类 git 失误。对 main 或任何受保护分支的 force-push,都可能抹掉团队其他成员的工作。下面这条 hook 拦截任何带 force 标志的 `git push` 命令,若目标分支落在受保护清单里,就拒绝。

```
#!/usr/bin/env bash
block-force-push.sh - refuses force pushes to protected branches.
CMD=$(jq -r '.tool_input.command')

if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease|-f[[:space:]]+); then
 if echo "$CMD" | grep -qE '(origin|upstream)[[:space:]]+(main|master|develop|production|release/>'; then
 echo "Refusing force push to protected branch." >&2
 echo "Force pushes to main/master/develop/production are blocked." >&2
 exit 2
 fi
fi

exit 0
```

(脚本含义:从 JSON 输入里抽出 Bash 命令,先匹配是否含 force 标志(--force / --force-with-lease / -f 后跟空白);若命中,再匹配 remote 与受保护分支对( main / master / develop / production / release/... );同时命中就向标准错误写拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

The script catches the force-push pattern in any of its common forms and checks the target branch against a protected list. The list can be extended for projects with additional long-lived branches. The protection is conservative, blocking even `--force-with-lease`, because in many real cases the developer's

这段脚本会捕捉各种常见形态的 `force-push` 模式,并把目标分支对照一份受保护清单去查。这份清单可以根据项目里额外的长期分支去扩展。这套保护偏保守——它连 `--force-with-lease` 也不放过,因为在很多真实场景中,开发者的

← 上一页

下一节 →

intent in using `--force-with-lease` is benign but the consequences when it goes wrong are still substantial; an explicit override path through manual command-line use outside Claude Code remains available.

用意本是无害的,但"一旦出错后果仍然不小"这件事不会因此改变;真要强推,得绕开 Claude Code 在命令行里手动执行——这条"显式 override"的路径始终是开放的。

The final security hook is a permission-confirmation gate for high-impact operations. Some actions are not strictly forbidden but should never happen without explicit human acknowledgment. Deploying to production. Running database migrations. Sending mass emails. Each of these can be encoded as a gate that prompts the user for confirmation rather than refusing outright. The mechanism uses a `PreToolUse` hook that, when it matches a high-impact pattern, opens a desktop notification or a small terminal prompt asking the user to confirm. If the user confirms, the hook exits with code zero and the action proceeds. If the user declines or the prompt times out, the hook exits with code two and the action is blocked.

最后一条安全 hook,是"对高影响操作做权限确认"的 gate。有些动作并不"严格禁止",但绝不该在"没有显式人工确认"的情况下发生:部署到生产、跑数据库 migration、群发邮件。这些都可以被编码成"提示用户确认而非直接拒绝"的 gate。机制是:一条 `PreToolUse` hook,一旦匹配到"高影响"模式,就弹一个桌面通知或一小段终端提示,问用户"是否允许"。用户允许,hook 以退出码 0 退出,动作放行;用户拒绝或提示超时,hook 以退出码 2 退出,动作被拦。

The implementation varies by operating system, because the notification mechanism differs. On macOS, the `osascript` command can produce a confirmation dialog that returns a result. On Linux, `zenity` or `kdialog` can do the same. On Windows, PowerShell can produce a confirmation dialog. The configuration below uses macOS `osascript` as an example, but the pattern translates directly to the other systems.

具体实现因操作系统而异——通知机制不同。macOS 上, `osascript` 命令可以弹一个会回送结果的确认对话框;Linux 上, `zenity` 或 `kdialog` 同样可以;Windows 上,PowerShell 可以做等价的确认对话框。下面这份配置以 macOS `osascript` 为例,但这套模式可以直接平移到其他系统。

```
#!/usr/bin/env bash
confirm-deploy.sh - requires explicit confirmation before deploy commands.
CMD=$(jq -r '.tool_input.command')

Patterns that require explicit confirmation.
PATTERNS=(
 'kubectl[[:space:]]+apply.*production'
 'terraform[[:space:]]+apply'
```

```

'aws[:space:]]+s3[:space:]]+sync.*production'
'flyway[:space:]]+migrate'
'alembic[:space:]]+upgrade'
)

for PAT in "${PATTERNS[@]"; do
 if echo "$CMD" | grep -qE "$PAT"; then
 RESULT=$(osascript -e "display dialog \"Claude Code is about to run a high-impact command:\n\n$CMD\n\nAllow it?\nCancel\" / \"Allow\""; then
 if echo "$RESULT" | grep -q "Allow"; then
 exit 0
 else
 echo "User declined high-impact command: $CMD" >&2
 exit 2
 fi
 fi
 fi
done

exit 0

```

(脚本含义:从 JSON 输入里抽出 Bash 命令,在一组"高影响命令"模式( `kubectl apply ... production` 、 `terraform apply` 、 `aws s3 sync ... production` 、 `flyway migrate` 、 `alembic upgrade` )上挨个走一遍;命中任一就弹一个 macOS 对话框,把命令原文摆出来,让用户在 "Cancel" / "Allow" 之间二选一;选 "Allow" 则退出码 0 放行,选 "Cancel"(或超时)则向标准错误写 "User declined high-impact command: ..."并以退出码 2 退出。)

← 上一页

下一节 →

The script reads the Bash command from the JSON input, checks it against a list of patterns that match high-impact operations, and if any match, opens a system dialog asking the user whether to allow the command. The dialog is modal; the script blocks until the user responds. If the user clicks `Allow`, the script exits with code zero and the command proceeds. If the user clicks `Cancel` or dismisses the dialog, the script exits with code two and the command is blocked. The user is now in the loop for any operation that touches production, with no possibility of a slip-up where the model decides on its own that a deployment is safe to run.

这段脚本从 JSON 输入里读出 Bash 命令,与一份"高影响操作"模式清单逐项比对,任一命中就弹出一个系统对话框,问用户"是否允许这条命令"。这个对话框是模态的——脚本会在用户回应之前一直阻塞。用户点 `Allow`,脚本以退出码 0 退出,命令继续;用户点 `Cancel` 或直接关掉对话框,脚本以退出码 2 退出,命令被拦。开发者由此被嵌入"任何触及生产的操作"的人在回路,杜绝了"模型自顾自判断某次部署可以安全跑"这种事。

The list of patterns in the confirmation gate is, like the lists in the earlier hooks, intended to be edited as the developer learns. New high-impact operations will surface over time, and each one should be added to the list as it is recognized. The hook is most valuable when its pattern list is comprehensive, because every gap is an operation that could happen without confirmation, and confirmation gates are most useful when they leave no significant gaps.

和前面那些 hook 一样,这份确认 gate 的模式清单也是"边学边改"的——新的高影响操作会随着时间浮现,每识别出一个,就应当补进清单。模式清单越齐全,这条 hook 越有价值;因为每一道缺口,都是一次"可能绕开确认"的潜在操作,而确认 gate 最有用的地方,恰恰是它"没留下什么明显缺口"。

A reflection on the threat model is worth including, because it clarifies what the hooks in this chapter do and do not protect against. The hooks protect against accidental damage caused by Claude itself or by the developer through Claude. They prevent Claude from running an `rm -rf` when the developer asked for help cleaning up files. They prevent Claude from writing a file to `.env` when the developer asked for help with environment variables. They prevent Claude from force-pushing to main when the developer asked for help reorganizing branches. The protection is against the failure mode where the developer's broad request gets translated by the model into an action she did not intend.

关于"威胁模型",值得专门说几句——它能讲清本章的 hook "在防什么、不在防什么"。这些 hook 防的是"由 Claude 自己,或经由 Claude 由开发者触发"的那一类意外损害。它们阻止 Claude 在"开发者只是请它帮忙整理文件"时去跑 `rm -rf`;阻止 Claude 在"开发者只是请它帮忙处理环境变量"时把内容写进 `.env`;阻止 Claude 在"开发者只是请它帮忙整理分支"时去 force-push 到 main。整道防线防的是这样一种失效模式:开发者给出一个比较宽泛的请求,被模型翻译成了她本不打算做的那个具体动作。

← 上一页

下一节 →

The hooks do not protect against a deliberately malicious user. A user with shell access to the developer's machine can edit the hook scripts, disable the hook configuration, or simply run the dangerous command directly outside Claude Code. The hooks are not a security boundary against an adversary. They are a safety boundary against accident, and that is exactly the boundary they are designed to be. A security architect who reads this chapter and concludes that hooks are insufficient as a complete security solution is correct. The right framing is that hooks are one layer in a defense-in-depth model, and the layer they occupy is the one closest to the developer, where most actual incidents in practice originate. The other layers, including operating system permissions, version control protections, and infrastructure access controls, remain necessary, and the hooks complement them rather than replacing them.

这些 hook 防不住"主动恶意的用户"。拿到这位开发者机器 shell 权限的人,可以直接改 hook 脚本、关掉 hook 配置,或者干脆绕开 Claude Code 在终端里直接跑那条危险命令。Hook 不是面向"敌手"的安全边界;它是面向"意外"的安全边界——而这恰恰是它设计要成为的那道边界。一位安全架构师读完本章,得出"hook 不足以构成完整的安全方案"这一结论,是正确的。更恰当的框架是:hook 是"纵深防御"模型里的一层,它占据的那一层,正是"最贴近开发者"的那一层——而绝大多数真实事故,正是在这一层发生的。其他层(操作系统权限、版本控制保护、基础设施访问控制)依然必要;hook 补足它们,而不是取代它们。

Together, the five hooks in this chapter form a security-hook bundle that addresses the most common categories of risk a developer faces in a Claude Code session. The hooks are not exhaustive; security is a continuous practice, not a fixed checklist, and the developer should expect to add to her bundle as her work surfaces new risks. But the five above cover the largest categories at the lowest cost. Installing them takes roughly an hour. The protection they provide compounds over every future session, and the cost of not having them, even once, is potentially substantial.

这五条 hook 合在一起,就构成了一套"安全 hook 套件",覆盖了 Claude Code session 中开发者最常碰到的那些风险类别。它们并不穷尽一切——安全是一项持续实践,不是一份固定清单;随着工作中浮现新的风险,开发者应当预期会持续往里加。但这五条已经以最低的成本覆盖了最大头的几类。装好它们大约要一小时;它们提供的保护,会在之后每一个 session 里持续复利;而"某一次没装"的代价,哪怕只一次,都可能大得惊人。

A second reflection concerns the experience of working with these hooks installed. The hooks are quiet most of the time. They sit in their gate positions, evaluating every relevant tool call, and they almost never block, because the developer almost never asks Claude to do anything dangerous. The cost of their presence is negligible. Then, one day, a hook fires. The developer asked for something that, on careful reading,

would have led Claude to write a secret to a file or to run a command she did not actually intend. The hook stops the action. The session continues without damage. The developer realizes, in retrospect, that she had asked for something she did not mean. The hook just bought her a substantial amount of cleanup time, or worse. This pattern, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even before any incident has demonstrated their value.

第二条反思,关于"装上这些 hook 之后的工作体感"。Hook 大多数时候都安安静静——它们守在 gate 的位置上,对每一条相关工具调用做评估,而几乎从不拦截,因为开发者几乎从来不会让 Claude 去干危险的事。它们常驻的代价几乎为零。然后某一天,某条 hook 跳了出来——开发者提了一个请求,仔细一读,本来会让 Claude 把一段秘密写进文件,或去跑一条她并不真打算跑的命令。Hook 拦下了这个动作,session 没有受到任何损害;开发者事后回看,意识到自己方才开口要的,其实并不是自己真正想做的。这条 hook 在那一刻,替她省下了一大笔收尾的时间——或者更糟。这个模式——"日常成本极低,单次事件回报极高"——正是安全 hook 在"还没出过任何事故来证明它的价值"之时,就值得装上的原因。

A fourth observation, more practical, concerns the maintenance of the security bundle over time. The patterns the hooks check for will need to

第四条观察更偏实操,关于"安全套件随时间的维护"。Hook 所检查的那些模式,要随着开发者的工作一起演化:新的危险命令会被发现;新的受保护路径会随项目生长而变得相关;新的秘密格式会随着接入新的服务、用上新的 key 形状而出现。Hook 套件不是"装完就忘"的一次性安装——它是一份"小但真实"的持续维护;开发者应当每几个月回头看一遍,往清单里加新模式、移掉不再适用的旧模式。每季度花十五分钟维护,就能让套件保持有效;而一旦把 hook 脚本写干净、让模式清单清楚地放在每个文件的最上面,维护起来就轻松得多。钩子写得"易于维护"这条纪律,归根结底,决定了这套件是多年里一直好用,还是悄无声息地漂成无用之物。

evolve as the developer's work evolves. New dangerous commands will be discovered. New protected paths will become relevant as projects grow. New secret formats will emerge as the developer adopts new services that use new key shapes. The hook bundle is not a fire-and-forget installation. It is a small but real piece of ongoing maintenance, and the developer should plan to revisit it every few months, adding new patterns to the lists and removing patterns that no longer apply. Fifteen minutes of maintenance per quarter is sufficient to keep the bundle effective, and the maintenance is much easier to perform if the hook scripts are written cleanly with their pattern lists clearly visible at the top of each file. The discipline of writing hooks that are easy to maintain is, in the end, the discipline that determines whether the bundle remains useful over the years or quietly drifts into uselessness.

evolve as the developer's work evolves. New dangerous commands will be discovered. New protected paths will become relevant as projects grow. New secret formats will emerge as the developer adopts new services that use new key shapes. The hook bundle is not a fire-and-forget installation. It is a small but real piece of ongoing maintenance, and the developer should plan to revisit it every few months, adding new patterns to the lists and removing patterns that no longer apply. Fifteen minutes of maintenance per quarter is sufficient to keep the bundle effective, and the maintenance is much easier to perform if the hook scripts are written cleanly with their pattern lists clearly visible at the top of each file. The discipline of writing hooks that are easy to maintain is, in the end, the discipline that determines whether the bundle remains useful over the years or quietly drifts into uselessness.

A fifth observation concerns the team dimension of the security bundle. Security hooks are valuable when one developer installs them, and they are dramatically more valuable when an entire team installs the same set across all of its projects. A team that has agreed on a standard security bundle, encoded in shared configuration that travels with each project, raises the floor of safety for everyone on the team simultaneously. New team members inherit the bundle automatically the moment they clone any project. The team's collective risk of an incident drops sharply, because the most common categories of accident are blocked across every developer's environment, not just in the environments of the developers who happened to install hooks personally. This is the team-level argument for hooks, and it is the argument that justifies treating the security bundle as a shared engineering asset rather than as a personal preference. Teams that adopt this view tend to have strikingly fewer incidents than teams that leave each developer to her own preferences, and the difference is not subtle.

第五条观察,关于安全套件的"团队维度"。单个开发者装上安全 hook,是有价值的;而当整个团队在所有项目上都装上同一套,价值会被大幅放大。一个团队若就"标准安全套件"达成共识,并把它编码进随每个项目走的共享配置里,就能同时抬高全员的安全底线。新成员一克隆任何项目,套件就自动继承到手;团队的事故总风险会急剧下降——因为最常见的那几类意外,在每一位开发者的环境里都被拦下,而不是只落在"碰巧个人装了 hook"的那几位开发者身上。这是"hook 必要性"的团队级论据,也正是它把"安全套件"从"个人偏好"提升为"团队工程资产"的那条理由。秉持这种看法的团队,通常比"把偏好留给每个人自己"的团队,事故少得多——这份差异,可不是"细微"。

A sixth and final observation concerns the relationship between security hooks and the other two pillars of the framework. Skills and subagents both shape what Claude does within the agent loop, and both can encode warnings about dangerous operations. A skill could include a directive to *never run rm -rf*, and a subagent system prompt could include a similar warning. These shapings are useful, but they are not enough on their own,

第六条、也是最后一条观察,关于"安全 hook"与框架另外两根支柱之间的关系。Skill 与 subagent,都塑造 Claude 在 agent loop 内部的行为,也都可以把"危险操作的警告"编码在内。一条 skill 可以包含"绝对不要跑 `rm -rf`"这种指令,一个 subagent 的 system prompt 也可以放一段类似的警告。这些塑造是有用的,但单靠它们并不够——

← 上一页

下一节 →

because they ride on the model's probabilistic compliance with its instructions. The security hook is what turns the warning into a guarantee. The right architectural pattern is to have all three: a skill that explains what good practice looks like, a subagent that operates within that practice, and a hook that enforces the boundary that even the best skill and subagent cannot guarantee. The three layers reinforce each other. The skill teaches. The subagent applies. The hook enforces. The combination is stronger than any one layer alone, and developers who think of security as a single-layer concern miss the architectural elegance that the three-pillar approach provides.

因为它们都"骑"在模型对其指令的概率性遵循之上。安全 hook,才是把"警告"变成"保证"的那一层。正确的架构模式,是三层都要有:一条 skill 把"好的实践是什么样的"讲清楚;一个 subagent 在那条实践里做事;一条 hook 把边界强制住——这条边界,即便再好的 skill 和 subagent 也无法保证自己不去踩。三层彼此增益:skill 教;subagent 照做;hook 强制。组合起来,比任一层单兵作战都要强;把"安全"当成"单层问题"看的开发者,会错过三支柱方法在架构上的那份优雅。

A third reflection concerns the relationship between hooks and trust. Without hooks, the developer has to trust the model on every action. Trust, in this context, means hoping that the model will interpret her requests correctly and will not produce dangerous side effects. Trust is appropriate for many things, but it is not appropriate for catastrophic failure modes, because the failure rate of any probabilistic system is non-zero, and a non-zero failure rate over many actions eventually produces a failure. Hooks change the relationship. With hooks, the developer

trusts the model for most things and relies on mechanism for the things that matter. This is a better division of labor than asking the model to be perfect, because perfection is not what models are good at, and mechanism is exactly what mechanism is good at.

第三条反思,关于 hook 与"信任"的关系。没有 hook,开发者就只能在每一次动作上信任模型。这里的"信任",其实是"指望着模型正确解读她的请求,且不产生危险的副作用"。对很多事,这种信任是恰当的;但对"灾难性失效"那一类,这种信任并不恰当——任何概率性系统,其失败率都不为 0;而"不为 0 的失败率"摊在足够多的动作上,迟早会引发一次失败。Hook 改变了这种关系:有了 hook,开发者把多数事交给模型去信任,把"真正要紧的事"交给机制去兜底。这比"要求模型完美"是更好的分工——因为"完美"本就不是模型所擅长的;而"机制",恰恰是机制所擅长的。

A reflection before this chapter closes. Security hooks have a quiet quality that distinguishes them from skills and subagents. Skills produce visible value: every commit message, every PR description, every refactor proposal is something the developer sees and uses. Subagents produce visible value too: every parallel review, every specialized output is a tangible artifact. Security hooks produce value that is mostly invisible. Most days, no security hook fires, because the developer is not asking Claude to do anything dangerous. The hooks sit silently. Their value reveals itself only on the day when something goes wrong and a hook stops it from going more wrong. That day, the value of the entire bundle is realized in a single block. The asymmetry, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even if they never fire. The developer who waits until something goes wrong to install them has discovered the value of hooks at the worst possible moment. The

本章收束前的反思:安全 hook 有一种"安静"的特质,把它与 skill、subagent 区分开。Skill 产出的是可见的价值——每一条 commit message、每一条 PR 描述、每一份重构建议,都是开发者看得见、用得上的东西。Subagent 也产出可见的价值——每一次并行评审、每一份专家产物,都是有形有质的工件。安全 hook 产出的价值,几乎全在暗处。大多数日子里,没有安全 hook 跳出来——因为开发者根本没让 Claude 去干危险的事。Hook 们安安静静地蹲着;它们的价值,只在某天"出了岔子、而某条 hook 把更大的岔子拦下"那一刻,才浮出水面。那一天,整套装件的价值,以一次拦截兑现。这种"日常成本极低、单次事件回报极高"的不对称,正是安全 hook 哪怕一次都没真正触发,也仍然值得装上的原因。等到"出了事再装"的开发者,是在最不该的时机才看清 hook 的价值。

developer who installs them as part of her standard environment has bought insurance she will probably never use, which is exactly the kind of insurance worth having.

把安全 hook 作为标准环境的一部分装上的开发者,买了一份自己大概永远用不到的"保险"——而这恰恰是"值得买"的本意。

The next chapter takes hooks one more step into composition. With single hooks installed, the question becomes how multiple hooks work together, how event pipelines emerge when hooks chain, and how the developer can construct sophisticated lifecycle workflows out of the same simple primitives. Hook chains are the final piece of hook design, and they are the subject of the chapter ahead.

下一章,把 hook 再推一步,推入"组合"。单条 hook 装好之后,真正的问题变成:多条 hook 如何协作?hook 链化时,事件流水线如何浮现?开发者又如何用同一批简单的原语,搭出精细的生命周期工作流?Hook 链是 hook 设计最后一块拼图,也是下一章要讲的事。

## CHAPTER EIGHTEEN · 第十八章

## Hook Chains and Event Pipelines · Hook 链与事件流水线

*"A pipeline is what happens when many small actions, each useful in itself, are arranged so that the output of one becomes the context of the next."*

— Doug McIlroy, paraphrased

*"所谓流水线,就是把许多本身各自有用的小动作,安排成'上一个的输出,成为下一个的上下文'。"*

—— 道格·麦克罗伊(意译)

This chapter is the final treatment of hooks before the book turns to composition across all three pillars. The previous three chapters built hooks one at a time: the mechanics, the design patterns, the security applications. This chapter puts hooks together. When multiple hooks fire on the same event, when hooks pass information forward through the lifecycle, when the developer wants a coordinated workflow that involves several lifecycle moments rather than just one, the resulting structure is a hook chain or an event pipeline. This chapter teaches both.

本章是关于 hook 的最后一章,之后书就要谈"三根支柱如何彼此组合"了。前三章,是一根一根地讲 hook:机制、设计模式、安全应用。本章要把 hook 装到一起。多条 hook 在同一事件上触发、hook 在生命周期里把信息往前传、开发者想要的不是一个生命周期节点而是几个——这些情况所衍生出来的结构,就是 hook 链或事件流水线。本章把两者都讲一遍。

Begin with the simpler case. A hook chain is what happens when more than one hook is configured under the same event with the same matcher. Claude Code runs them in the order they are listed in the configuration file. Each hook receives the same input on standard input. Each hook's exit code is independent: one hook's decision to allow or block does not affect what the next hook does. The chain proceeds through all hooks unless one of them blocks, in which case the action is blocked and subsequent hooks in the same chain do not run. The output of each hook is collected, but the hooks themselves do not communicate directly with each other. Communication, when it is wanted, happens through side effects on the file system or through environment variables.

从简单的那种情形讲起。Hook 链,就是"同一个事件、同一个 matcher 下配了不止一条 hook"时发生的事。Claude Code 按配置文件中列出的顺序执行它们。每条 hook 都在标准输入上收到同样的输入;每条 hook 的退出码彼此独立——一条 hook 的"放行/拦截"决定,不影响下一条 hook。链会一路走完整段,除非其中某条拦下,这时动作被拦、链上后续的 hook 不再跑。每条 hook 的输出会被收集,但 hook 彼此之间并不直接通信。需要通信的时候,就通过文件系统的副作用,或通过环境变量。

The Prettier-and-ESLint example from Chapter Sixteen is a small chain. Two hooks fire on the same `PostToolUse` event with the same matcher. The first runs Prettier; the second runs ESLint. The order matters in this case, because Prettier reformats the file and ESLint then applies its fixes on the reformatted version. If the order were reversed, the result would be subtly different in some edge cases. The developer who designs a chain should think about whether the order of hooks is meaningful, and if it is, she should comment the configuration to make the order explicit for future readers.

第十六章的 Prettier + ESLint 例子,就是一条小链:两条 hook 在同一个 `PostToolUse` 事件、同一个 matcher 下触发。第一条跑 Prettier,第二条跑 ESLint。这里顺序重要——Prettier 把文件重新格式化,ESLint 再在格式化后的版本上施加它的修复;若顺序反过来,某些边界情形会得到微妙不同的结果。设计 hook 链的开发者,应当想清楚"顺序是否有意义";若有,就该在配置里加注释,让顺序对未来的读者也是显式的。

A second example, slightly more elaborate, shows a chain that combines a notifier with a guard. The configuration runs after every `Bash` command that Claude executes: it logs the command to an audit file, then checks whether the command has produced any new git changes, and if so, writes a small note to a session-summary file that the developer can review at the end of the day.

第二个例子稍复杂一点,展示一条把 `notifier` 与 `guard` 组合起来的链。配置在 Claude 跑完每一条 `Bash` 命令后触发:先把命令写进审计文件,再检查该命令是否产生了新的 git 变更;若有,就在一份"当日 session 摘要"文件里写一条小备注,供开发者晚些时候回看。

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '\"\\(.session_id)\\t\\(.tool_input.command)\"' >> ~/.claude/bash-history.log"
 },
 {
 "type": "command",
 "command": "if git status --porcelain 2>/dev/null | grep -q . ; then jq -r '.tool_input.command' >> ~/.c"
 }
]
 }
]
 }
}
```

(配置含义:在 `PostToolUse` 上挂两条 hook 形成的链,matcher 限定 Bash 工具;第一条以制表符分隔的 "`\t`" 格式追加到 `~/.claude/bash-history.log` ;第二条检查 `git status --porcelain` 若有变更,就把该命令追加到 `~/.claude/session-summary.txt` 。)

## Two Hooks in Sequence · 两条 hook 串行

The two hooks fire in sequence on the same event. The first appends a tab-separated record to the audit log. The second checks the git status and, if changes are present, appends the command to a session summary. Both run on every Bash call. Both are silent in the success case. Together, they produce an audit trail and a daily summary without any conscious intervention from the developer.

这两条 hook 在同一事件上串行触发。第一条向审计日志追加一条 tab 分隔的记录;第二条检查 git 状态,若有变更就把命令追加到 session 摘要。两者在每次 Bash 调用时都跑;两者在成功路径上都沉默。合起来,就能在开发者完全无须主动介入的情况下,产出一份审计轨迹和一份当日摘要。

Move to event pipelines, which is the more sophisticated case. An event pipeline is a workflow that spans multiple lifecycle events, with hooks at each event reinforcing or extending the others. The classic example is a git pipeline: a `SessionStart` hook ensures the working tree is clean and reports the

进入事件流水线——这是更复杂的那一种情形。事件流水线,是一条跨越多个生命周期事件的工作流;每个事件上都挂着 hook,彼此强化、彼此延展。经典例子就是一条 git 流水线: `SessionStart` hook 确保工作树是干净的,并汇报

← 上一页

下一节 →

current branch, a `PreToolUse` hook on Bash blocks force-pushes to main, a `PostToolUse` hook on Edit or Write runs the formatter, a `Stop` hook checks whether tests pass before allowing the session to end with uncommitted changes. None of these hooks knows about the others directly. But together they form a pipeline that wraps every git-related action in a consistent set of guarantees, and the developer who has installed the pipeline experiences her git work differently because every step is now defended in the right place.

当前分支;Bash 上的 `PreToolUse` hook 拦截 force-push 到 main;Edit/Write 上的 `PostToolUse` hook 跑格式化; `Stop` hook 在 session 结束前检查测试是否通过,以免"session 结束时还存在未提交的变更"。这些 hook 彼此并不直接"知道"对方的存在;但合在一起,它们构成了一条流水线,把每一项与 git 相关的动作,都包进同一套一致的保证里。装上这条流水线的开发者,她的 git 工作会"感觉不一样"——每一步都恰好守在它该被守的地方。

A worked event pipeline is presented below. The configuration is more substantial than any in the previous chapters, because it is doing real cross-event coordination. The pipeline assumes a project where tests are run with `npm test`, the main branch is named `main`, and the developer wants Claude Code to be especially careful with anything that touches that branch.

下面是一条完整的事件流水线。配置比前几章任何一条都更厚实——它在跨事件做实在的协调。这条流水线假定:测试用 `npm test` 跑、主分支名为 `main`、开发者希望 Claude Code 对"触及主分支"这件事格外小心。

```
{
 "hooks": {
 "SessionStart": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --short"
 }
]
 }
],
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
 },
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-force-push.sh"
 }
]
 }
]
 }
}
```

```

 },
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
 }
]
 }
],
 "PostToolUse": [
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\" 2>/dev/null || true"
 }
]
 }
],
 "Stop": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "if [\"$(git status --porcelain | wc -l)\" -gt 0]; then echo '{\"decision\":\"block\",\"reason\":\"'
 }
]
 }
]
}
}

```

(配置含义:这是一个跨四个生命周期事件的完整 git 流水线。SessionStart:打印当前分支和 `git status --short`,注入到 Claude 的启动上下文;PreToolUse(Bash):同时挂 `block-dangerous.sh` 和 `block-force-push.sh` 两条 gate;PreToolUse(Edit|Write|MultiEdit):挂 `protect-files.sh` 一条 gate;PostToolUse(Edit|Write|MultiEdit):跑 Prettier 格式化;Stop:若有未提交变更,向 stdout 打 JSON 决策块 `{"decision":"block","reason": "..."}` ,让模型无法在带未提交变更的情况下结束 session。)

← 上一页

下一节 →

Read the configuration carefully, because it covers four lifecycle events and demonstrates how a serious pipeline holds together. `SessionStart` runs once when the session begins and prints the current branch and any uncommitted changes; the output is injected into Claude's starting context, so Claude knows the state of the world at the start of every session. `PreToolUse` on `Bash` runs the dangerous-command and force-push gates from Chapter Seventeen. `PreToolUse` on `Edit` and `Write` runs the protected-files gate. `PostToolUse` on `Edit` and `Write` runs Prettier. `Stop` runs a check that uses the JSON output format to refuse the session's end if there are uncommitted

请把这份配置仔细读一遍——它覆盖了四个生命周期事件,示范了一条严肃的流水线是怎么"粘合"起来的。`SessionStart` 在 session 启动时跑一次,打印当前分支以及任何未提交的变更;输出会被注入到 Claude 的启动上下文里,因此 Claude 在每个 session 一开始就"知

道外面世界长什么样”。PreToolUse 在 Bash 上挂第十七章讲过的"危险命令"与"force-push"两条 gate。PreToolUse 在 Edit / Write 上挂"受保护文件"的 gate。PostToolUse 在 Edit / Write 上跑 Prettier。Stop 做一项"用 JSON 输出格式"做判定:若存在未提交

changes; the JSON output format is what allows the Stop event to actually refuse the end of the session, because the standard exit-code mechanism cannot stop a session that is already ending. The pipeline as a whole ensures that: Claude always knows the branch and clean-status state of the world; dangerous shell commands are blocked; force pushes to main are blocked; protected files cannot be edited; every file is formatted; and the session cannot end with uncommitted changes. The combination is a substantial improvement over the unconfigured environment, and it costs about a hundred lines of JSON to install.

变更,就拒绝 session 结束。这里之所以要"用 JSON 输出格式"——是因为 Stop 事件对应的"会话即将结束"这件事,光靠"退出码"那条机制拦不住,只有 JSON 决策块才拦得住。这条流水线作为整体,保证了:Claude 总能知道当前分支和 clean-status 状态;危险的 shell 命令被拦;force-push 到 main 被拦;受保护文件改不了;每一个文件都被格式化;session 不会在"带未提交变更"的状态下结束。这套组合,比未配置的环境是一个显著升级;而装好它总共才百来行 JSON。

The Power of Coordination · 协调的力量

The key insight from this pipeline is the coordination across events. None of the individual hooks is doing something the developer could not do manually. She could, in principle, run Prettier by hand after every Claude edit, run git status by hand before every commit, refuse to allow force pushes by typing the wrong command into her own terminal. The point of the pipeline is not that any individual hook is doing something new. The point is that the developer no longer has to remember to do any of it. The pipeline remembers for her. The cumulative effect of having all these little guarantees enforced mechanically, rather than remembered, is that the developer's attention is freed for the parts of the work that genuinely require attention. The mechanical parts run mechanically. The interesting parts get the focus they deserve.

这条流水线带出的关键洞见,是"跨事件协调"本身。任一单条 hook 都没有做开发者"手动做不了"的事——她原则上可以每次 Claude 改完手动跑一次 Prettier、提交前手动跑一次 git status、想 force-push 时自己在终端里敲错命令拒绝自己。流水线真正的意义,并不在"哪一条 hook 做了什么新鲜事";而在于"开发者再也不必记得去做这些"。流水线替她记着。所有的这些"小事"由机制强制而不是靠记忆——其累积效果是:开发者的注意力,被释放出来,留给那些真正需要她注意的工作。机械的部分,机械地跑;有意思的部分,得到它应得的专注。

changes, returning a decision of block with a clear reason that surfaces to Claude.

变更,就以 block 决策回送,并附上一段清晰的 reason,送达 Claude。

A Coherent Workflow · 一条内洽的工作流

The pipeline together enforces a coherent workflow. Sessions begin with awareness of state. Tool calls are filtered through gates appropriate to their kind. Edits are followed by formatting. Sessions cannot end with the working tree dirty. The developer has not had to remember any of these rules, because each rule is enforced by a hook at the right lifecycle moment. The development experience is, in a sense, supervised, but the supervision is mechanical and silent, and it intervenes only when the developer was about to do something the pipeline considers a mistake.

这条流水线,作为一个整体,强制的是一条内洽的工作流:session 启动时即知"世界状态";工具调用按各自类型被相应的 gate 过滤;编辑后即格式化;session 不会在 dirty 工作树的状态下收尾。开发者不再需要记得这些规则中的任何一条——每条规则都被一个 hook 守在它"该被守的"那个生命周期节点上。从某种意义上,这种开发体验是被"监督"着的;但这种监督是机械的、沉默的,只在开发者正要做"流水线认为是个错"的某件事时,才插一手。

A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has not earned its place after a month of use. The pipeline should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

关于事件流水线,有一条值得直接点出的纪律:流水线很强,但它同时也是"累积复杂度"的一种形态;流水线里每条 hook,都是开发者承诺要维护的。一条三四条 hook 精心挑选的小流水线,比"完全没有流水线"已是显著升级;而十五条 hook 的大流水线,可能反而不如七条——因为失效模式成倍增长,累计延迟叠加,多 hook 共同运行时开发者 debug 异常行为的能力也会退化。推荐做法是:小起步、一次加一条、用了一个月还没证明自身价值的 hook 就拿掉。流水线应当服务于开发者,而不是反过来;一条"长成了维护负担"的流水线,就是被过度工程化的流水线。

A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

第二条纪律关于测试。每条 hook 都应当在"提交"前测过。标准测试模式:在 shell 里直接调用它,在标准输入上喂一段"有代表性"的 JSON 输入,然后观察它的退出码和标准错误输出。在直接测试下行为正确的 hook,通常在 Claude Code 内也会正确;而直接测试下都不对的 hook,进了 session 大概率会吐出莫名其妙的失败。安装时花的这十五分钟直测,免去了日后的一整轮 debug。这套模式,任何"做的不止一行命令"的事,都值得无条件采用。

A second worked pipeline is worth presenting, because pipeline design is the kind of skill that matures only after a developer has seen several

第二条完整流水线值得摆出来——因为流水线设计这门手艺,只在看过若干条

substantial examples. The pipeline below is for a different shape of work: a content-generation project, where the developer is using Claude Code to produce written material rather than to write code. The pipeline reflects the different concerns of writing work. Drafts should be backed up automatically. Drafts should be passed through a humanizer pass after every edit. Sessions should end with a daily summary appended to a project journal. The pipeline below implements each of these.

比较"实在"的例子之后,才会真正成熟。下面这条流水线服务的,是一种不同形态的工作——一个"内容生成"项目,开发者用 Claude Code 产出的不是代码而是书面材料。流水线反映了写作这件事的关切:草稿应当自动备份;每次编辑之后,应当过一次"人味化"pass;session 结束时,应当向项目日志追加一份当日小结。下面这条流水线,把每一项都实现了。

```
{
 "hooks": {
 "SessionStart": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "echo '## Today, you are working on: '; cat $CLAUDE_PROJECT_DIR/.claude/today.txt 2>/dev/null"
 }
]
 }
],
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); cp \"$FILE\" \"$CLAUDE_PROJECT_DIR/.backups/${basename \"$FILE\"}";",
 "async": true
 },
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); if echo \"$FILE\" | grep -qE '\\.(md|txt)$'; then $CLAUDE_CODE_EDITOR \"$FILE\" --check-em-dashes"
 }
]
 }
],
 "SessionEnd": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "echo \"$(date '+%Y-%m-%d %H:%M') session ended. Files touched today:\" >> $CLAUDE_PROJECT_DIR/journal.md"
 }
]
 }
]
 }
}
```

(配置含义:这是一条"内容生成"项目的事件流水线。SessionStart:读 .claude/today.txt 焦点文件,把它注入到 Claude 的启动上下文;若文件不存在则给一条提示。PostToolUse(Edit|Write):挂两条 hook——第一条以 async: true 异步地把被编辑的文件备份到 .backups/<文件名>.<时间戳>,不影响 session 速度;第二条仅在文件后缀为 .md / .txt 时,调用 check-em-dashes.sh 检查"em-dash"等被禁用的写作习惯。SessionEnd:向项目 journal.md 追加一行"session 结束"加时间戳,再附上近 5 次提交里被改过的文件名清单。)

Read the pipeline carefully, because its shape is informative. `SessionStart` loads a focus file, which the developer has placed in the project to remind herself what she is working on today. The output is injected into Claude's starting context, so Claude knows the day's focus from the moment the session begins. `PostToolUse` fires two hooks on every `Edit` or `Write`. The first is asynchronous: it backs up the file to a timestamped copy in a backups directory, without slowing down the session. The second runs only on Markdown or text files, and it invokes a custom script that checks for em-dashes and other patterns the developer wants to avoid in her writing. `SessionEnd` appends a record to a project journal, giving the developer a continuous log of what she worked on across sessions. The pipeline does not block any of these actions. It does not interfere with Claude's flow. It quietly produces backups, applies the writing standard, and maintains the journal, all without conscious intervention.

请把这条流水线仔细读一遍——它的形状本身就是有信息量的。`SessionStart` 加载一份"焦点文件",这是开发者放在项目里、用来提醒自己"今天做什么"的;其输出会被注入到 Claude 的启动上下文,于是 Claude 在 session 一开始就掌握了"今日焦点"。`PostToolUse` 在每次 `Edit` / `Write` 上挂两条 hook——第一条是异步的:它把文件复制成一份带时间戳的备份,放进 backups 目录,并不拖慢 session;第二条仅对后缀为 Markdown 或 txt 的文件触发,调用一个自定义脚本,检查 em-dash 以及其他"开发者希望在自己的写作里回避"的模式。`SessionEnd` 向项目日志追加一条记录,给开发者留下跨 session 的一份"自己今天做了什么"的连续流水。流水线不拦截任何动作,不打断 Claude 的节奏,只是悄悄产出备份、应用写作规约、维持日志——一切无须开发者主动介入。

## What Pipelines Buy You · 流水线真正"买到"的是什么

The pipeline is small, but it changes the developer's experience of working on the project substantially. She knows that backups exist. She knows that her em-dash policy is being enforced on every edit. She knows that her journal is up to date. The mental load of remembering these things has shifted from her to the pipeline, and the load that has been removed from her is now available for the actual writing work. This is the deeper benefit of pipelines: they take ambient concerns off the developer's mind and handle them as standing background processes, freeing the developer's attention for the work that requires attention.

这条流水线虽小,却显著地改变了开发者在该项目上的工作体验:她知道有备份存在;她知道每次编辑 em-dash 规约都在生效;她知道日志是同步的。"记得这些事"的心智负担,从她身上挪到了流水线上;而腾出来的那份注意力,如今可以投到真正的写作工作上。这正是流水线更深一层的收益——它把"背景里的事"从开发者脑子里搬走,当成"长期运行的后台进程"去处理,把开发者的注意力释放给那些"真正需要注意力"的工作。

A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to

关于事件流水线,有一条值得直接点出的纪律:流水线很强,但它同时也是"累积复杂度"的一种形态;流水线里的每一条 hook,都是开发者承诺要去维护的——

[< 上一页](#)[下一节 >](#)

maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has

not earned its place after a month of use. The pipeline should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

一份三四条 hook 精心挑选的小流水线,比"完全没有流水线"已是显著升级;而十五条 hook 的大流水线,可能反而不如七条——因为失效模式成倍增加,累计延迟叠加,多 hook 共同运行时开发者 debug 异常行为的能力也会退化。推荐做法是:小起步、一次加一条、用了一个月还没证明自身价值的 hook 就拿掉。流水线应当服务于开发者,而不是反过来;一条"长成了维护负担"的流水线,就是被过度工程化的流水线。

A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

第二条纪律关于测试。每条 hook 都应当在"提交"前测过。标准测试模式:在 shell 里直接调用它,在标准输入上喂一段"有代表性"的 JSON 输入,观察退出码和标准错误输出。直接测试下行为正确的 hook,通常在 Claude Code 内也正确;而直接测试下都不对的 hook,进了 session 大概率会吐出莫名其妙的失败。安装时花的十五分钟直测,免去日后的一整轮 debug。这套模式,任何"做的不止一行命令"的事,都值得无条件采用。

A third discipline concerns version control. The `.claude` directory of a project, including the `settings.json` file that holds the hook configuration, should be version-controlled alongside the source code. Hooks are part of the project's contract for how work happens in the repository. A team member who clones the project for the first time should receive the entire pipeline automatically, without needing to configure anything herself. The version control discipline ensures this. Pipeline changes should be reviewed in pull requests like any other code change, because a hook change can affect every future session in the project, and the impact of a bad hook can be substantial. The same engineering hygiene that applies to source code applies to hook configuration, and projects that treat their `.claude` directory with this discipline end up with stable, predictable, well-understood pipelines that genuinely serve the team.

第三条纪律关于版本控制。项目的 `.claude` 目录(含那份装 hook 配置的 `settings.json`)应当与源码一起进版本控制。Hook 是项目"工作如何发生"这份契约的一部分。第一次 clone 项目的团队成员,应当自动拿到整条流水线,而不必自己再配置什么——版本控制纪律保证了这一点。流水线的变更,应当像其他代码变更一样走 PR 评审;因为一条 hook 的变更会作用于该项目里之后所有的 session,而一条"坏 hook"的影响可能非常显著。源码适用的那套工程卫生,同样适用于 hook 配置;以这种纪律对待自家 `.claude` 目录的项目,最终都会得到"稳定、可预测、被充分理解"的流水线,真正服务于团队。

## Pipelines and Engineering Practice · 流水线与工程实践

A fourth observation, more philosophical, concerns what pipelines mean for the practice of software engineering. The traditional way of enforcing

第四条观察更偏哲学,关于"流水线"对软件工程实践本身意味着什么。传统上,强制项目规约的方式是——

← 上一页

下一节 →

project conventions has been documentation, training, and review. Documentation tells people what the conventions are. Training builds the habits. Review catches violations. Each layer is necessary, and each layer is fallible. A pipeline shifts some of the enforcement from human judgment to mechanism. The conventions that the pipeline enforces no longer need to be documented in a wiki, because they are enforced at the moment they would be violated. The training for new team members shrinks accordingly, because the environment teaches the conventions through its behavior. Review focuses on the things that pipelines cannot enforce, like architectural choices and code quality, rather than on style and formatting issues that mechanism handles cleanly. The result is a team where attention is concentrated on what is genuinely human work, and the mechanical work is done by the mechanism. This is, in the end, the deeper promise of the engineered Claude Code environment, and pipelines are the mechanism that delivers it.

文档、培训、评审。文档把“规约是什么”写下来;培训把习惯建立起来;评审把违规抓出来。每一层都必要,每一层也都靠不住。流水线把一部分强制工作从“人的判断”挪到了“机制”。被流水线强制住的规约,不再需要写到 wiki 里——因为它会在“差点被违反”的那一刻被强制住;对新成员的培训也相应瘦身——因为环境会通过自身的行为“教”出规约;评审则得以聚焦在“流水线兜不住的事”上——比如架构选择、代码质量——而不是把力气花在机制本就能干净处理的风格与格式化上。结果是:整个团队的注意力,被集中到那些真正属于“人”的工作;机械的部分由机制去做。这,正是“工程化的 Claude Code 环境”更深一层的承诺;而流水线,是把这份承诺兑现出来的那条机制。

A final observation about pipelines, before this chapter ends, concerns what happens when several pipelines coexist. A user-level pipeline applies to every project. A project-level pipeline applies inside the specific project. The two pipelines combine: every event that fires runs the hooks from both levels, in a defined order. The combined behavior is the union of the two pipelines, which means that the developer can build a stable personal pipeline at the user level and let projects extend it as needed. A team that has agreed on certain project-level conventions can encode them in the project pipeline. Each developer's personal preferences are layered on top through the user pipeline. The combination is a personalized variant of the team standard, and the developer experiences both layers as a single coherent environment. This layered architecture is one of the quieter strengths of the hook system, and developers who design with it in mind end up with environments that are both standardized and personalized, without conflict between the two.

本章收束前,再补一条关于流水线的观察——关于“多条流水线并存”时会发生什么。用户级流水线作用于所有项目;项目级流水线作用于该项目内部。两者是组合的关系:任一事件触发时,两层级的 hook 都会按既定顺序跑一遍。组合行为就是两层流水线之“并”,也就是说:开发者可以在用户层搭一条稳定、长期的个人流水线,再让项目按需扩展它。团队就“项目级规约”达成的共识,被编码进项目流水线;每位开发者的个人偏好,通过用户流水线叠在它上面。组合起来,就是“团队标准的个性化变体”;而开发者本人体验到的,是两层合为一块“内洽的环境”。这种“分层”架构,是 hook 系统里相当安静的一项强项;按它来设计的开发者,最终会得到一个“既标准化、又个性化”的环境,两层之间互不冲突。

One more observation deserves a place before this chapter closes, about the felt experience of working with a substantial pipeline installed. The pipeline does its work invisibly. The developer does not see the formatting hook fire, because the file simply ends up formatted. The developer does not see the test guard run, because the tests pass and the session continues normally. The developer does not see the audit logger record her

本章收束前,再放一条观察——关于“装上一条相当规模的流水线之后的工作体感”。流水线做的事是“看不见”的:开发者不会“看见”那条格式化 hook 触发,因为文件“自然而然”就格式化好了;不会“看见”那条测试 guard 跑过,因为测试通过,session 照常推进;也不会“看见”审计 logger 把她的

commands, because the log file fills silently in the background. What the developer experiences is a working environment that simply behaves the way she wants it to, as if the conventions she cares about were the natural shape of the tool itself. The configuration is invisible, and the behavior is what remains. After a few weeks of working in such an environment, the developer often forgets that the behavior is the result of configuration at all; the environment feels like the default, and the absence of the configuration on a fresh installation feels strange. This is the deepest sign of a successful pipeline. It has receded into the background, and the work has become foreground. The architecture, once built, asks nothing further of the developer, and the developer's attention is free to focus entirely on what she is making.

命令记下来,因为那份日志是在后台默默填满的。开发者真正"体验到"的,是一个"她想要什么它就怎么表现"的工作环境——仿佛她所看重的那些规约,本就是工具自己的天然形状。配置是不可见的,剩下的只有行为。在这样的环境里工作几周之后,开发者常常会忘记:这种行为其实是配置出来的。环境成了"默认",而一个全新安装、没装这套配置的环境,反倒显得别扭。这就是一条成功流水线最深处的标志:它已经退到背景里,工作被推到了前景。这副架构一旦搭好,不再向开发者索取什么;她的注意力,可以完全聚焦于"自己在造的那个东西"。

A reflection before this chapter and this part of the book end. Hooks are quieter than skills and subagents. They do not produce visible artifacts in most sessions. They sit at the edges of the lifecycle, doing their work without drawing attention, and they earn their place by what they prevent rather than by what they produce. The developer who has installed a thoughtful set of hooks may not notice them for weeks at a time, and that invisibility is exactly what good hook 设计 produces. The hooks that are invisible are the hooks that are working. The day comes, eventually, when a hook fires on a real action that needed to be blocked, or a guard catches a problem that would have shipped, or a notifier surfaces an event that needed attention. On that day, the entire investment in the hook bundle is repaid in a single moment, and the developer recognizes, sometimes for the first time, that the architecture has been protecting her all along.

本章、也是本书这一部分收束前的反思:hook 比 skill、subagent 都更"安静"。在大多数 session 里,它们并不产出可见的工件。它们蹲在生命周期的边缘,默默做事,不引人注目;它们赢得位置,靠的是"它们拦下了什么"而非"它们产出了什么"。装上一套"用心挑选过"的 hook 的开发者,可能连续几周都想不起它们;而这种"隐形",恰恰是好 hook 设计所产生的结果。"看不见"的 hook,才是"在工作"的 hook。终有一天,某条 hook 在一次真该拦的动作上跳出来,或者某条 guard 抓住了一个本该被发出去的问题,又或者某条 notifier 把一件确实需要关注的事顶到台面——就在那一天,装这套 hook 套件的全部投入,以一个瞬间被一次性结清;开发者常常是头一次意识到:这套架构,其实一直在保护她。

Part Four ends here. The reader has, over four chapters, learned the mechanics of hooks, internalized the gate-guard-notifier vocabulary that organizes hook 设计, installed a security-hook bundle, and seen how multiple hooks compose into event pipelines that wrap entire workflows in deterministic guarantees. The third pillar of the framework is now in place. Three pillars stand. The next part of the book takes the architecture as a whole and shows how to build a complete environment in which all three pillars work together. Two end-to-end case studies, each substantial, will demonstrate the architecture in concrete form. By the end of the next part, the reader will have seen the framework not as three separate sets of techniques but as a single coherent system, and she will have the blueprint

第四部分至此收束。读者用四章时间,学会了 hook 的机制、内化了"gate-guard-notifier"这套组织 hook 设计的词汇、装好了一套安全 hook 套件,又亲眼见证了多条 hook 是如何组合成事件流水线、把整条工作流包在确定性保证之中。框架的第三根支柱,至此就位。三根支柱,立住了。本书的下一部分,要把架构作为一个整体来谈:如何在"三根支柱一起工作"的前提下,搭建出一套完整的环境。会有两个端到端的、份量扎实的案例研究,把这套架构以具体形式演给你看。读完下一部分,读者将看到的不再是"三套彼此分立的技术",而是"一个内治的整体系统";她将拿到一份蓝图,

← 上一页

下一节 →

for building such a system in her own work. That synthesis begins in the chapter ahead.

足以在自己的工作里搭出这样一套系统。这场"综合",从下一章开始。

← 上一页

PART FIVE (Ch 19) →

## PART FIVE · 第五部分

### The Composition Layer: Building a Self-Governing Development Environment · 组合层:搭一个"自我治理"的开发环境

Three pillars, one architecture. The whole begins to mean more than the sum of the parts.

三根支柱,一套架构。整体开始意味着"比各部分之和更多"的东西。

## CHAPTER NINETEEN · 第十九章

### The Three-Pillar Architecture: Designing a Complete Claude Code System · 三支柱架构:设计一套完整的 Claude Code 系统

← 上一页

下一节 →

"Architecture is the art of arranging elements so that the whole has properties none of the elements possess. Without arrangement, there is only inventory."

— Christopher Alexander, paraphrased

"所谓架构,就是把元素安排成一种形态,使整体具有任何单一元素都不具备的性质。没有安排,就只剩一份清单。"

—— 克里斯托弗·亚历山大(意译)

Up to this point in the book, the three pillars have been treated separately. Skills were the subject of Part Two, subagents the subject of Part Three, hooks the subject of Part Four. Each part developed its pillar in depth, with mechanics, design patterns, and a working library installed by the chapter's end. The treatment was deliberately separate, because each pillar carries its own design discipline and learning each one in

isolation is the only way to develop the discipline cleanly. But the separation is now over. The remainder of the book treats the three pillars as components of a single architecture, and this chapter is where the architecture itself gets explicit attention.

到本书此前的位置为止,三根支柱是分开讲的。Skill 是第二部分的主题,subagent 是第三部分的主题,hook 是第四部分的主题。每一部分都把各自那根支柱讲得很深——含机制、设计模式,以及"到章末装好的一套能跑起来的库"。这种分开是有意为之——因为每根支柱都自有一套设计纪律,而"一门门分开学",是把纪律练得干净的唯一办法。但"分开"这件事,到此为止。书剩下的部分,会把三根支柱当作一套架构里的若干组件来对待;而本章,正是让这套架构本身"被显式注意"的地方。

The argument of this chapter is that any sophisticated Claude Code environment uses all three pillars, that the pillars have a natural division of responsibility, that the composition between them follows recognizable patterns, and that the resulting environment has properties that no single pillar could produce on its own. The architecture is not arbitrary. It reflects the structure of the tool, the structure of real engineering work, and the structure of the failure modes that the architecture is built to prevent. Once the reader has internalized the architecture, the design of any new Claude Code environment becomes a matter of applying the same patterns to her specific work, with the patterns themselves as a stable starting point.

本章的论点是:任何一套"够复杂"的 Claude Code 环境,都会同时用上三根支柱;三根支柱之间存在一种自然的职责划分;它们彼此组合时,会遵循若干可识别的模式;而组合出来的环境,具有"任一根支柱单兵作战"都达不到的性质。这套架构并非随意——它反映的是工具本身的结构、真实工程工作的结构、以及"这套架构要去防住"的那几类失效模式的结构。一旦读者把架构内化,设计任何新的 Claude Code 环境,就变成"把这套同样的模式套到自己的具体工作上";模式本身,则成为一个稳定的起点。

Begin with the division of responsibility. Each pillar handles a class of problem that the other two cannot handle as well. Skills handle capability; they teach Claude how to do specific kinds of work consistently. Subagents handle specialization and parallelism; they give specific kinds of work their own focused context, their own tuned role, and the option to run alongside other work simultaneously. Hooks handle determinism; they impose mechanical guarantees on the lifecycle, ensuring that what must happen does happen, and what must not happen cannot happen. The three together cover capability, specialization, and boundary, which are the three categories of concern in any serious workflow.

从"职责划分"讲起。每一根支柱,各自处理一类"另两根处理得没那么好"的问题。Skill 处理"能力"——教 Claude 如何把某类工作一致地做对。Subagent 处理"专门化"与"并行"——为某类工作配上聚焦的上下文、调好的角色,以及"和其他工作并行跑"的可能性。Hook 处理"确定性"——对生命周期施加机械性的保证,让"必须发生的"确实发生,"绝不能发生的"绝不能发生。三者合起来,覆盖了"能力 / 专门化 / 边界"这三类——它们正是任何严肃 workflow 都绕不开的三个关切类别。

← 上一页

(下一批翻译) →

A simple way to remember the division is by the question each pillar answers. Skills answer the question, how should this be done. Subagents answer the question, who should do this. Hooks answer the question, what is permitted. A workflow that uses only skills can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete one.

记住三者分工的一个简单办法,就是看每一柱回答的是哪个问题。Skills 回答的是"这件事该怎么做"。Subagents 回答的是"这件事该由谁做"。Hooks 回答的是"这件事什么被允许做"。只装 skills 的工作流,能教会 Claude 把活做漂亮,但既无法把不同类的工作隔离开来,也

无法保证规约被执行。只装 subagents 的工作流,有专门化但没有沉淀下来的知识,也没有强制力。只装 hooks 的工作流,有强制力但没有能力,也没有专门化。任一柱单独存在,都只是一个"局部解";三柱一起,才构成一个完整的答案。

## Two Layers of the Architecture · 架构的两层

The next observation, which is more subtle, is that the three pillars sit at different layers of the architecture. Skills and subagents both operate inside the agent loop. They are read by the model. They shape the model's behavior through prose. They live in the probabilistic space where the model interprets, reasons, and decides. Hooks operate outside the loop. They run as shell commands, not as language. They are not interpreted by the model. They are mechanical. The two layers, probabilistic inside and deterministic outside, are the natural partitioning of responsibility in a well-designed environment. The model is given freedom and capability inside the loop. The boundaries of the loop are deterministic. This shape, what one practitioner has called the deterministic outer and the probabilistic inner, is the central architectural pattern of the book, and it is worth holding in working memory because it explains almost every design decision in any complete environment.

下一条观察更微妙一些:三柱其实处在架构的不同层面上。Skills 和 subagents 都是在 agent loop 内部工作的——它们由模型读取,通过对模型吐出的文字(prose)塑形;它们栖身在那个"模型在其中解读、推理、做决定"的概率性空间里。Hooks 则是在 loop 外部工作的——它们以 shell 命令的形式运行,不是语言,也不会被模型"解读";它们是机械的。这两层——内部概率性、外部确定性——正是一个设计良好的环境里"职责"最自然的切分方式。模型在 loop 内被给予自由和能力;loop 的边界则是确定性的。这个形状(有位实践者称之为"外层确定、内层概率")是本书的中心架构范式,值得放在工作记忆里——因为它能解释任何一个完整环境里几乎所有的设计决策。

A worked illustration brings the architecture to life. Consider a developer who needs to perform a code review. The work has several components. The reviewer needs to know how good code reviews are written in this codebase. The reviewer needs to apply different kinds of attention to different aspects of the change. The reviewer needs to be prevented from accidentally approving changes that touch protected paths. Each of these components maps cleanly to one of the three pillars. The how is encoded in skills: a project-level `pr-review` skill that knows the team's preferred review format. The who is encoded in subagents: parallel security, performance, and architecture specialists who each handle their dimension.

举一个落地的例子,可以让这套架构活起来。想象一位开发者要执行一次 code review。这项工作包含好几个组件:reviewer 需要知道"在这个代码库里,一份好的 code review 应当怎么写";reviewer 需要对变更的不同方面施加不同种类的注意力;reviewer 还需要被防止在"被改的文件刚好落在受保护路径上"时不小心点了 approve。这每一个组件,都能干净地映射到三柱的其中一柱上。"怎么做"编码在 skill 里:一个 project 级别的 `pr-review` skill,知道本团队偏好的评审格式。"由谁做"编码在 subagent 里:并行运行的安全、性能、架构专员,各自处理自己那一维。

← 上一节

下一节 →

The *what-is-permitted* is encoded in hooks: a gate on `PreToolUse` that blocks any tool call that would mark a PR approved without explicit human confirmation when the PR touches authentication code.

"什么被允许"编码在 hooks 里:一道挂在 `PreToolUse` 上的 gate,会在"PR 触及身份认证相关代码、且没有任何人工确认时"拦截任何把它标为 approved 的工具调用。

The composition runs naturally. The developer asks for a review. Skills fire because the request matches their descriptions. Subagents are dispatched in parallel because the orchestrator pattern recognizes the multidimensional shape of the work. Hooks watch the lifecycle silently. The review unfolds in two minutes, the unified result is presented to the developer, and the deterministic guarantees protect the boundary that the model alone could not be trusted to respect. None of this required the developer to think about the architecture during the request. She typed a sentence. The architecture did the work.

三柱的组合自然地跑起来。开发者请求一次评审;skill 因为请求匹配了 description 而触发;subagent 因为 orchestrator 范式识别出了"这件事是多个维度"而被并行派出去;hooks 在生命周期里静默地守着。两分钟后,review 跑完,统一的结果呈到开发者面前,确定性的那条边界——模型本身无法被信赖会去守的那条——被 hooks 守住了。整件事,并不要求开发者在请求时心里还惦记着这套架构。她只敲了一句话,架构就替她把活干了。

A second observation about the architecture concerns the order of construction. The natural order in which a developer should build her environment is the order in which the pillars were taught in this book: skills first, subagents second, hooks third. The order is not arbitrary. Skills are the broadest and most generally useful pillar; they pay back immediately in almost every session, and they are the easiest to write well on the first attempt. Subagents are the next layer; they require slightly more design effort but produce dramatic benefits when the work has the right shape. Hooks are the most specialized; their value is concentrated in specific safety-critical concerns and in workflow enforcement, and they require the most careful design to install correctly. Building in this order means that the early investment produces immediate benefits, the middle investment produces architectural benefits, and the late investment produces the deterministic guarantees that complete the system. A developer who tries to build in reverse order, hooks first and skills last, often finds the experience frustrating, because hooks without skills and subagents are just constraints on a vanilla environment, and the constraints feel restrictive rather than enabling.

关于架构的第二条观察,是关于"建造顺序"的。开发者搭建她环境时,自然的顺序,就是本书讲述这三柱的顺序:先 skills,再 subagents,最后 hooks。这个顺序不是随意的。Skills 是覆盖面最广、最普适的那一柱;它几乎在每一个 session 里都能立刻见回报,而且第一次写就写得不错的难度也最低。Subagents 是下一层;它需要稍微多花点设计力气,但只要工作形状对得上,带来的收益是戏剧性的。Hooks 则是最专门化的;它的价值集中在某些"安全攸关"的具体关切,以及 workflow 强制,装的时候也需要最仔细的设计。按这个顺序搭,意味着前期的投入立刻就见到效益,中期的投入换来架构层面的效益,后期的投入换来"让系统真正闭环"的那些确定性保证。要是反过来——先装 hooks,最后才装 skills——体验常常很令人沮丧:没有 skills 和 subagents 的 hooks,只是套在一个"素面"环境上的约束,这种约束会让人感到被束缚,而不是被赋能。

## Failure Modes Prevented · 架构所规避的失效模式

A third observation, this one about the failure modes the architecture prevents. Each pillar prevents a different category of failure. Skills prevent the failure of inconsistency: without skills, the developer's own attention has to carry every convention, and her attention is fallible. Subagents

第三条观察,是关于"这套架构规避了哪些失效模式"。每一柱规避的是不同类别的失效。Skills 规避的是"不一致"这种失效:没有 skills,每一个规约都得靠开发者自己的注意力去扛,而人的注意力是会出错的。Subagents

← 上一节

下一节 →

prevent the failure of context pollution: without subagents, the main session's context fills with the residue of every phase of work, and the quality of every phase suffers. Hooks prevent the failure of probabilistic breach: without hooks, the model's judgment is the only line of defense

against catastrophic mistakes, and the model's judgment is non-zero failure rate. The architecture, taken as a whole, is a defense in depth against the three most common ways that AI-assisted workflows go wrong. A developer who has installed all three pillars has, in effect, hardened her workflow against the three failure modes that would otherwise haunt it.

规避的是"上下文污染"这种失效:没有 subagents,主会话的上下文会被每一阶段工作的残留物填满,每一阶段的质量都会跟着掉。Hooks 规避的是"概率性破防"这种失效:没有 hooks,模型自己的判断就成了抵御灾难性错误的唯一防线,而模型的判断是有一个非零失败率的。把三柱合起来看,这套架构对"AI 辅助工作流最常见的三种出错方式"形成了一道纵深防御。一个装好了三柱的开发者,事实上是在把她的工作流硬化,以抵御那三种若不这样就会始终缠着她的失效模式。

A fourth observation concerns the design process for adding new capabilities to the environment. When the developer notices a friction in her work that she wants to address, she should ask herself which pillar fits. The classification questions are simple. Is this a knowledge gap, where the developer wishes the environment knew something specific? That is a skill. Is this an attention gap, where the developer wishes a different kind of attention were applied to a kind of work? That is a subagent. Is this a guarantee gap, where the developer wishes that something would always or never happen? That is a hook. Asking the right question first prevents the most common design mistakes, which are building skills that should have been hooks, building hooks that should have been skills, and building subagents that should have been narrow skills inside a regular session. The classification is fast, the right answer is usually obvious once the classification has been made, and the cumulative effect of asking the question every time is an environment whose components live in the right places.

第四条观察,是关于"向环境里加新能力"的设计流程。当开发者在工作中留意到一处她想去消除的摩擦时,她应当先问自己"这该落到哪一柱上"。分类问题本身很简单:这是一个"知识缺口"吗——也就是你希望环境知道某件具体的事?那就是 skill。这是一个"注意力缺口"吗——也就是你希望"某类工作被施加一种不同种类的注意力"?那就是 subagent。这是一个"保证缺口"吗——也就是你希望某件事要么永远发生、要么永远不发生?那就是 hook。先问对问题,可以规避最常见的设计错误:把本该做成 hook 的事做成 skill,把本该做成 skill 的事做成 hook,把本该是"普通 session 里一个窄 skill"的事做成 subagent。分类很快,一旦分类完成,正确答案通常就一目了然;而每次都问一遍这个问题,长年累月下来,环境里的每一个组件都会落在该落的位置上。

A fifth observation concerns over-engineering. The temptation to build an elaborate environment is real, and most developers go through a phase where they install too many skills, too many subagents, too many hooks, and the environment becomes harder to maintain than the work it was supposed to support. The remedy is to start small, add components only when their need is concrete, and remove components that have not been used in a few months. A small, well-designed environment of fifteen skills, six subagents, and eight hooks is dramatically more useful than a sprawling one with fifty of each. The compounding of component count past a certain point becomes a tax rather than a benefit, because the developer has to keep more in her head, the failure modes multiply, and the cost of maintenance

第五条观察是关于"过度工程化"的。搭建一个庞大环境的诱惑是真实的;大多数开发者都会经历这么一个阶段——装了太多 skills、太多 subagents、太多 hooks,环境变得比它本该支持的工作还难维护。解法是:起步要小,只在需求具体且明确时才加新组件,几个月没用上的就清掉。一个轻巧、设计良好的环境(15 个 skills、6 个 subagents、8 个 hooks)远比一个松散铺开的 50 个同款有用得多。组件数一旦越过某个临界点,就开始变成"税"而不是"福利"——因为开发者脑子里要装的东西变多,失效模式也成倍增加,而维护的成本,

← 上一节

下一节 →

starts to consume the benefit of the components themselves. The discipline of restraint is part of the architecture.

开始反噬组件本身带来的好处。"克制"这条纪律,本身也是这套架构的一部分。

A sixth observation concerns the maintenance posture of the environment. A well-built environment is not a static artifact. It evolves with the developer's work. New skills get added when new frictions emerge. Old skills get refined as the developer's preferences change. Subagents get reshaped as the developer learns which roles benefit most from isolation. Hooks get updated as new threats and conventions emerge. The `.claude` directory of any active developer should look noticeably different at the end of a year from how it looked at the start, because the developer's environment has been growing alongside her work. The maintenance investment is small; thirty minutes per week is sufficient to keep an environment current and useful. The compounding of that investment, across the years a developer uses her environment, is what produces the gap between developers who treat their tooling seriously and developers who do not.

第六条观察是关于“环境的维护姿态”。一个搭建得好的环境,不是一个静态的产物——它会跟着开发者的工作一起演化。新摩擦出现时,会有新 skill 加进来;开发者偏好变了,旧 skill 会被精修;在实践中摸清“哪些角色最受益于隔离”之后,subagent 会被重新塑形;新的威胁、新的规约浮现时,hook 会被更新。任何一位活跃开发者的 `.claude` 目录,在一年末尾的样子,都应当和年初的样子明显不同——因为她的环境,一直在和她的工作一起长大。维护的投入并不大:每周 30 分钟,就足以让一个环境保持鲜活、好用。而这份额不大的投入,在开发者使用她环境的这些年里不断复利,正是“认真对待自己工具的开发者和”不这么做的开发者”之间那条鸿沟的来源。

## What This Architecture Means · 这套架构意味着什么

The final observation in this chapter is the one that matters most for the rest of the book. The composition principle, repeated throughout these chapters, is not just a design pattern. It is the difference between using Claude Code and engineering with it. A developer who has built a thoughtful three-pillar environment is doing something fundamentally different from a developer who has not. She is operating inside an environment that has been shaped to her work, that absorbs frictions she would otherwise carry, that protects her from mistakes she would otherwise make, and that produces a level of consistency and quality across her output that no amount of careful prompting alone could match. The investment to build this environment is real but bounded. The benefit, once built, compounds for as long as the developer continues to work in the environment. There is no comparable investment available to a software engineer in 2026 that produces anything close to the leverage of a well-designed Claude Code environment, because no other tool sits at the intersection of capability, specialization, and enforcement that Claude Code does.

本章的最后一条观察,是往后整本书里最关键的一条。在前面这些章节里反复出现的“组合原则”,不只是一个设计范式——它是“用 Claude Code”和“用 Claude Code 做工程”之间那条分水岭。一个搭建了深思熟虑的“三柱环境”的开发者,和一个没有这样做的开发者,做着本质不同的事。前者是在一个已经被塑造得贴合她工作的环境里运转:环境吸收她原本要自己扛的摩擦、保护她免于她原本会犯的错、并让她产出的一致性和质量达到“再仔细的 prompt 工程”也匹配不上的水平。搭建这个环境的投入是真实但有限的;一旦搭好,收益会在她继续使用环境的每一天里不断复利。在 2026 年,软件工程师能做的投资里,没有哪一项能在杠杆效率上接近“一个设计良好的 Claude Code 环境”——因为没有任何别的工具,同时坐在能力、专门化、强制这三件事的交汇点上,而 Claude Code 坐在那里。

The next two chapters take the architecture from principle to practice. Each chapter is a substantial case study in which a complete environment is

接下来两章,要把这套架构从原则带到实践。每一章都是一个分量十足的案例研究——会从头到尾为一个特定种类的工作,搭出一个完整的环境——

← 上一节

下一节 →

built end-to-end for a specific kind of work. The first case study, in the chapter that follows, is a full-stack development environment for a TypeScript engineer. The second, in the chapter after that, is a solopreneur's environment for a technical founder running product, marketing, and operations from a single Claude Code installation. Together, the two case studies demonstrate the architecture across two very different shapes of work, and they leave the reader with concrete blueprints she can adapt to her own situation. The architectural principles end here. The applied builds begin in the chapter ahead.

端到端地搭出来。第一个案例在紧接的这一章:为一位 TypeScript 工程师搭的全栈开发环境。第二个案例在再后面那一章:为一位独立创业者搭的、用于"从一份 Claude Code 安装里同时跑产品、营销、运营"的环境。两个案例合起来,把同一套架构展示在两种非常不同的工作形状上,也会给读者留下可以拿回去套自己情况的、具体可用的蓝图。架构原则到此为止;实战搭建,从下一章开始。

Before turning to the case studies, a few additional architectural patterns deserve explicit attention, because they show up in almost every well-built environment and the reader should recognize them when she sees them in the chapters ahead.

在转向案例之前,还有几条额外的架构范式值得明说——它们几乎会出现在每一个搭得好的环境里,读者应当能在接下来两章里一看到它们就认出来。

The first pattern is what might be called the project-skill, user-subagent split. In well-designed multi-developer environments, project-specific knowledge tends to live in skills at the project level, while role specialists tend to live as subagents at the user level. The reason is that knowledge is project-specific (a Python project has different conventions from a TypeScript project) but roles are usually personal (a developer's preferred way of approaching code review does not change much when she switches projects). The split allows the developer to maintain a stable personal subagent library that she carries from project to project, while each project contributes its own skills that augment her subagents with project-specific knowledge. The composition pattern from Chapter Fourteen depends on this split working cleanly, and developers who set up their environments with the split in mind from the start avoid a common refactoring later when they realize their subagents have accumulated project-specific knowledge that should have lived elsewhere.

第一条范式,可以叫做"项目级 skill、用户级 subagent"的拆分。在设计良好的多开发者环境里,"项目特有的知识"倾向于作为 skill 落在 project 级别;"角色专家"则倾向于作为 subagent 落在 user 级别。理由在于:知识是项目特有的(Python 项目和 TypeScript 项目的规约不一样),而角色通常是个人的(开发者偏好的 code review 方式,她换了项目也基本不变)。这种切分让开发者可以维护一个稳定的"个人 subagent 库",从项目带到项目;而每个项目自己贡献自己的 skill,在她的 subagent 之上"长出"项目特有的知识。第十四章讲过的"组合范式"依赖这种切分能干干净净地跑;从一开始就按这个切分来组织环境的开发者,会避免掉后来一种很常见的重构:她突然发现,自己的 subagent 不知道什么时候塞进了"本不该属于它"的项目特有知识,得重新拆出来。

## Ambient Versus Targeted · 环境级 vs 触发级

The second pattern is the ambient-versus-targeted split, which was introduced in Chapter Two but deserves restatement in the architectural context. Information that is true at all times in a project belongs in `CLAUDE.md`, where it is loaded into every session's context unconditionally. Information that is true only when a specific kind of work is being done belongs in a skill, where it is loaded only when the skill activates. The line

第二条范式是"环境级 vs 触发级"的切分,第二章引入过,但放在架构的上下文里值得再讲一遍。一个项目里"任何时候都为真"的信息,放在 `CLAUDE.md` 里——它会在每个 session 的上下文里无条件被加载。"只在做某类特定工作时才为真"的信息,放在 skill 里——只有当这个 skill 被触发时,它才会被加载。这条线

between the two is sometimes blurry, but a useful rule is that any piece of knowledge that costs more than fifty words to express, and that is relevant to less than a quarter of the work in the project, almost certainly belongs in a skill rather than in `CLAUDE.md`. Putting too much in `CLAUDE.md` inflates the baseline context cost of every session and degrades the model's attention. Putting too little in `CLAUDE.md` makes the developer brief Claude on the same project facts every session, which defeats the purpose of having ambient context at all. The architectural discipline is to push as much as possible into skills and keep `CLAUDE.md` to genuinely ambient material.

有时候画得不太清,但有一条好用的经验:任何"表达起来超过 50 个字"、且"相关性覆盖不到项目四分之一工作"的知识块,几乎可以肯定该放在 skill 里,而不是 `CLAUDE.md` 里。`CLAUDE.md` 里塞太多,会撑大每个 session 的基线上下文成本,让模型的注意力变差; `CLAUDE.md` 里塞太少,又会让开发者每个 session 都要把同一组项目事实再向 Claude 复述一遍——这就完全失去了"环境级上下文"存在的意义。架构层面的纪律是:能推到 skill 里的就尽量推到 skill 里,把 `CLAUDE.md` 留给真正"环境级"的那些内容。

The third pattern is the hooks-as-the-last-resort principle. When the developer is considering whether to enforce some convention with a hook or with a skill, the right default is to start with a skill and only escalate to a hook if the skill is not enough. Skills are softer, more flexible, easier to maintain, and more conversational. Hooks are harder, more rigid, and harder to debug when they go wrong. A convention that the model can reliably honor when reminded should live in a skill. A convention that the model genuinely cannot be trusted to honor, where the consequences of a single failure are substantial, should live in a hook. This default keeps the architecture honest. The developer who reaches for hooks first ends up with a brittle environment that is hard to evolve. The developer who reaches for skills first and escalates to hooks only when necessary ends up with a flexible environment whose enforcement layer is concentrated where it actually earns its place.

第三条范式是"hooks 作为最后手段"原则。当开发者面临"用 hook 还是用 skill 来强制一条规约"的选择时,正确的默认是先 skill,只有当 skill 不够用时才升级到 hook。Skill 更软、更灵活、更易维护、也更像对话;hook 更硬、更刚性、出错时也更难排查。"模型只要被提醒一下,就能稳定地遵守"的规约,应当住在 skill 里;"模型真的不能被信赖去遵守、且一旦失败代价极大"的规约,才应当住到 hook 里。这个默认值守住了架构的诚实。先伸手去抓 hook 的开发者,最后会得到一个脆弱、难以演进的环境;先伸手去抓 skill,只在必要时才升级到 hook 的开发者,会得到一个灵活的环境,其强制层恰好集中在它真正值得出场的地方。

The fourth pattern concerns the lifecycle of any specific configuration entry. Every skill, subagent, and hook the developer adds to her environment goes through a predictable lifecycle. It is created in response to a specific friction. It is used heavily for some period. It either earns its place by being used continuously or fades into disuse. Either way, after roughly six months, the developer should review the entry and ask whether it is still pulling its weight. Entries that have not been used in months should be removed. Entries that have been used heavily but slightly off-fit should be refined. Entries that have been used as a starting point for several variants should perhaps be split into more specific siblings. This kind of pruning maintenance is what keeps an environment from accumulating dead weight, and the developers who treat their `.claude` directories as living

第四条范式是关于"任意一条配置项的生命周期"的。开发者每加进环境里一个 skill、一个 subagent、一个 hook,都会走一条可预测的生命周期:因某个具体摩擦而被创建;有一段时间被频繁使用;之后要么靠持续被使用而"赢得自己的位置",要么慢慢淡出。不管哪种,大概 6 个月,开发者都应当回头 review 一下这条项,问问"它还在撑着场面吗"。几个月没被用过的项,删掉;被频繁用但用得有点别扭的,精修;已经被当作多个变体起点的,考虑拆成几个更专门的兄弟项。这种"剪枝式"的维护,正是让一个环境不至于积累"死重"的关键——而那些把 `.claude` 目录当"活的工件"对待、

projects, gardened regularly, end up with environments that stay sharp for years rather than gradually decaying into clutter.

像对待项目一样、定期打理它的开发者,他们环境会保持锋利好些年,而不是慢慢腐烂成一堆乱糟糟的东西。

The fifth pattern, and the last to name explicitly before the case studies, is the small-and-good principle. A small environment of fifteen well-designed components beats a large environment of fifty mediocre ones, almost always. The temptation to build large is real, especially after the first few components produce visible benefits. The discipline is to resist building large for its own sake, and to add components only when their need is concrete and undeniable. Many of the most effective environments in practice contain fewer than thirty components total. Their effectiveness comes from how well-tuned the components are, not from their count. The reader who internalizes the small-and-good principle will produce environments that work beautifully and remain maintainable. The reader who does not will produce environments that impress at first glance and frustrate at the first attempt to evolve them six months later.

第五条范式,也是进入案例前最后要明说的,叫"小而精"原则。一个 15 个组件、个个设计得好的小型环境,几乎在所有情形下都胜过 50 个组件、但个个平庸的大型环境。诱惑是真实存在的——尤其是前几个组件带来了可见收益之后,人会很容易飘起来。要守的纪律是:不为"大"而大,只在需求具体且不可否认时才加新组件。实践中很多最有效的环境,总数都不到 30 个。这些环境之所以有效,靠的是每个组件都被调得有多好,而不是组件的数量。把"小而精"原则真正吃进去的读者,会造出用起来漂亮、维护起来也轻的环境;吃不进去的,会造出"乍一看令人印象深刻、但六个月后一要演进就让人抓狂"的环境。

A final architectural observation, before the case studies begin, concerns the relationship between the three-pillar architecture and the deeper craft of software engineering. Engineers who have worked on production systems for years recognize the architecture in this book as familiar. Skills resemble the configuration files that have always shaped good engineering systems. Subagents resemble the worker pools and microservice patterns that decompose monolithic systems into focused specialists. Hooks resemble the middleware, the policy enforcement points, and the observability layers that wrap deterministic guarantees around probabilistic services. The three-pillar architecture is not, in its bones, a new invention. It is the application of well-established software-architecture patterns to the specific problem of working with an AI assistant. This continuity is reassuring, because it means that the discipline this book asks the reader to develop is the same discipline that produces good software in general. A developer who builds her Claude Code environment well becomes a slightly better software engineer along the way, because the patterns she practices on her environment are the same patterns she should practice on her production systems. The book's teaching, in this sense, transfers beyond the tool.

进入案例之前,最后一条架构观察,是关于"三柱架构"和"软件工程这门更深的手艺"之间关系的。在生产系统上工作过多年的工程师,会在本书这套架构里认出自己的老熟人。Skills 像那些一直以来"塑造优秀工程系统"的配置文件;subagents 像把单体系统拆成专门化"小而专"的 worker pool 和微服务范式;hooks 像那些"把确定性保证包在概率性服务外头"的中间件、策略执行点和可观测性层。三柱架构骨子里并不是什么新发明;它是把已经成熟的软件架构范式,应用到了"和 AI 助手协作"这个具体问题上。这种连续性让人安心——它意味着本书让读者培养的纪律,和"做好的软件"这件事本身所要求的纪律,其实是同一种。开发者把 Claude Code 环境搭好,顺带会成为稍微好一点的软件工程师,因为她在环境上练的这些范式,正是她应该在生产系统上也练的那些。从这个意义上,本书所教的东西,溢出了工具本身。

## CHAPTER TWENTY · 第二十章

### Case Study: Building a Full-Stack Development Environment

#### 案例研究:搭建一个全栈开发环境

*"The proof of the architecture is in the building. Until something stands, the design is only an opinion."*

— a structural engineer, anonymous

*"架构是否成立,要在盖起来之后才算数。在有什么东西真正立在那儿之前,设计只是意见。"*

—— 一位结构工程师(匿名)

This chapter is a build. The reader will follow, end to end, the construction of a complete Claude Code environment for a single specific kind of work: full-stack web development in TypeScript, with React on the front end and Node with PostgreSQL on the back end. The reader does not need to be a TypeScript developer for the chapter to be useful. The point of the case study is the shape of the construction, not the specific technologies. A reader who works in a different stack can read the chapter as a template and substitute her own languages, her own frameworks, her own conventions. The skeleton is what matters. The flesh is illustrative.

这一章就是一场“搭”。读者会从头到尾跟着,把一份完整的 Claude Code 环境搭出来——针对一种非常具体的工作:TypeScript 全栈 Web 开发,前端 React,后端 Node + PostgreSQL。读者本人并不需要是 TypeScript 开发者,这一章才有价值。这个案例研究的重点,是“搭建的形状”,而不是具体技术栈。读者在用别的技术栈,完全可以把这一章当作一个模板来读,把语言、框架、规约换成自己的。骨架才是重点;血肉只是示例。

The developer in this case study is a senior engineer at a small product company. She works in a single repository that contains a Next.js front end, a Node API server, a PostgreSQL database with Prisma as the ORM, a test suite written in Vitest, and deployment configuration in Docker and Kubernetes. The team has agreed conventions for how new features are scaffolded, how tests are written, how database migrations are produced, how PR descriptions are written, and how releases are tagged. The conventions are presently maintained in a wiki that nobody reads after their first week on the team. The developer wants to encode the conventions into her Claude Code environment so that the environment enforces and applies them automatically, freeing her from having to remember them and freeing her teammates from having to be reminded of them in every code review.

案例里的这位开发者,是某小型产品公司的一位资深工程师。她在单一仓库里工作:Next.js 前端、Node API 服务、用 Prisma 作为 ORM 的 PostgreSQL 数据库、用 Vitest 写的测试套件、Docker + Kubernetes 的部署配置。团队已经就一些规约达成了一致:新 feature 怎么搭脚手架、测试怎么写、数据库 migration 怎么生成、PR description 怎么写、release 怎么打 tag。这些规约目前被维护在一个 wiki 里——而入职第一周之后,没人再去看那个 wiki。这位开发者希望把规约编码进她的 Claude Code 环境,让环境自己去强制、自己去应用,免得她每次都要去记、也免得队友每次在 code review 里被反复提醒。

The build will produce, in the project's `.claude` directory, a configuration that includes eight skills, four subagents, and six hooks. By the end of the chapter, the directory will be committed to the repository, every developer who clones the project will inherit the environment

这次搭建会在项目里的 `.claude` 目录下,产出一份包含 8 个 skills、4 个 subagents、6 个 hooks 的配置。等这一章结束时,这个目录会被提交进仓库;之后凡是 clone 这个项目的开发者,都会自动继承这一环境——

automatically, and the daily experience of working in the project will be substantially different from what it was before the build began.

在这个项目里"日常工作的体验",会和搭建开始之前有明显不同。

Begin with the directory structure. The `.claude` directory at the project root is organized as follows.

先从目录结构开始。项目根目录下的 `.claude` 长这样:

```
.claude/
├─ settings.json
├─ skills/
│ ├─ component-scaffold/
│ ├─ api-route-scaffold/
│ ├─ prisma-migration/
│ ├─ pr-description/
│ ├─ commit-message/
│ ├─ test-conventions/
│ ├─ api-doc/
│ └─ error-format/
├─ agents/
│ ├─ security-auditor.md
│ ├─ performance-reviewer.md
│ ├─ test-writer.md
│ └─ migration-planner.md
└─ hooks/
 ├─ block-dangerous.sh
 ├─ protect-files.sh
 ├─ format-on-save.sh
 └─ tests-must-pass.sh
```

Each of the components has a specific job in the larger system. The skills encode the project's knowledge. The subagents provide specialized roles for the major review and authoring tasks. The hooks enforce the deterministic guarantees. Together, the configuration makes a substantial development workflow possible to perform in a single Claude Code session, with the environment carrying most of the load that the developer used to carry herself.

每一个组件,都在这套大系统里承担着特定职责。Skills 编码的是项目里的知识;subagents 为"主要的 review 与撰写任务"提供专门化角色;hooks 强制的是确定性保证。三者合起来,使得"在一次 Claude Code 会话里完成一份相当分量的开发工作流"成为可能——原本由开发者自己扛的那一部分负担,现在由环境接管了。

Begin building with the skills, because skills are the foundation. The first skill is `component-scaffold`, which generates a new React component in the project's conventional layout. A React component in this codebase is a folder containing the component file, a test file, a Storybook story, and an index re-export. The folder convention is that each component lives at `src/components/[ComponentName]/`, with PascalCase naming throughout. The skill captures this convention and the templates for each generated file.

先从 skills 搭起——skills 是地基。第一个 skill 是 `component-scaffold` ,作用是按项目的规约布局生成一个新的 React 组件。这个代码库里,一个 React 组件是一个目录,里面装:组件文件本身、一份测试文件、一份 Storybook story、以及一个 index 的再导出。规约上的目录约定是:每个组件住在 `src/components/[ComponentName]/` 下,全程 PascalCase 命名。这个 skill 把这条规约和每个被生成文件的模板一起,都收进了它的定义里。

← 上一节

下一节 →

The second skill is `api-route-scaffold` , which generates a new API route in the Next.js conventional layout, with the route handler, the validation schema, the test file, and the OpenAPI documentation fragment. The skill captures the team's convention that every route has explicit validation using Zod, every route has a test that exercises the happy path and at least two error cases, and every route has a corresponding OpenAPI fragment that lives alongside the route file.

第二个 skill 是 `api-route-scaffold` ,作用是按 Next.js 的规约布局生成一条新的 API 路由:包含路由 handler、validation schema、测试文件、以及 OpenAPI 文档片段。这个 skill 把团队规约也一起收进去了:每条路由必须用 Zod 做显式校验;每条路由的测试必须覆盖 happy path 加至少两种错误情形;每条路由必须有一份与之同址存放的 OpenAPI 片段。

## The Component Scaffold Skill · 组件脚手架 skill

```

name: component-scaffold
description: Scaffolds a new React component in the project's conventional layout, complete with test file, Storybook
allowed-tools: Read, Write, Glob

Component Scaffold

When asked to scaffold a component, follow this procedure.

1. Get the component name from the user. Convert to PascalCase if needed.
2. Create the folder at `src/components/[ComponentName]/`.
3. Create the four files using the templates in `assets/`:
 - `[ComponentName]/index.ts`, re-exports the component as a named export
 - `[ComponentName]/[ComponentName].tsx`, the component itself
 - `[ComponentName]/[ComponentName].test.tsx`, Vitest test file
 - `[ComponentName]/[ComponentName].stories.tsx`, Storybook story
4. Substitute the placeholder `__COMPONENT_NAME__` with the actual name in each file.
5. Confirm by listing the four files created.

Use functional components with hooks. Use TypeScript strict mode. Define props as a named interface above the compon
```

## The API Route Scaffold Skill · API 路由脚手架 skill

```

name: api-route-scaffold
description: Scaffolds a new API route in the Next.js conventional layout, complete with handler, Zod validation sch
allowed-tools: Read, Write, Glob

API Route Scaffold
```

When asked to scaffold an API route, follow this procedure. (正文接下页)

注:上面这段 YAML 是该 skill 的核心定义。name 字段是 skill 的标识; description 字段是触发语言; allowed-tools 限制 skill 激活时可用的工具集;body 部分( --- 之后)则是 Claude 在执行时遵循的步骤与规约。

注:上面这段 YAML 是该 skill 的核心定义。name 字段是 skill 的标识; description 字段是触发语言; allowed-tools 限制 skill 激活时可用的工具集;body 部分( --- 之后)则是 Claude 在执行时遵循的步骤与规约。

← 上一节

下一节 →

## The Migration Skill · Migration skill

The third skill is `prisma-migration`, which produces a Prisma migration file from a description of the schema change, with the up migration, the down migration, and a note about whether the change is backward-compatible.

第三个 skill 是 `prisma-migration`,作用是:给定一段对 schema 变更的描述,产出一份 Prisma migration 文件——包含 up migration、down migration、以及一条关于"这次变更是否向后兼容"的说明。

## API Route Scaffold(续) · API 路由脚手架(续)

When asked to scaffold an API route, follow this procedure.

1. Get the route path and HTTP method from the user.
2. Create the route handler at ``app/api/[path]/route.ts``.
3. Create the validation schema at ``app/api/[path]/schema.ts``. All input must be validated with Zod before being processed.
4. Create the test file at ``app/api/[path]/route.test.ts``. Cover happy path, validation failure, and one business-logic failure.
5. Create the OpenAPI fragment at ``app/api/[path]/openapi.yaml``. Document the request shape, the response shape, and the error shape.
6. Wire the route into the API surface following the project's convention.

Every route must validate its input with Zod. Every route must produce errors in the project's standard error format.

## Prisma Migration Skill · Prisma Migration skill

```

name: prisma-migration
description: Produces a Prisma migration from a described schema change, with up and down migrations and a backward-compatible note.
allowed-tools: Read, Write, Bash

Prisma Migration
```

When asked to create a migration, follow this procedure.

1. Read the current schema at ``prisma/schema.prisma``.
2. Apply the change to the schema. Be conservative: prefer additive changes (new columns, new tables) over destructive changes.
3. Generate the migration file using ``npx prisma migrate dev --create-only --name [descriptive_name]``.
4. Read the generated migration. Verify it does what was intended.
5. If the migration is destructive, write a note at the top of the migration file explaining what is being changed and why.

6. Report:
- What changed

注:以上是 `prisma-migration` skill 的核心定义,body 的第 6 步"报告什么变了"在下一页继续展开。

注:以上是 `prisma-migration` skill 的核心定义,body 的第 6 步"报告什么变了"在下一页继续展开。

← 上一节

下一节 →

The fourth skill is `pr-description`, which produces a pull request description in the team's standard format. This is a near-copy of the personal `pr-description` skill from Chapter Nine, adjusted for the team's specific conventions.

第四个 skill 是 `pr-description`,作用是按团队的标准格式生成一份 pull request 描述。这个 skill 和第九章里的"个人版 `pr-description` skill"几乎是同一份,只是按本团队的具体规约做了几处微调。

## Prisma Migration(续) · Prisma Migration(续)

- Whether the change is backward-compatible (can old code run against the new schema)
- Whether a data migration is needed in addition to the schema migration
- Recommended deploy order (migration before code, or code before migration)

Do not run ``prisma migrate deploy`` or apply the migration. The user reviews and applies it manually.

## PR Description Skill · PR Description skill

```

name: pr-description
description: Drafts a pull request description in the team's standard format. Use whenever the user asks for a PR de
allowed-tools: Bash, Read, Grep

PR Description

When asked for a PR description, follow this procedure.

1. Identify the diff. Run `git log` and `git diff` against the merge base on main.
2. Read the diff in full.
3. Produce the description in this exact format:

Summary
A single sentence in the imperative mood describing the change.

What changed
A bulleted list of concrete changes, one bullet per logical unit.

Schema changes
If the diff includes any Prisma migrations, summarize the schema changes here. State whether they are backward-compa

Testing
Describe how the change was tested. List any new test files created.
```

```
Risk
One sentence categorizing risk as low, medium, or high with a one-line reason.
```

注:上面这段是 `pr-description` skill 的核心定义,body 中"Deploy notes"小节将在下一页继续。

注:上面这段是 `pr-description` skill 的核心定义,body 中"Deploy notes"小节将在下一页继续。

← 上一节

下一节 →

The fifth skill is `commit-message`, identical to the Chapter Five version but installed at the project level rather than the user level so that every developer in the team uses the same conventions.

第五个 skill 是 `commit-message`,和第五章那一份完全一致,只是装在 `project` 级别而不是 `user` 级别——这样团队里的每一位开发者都会使用同一套规约。

The sixth skill is `test-conventions`, which encodes the project's specific test conventions: Vitest, the location pattern for test files, the use of MSW for HTTP mocking, the convention of one `describe` block per function and one `it` block per scenario. The skill is loaded by the `test-writer` subagent when it begins working, so that the subagent automatically applies the project's conventions without needing to be briefed every time.

第六个 skill 是 `test-conventions`,作用是把本项目特有的测试规约编码进去:Vitest、测试文件的存放位置模式、用 MSW 做 HTTP 模拟、每个函数对应一个 `describe` 块、每个场景对应一个 `it` 块等。当 `test-writer` subagent 一开始工作,就会加载这个 skill ——这样 subagent 就能自动套用本项目规约,不必每次被人提醒。

## PR Description(续) · PR Description(续)

```
Deploy notes
Any non-default deployment considerations, such as environment variable changes, manual data migrations, or required

Do not invent changes that do not appear in the diff.
```

## Test Conventions Skill · Test Conventions skill

```

name: test-conventions
description: Encodes this project's test conventions, including framework choice, file layout, mocking patterns, and
allowed-tools: Read

Test Conventions

This project uses Vitest. Test files are colocated with the file under test, with a `.test.ts` or `.test.tsx` suffix

Structure
- One `describe` block per public function or component.
- One `it` block per scenario. Every public function should have at minimum: happy path, one boundary, one error.
```

– Use `beforeEach` for setup that resets state. Avoid `beforeAll`.

### ## Mocking

- Use Vitest's `vi.mock` for module mocks.
- Use MSW (Mock Service Worker) for HTTP mocks. Set up handlers in the test file's `beforeAll`.
- Never make real network calls in tests.
- Never write to the real file system in tests. Use `memfs` if file operations are needed.

注:上面这段是 `test-conventions` skill 的核心定义,剩余的"## Database"、"## Async"、"## Naming"三个小节将在下一页继续。

注:上面这段是 `test-conventions` skill 的核心定义,剩余的"## Database"、"## Async"、"## Naming"三个小节将在下一页继续。

← 上一节

下一节 →

The seventh skill is `api-doc`, which produces an OpenAPI documentation fragment for a given route. It is similar to the `api-route-scaffold` skill in that it produces OpenAPI output, but its job is to update existing documentation rather than to create scaffolding for new routes.

第七个 skill 是 `api-doc`,作用是:给定一条路由,产出它对应的 OpenAPI 文档片段。它和 `api-route-scaffold` 相似——两者都产出 OpenAPI 文档——但它的工作是"更新已存在的文档",而不是为新路由搭脚手架。

The eighth skill is `error-format`, which encodes the team's standard error format. The format includes a numeric code, a human-readable message, and an optional details object. The skill is short but loaded by other skills and subagents whenever they need to produce or parse errors.

第八个 skill 是 `error-format`,作用是:把团队标准的错误格式编码进去。格式包含一个数字 code、一条人类可读的消息、以及一个可选的 details 对象。这个 skill 本身不长,但其他 skill 和 subagent 一旦需要产出或解析错误,就会去加载它。

With the eight skills in place, the project has its conventions encoded as a small library that any session in the project can draw on automatically. Move next to the subagents, four of which complete the specialist team.

8 个 skills 各就各位之后,项目的规约就被编码成了一座小巧的库——项目中任何一个 session 都可以自动取用。接下去是 subagents,4 个 subagent 凑齐这支"专家小队"。

The first subagent is `security-auditor`, identical in shape to the version from Chapter Eleven. The second is `performance-reviewer`, also identical to its Chapter Thirteen counterpart. Both are general-purpose specialists that benefit from the project's skills when they activate.

第一个 subagent 是 `security-auditor`,结构上和第十一章的版本完全一致。第二个是 `performance-reviewer`,也和第十三章的对应物完全一致。这两个都是通用型的专家,激活时还能享受到项目里 skills 的加持。

## The Test Writer Subagent · Test Writer subagent

The third subagent is `test-writer`, customized slightly for this project. The customization is in the description, which now mentions Vitest and the project's specific test conventions, and in the system prompt, which directs the subagent to load the `test-conventions` skill at the start of its work.

第三个 subagent 是 `test-writer`, 针对本项目做了轻微定制: 定制落在两处——description 中点名了 Vitest 和本项目特有的测试规约; system prompt 中指示这个 subagent 一开工就先把 `test-conventions` skill 加载起来。

Test Conventions(续) · Test Conventions(续)

```
Database
- Tests that touch the database use a transaction that is rolled back after the test.
- Never use the production or staging database for tests.

Async
- Always await async operations explicitly. Do not rely on returned promises being awaited by the test runner.
- Use `vi.useFakeTimers()` for tests that depend on time.

Naming
- Test files: `[FileName].test.ts` or `[ComponentName].test.tsx`.
- Test descriptions: imperative, e.g. "creates a user when given valid input," not "should create a user."
```

The fourth subagent is `migration-planner`, used when the developer is contemplating a substantial migration such as a Next.js major-version upgrade or a switch from Prisma to a different ORM. The subagent is identical in shape to its Chapter Thirteen version.

第四个 subagent 是 `migration-planner`, 用于“开发者正在认真考虑一次大型 migration”的情形——比如 Next.js 的主版本升级, 或者把 Prisma 换成别的 ORM。它的结构和第十三章的版本完全一致。

With four subagents installed, the project has access to specialized parallel review and authoring capabilities for security, performance, testing, and migration planning. The orchestrator pattern from Chapter Twelve

4 个 subagent 装好之后, 这个项目就有了“安全、性能、测试、migration 规划”这四个维度上的专门化并行 review 与撰写能力。第十二章的 orchestrator 范式——

Test Writer Subagent · Test Writer subagent 定义

```

name: test-writer
description: Writes Vitest test files for a given function or module, following this project's test conventions. Use
tools: Read, Write, Bash, Glob
model: sonnet

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the ha
```

Your method:

1. Activate the project's test-conventions skill at the start of your work.
2. Read the file under test and any closely related files needed to understand its behavior.
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
  - The happy path
  - At least one boundary condition
  - At least one error or invalid-input case
5. Mock external dependencies using the project's mocking patterns (Vitest `vi.mock` for modules, MSW for HTTP).
6. Place test files alongside the code under test, following the project's naming convention.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in
- You do not write tests that only call the function and assert that it does not throw.
- You do not mock the function under test. You mock its dependencies.
- You do not produce excess of nearly-identical tests. Three meaningful tests beat ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

注:以上是 `test-writer` subagent 的完整定义。`tools` 字段限定它可用的工具集(本案例中拿不到 `WebFetch`); `model: sonnet` 指明它跑在 Sonnet 模型上;System prompt 用了"方法步骤 + 显式禁止项"的双层结构,以避免它写出"只验证函数不抛异常"那种空壳测试。

注:以上是 `test-writer` subagent 的完整定义。`tools` 字段限定它可用的工具集(本案例中拿不到 `WebFetch`); `model: sonnet` 指明它跑在 Sonnet 模型上;System prompt 用了"方法步骤 + 显式禁止项"的双层结构,以避免它写出"只验证函数不抛异常"那种空壳测试。

← 上一节

下一节 →

activates naturally when the developer asks for broad reviews, and the four subagents work in parallel to produce a multidimensional report.

在开发者提出"宽口径 review"请求时就会被自然触发,4 个 subagent 并行工作,产出一份多维度的报告。

Move next to the hooks. Six hooks complete the configuration, and they live in the `.claude/hooks` directory as separate scripts referenced from the `settings.json` file. The `settings.json` itself looks like this.

接下去是 hooks。6 个 hooks 收尾整个配置;它们住在 `.claude/hooks` 目录下,作为独立的脚本被 `settings.json` 引用。  
`settings.json` 本身长这样:

## Settings.json · settings.json(节选,接续下页)

```
{
 "hooks": {
 "SessionStart": [
 {
 "hooks": [
 {
```

```

 "type": "command",
 "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --short"
 }
]
},
"PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
 }
]
 },
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
 }
]
 }
],
"PostToolUse": [
 {
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",

```

注:这段 JSON 是 `settings.json` 的开头。完整结构由四个事件构成: `SessionStart` (打 git 状态)、`PreToolUse` (两道门: `block-dangerous.sh` 给 Bash、`protect-files.sh` 给文件编辑)、`PostToolUse` (运行格式化与测试)、`Stop` (检查未提交变更)。余下内容在下一页继续。

注:这段 JSON 是 `settings.json` 的开头。完整结构由四个事件构成: `SessionStart` (打 git 状态)、`PreToolUse` (两道门: `block-dangerous.sh` 给 Bash、`protect-files.sh` 给文件编辑)、`PostToolUse` (运行格式化与测试)、`Stop` (检查未提交变更)。余下内容在下一页继续。

← 上一节

下一节 →

The `block-dangerous.sh` and `protect-files.sh` scripts are the same security gates from Chapter Seventeen, slightly expanded for this project to add the production deployment paths and the `.env.production` file as protected. The `format-on-save.sh` script runs Prettier and ESLint on every file that Claude edits or writes. The `tests-must-pass.sh` script runs the test suite scoped to the changed file's area, and if any tests fail, returns a JSON decision-block that surfaces the failure to Claude with a directive to fix the failing tests before continuing. Together, the four `PreToolUse` and `PostToolUse` hooks form a tight enforcement pipeline that runs on every action Claude takes inside the project.

`block-dangerous.sh` 和 `protect-files.sh` 这两道门和第十七章那两道一样——只是为本项目做了点小扩展:把"生产部署路径"和 `.env.production` 也加进了受保护名单。`format-on-save.sh` 会在 Claude 每次编辑或写文件时跑一遍 Prettier 和 ESLint。`tests-must-pass.sh` 则会跑"限定在被改文件所属区域"的测试套件;一旦有失败,会回一份 JSON decision-block,把失败暴露给 Claude,要求它在继续之前把失败的测试修好。四道 PreToolUse + PostToolUse hook 合起来,组成了一个紧贴的强制流水线——Claude 在项目里每动一下,都会被它"过一遍"。

The `Stop` hook is a final guard against ending a session with uncommitted changes. The hook checks the `git status` at session end, and if any changes are present, it returns a JSON decision-block that prevents the session from ending and surfaces a clear instruction to commit or stash before continuing. This is a small but real piece of workflow enforcement; it makes it impossible to walk away from the terminal with work in an inconsistent state.

`Stop` hook 是最后一道门,防止你"带着未提交的变更就退出 session"。这个 hook 会在 session 结束时检查 `git status`;一旦发现有任何未提交的变更,就回一份 JSON decision-block,拦住 session 退出,并把一条清晰的指令推到面前——"先 commit 或 stash 才能继续"。这是一条小但真实的"工作流强制":它让人不可能从终端前走开时,工作还处在不一致的状态里。

## Settings.json(续) · settings.json(续)

```
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/format-on-save.sh"
 },
 {
 "type": "command",
 "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
 }
]
 },
 "Stop": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "if [\"$(git status --porcelain | wc -l)\" -gt 0]; then echo '{\"decision\":\"block\",\"rea"
 }
]
 }
]
}
```

注:以上是 `settings.json` 的下半段。PostToolUse 在文件被改/写后跑 `format-on-save.sh` + `tests-must-pass.sh`; Stop 在 session 结束那一刻检查 `git status --porcelain` ——只要行数大于 0,就把"未提交变更"这条理由回给 Claude,强制它继续直到退出条件满足。

注:以上是 `settings.json` 的下半段。PostToolUse 在文件被改/写后跑 `format-on-save.sh` + `tests-must-pass.sh`; Stop 在 session 结束那一刻检查 `git status --porcelain` ——只要行数大于 0,就把"未提交变更"这条理由回给 Claude,强制它继续直到退出条件满足。

The configuration is now complete. Eight skills, four subagents, six hooks. The `.claude` directory is committed to the repository, and from the next session forward, every developer who works in this project inherits the entire environment. The work that previously required the developer to remember the conventions, apply them manually, and verify them after the fact, now runs automatically. The conventions are encoded in skills. The specialist work is dispatched to subagents. The boundaries are enforced by hooks.

配置到此完成。8 个 skills、4 个 subagents、6 个 hooks。`.claude` 目录被提交进仓库;从下个 session 开始,在本项目里工作的每一位开发者,都会继承这一整套环境。原本要由开发者自己"记规约、手工应用、再事后核对"的那一大堆工作,现在自动就跑了。规约编码在 skills 里;专家工作派给 subagents;边界由 hooks 守着。

## A Day in the Environment · 在这个环境里的一天

A walk through a representative day in the project will make the difference visible. The developer opens Claude Code in the morning. The `SessionStart` hook fires and shows her the current branch and any uncommitted changes from the previous day, which there are none of, because the `Stop` hook from the previous session would have prevented her from leaving the terminal with uncommitted work. She decides to add a new feature: a user profile endpoint with avatar upload support. She types into Claude Code, in plain language, scaffold an API route at `/api/users/profile` that accepts PUT requests with a multipart body containing an avatar file.

走一遍这个项目里"代表性的一天",可以让这种差别变得可见。开发者早晨打开 Claude Code。`SessionStart` hook 触发,把当前 branch 和前一天留下的未提交变更显示给她——这次什么也没有,因为上一个 session 的 `Stop` hook 早就不允许她"带着未提交的工作走开"。她决定加一个新 feature: 一个支持上传头像的 user profile 接口。她用大白话在 Claude Code 里敲:给我搭一个 `/api/users/profile` 路由,接受带 multipart body 的 PUT,body 里装一个头像文件。

The `api-route-scaffold` skill fires automatically because the request matches its description. Claude creates the route handler, the Zod schema, the test file with three tests, and the OpenAPI fragment. The `format-on-save` hook runs Prettier on each new file as it is written. The `tests-must-pass` hook runs the project's tests after each write; the new tests pass, because the scaffold includes them complete, and the hook is silent.

`api-route-scaffold` skill 自动触发——因为请求匹配上了它的 description。Claude 创出 route handler、Zod schema、含三个测试的测试文件、还有 OpenAPI 片段。`format-on-save` hook 在每个新文件落地时跑一遍 Prettier。`tests-must-pass` hook 在每次写完后跑一遍项目测试;新测试全部通过(因为脚手架把它们都生成了完整),于是 hook 静默。

The developer then asks for a commit. The `commit-message` skill fires. Claude reads the staged changes, produces a conventional commit message in the project's format, and presents it. The developer approves it. The commit happens. The `format-on-save` hook runs again on each file, finds nothing to change, and exits silently.

开发者接着要一次 commit。`commit-message` skill 触发。Claude 读 staged changes,按项目格式生成一条 conventional commit 消息,呈到面前。开发者点了 ok,commit 完成。`format-on-save` hook 又跑一遍每个文件——没东西要改——于是又静默退出。

She then asks for a PR description. The `pr-description` skill fires. Claude reads the branch's diff against main, produces a PR description in the team's exact format with all five sections including the deploy notes section that flags the new endpoint as needing the file-upload size limit increased in the production environment, and presents it. She copies it into the GitHub PR.

接着要一份 PR 描述。`pr-description` skill 触发。Claude 读 branch 相对 main 的 diff,按团队精确的格式生成 PR 描述——五个小节一个不少——其中"Deploy notes"小节特别指出:新接口上线需要在生产环境调高文件上传大小限制。她把它原样粘到 GitHub PR

She wants a thorough review of the change before requesting reviewers. She types, perform a full review of this branch covering security, performance, testing, and architecture. The orchestrator pattern recognizes the multidimensional shape of the request and dispatches four subagents in parallel: `security-auditor`, `performance-reviewer`, the `test-writer` (in review mode), and an `architecture-reviewer`. Each runs in its own context. Each loads the project skills it needs. Two minutes later, four reports return. The unified review notes that the file upload code does not validate the file size on the server side (security flag), that the avatar processing happens synchronously in the request handler when it should be queued (performance flag), and that the test coverage is solid. She addresses the security and performance flags before requesting reviewers.

在叫 review 之前,她想对这次变更做一次彻底的审查。她敲:对这个 branch 做一次完整 review,覆盖安全、性能、测试、架构。Orchestrator 范式识别出请求的"多维度"形状,并行派出 4 个 subagent: `security-auditor`、`performance-reviewer`、`test-writer` (review 模式)、以及 `architecture-reviewer`。每个 subagent 跑在各自的上下文里,各自加载项目里它需要的那些 skill。两分钟后,4 份报告回来。综合 review 指出:文件上传那段代码在服务端没校验文件大小(安全 flag);avatar 处理在 request handler 里同步发生,本该走队列(性能 flag);测试覆盖是扎实的。她在叫 reviewer 之前,先把安全和性能两条 flag 处理掉了。

The whole interaction has taken her twenty-five minutes. Without the environment, the same work would have taken closer to two hours, and several of the issues the review caught would likely have been merged in and discovered later. The environment did the conventions work. The environment did the review work. The environment enforced the boundaries. She did the design and judgment work, which is what she was hired for in the first place.

整个交互花了 25 分钟。没有环境的话,同样的活会接近两小时,而且 review 抓出来的那几条问题很可能就这么被合进去,之后才被发现。环境做了规约的工作;环境做了 review 的工作;环境守住了边界。她做的,是"设计和判断"那部分工作——这本来就是公司雇她来做的事。

A reflection on what the case study demonstrates. The configuration above is not large. The `.claude` directory at the end of the build is roughly three thousand lines of configuration spread across twenty files. A single developer can build it in a long afternoon. Once built, it persists. It travels with the project to every developer who clones the repository. It saves substantial time on every session in the project, indefinitely. The investment-to-benefit ratio is exceptional, and the case study is meant to convince the reader that her own project has a similar configuration latent inside it, waiting to be built.

回头想想这个案例证明了什么。上面这份配置并不大。搭完之后,这个 `.claude` 目录大致是 3000 行配置,分布在 20 个文件里。单个开发者花一个长一点的下午就能搭起来。一旦搭好,它就持久存在——它会随项目走到每一位 clone 仓库的开发者那里。它会在项目里的每一个 session 上持续省下大把时间,没有止境。投入产出比出奇地高;这个案例的目的是说服读者:你自己的项目里,也有一个类似配置在潜伏着,等着被搭出来。

A second walkthrough of a different kind of session is worth including, because the morning scenario above showed the environment helping with a relatively well-defined feature build. A different kind of session, the bug investigation, exercises different parts of the environment and is worth seeing in concrete form.

第二种 session 演示也值得放进案例里。上面那个"早晨场景"展示的是环境帮上"形态良好、定义清晰的 feature 搭建";另一种 session——bug 调查——会让环境里不同的部分被锻炼到,也值得以具体形态呈现一遍。

A Bug Investigation · 一次 bug 调查

← 上一节

下一节 →

The developer notices that errors from the avatar upload endpoint have been spiking in the production monitoring dashboard. She opens Claude Code in the project. The `SessionStart` hook fires and shows her the current branch (a feature branch she had been working on yesterday) and any uncommitted changes (none, because the `Stop` hook from the previous session prevented her from leaving with dirty state). She switches to main and pulls the latest. She types into Claude Code, recent errors from `/api/users/profile` suggest something is failing on large file uploads; investigate.

开发者注意到生产监控面板上,avatar upload 接口的错误数在飙升。她在项目里打开 Claude Code。`SessionStart` hook 触发,把当前 branch(她昨天在写的一个 feature branch)和未提交变更(没有——上一个 session 的 `Stop` hook 早就不允许她"带着脏状态走")显示给她。她切到 main,把最新代码拉下来。她在 Claude Code 里敲:最近 `/api/users/profile` 的错误说明,大文件上传那条路径上有什么东西在挂;请查。

The orchestrator pattern recognizes the investigative shape of the request, but unlike the morning's feature work, no existing subagent perfectly matches investigation. The main session handles the work directly, reading the route handler, reading the recent commits to the file, reading the error logs the developer points it at. It identifies a likely cause: the Zod schema does not enforce a maximum file size, and the underlying middleware that should have caught oversized uploads has a configuration drift between staging and production. The main session proposes a fix: tighten the schema to include explicit size validation, and adjust the middleware configuration. The developer agrees with the diagnosis.

Orchestrator 范式识别出这是"调查型"的请求——但和早上的"feature 搭建"不同,这次没有任何一个现成的 subagent 能完美匹配"调查"这件事。主会话直接接手:读 route handler、读这份文件最近的 commits、读开发者指过来的错误日志。它定位到一个可能的原因:Zod schema 没有强制最大文件大小,而本来应该兜住"过大上传"的那道中间件,在 staging 和 production 之间有配置漂移。主会话提出修复方案:把 schema 收紧,显式校验大小;再调一下中间件配置。开发者同意这个诊断。

She types, write the fix and add a test for the size validation. The `api-route-scaffold` skill is not the right fit here, because the route already exists. The `test-writer` subagent activates because the request matches its description. The subagent reads the route, reads the existing test file, adds three new test cases for size validation including one boundary, one over-size, and one with the new error format, and ends with a note about the test it wrote. The `format-on-save` hook runs Prettier on the modified test file. The `tests-must-pass` hook runs the tests; they pass. She asks for a commit. The `commit-message` skill produces a message: `fix(users): enforce avatar size limit at the API boundary`. She commits, pushes, opens a PR, asks for a PR description, the `pr-description` skill produces it. The whole investigation takes forty minutes, from noticing the error spike to opening the PR.

她敲:写这个修复,再加一个 size 校验的测试。`api-route-scaffold` skill 不合适——因为这条路由已经存在。`test-writer` subagent 被激活,匹配上了 description。Subagent 读路由、读现有测试文件,补了 3 个 size 校验的新 case:一个边界、一个超限、一个新错误格式;最后附一句对所写测试的说明。`format-on-save` hook 在被改的测试文件上跑一遍 Prettier。`tests-must-pass` hook 跑测试——全过。她要 commit; `commit-message` skill 吐出一条消息: `fix(users): enforce avatar size limit at the`

API boundary。Commit、push、开 PR、要 PR 描述——pr-description skill 吐出来。她从注意到错误飙升,到开 PR,整条链路 40 分钟。

The example shows the environment functioning at a different scale from the morning scenario. No single subagent dominated the work. No single skill was the centerpiece. The environment's value showed up in the small accelerations along the way: the hook that surfaced the branch state at session start, the subagent that wrote the tests in the project's conventions, the formatter and test runner that ran without prompting, the commit-

这个例子说明:环境和"早晨场景"里的运作尺度是不同的。这次没有哪个 subagent 是绝对主角,也没有哪个 skill 占据舞台中央。环境的价值,体现在这一路上无数处"小加速"上:session 一开就把 branch 状态摆出来的 hook、按项目规约写测试的 subagent、不用催就自动跑的格式化和测试、commit 时那

← 上一节

下一节 →

message and PR-description skills that produced consistent outputs at the moments they were needed. The cumulative effect of these small accelerations is what makes the environment substantially better than vanilla Claude Code, and the cumulative effect is most visible in routine sessions like this one rather than in the showcase orchestration of the morning scenario.

刻吐出来、随用随到的 commit 消息和 PR 描述 skill。这些"小加速"加在一起的累积效应,正是这个环境能比 vanilla Claude Code 好出一截的关键——而且这种累积效应,在"日常 session"里反而比"早上的 showcase 编排"更看得见。

A third reflection concerns the team-adoption arc that this kind of environment enables. Once the configuration is committed to the repository, every developer on the team experiences the same environment. New team members inherit it on their first clone. Conventions that previously had to be taught through code review or onboarding documentation are now enforced by the tooling itself. The team's collective output becomes more consistent. The collective time saved compounds across every developer. A team of five developers, each saving ninety minutes a day through their inherited environment, recovers nearly forty hours a week of engineering time. That recovered time can be spent on new features, on technical debt, on the hard problems that the conventions could never have solved on their own. This is the deeper argument for treating the .claude directory as a first-class engineering asset alongside the source code: the leverage compounds across the team, not just within any one developer.

第三条反思,是关于"这类环境所支持的团队采纳曲线"。配置一旦进了仓库,团队里每一位开发者就都处在同一个环境里。新成员一 clone 就自动继承。曾经必须通过 code review 或 onboarding 文档来教的那些规约,现在由工具自己强制。团队整体的产出变得更一致;每一份被省下来的时间,在所有开发者身上一一起复利。一个 5 人的团队,每个人每天靠这个继承来的环境省下 90 分钟,等于一周给团队回收将近 40 小时的工程时间。回收出来的时间,可以投到新 feature、技术债、或规约本身永远解不开的那些难题上。这正是"把 .claude 目录当成和源代码并列的一等工程资产"更深的理由:这种杠杆,是在整个团队层面复利,而不是只复利在某一个开发者身上。

A second reflection concerns the team dimension. The configuration above is committed to the repository, which means that every developer on the team uses the same skills, the same subagents, and the same hooks. When a new developer joins the team, she receives the entire environment automatically when she clones the project for the first time. She does not need to be onboarded into the conventions, because the conventions are enforced by the environment. The wiki that nobody read becomes irrelevant, because the wiki has been replaced by configuration that the tool itself honors. This is how engineering conventions actually take hold in a team: not through documentation, not through training, but through tooling that makes the right thing automatic and the wrong thing impossible. The Claude Code environment is a

particularly powerful version of this pattern, because it operates at the level of the developer's primary AI assistant, which means that almost every action a developer takes inside Claude Code is shaped by the environment.

第二条反思,是关于"团队维度"的。上面这份配置被提交进了仓库,这就意味着团队里每一位开发者都在用同一套 skills、同一套 subagents、同一套 hooks。当一位新开发者加入团队,她第一次 clone 项目时,就把整个环境自动收到了手。她不需要被"onboarding 进规约"——因为规约已经由环境在强制了。那份没人读的 wiki 也就变得无关紧要:它已经被"工具自己会遵守的配置"取代了。这才是工程规约在团队里真正扎根的方式——不是靠文档,不是靠培训,而是靠那种"让对的事自动发生、让错的事无法发生"的工具。Claude Code 环境是这种范式的一个特别有力的版本,因为它作用在"开发者的主要 AI 助手"这一层——也就是说,开发者在 Claude Code 里几乎做的每一件事,都会被环境塑形。

← 上一节

下一节 →

A final reflection on the case study concerns the cost of building it. The afternoon investment to produce the configuration above is real but bounded. A developer who has read this book carefully and who has practiced the patterns on a few personal skills can produce an environment of this size in roughly six to eight hours of focused work. The same developer working on a fresh codebase she has not used Claude Code in before would need closer to twelve hours, because the project conventions have to be written down somewhere before they can be encoded in skills. Either way, the investment is on the order of a single working day, and the return on that investment compounds across every future session. A developer who saves an average of ninety minutes per working day through her environment recovers her initial investment in less than a working week. After that, the savings are pure leverage. This is the practical case for treating the build of the `.claude` directory as a high-priority engineering task rather than as a side hobby. The numbers favor the build, and they favor it dramatically.

关于这个案例,最后一条反思是"搭建成本"。上面这份配置,花一个下午搭出来——投入真实,但有上限。一个认真读完本书、且已经把若干范式在几个人 skill 上练过手了的开发者,大概 6~8 小时专注工作就能搭出这个体量的环境。同一位开发者,在一个她从未用 Claude Code 跑过的全新代码库上,需要接近 12 小时——因为项目规约总得先在某处被写下来,然后才能编码进 skill。两种情形,投入的量级都在"一个工作日"这个量级;而回报会在此后每一个 session 里持续复利。一个开发者,如果通过她的环境每个工作日平均省下 90 分钟,那不到一个工作周就能回本。回本之后,省下来的每一分钟都是纯杠杆。这正是"把搭 `.claude` 目录当作高优先级工程任务、而不是业余消遣"的最朴素理由。数字站在"搭"这一边,而且站得很有力。

The next chapter takes the same architecture and applies it to a very different shape of work. The full-stack developer in this case study had a clearly engineering-shaped job. The next case study features a solo technical founder whose work spans product, marketing, and operations, and whose Claude Code environment looks substantially different from this one even though the underlying architecture is identical. The contrast is instructive, because it shows that the three-pillar framework is not specific to engineering work; it is a general pattern for engineering one's relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. The second case study begins in the chapter ahead.

下一章把同一套架构套到了一种非常不同的工作形状上。本案例里的全栈开发者,做的是"形态明显的工程工作"。下一个案例的主角,是一位独立技术创业者——她的工作横跨产品、营销、运营,她的 Claude Code 环境看上去会和这一份相当不同,但底下那套架构是同一个。这种对比很有教益:它说明三柱框架并不是工程工作特有的——它是"工程化地处理你和 AI 助手关系"的一条通用范式,几乎能干净地适配到任何一种知识工作上。第二个案例,下一章开始。

## CHAPTER TWENTY-ONE · 第二十一章

## Case Study: The Solo Technical Founder's Claude Code Setup

## 案例研究:独立技术创业者的 Claude Code 配置

*"The same architecture, applied to different work, produces different environments. The architecture is the constant. The work is the variable."*

— a software architect, anonymous

*"同一套架构,套到不同的工作上,会产生不同的环境。架构是常量;工作才是变量。"*

—— 一位软件架构师(匿名)

This chapter is the second case study, and it is deliberately different in shape from the first. The previous chapter built an environment for a single specific kind of engineering work. This chapter builds an environment for a solo founder whose work spans product, engineering, marketing, customer support, and operations, all from a single Claude Code installation. The shift in shape is the point of the chapter. The three-pillar framework was developed in this book using engineering examples, but the framework is not specific to engineering. It is a general pattern for engineering one's relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. By the end of this chapter, the reader will have seen the same architecture produce two very different environments, and she will have the conceptual tools to design a third for her own situation, whatever that situation looks like.

这一章是第二个案例研究,它在形态上故意和第一个拉开了距离。上一章搭的是一个面向"单一特定种类工程工作"的环境;这一章要搭一个独立创业者的环境——她的工作横跨产品、工程、营销、客户支持、运营,所有这些事都从同一份 Claude Code 安装里跑。这种形态上的位移,正是本章的重点。三柱框架在本书里是用工程例子展开的,但它并不专属工程。它是一条"工程化地处理你和 AI 助手关系"的通用范式,几乎能干净地适配到任何一种知识工作上。到本章结束时,读者会看到同一套架构产出两种相当不同的环境,也会握有设计"第三份给自己的环境"的概念工具——无论那第三份的形态长成什么样。

The founder in this case study runs a small SaaS company alone. The product is a developer tool, a paid service that helps engineering teams analyze and visualize their CI/CD pipelines. She wrote the original prototype, she runs the marketing, she handles customer support, she manages the infrastructure, and she does the bookkeeping. The company has roughly two hundred paying customers, the revenue is enough to support her full-time, and she is at the stage where her single biggest constraint is her own time. She has been a heavy Claude Code user for several months, and she has decided that her environment is overdue for serious engineering. The case study walks through the build.

本案例里的这位创业者,独自经营着一家小型 SaaS 公司。产品是一款开发者工具——一项付费服务,帮工程团队分析并可视化他们的 CI/CD 流水线。她写出了最初的原型;她跑营销;她处理客户支持;她管基础设施;她也做账。公司大约有 200 个付费客户,营收足够支撑她全职投入;她目前的阶段,最大的瓶颈就是她自己的时间。她已经重度使用 Claude Code 几个月,并认定自己的环境早该被好好"工程化"一遍。这个案例会走一遍搭建的过程。

Begin with the diagnosis. The founder's work has a distinctive shape that is worth naming explicitly, because it influences every later design decision. Her work is multi-domain. In a single working day, she might

先从"诊断"开始。这位创业者的工作有一种值得明确点出的形态——因为它会决定后面所有的设计决策。她的工作是多领域的。一个工作日里,她可能

← Chapter 20 上一节

下一节 →

write a product roadmap section, draft a sales email to a prospective customer, debug a production issue in the ingestion service, write a blog post about a recent feature, respond to a support ticket about API rate limits, update the company's investor dashboard, and produce next week's marketing calendar. No single role-shaped specialist would cover this range, and a generic Claude Code session that handled all of it would suffer from the context-pollution problem in an extreme form, because the residue of writing a sales email would still be in the context when she pivoted to debugging a production issue.

写一段产品 roadmap 章节;给一个潜在客户一封销售邮件;debug 一下 ingestion service 的一个生产问题;就最近一个 feature 写篇 blog;回一张关于 API rate limit 的支持工单;更新一下公司投资方看板;再做出下周的营销日历。这种范围,没有任何"单一角色"专家能覆盖;而一个把这一切都包在里面的、通用型的 Claude Code session,会以最极端的形式遭受"上下文污染"问题——因为写完销售邮件留下的那些残留,会一直在她切到 debug 生产问题时赖在上下文里不走。

Her environment, therefore, needs to be designed around clean context separation between domains. Skills will encode the conventions that apply within each domain. Subagents will provide specialist roles for the major recurring tasks. Hooks will enforce boundaries at the lifecycle moments where the founder's attention is least reliable. The directory structure will reflect the multi-domain nature of the work, with skills organized so that the right ones activate for the right kind of work without firing in inappropriate contexts.

所以,她的环境,必须围绕"领域间干净的上下文分隔"来设计。Skills 编码各领域内部适用的规约;subagents 为主要反复出现的任务提供专家角色;hooks 在生命周期里那些"她注意力最不可靠"的节点强制边界。目录结构要反映这种"多领域"的本性——skills 要组织得能让"对的 skill 在对的工作上触发,而不会在不恰当的场景下乱触发"。

The `.claude` directory at the user level (since this is a personal environment that travels across all of her projects) is organized as follows.

user 级别的 `.claude` 目录(因为这是一份跟着她走遍所有项目的"个人环境")长这样:

```
~/ .claude/
├─ settings.json
├─ skills/
│ └─ product/
│ ├── prd-section/
│ ├── feature-roadmap-entry/
│ └─ decision-log/
│ └─ marketing/
│ ├── blog-post-outline/
│ ├── feature-announcement/
│ └─ twitter-thread/
```

```
| | | newsletter-section/
| | | sales/
| | | | prospect-email/
| | | | follow-up/
| | | support/
| | | | support-reply/
| | | | ticket-summary/
| | | ops/
| | | | investor-update/
| | | | financial-summary/
| | | | weekly-review/
| | | | writing/
```

注:目录结构将在下一页继续,展示剩余的 `writing/` 、 `agents/` 、 `hooks/` 部分。

注:目录结构将在下一页继续,展示剩余的 `writing/` 、 `agents/` 、 `hooks/` 部分。

The most striking difference from the previous case study is the breadth of skills. The founder has roughly fifteen skills organized into six domain folders. The folder organization is a convention rather than a technical requirement; Claude Code does not care how skills are organized inside the skills directory. The folders exist for the founder's own benefit, so that when she opens the directory to add or edit a skill, she can find what she is looking for at a glance. The discipline of organizing skills by domain pays off as the library grows, because a flat directory with thirty skills becomes hard to navigate, while a folder-organized one stays legible indefinitely.

和上一个案例最显眼的差异,是 skills 的广度。这位创业者大致有 15 个 skill,组织在 6 个领域目录下。按目录来组织,更多是一种约定,而不是技术上的硬性要求——Claude Code 并不在意 skills 目录里怎么排。这些目录是为了她自己的便利而存在的:当她打开目录加 skill 或改 skill 时,一眼就能找到要找的东西。随着库变大,这种"按领域组织"的纪律就显出价值了——一个扁平目录里塞 30 个 skill,会变得难以导航;而按文件夹分层的可以一直保持清晰。

Begin building with the marketing domain, because it is the most distinctive of the founder's use cases. The `marketing-writer` subagent is the role specialist that handles most marketing-related tasks. It has its own system prompt, tuned for the founder's voice and the company's tone, and it loads relevant skills as it works.

从 marketing 领域开始搭起,因为它是这位创业者用法里"最不一样"的那一块。`marketing-writer` subagent 是处理大多数营销相关任务的"角色专家"。它有自己的 system prompt,针对这位创业者的口吻和公司调性调过;它在工作中会加载相关的 skill。

目录结构(续) · Directory Structure (continued)

```
| | | writing/
| | | | humanize-text/
| | | | shorten/
| | | | (minimal, see below)
| | | agents/
| | | | product-strategist.md
| | | | marketing-writer.md
```

```
| |— support-helper.md
| |— financial-analyst.md
| |— code-reviewer.md
|— hooks/
| (minimal, see below)
```

Marketing Writer Subagent · Marketing Writer subagent 定义

```

name: marketing-writer
description: Drafts marketing content including blog posts, feature announcements, Twitter threads, newsletter sections
tools: Read, Write, Glob
model: sonnet

You are the marketing voice of a small developer-tools company. The company sells a service that helps engineering t
```

注:以上是 `marketing-writer` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"你写东西的口吻"以及"方法/不做什么"两段在下一页继续。

注:以上是 `marketing-writer` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"你写东西的口吻"以及"方法/不做什么"两段在下一页继续。

Marketing Workflows · 营销 workflow

The subagent has access to the company's marketing skills, including the format-specific ones ( `blog-post-outline` , `feature-announcement` , `twitter-thread` , `newsletter-section` ), and the general writing skills ( `humanize-text` , `shorten` ). When the founder asks for a marketing piece, the `marketing-writer` subagent activates, the relevant format skill fires inside it, the `humanize-text` skill applies its pass at the end, and the result comes back as a draft in the founder's voice with the company's conventions applied.

这个 subagent 拿得到公司的 marketing skills——既包括"按形态分"的那几个( `blog-post-outline` 、 `feature-announcement` 、 `twitter-thread` 、 `newsletter-section` ),也包括"通用写作"那两个( `humanize-text` 、 `shorten` )。当创业者要一段营销内容, `marketing-writer` subagent 激活;相关的"形态 skill"在它内部触发; `humanize-text` skill 在最后做一遍"去机器味"的过稿;最终回来的,是一段套用了公司规约、又带着创业者本人声音的草稿。

The `blog-post-outline` skill is small but important, because it captures the founder's preferred structure for a blog post.

`blog-post-outline` skill 不大但重要——它把这位创业者偏好的 blog 结构沉淀了下来。

Marketing Writer(续) · Marketing Writer(续)

The voice you write in is honest, technically precise, and slightly understated. You do not use marketing superlativ

Your method:

1. Activate the relevant skill for the format requested (blog post outline, feature announcement, twitter thread, ne
2. Activate the humanize-text skill at the end of every draft to remove AI-flavored vocabulary and patterns.
3. Read the existing marketing material in the company's content directory to match the voice.
4. Produce the draft in the requested format.
5. End with a short note about the angle you took and a question if you are uncertain whether the angle matches the

What you do not do:

- You do not invent product features. If the request implies a feature you have not seen documentation for, ask the
- You do not use language the founder has explicitly excluded.
- You do not produce vague marketing-speak. Every sentence should communicate something specific.
- You do not pad. A sharp short post beats a padded long one.

End with the draft and a brief note about voice or angle.

注:上面这段 system prompt 用了"口吻 + 方法 + 显式禁止"三层结构。口吻一段明确点了"禁词清单"(powerful / robust / elegant / industry-leading);方法一段规定了"先激活相关 skill、最后跑 humanize-text"的工序;显式禁止一段把"会出现的失败模式"列成可枚举的负面条款。

注:上面这段 system prompt 用了"口吻 + 方法 + 显式禁止"三层结构。口吻一段明确点了"禁词清单"(powerful / robust / elegant / industry-leading);方法一段规定了"先激活相关 skill、最后跑 humanize-text"的工序;显式禁止一段把"会出现的失败模式"列成可枚举的负面条款。

← 上一节

下一节 →

Move to the support domain. The `support-helper` subagent handles customer support tasks, with the `support-reply` skill providing the format and the `ticket-summary` skill providing a way to compress long ticket threads into actionable summaries.

接下去是 support 领域。`support-helper` subagent 处理客户支持任务——`support-reply` skill 提供格式, `ticket-summary` skill 提供把长 ticket 线程压缩成可行动摘要的能力。

## Blog Post Outline Skill · Blog Post Outline skill 定义

```

name: blog-post-outline
description: Produces a blog post outline in the company's standard structure. Use whenever the user asks for a blog
allowed-tools: Read

Blog Post Outline

When asked for a blog post, structure it as follows.

1. Hook (one paragraph): Open with a specific, concrete observation or question. Avoid abstract framing. The hoo
2. The problem (one or two paragraphs): Describe the actual pain the post will address, in technical terms. Use
3. The mental model or framework (two or three paragraphs): Lay out the way of thinking about the problem that t
4. The application (one or two examples): Show the framework applied to one or two concrete cases. Use real code
5. The trade-offs (one paragraph): Be honest about what the framework does not solve, where it does not apply, a
6. The takeaway (one or two sentences): A single concrete statement of what the reader should do or remember.
```

Length target: 1,200 to 1,800 words. Longer is acceptable when the topic justifies it. Shorter is acceptable when the topic is narrow.

After producing the outline, ask the founder to confirm the angle before drafting the full post.

Support Helper Subagent · Support Helper subagent 定义

```

name: support-helper
description: Drafts customer support replies and summarizes long support threads. Use whenever the user mentions a customer support issue.
tools: Read, Write
model: sonnet

You are a senior customer support engineer for a small developer-tools company. The customers are technical: engineers and product managers. You are helpful, concise, and clear. You are not afraid to ask for clarification if needed. You are not afraid to ask for more context if needed. You are not afraid to ask for more context if needed.
```

注:以上是 support-helper subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"口吻 / 方法 / 不做什么"段在下一页继续。

注:以上是 support-helper subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"口吻 / 方法 / 不做什么"段在下一页继续。

Move to the operations domain. The financial-analyst subagent handles tasks like generating investor updates, summarizing the company's monthly financials, and producing weekly review reports.

接下去是 operations 领域。financial-analyst subagent 处理诸如"投资方更新生成、公司月度财务摘要、周报"等任务。

Support Helper(续) · Support Helper(续)

```
and substantive. You do not use phrases like "I completely understand your frustration." You acknowledge the issue and express empathy. You do not use phrases like "I'm sorry to hear that." You acknowledge the issue and express empathy. You do not use phrases like "I'm sorry to hear that." You acknowledge the issue and express empathy.

Your method:

1. Read the customer's message carefully. Identify what they actually need: a fix, an explanation, a workaround, an apology, or a combination of these.
2. If the answer involves product knowledge you are not sure about, flag the uncertainty rather than guess. The founder is your best resource for this.
3. Activate the support-reply skill for format guidance.
4. Draft the reply.
5. If the issue is a bug or a known limitation, end with a sentence that indicates whether it is on the roadmap to fix.

What you do not do:
- You do not promise fixes you cannot guarantee.
- You do not use customer service platitudes.
- You do not pad. Customers' time is valuable.
- You do not ignore technical specifics. If the customer mentions a version number, an error code, or a configuration, you must use it.

End with the draft reply and a one-line note about whether you are confident in the technical content or whether the issue is a bug or a known limitation.
```

Financial Analyst Subagent · Financial Analyst subagent 定义

```

name: financial-analyst
description: Produces investor updates, monthly financial summaries, and weekly business reviews using data from the
tools: Read
model: opus

```

You are a senior financial analyst working with a small SaaS founder. Your job is to take raw metrics and accounting

Your principles:

- Numbers without context are not useful. Every metric should be paired with a comparison: month-over-month, year-over-year

注:以上是 `financial-analyst` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"原则 / 方法 / 不做什么"在下一页继续。注意它选的是 `model: opus` ——这位创业者在"涉及数字精度"的任务上,愿意多花 token 换更稳的判断。

注:以上是 `financial-analyst` subagent 的 YAML frontmatter 和 system prompt 开头;剩下的"原则 / 方法 / 不做什么"在下一页继续。注意它选的是 `model: opus` ——这位创业者在"涉及数字精度"的任务上,愿意多花 token 换更稳的判断。

← 上一节

下一节 →

The other subagents in this environment include `product-strategist`, which handles product roadmap and PRD work, and `code-reviewer`, which is a near-copy of the engineering-focused subagent for the times when the founder is reviewing her own code.

本环境里其他的 subagent 包括 `product-strategist` ——它处理产品 roadmap 和 PRD 相关工作;以及 `code-reviewer` ——它是"工程向 subagent 的一个近复制版",在创业者 review 自己代码时出场。

Move now to the hooks. The hooks for this environment are deliberately minimal, because the founder's work does not have the same enforcement requirements as the team-engineering project from the previous chapter. Her work is largely solo, the consequences of mistakes are concentrated on her own time rather than on a team's infrastructure, and over-engineering the hook layer would produce friction without proportionate benefit. Her `settings.json` contains only three hooks.

接下来是 hooks。这个环境的 hooks 是有意做得极简的——因为这位创业者的工作并不像上一章的"团队工程"那样有同等的强制需求。她基本是单干;错误的后果集中在她自己的时间上,而不是团队的基础设施上;过度工程化 hook 层只会带来和收益不成比例的摩擦。她的 `settings.json` 里只有 3 个 hook。

## Financial Analyst(续) · Financial Analyst(续)

- Be honest about bad numbers. A report that hides a slowdown is a report that delays the founder's response to a re
- Be specific about what is changing. "Revenue grew" is not a useful sentence. "Revenue grew 8% MoM, driven by three
- Distinguish leading indicators from lagging indicators. New trial signups are leading. MRR is lagging. Both matter

Your method:

1. Activate the relevant format skill (investor-update, financial-summary, weekly-review).
2. Read the data sources the founder has pointed you at.
3. Produce the report in the format the skill specifies.

4. End with a short section labeled "Things I would watch," where you flag anything in the data that you think deserves

What you do not do:

- You do not invent numbers. If a number is not in the data, you do not include it.
- You do not editorialize beyond the "things I would watch" section.
- You do not produce reports longer than necessary. Density beats length.

End with the report and the "things I would watch" section.

Settings.json(节选) · Settings.json(节选)

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "$HOME/.claude/hooks/block-dangerous.sh"
 }
]
 }
]
 }
}
```

注:这段是 settings.json 的开头;剩下的 Edit|Write|MultiEdit 门和 Notification 通知 hook 将在下一页继续。

注:这段是 settings.json 的开头;剩下的 Edit|Write|MultiEdit 门和 Notification 通知 hook 将在下一页继续。

The first hook blocks dangerous shell commands. The second protects sensitive file paths, including the founder's .env files, her accounting exports, and her investor communications archive. The third produces a desktop notification whenever Claude needs her attention, because she often has Claude Code running in a background terminal while she works on something else and would otherwise miss the prompt for input.

第一个 hook 用来拦危险的 shell 命令。第二个保护敏感文件路径——包括她的 .env 文件、她的会计导出、她的投方沟通存档。第三个在 Claude 需要她注意时弹一条桌面通知,因为她经常把 Claude Code 挂在后台 terminal 里,自己同时在干别的事,如果不弹通知,她很容易错过 Claude 抛过来的提问。

A Minimal Hook Layer · 极简的 hook 层

The minimalism of the hook layer is a design choice, not an oversight. The founder considered adding test-running hooks, formatting hooks, and several other guards from the previous case study, but concluded that the friction of those hooks would outweigh their benefit in her solo, multi-domain context. Her work moves between writing, code, and analysis throughout the day, and a hook that runs the test suite after every Edit would slow down a writing session where she is editing markdown files that have nothing to do with code. The right call for her was to keep the

Hook 层的极简,是一个设计选择,不是疏漏。这位创业者考虑过加"跑测试的 hook""格式化的 hook"以及上一章里的另外几道门;但她最后判断:在她的"单干、多领域"语境下,这些 hook 带来的摩擦大于收益。她一天里在写作、代码、分析之间来回切换;一个"每次 Edit 之后跑测试套件"的 hook,会拖慢一次"她在改完全跟代码无关的 markdown 文档"的写作 session。对她来说,正确的选择是——

```

 }
]
},
{
 "matcher": "Edit|Write|MultiEdit",
 "hooks": [
 {
 "type": "command",
 "command": "$HOME/.claude/hooks/protect-files.sh"
 }
]
}
],
"Notification": [
 {
 "hooks": [
 {
 "type": "command",
 "command": "osascript -e 'display notification \"Claude needs your input\" with title \"Claude Code\"'"
 }
]
 }
]
}
}
}

```

注:这段 `settings.json` 比上一章那份明显瘦一圈——只有 1 个 Bash 门(拦危险命令)、1 个文件门(保护 `.env` /会计导出/投方存档)、再加 1 个 macOS 桌面通知(`osascript -e 'display notification ...'`)。这种"够用就好"的选择,正是为单干多领域的工作量身定的。

注:这段 `settings.json` 比上一章那份明显瘦一圈——只有 1 个 Bash 门(拦危险命令)、1 个文件门(保护 `.env` /会计导出/投方存档)、再加 1 个 macOS 桌面通知(`osascript -e 'display notification ...'`)。这种"够用就好"的选择,正是为单干多领域的工作量身定的。

[< 上一节](#)
[下一节 >](#)

hook layer light and let the discipline live in her skills and subagents instead. A different founder, with a different shape of work, might make different choices. The architecture accommodates both.

把 hook 层保持轻量,让"规约纪律"由 skills 和 subagents 去承担。一位形态不同的创业者,可能做出不同的取舍。这套架构两边都容得下。

A walk through a representative day in this environment will make the difference visible. The founder opens Claude Code in the morning. She has a busy day planned: a customer support reply, a section of next month's investor update, a marketing post about a feature she shipped last week, and some debugging of a production issue she noticed in last night's metrics.

走一遍这个环境里"代表性的一天",可以让这种差别可见。创业者早上打开 Claude Code。今天安排得很满:一封客户支持回复、下月投资方更新里的一节、上周发布的那个 feature 的 marketing post、再加上昨晚 metrics 上注意到的一个生产问题,要 debug 一下。

She starts with the support reply. She reads the ticket, then types into Claude Code, draft a reply to this customer about the rate-limiting issue. The `support-helper` subagent activates because the request matches its description. The subagent reads the customer's message, drafts a reply that acknowledges the issue concretely and offers a workaround, and ends with a note that the rate-limit configuration change the customer is asking about is on the roadmap for the next release. She reviews the draft, makes a small wording change, and sends it.

她从支持回复开始。她读了 ticket,然后在 Claude Code 里敲:给这个客户草一份关于 rate-limiting 问题的回复。`support-helper` subagent 因为请求匹配上 description 而激活。Subagent 读客户消息,草出一份"具体承认问题 + 提供临时方案"的回复,末尾加一句:客户想要的 rate-limit 配置调整,已经排在下一版的 roadmap 上。她 review 草稿,改了一两个词,发出去了。

She moves to the investor update. She types, write the metrics section of this month's investor update; data is in `/investors/data/january.csv`. The `financial-analyst` subagent activates. It reads the data, produces a section that opens with the headline metric, follows with the MoM and YoY comparisons, breaks down the drivers, and ends with the "things I would watch" section that flags an unusual dip in trial-to-paid conversion in the second week of the month. She thanks the analyst for the catch, makes a note to investigate, and copies the section into her investor update document.

她接着做投资方更新。她敲:写本月投资方更新的 metrics 章节;数据在 `/investors/data/january.csv`。`financial-analyst` subagent 激活。它读数据,生成一节内容:头条指标打头,接着是 MoM 和 YoY 对比,然后拆解 driver,最后是"things I would watch"小节——点出本月第二周一次不寻常的"试用转付费"下滑。她感谢这个 subagent 抓到了这条,记下来要查一下,然后把这一节粘进投资方更新文档。

She moves to the marketing post. She types, draft a blog post about the new pipeline visualization feature we shipped last week. The `marketing-writer` subagent activates. The `blog-post-outline` skill fires inside it. The subagent produces an outline following the company's standard structure, asks her to confirm the angle, she confirms, and the subagent drafts the full post. The `humanize-text` skill applies its pass at the end, removing AI-flavored vocabulary and tightening the prose. She reads the draft, finds it about ninety percent of where she wants it, makes a few specific edits, and schedules it for publication.

接着是 marketing post。她敲:给上周发布的那个新的 pipeline 可视化 feature 草一篇 blog。`marketing-writer` subagent 激活。`blog-post-outline` skill 在它内部触发。Subagent 按公司标准结构出一份大纲,问她确认"切入点",她确认,然后 subagent 写完整篇。`humanize-text` skill 在最后跑一遍,去掉 AI 味、收紧句子。她读一遍,觉得约九成到位,改了几处,排期发布。

← 上一节

下一节 →

She moves to the production debugging. The production code lives in a separate project directory, and she opens Claude Code there. The project-level configuration in that directory is much closer to the engineering case study from the previous chapter, with skills, subagents, and hooks tuned for the engineering work. The founder's personal user-level configuration from this chapter still applies, but the project-level configuration extends it with engineering-specific tooling. She debugs the production issue, fixes it, gets a code review from her `code-reviewer` subagent, and ships the fix. The whole cycle takes thirty minutes.

她接着处理生产 debug。生产代码住在另一个项目目录里,她在那里打开 Claude Code。那个目录的 project-level 配置,和上一章那个工程案例非常接近——有针对工程工作调好的 skills、subagents、hooks。本章讨论的"user 级别"个人配置仍然生效,但 project-level 配置在它之上又"接"了一层专门给工程用的工具。她 debug 完生产问题、改好、请 code-reviewer subagent 给她做一轮 review,然后把修复推上去。整个闭环 30 分钟。

The total time across the four tasks is roughly two and a half hours. Without the environment, the same four tasks would have taken her closer to five hours, and several of them would have come out worse. The investor update would have lacked the "things I would watch" framing. The marketing post would have sounded like every other AI-drafted marketing post. The support reply would have either been too long or too short, depending on her mood. The environment did not make her smarter. It made her output more consistent and her time more efficient, by absorbing the conventions, the formats, and the role-shaping that her work requires.

四件事加起来的总时间,差不多两个半小时。没有这个环境的话,同样的四件事会更接近五个小时;而且其中几件出来的质量会更差。投方更新会缺那层"things I would watch"的框架;marketing post 会听起来跟所有别的"AI 起草"营销文一样;支持回复——视她当时心情——要么过长、要么过短。环境没让她变聪明——环境通过吸收她工作里那些"规约、格式、角色塑形"的部分,让她的产出更一致、她的时间更高效。

## Generalizing Beyond This Case · 跨出本案例去推广

A reflection on what this case study demonstrates relative to the previous one. The two environments are strikingly different in surface appearance. The full-stack engineering environment has eight skills focused on engineering tasks, four engineering specialists, and six enforcement hooks. The solo founder environment has fifteen skills spanning six domains, five role specialists across product, marketing, support, ops, and engineering, and three minimal hooks. The two environments could not look less alike on first glance. But the underlying architecture is identical. Both use the three-pillar framework. Both organize knowledge into skills, roles into subagents, and boundaries into hooks. Both compose the three pillars to produce capabilities that no single pillar could provide. The framework is what makes the two environments coherent, even though their content is entirely different.

先回头看一下:相对于上一个案例,这个案例究竟说明了什么。两个环境在"表面长相"上的差距是惊人的。全栈工程环境有 8 个 skills 全部面向工程任务、4 个工程方面的专家、6 个强制 hook。独立创业者环境有跨 6 个领域的 15 个 skills、跨产品/营销/支持/运营/工程 5 个角色专家、3 个极简 hook。两者乍一眼看上去,没有比这更不像的了。但底下的架构是同一套。两者都用了三柱框架;都把知识组织进 skills、角色塑形进 subagents、边界强制进 hooks;都通过三柱组合产出"任何一柱单干都拿不出的"能力。框架,正是让这两份环境在"内容天差地别"的前提下仍保持连贯的那条线。

A second reflection concerns the generalizability of the framework. The two case studies in this part of the book are deliberately different to make the generalizability point. A reader who works in a third shape of work,

第二条反思是关于"框架的可推广性"。本书这一部分的两个案例,故意做得不一样,就是为了把"可推广"这一点立起来。一位读者,如果从事的是第三种形态的工作——既不是全栈工程,也不是独立 SaaS 创业——

← 上一节

下一节 →

neither full-stack engineering nor solo SaaS founding, can take the framework and apply it to her own context. The classification questions still apply: knowledge gap means skill, attention gap means subagent, guarantee gap means hook. The composition principle still applies: skills inside subagents, subagents in parallel orchestration, hooks at the lifecycle boundaries. The maintenance discipline still applies: start small, add components when their need is concrete, remove components that have not earned their place. The reader who has read this book carefully has a complete blueprint for engineering her own Claude Code environment, regardless of what kind of work she does.

也可以拿这套框架,套到自己的语境上。分类问题依然成立:知识缺口→skill,注意力缺口→subagent,保证缺口→hook。组合原则依然成立:skills 嵌入 subagents、subagents 并行编排、hooks 守在生命周期边界。维护纪律依然成立:起步要小、需求具体时才加、用得不够就清。认真读过本书的读者,手里已经握有一份"为自己工作工程化一份 Claude Code 环境"的完整蓝图,不管她的工作是哪一种。

A third reflection concerns the founder's specific situation as illustrative of a broader pattern. The founder in this case study is the entire labor force of her company. Her time is the binding constraint. The Claude Code environment is, in her case, not just a productivity improvement; it is the difference between a one-person company that can sustain its operations and one that cannot. The leverage that the environment provides is not optional for her. It is the precondition for the company existing at all in its current form. A reader who is also operating at this kind of scarce-time, multi-domain shape of work should recognize her own situation in the case study and recognize that the architecture this book has been teaching is, for her, a strategic asset rather than a developer convenience.

第三条反思,是关于"这位创业者的具体处境作为更广范式的例证"。本案例里的创业者就是她公司的全部劳动力——她的时间是真正的紧约束。对她来说,Claude Code 环境不只是"生产力改进",它是有和没有之间——一家"一个人撑着、且能维持运转"的公司 vs 一家"维持不了"的公司。环境提供给她的杠杆,对她而言不是"可选"——它是"公司以当前形态存在"这件事本身的前提条件。一位同样处于"时间稀缺、多领域"这种工作形状的读者,应当在这个案例里看到自己的处境,看到本书一直在教的这套架构——对她来说——是战略性资产,而不是开发者的便利。

A second worked walkthrough is worth including, because the Friday weekly-review session exercises a different combination of the founder's environment than the daily-task scenario above and shows the architecture from another angle.

第二个落地演示也值得放进案例。"周五周会回顾"这个 session,会锻炼到上面那个"日常任务"场景没有锻炼到的那一组环境组合,可以从另一个角度把这套架构展示出来。

It is Friday afternoon. The founder sits down to do her weekly review, which she does every Friday before she logs off for the weekend. The review has three components: a financial weekly summary, a writeup of what shipped during the week, and a planning section for the following week. She opens Claude Code in the directory where she keeps her business operations files. She types, run my weekly review.

周五下午。这位创业者坐下来做她的"周会回顾"——她每个周五、在周末下线前都会做一次。这次回顾包含三块:一份财务周摘要、一段本周已交付的总结、以及下一周的规划。她在"业务运营文件"所在的目录里打开 Claude Code,敲:跑一遍我的周会回顾。

The financial-analyst subagent activates first, because the weekly-review skill it is paired with carries the highest priority in the matching. The subagent reads the metrics dashboard export from earlier that day, reads the previous week's summary for comparison, and produces the financial

financial-analyst subagent 首先被激活——因为和它配对的 weekly-review skill,在匹配里拥有最高优先级。Subagent 读当天早些时候导出的 metrics dashboard、读上一周的周报做对比,然后产出"财务"那块——

section: revenue change, customer count change, churn analysis, and the "things I would watch" section. The "things I would watch" section flags an unusual cluster of support tickets all related to the same API endpoint. The founder makes a mental note.

营收变化、客户数变化、churn 分析、再加"things I would watch"小节。这最后这一节点出了一组不寻常的、与同一 API endpoint 相关的支持工单。这位创业者心里记了一笔。

The `marketing-writer` subagent activates next, because the founder also wants the weekly review to capture what shipped on the marketing side. The subagent reads the blog posts published that week, the social media posts, the newsletter that went out, and produces a short summary of marketing output with attached engagement metrics where available.

接着轮到 `marketing-writer` subagent 激活——因为这位创业者也希望周报把"marketing 这一侧这周发了什么"记下来。Subagent 读当周发布的 blog posts、社交媒体上的帖子、发出去的 newsletter,产出一段"marketing 产出"的短摘要,能附上 engagement metrics 的地方都附上。

## The Weekly Review · 周会回顾

The `product-strategist` subagent then handles the planning section. It reads the founder's product roadmap document, reads the recent decision logs from the product folder, and produces a candidate list of three to five things to consider tackling next week, each with a one-sentence rationale.

然后是 `product-strategist` subagent 接手"规划"那一节。它读这位创业者的产品 roadmap 文档、读 product 文件夹里最近的决策日志,产出 3~5 件"下周可以考虑推进"的候选清单,每件附一句"为什么值得做"的理据。

The three sections come together as a single document. The founder reads it, edits a few wording choices in the marketing section, expands the rationale for one of the planning items, and saves it to her ops folder. The whole review takes twenty minutes, where it would have taken closer to ninety without the environment, and the resulting review is more disciplined and more honest than the ones she used to write by hand, because the `financial-analyst` subagent does not let her gloss over uncomfortable numbers.

三节合起来,成了一份单一文档。她通读一遍,在 marketing 那节改了几处措辞,把"规划清单"里某一条的理据展开了一句,然后存到她的 ops 文件夹里。整个回顾 20 分钟;没有这个环境会更接近 90 分钟。而且出来的这份周报比她过去手写时更守纪律、更诚实——因为 `financial-analyst` subagent 不会让她糊弄那些"看着不舒服"的数字。

The cluster of support tickets the `financial-analyst` flagged turns out to be the most consequential output of the review. She investigates the cluster, finds that the affected endpoint has a subtle race condition under high concurrent load, and ships a fix that following Tuesday. Without the `financial-analyst`'s "things I would watch" framing, the cluster would have been buried in the support volume and probably not investigated for several more weeks, by which point a few of the affected customers might well have churned. The investment in the `financial-analyst` subagent has, in effect, paid for the entire environment in this single session, by surfacing a pattern that human attention alone might have missed.

被 `financial-analyst` 点出来的那组支持工单,最后成了整份周报里"最值钱的产出"。她追查了这组工单,发现受影响的 endpoint 在高并发下有一个微妙的竞态;她下了个修复,在接下来的那个周二推上线。如果没有 `financial-analyst` 那条"things I would watch"框架,这一组工单大概会被埋在支持工单的整体量里、几周之内都不会被仔细看;等到几周之后才被发现时,几位受影响的客户很可能已经流失了。为 `financial-analyst` subagent 投入的那些功夫,事实上就靠这一次的发现,就把整个环境的成本收回来了——它把"光靠人的注意力很可能漏掉"的那一类模式,主动浮到了水面上。

The example shows the multi-domain nature of the founder's environment functioning at full strength. Three subagents from three

这个例子,把这位创业者的环境"多领域"的本性,完整地展示了出来。来自三个不同领域的 3 个 subagent——

← 上一节

下一节 →

different domains coordinated on a single integrated task. Each contributed its specialty. The composition produced an output that exceeded what any single subagent could have produced. The founder's role in the work shifted from drafting all three sections by hand to reviewing and editing three drafts, which is a higher-leverage form of attention. This is what the architecture is supposed to produce: not the elimination of the founder from the loop, but the elevation of her contribution from drafting to direction.

协调地跑同一件综合性任务。每一个都贡献自己的专长;组合起来产出的成果,超出了任何单一 subagent 单独能干的事。这位创业者在其中的角色,从"自己手写三节"转变为"读三份草稿并修订"——这才是更高杠杆形态的注意力。这正是这套架构想要做的事:不是把创业者从循环里拿掉,而是把她的贡献从"亲笔起草"提升到"把握方向"。

A fourth reflection concerns the limits of what this kind of environment can do. The architecture does not make the founder smarter. It does not produce ideas she would not have produced. It does not make strategic decisions for her. The architecture absorbs the conventions, the formats, and the role-shaping that her work requires, leaving her attention free for the work that actually requires her judgment. The strategic decisions, the brand voice that gives the marketing its character, the product instincts that decide which feature to build next, all of those remain hers. The environment is a tool, and like all tools, it amplifies what is already there rather than replacing it. A founder with poor strategic instincts will not become a great strategist by installing a Claude Code environment. A founder with strong instincts will find that her instincts now reach further, because the surrounding labor of executing on them is partly absorbed by the environment.

第四条反思是关于"这类环境能做到的边界"。这套架构不会让创业者变聪明——不会替她生出她本来没有的创意,也不会替她做战略决策。它吸收她工作里那些"规约、格式、角色塑形"的部分,把她的注意力腾出来,留给那些真正需要她判断的事。战略决策、构成 marketing 性格的品牌调性、决定下一个该做哪个 feature 的产品直觉——这些都还是她的事。环境是工具;工具和所有工具一样,放大它已经在那里的东西,而不会去替代它。一位战略直觉差的创业者,装一套 Claude Code 环境不会让她变成伟大的战略家。一位直觉强的创业者,会发现自己的直觉能到达更远的地方——因为执行那条直觉所围绕的那些"周边劳动",有一部分被环境吸收了。

A fifth reflection concerns the readiness of this kind of environment for evolution. The founder's environment will not stay the same. As her business grows, the work will shift. She might hire her first employee, at which point some of the personal user-level configuration will need to be migrated to project-level so that the new hire inherits the relevant pieces. She might launch a second product line, at which point new domain folders will appear in her skills directory. She might shift from monthly to weekly investor updates, at which point the `financial-analyst` subagent will need its description and its system prompt updated. The architecture accommodates all of these evolutions naturally, because each component is independently editable and the relationships between components are loose enough to evolve gradually. The environment that the

founder has at the end of her first year will not look identical to the environment at the start of the year, and the difference will be a record of how her work has evolved. This is the deeper meaning of treating the `.claude` directory as a living artifact: the

第五条反思是关于"这类环境对演进的准备度"。这位创业者的环境不会一成不变。她的业务一旦长大,工作本身就会变。她可能会招第一名员工——那时一部分 `user` 级别的个人配置就需要迁到 `project` 级别,让新同事能继承到对应的那部分。她可能会开第二条产品线——那时她的 `skills` 目录里会出现新的领域文件夹。她可能会把"投方更新"从月度改成周度——那时 `financial-analyst` subagent 的 `description` 和 `system prompt` 都要更新。这套架构对所有这些演化都是天然友好的——因为每一个组件都可以独立编辑,组件之间的关系也足够松,可以逐步演化。这位创业者在第一年年末手里那份环境,看上去不会和年初那份一样;而这两份之间的差异,本身就是她工作演化的记录。这正是"把 `.claude` 目录当作活着的工件"更深的含义:这个目录——

← 上一节

下一节 →

directory becomes a mirror of the work, and the work becomes more legible by being mirrored.

变成了一份工作的镜子;而被镜照过之后,工作本身也变得更可读了。

A sixth and final reflection concerns the broader implication of multi-domain Claude Code environments for the way solo founders and small teams operate in the present moment. The combination of cheap intelligence and well-engineered environments has shifted the boundary of what a single person can sustain operationally. Companies that would have required a team of five a decade ago can now be run by a single founder with a thoughtful Claude Code environment, because the environment absorbs the operational labor that used to require additional headcount. The leverage is real and substantial. Founders who recognize this shift and invest in their tooling accordingly are operating with a structural advantage over founders who treat their tools as commodities. The architecture this book has been teaching is, in this sense, a piece of strategic infrastructure for solo and small-team operators in the current era, and the case study above is meant to demonstrate not merely a productivity gain but a different kind of company that becomes possible to run when the architecture is in place.

第六条也是最后一条反思,是关于"多领域 Claude Code 环境"对"独立创业者和小型团队当下如何运作"这件事更广泛的意义。"廉价的智能 + 工程化良好的环境"这一组合,已经移动了"一个人能在运营上撑住多大规模"那条边界。十年前需要一个 5 人团队才能撑起来的公司,如今可以由一位独立创业者 + 一份经过深思熟虑的 Claude Code 环境撑起来——因为环境吸收了过去需要额外人手才能消化的运营劳动。这种杠杆是真实且可观的。看清这一变化、并据此投资自己工具的创业者,比起把工具当大宗商品对待的创业者,具备一种结构性的优势。本书一直在教的架构,在这个意义上,是当下"独立 / 小型团队"运营者手头的一块战略性基础设施;上面这个案例,想说明的也不仅是一份"生产力收益",而是一种"在架构到位之后才成为可能"的、不一样的公司形态。

Part Five ends here. The architecture has been laid out, and two case studies have shown it producing complete environments for very different shapes of work. The reader has, by this point in the book, both the conceptual framework and the practical patterns to build her own environment for her own work. The remaining chapter of the book takes the focus from building the environment to evolving it over time, sharing it with others, and fitting it into the broader Claude Code ecosystem that is forming around the framework. That closing work is the subject of the chapter ahead.

第五部分到此收束。架构已经铺开;两份案例已经演示了它为形态迥异的工作产出完整环境。到本书这里,读者手里已经既有了概念框架,也积累了实践范式,可以为"自己的那份工作"搭出对应的环境。剩下的一章,焦点从"搭"转到"演化"——随时间推移怎么长、和他人怎么分享、以及怎么接入"围绕这套框架正在形成"的更广 Claude Code 生态。这份收尾的工作,就是下一章的主题。

## PART SIX · 第六部分

## Shipping, Sharing, and Evolving

## 发布、分享、演进

*The environment built once is the foundation. The environment maintained over time is the asset.*

环境"被搭起来一次"是地基;"随着时间被维护"才是资产。

## CHAPTER TWENTY-TWO · 第二十二章

## Distribution, Team Adoption, and the Claude Code Marketplace

## 分发、团队采纳,以及 Claude Code 市场

*"The garden you tend for years is not the garden you planted. It is the garden you allowed to become itself, with help."*

— a gardener, anonymous

"你照看了多年的花园,已经不是你当初种下的那个花园了。它是你在助力之下,被允许成为自己的那个花园。"

—— 一位园丁(匿名)

This is the closing chapter of the book. The work of building has been the focus of every part up to now. The work of evolving is the focus of this one. A Claude Code environment that has been built and never touched again will degrade slowly, as the developer's work shifts and the environment fails to shift with it. A Claude Code environment that has been treated as a living artifact, gardened regularly, shared with others, and integrated into the broader ecosystem of available tooling, will continue producing benefits for years, and those benefits will compound in ways that no static environment can match. The chapter ahead is about that gardening.

这是本书的收尾一章。到此为止的每一部分,重心都落在"搭"上;本章的重心是"演化"。一份 Claude Code 环境,搭好之后再也不动——会随着时间的慢慢失能:开发者的工作在迁移,环境却没能跟着迁移。一份 Claude Code 环境,如果被当作活的工件对待、定期打理、与他人共享、并接入更广的工具生态——则会在多年里持续产出效益,这种效益会以任何静态环境都匹配不了的方式复利。本章讲的就是"打理"。

Begin with the first concern, which is version control. The `.claude` directory is part of a project's contract for how work happens in the project. It belongs in source control, alongside the source code. Every project the developer works in should have its `.claude` directory committed to its repository, with changes reviewed in pull requests like any other code change. This is not an aesthetic recommendation. It is the mechanism by which configuration travels with the code, by which teammates inherit the environment automatically when they clone, and by which the configuration evolves in a reviewed, accountable, reversible way. Projects that treat their `.claude` directories as ephemeral local configuration

fail to capture most of the value the architecture is capable of providing. Projects that treat them as first-class engineering artifacts capture the full value, and the difference compounds over months and years.

第一件事,版本控制。`.claude` 目录是项目"工作如何发生"那份契约的一部分——它应当和源代码一起住进版本控制。开发者工作的每一个项目,都应当把 `.claude` 目录提交进仓库,并按"和其他代码变更一样"的方式在 PR 中 review。这不是审美建议。它是"配置随代码走"的机制——队友 clone 时自动继承环境的机制——配置以一种"可 review、可追责、可回滚"的方式演化的机制。那些把 `.claude` 目录当"一次性本地配置"对待的项目,捕获不到这套架构本可提供的大部分价值。那些把它当一等工程工件对待的项目,捕获到的是它的全部价值;而这种差别会在一年又一年的复利下去。

The user-level configuration, which lives in the home directory and applies across every project the developer opens, deserves the same treatment, in a slightly different form. The user-level `.claude` directory should also be version-controlled, but typically in a personal repository rather than in any specific project's repository. The personal repository becomes a record of how the developer's tooling has evolved over time. It can be cloned to a new machine when the developer moves laptops. It can be shared with peers as a starting point when they ask how she has set up her environment. It can be referenced years later as a working portrait of how the developer's craft has grown. The version control of the user-level configuration is a small habit, but the developers who adopt it consistently end up with substantially more durable tooling than those who treat their home directories as ephemeral.

User 级别的配置——住在 home 目录、对开发者打开的每一个项目都生效——也值得同等对待,只是形态略有不同。User 级别的 `.claude` 目录也应当被版本控制,但通常是放一个"个人仓库"里,而不是塞进某个具体项目的仓库。这个个人仓库,就成了"开发者工具如何随时间演化"的一份记录。开发者换笔记本时,可以直接把这份配置 clone 到新机。当同事问"你怎么配的环境"时,她可以把这份配置当起点分享出去。几年后,这份仓库还可以被翻出来,作为"她的手艺是如何成长"的一份可工作画像。给 user 级配置做版本控制,只是一条小习惯;但稳定采纳这条习惯的开发者,手里的工具会比那些把 home 目录当一次性用品的人耐久得多。

← 上一节

下一节 →

A natural extension of version control is the practice of writing meaningful commit messages for `.claude` directory changes. A commit that says "added prettier hook" tells the developer in six months what she changed and why. A commit that says "config update" tells her nothing. The same discipline that produces good commit messages for source code applies to commit messages for configuration, and the same skill the developer has installed for source-code commits, the `commit-message` skill from Chapter Five, applies cleanly to configuration commits as well. The configuration is code, in every meaningful sense, and treating it as code produces the consistency that long-term maintenance requires.

版本控制顺理成章的下一步,就是"为 `.claude` 目录的变更也写有意义的 commit 消息"。一条写着"added prettier hook"的 commit,让六个月后的你一眼看明白改了什么、为什么改;一条只写着"config update"的,什么都看不出来。"为源代码写好 commit 消息"那条纪律,直接套到"为配置写 commit 消息"上即可;而开发者为源代码 commit 装的那条 skill——第五章的 `commit-message` skill——也能干干净净地用在"配置 commit"上。配置在任何有意义的层面上都是代码;把它当代码对待,正是产生"长期维护所需的一致性"的那条路。

Move to the second concern, which is sharing. A skill, subagent, or hook that has proven valuable in one developer's environment is often valuable in many others. The mechanism for sharing is straightforward. The component is a file or a small folder. The file or folder can be copied into another developer's `.claude` directory, or distributed through a public repository that other developers clone, or installed through one of the package managers that have begun to emerge around the Claude Code ecosystem. The friction of sharing is low, and the benefit of receiving someone else's well-designed components is substantial.

第二件事,分享。一条 skill、一个 subagent、一个 hook——在某个开发者环境里被证明是有价值的,常常在很多其他人的环境里也值钱。分享的机制很直接:组件本身就是一个文件或一个小目录;这个文件/目录可以被拷到另一个开发者的 `.claude` 目录里,可以通过一个公共仓库被其他人 clone 走,也可以通过 Claude Code 生态里正在冒出来的某个包管理器来装。分享的摩擦很低;拿到别人设计良好的组件,所获得的收益却很可观。

## The Marketplace · 市场

The Claude Skills Marketplace is the most prominent public channel for sharing. Anthropic has begun curating skills produced both by its own engineering team and by third-party developers and organizations, with skills available for installation through a small command-line interface. The marketplace is in its early days as of this writing, and the available catalog is modest, but the trajectory is clear. Skills that solve common problems will continue to accumulate in the marketplace, and the developer who watches the marketplace will find ready-made solutions for problems she might otherwise have built skills for from scratch. The right relationship with the marketplace is to use it as a starting point rather than as a complete library; off-the-shelf skills rarely fit a specific developer preferences perfectly, but they often save substantial design time when used as a base that the developer adapts.

Claude Skills Marketplace 是分享这件事上最显眼的公共渠道。Anthropic 已经开始"策展"由自家工程团队、以及由第三方开发者和组织产出的 skills——通过一个小型的 CLI 就能装。在本书写作的当下,这个市场还处在早期,可用目录也不算丰富,但方向已经很清晰。能解决常见问题的 skills 会持续在市场里沉淀;而关注市场的开发者,会为"那些她本来要从头搭起 skill 的问题"找到现成的解。和市场打交道的正确姿态,是把它当作"起点",而不是"完整的库"——现成的 skill 很少能完美匹配某一具体开发者的偏好;但当作"被自己再去适配"的基础时,常常能省下可观的"从零设计"时间。

Subagents and hooks have not yet developed marketplace structures of equivalent maturity, but the underlying mechanism is the same. Public

Subagents 和 hooks 还没有发展到同等成熟度的市场结构,但底下的机制是同一种。公共——

← 上一节

下一节 →

repositories of subagent definitions and hook scripts are appearing in greater numbers, often shared on GitHub under permissive licenses, and the developer who participates in this informal exchange both contributes to and benefits from the collective body of work. Sharing publicly is also a way of making one's own work better, because the act of writing a component that strangers will read forces a level of clarity that internal-only components do not require.

subagent 定义、hook 脚本的公共仓库正在越来越多地出现——通常以宽松许可证托管在 GitHub 上。参与这种非正式交换的开发者,既为这份集体工作添砖,也从这份集体工作里获益。公开分享,还是一种"让自己手上的活变得更好"的方式:为陌生人写一个组件这件事本身,就逼出一种"只写给内部人"时所不需要的清晰度。

A third concern is monetization. Some developers will, naturally, want to sell their work. The economics of this are still settling in 2026, but several patterns have emerged that look likely to persist. Skill packs targeted at specific niches, sold as one-time purchases or as subscriptions, are appearing on platforms like Gumroad and Lemon Squeezy. Subagent libraries customized for specific industries are being licensed by consultancies to their clients. Configuration packages that combine skills, subagents, and hooks into complete environments for specific roles are

being sold as productivity products. The market is small but growing, and the developers who build genuinely high-quality components have a real path to monetization that did not exist a few years ago.

第三件事,商业化。一些开发者自然会想把自己手上的活卖出去。这件事的经济形态在 2026 年仍处于成形期,但已经浮现出几种看上去会持续存在的范式:针对特定 niche 的 skill pack,以"一次性买断"或"订阅"的形式,在 Gumroad、Lemon Squeezy 这类平台上出现;按特定行业定制的 subagent 库,正被咨询公司授权给它们的客户;把 skills、subagents、hooks 组合成"针对特定角色的完整环境"的配置包,正作为生产力产品在出售。这个市场还小,但在增长;真正能造出高质量组件的开发者,面前摆着一条几年前还不存在的商业化通路。

The right relationship with monetization, for most developers, is to think of it as an option rather than as an objective. Building components for one's own work is the primary purpose. If, over time, certain components turn out to have wider value, the path to selling them is open. Building components for sale from the start, before any internal use has validated their quality, is harder, and the components produced this way often turn out to be less useful than components that emerged from real work. The advice that emerges is to build first, sell later, and to let the components prove themselves in actual use before any commercial considerations enter the picture.

对大多数开发者来说,和"商业化"打交道的正确姿态,是把它当一个"选项",而不是"目标"。为自己的工作搭组件,才是第一目的;如果在过程中,某些组件被发现具有更广的价值,把它们卖出去的路是敞开的。一上来就奔着"卖"去搭组件——还没经过自己真实使用验证的那种——往往更难;这样搭出来的组件,实用度常常比不上"从真实工作中长出来"的那些。从中提炼出的建议是:先搭,后卖;让组件先在真实使用里证明自己,再让商业考虑入局。

Move to the fourth concern, which is team standardization at scale. The case study in Chapter Twenty showed a small team adopting a shared configuration. The same pattern scales to larger organizations, but with additional considerations. A team of fifty engineers has more diversity of working style than a team of five, and a configuration that works for the entire team needs to be either narrow enough to fit everyone or layered carefully so that the shared layer represents the team's genuine common ground while individual variation lives in personal user-level configuration.

第四件事,规模化下的团队标准化。第二十章那个案例展示的是"小团队采纳一份共享配置"。同一套范式可以放大到更大的组织,但要补一些新的考量。一个 50 人的工程师团队,在"工作风格多样性"上远超 5 人团队;要让一份配置对全队都管用,要么它得"足够窄",能装下所有人,要么它得"仔细分层":共享层只承载团队真正的公约数,个人差异则放在每个人的 user 级配置里。

## Team Standardization at Scale · 规模化下的团队标准化

The pattern that has worked best in practice is to keep the shared project-level configuration tight and conservative, including only the components that genuinely apply to all engineers in the project, and to encourage individual engineers to maintain their own personal user-level configuration on top of it. The shared layer captures conventions, security guarantees, and project-specific knowledge. The personal layer captures preferences, role-shaping, and personal-style choices. The combination produces a personalized environment for each engineer that nonetheless honors the team's common standards. The split also makes the shared layer easier to evolve, because changes to the project-level configuration affect only what is genuinely shared, not the personal preferences that should not be overridden.

实践中真正管用的范式是:把"项目级共享配置"做得紧凑而克制——只装那些"对项目里所有工程师都适用"的组件;同时鼓励每位工程师在自己 user 级别的个人配置里再长一层。共享层抓的是规约、安全保证、项目特有知识;个人层抓的是偏好、角色塑形、个人风格选择。两者合起来,给每位工程师一个"既尊重团队公约数、又带个人色彩"的环境。这种切分也让共享层更容易演进——因为对 project 级配置的改动,只影响"真正属于共享"的那部分,而不会动到那些"不该被覆盖"的个人偏好。

A fifth concern is the long-term evolution of the developer's environment. After a few years of working with the architecture, the developer's `.claude` directory becomes a substantial artifact. It might contain forty skills, fifteen subagents, and a dozen hooks across her various projects and her user-level configuration. The maintenance posture for an environment of this size is different from the posture for a small new environment. The questions shift from what to add to what to keep, what to remove, and what to refactor. A quarterly review becomes a useful discipline. The developer sets aside an hour every three months to walk through her configuration, identify components that have not been used recently, refine components that have been used heavily, and remove components that have outlived their purpose. The hour pays for itself many times over in the months that follow, because it keeps the environment from accumulating dead weight that would otherwise quietly degrade the experience.

第五件事,是开发者环境的长期演化。在和这套架构共处几年之后,开发者的 `.claude` 目录会长成一份可观的工件——可能跨多个项目和 user 级配置,装着 40 个 skills、15 个 subagents、十几个 hooks。维护这种体量的环境,姿态和"新搭起来一份小型环境"是不同的:问题从"加什么"转向"留什么、删什么、重构什么"。一种有用的纪律是"季度回顾"——每三个月花一个小时,把自己的配置过一遍:标出最近没用过的组件、精修被频繁用到的组件、删掉已经走过其生命周期的组件。这一个小时会在随后的几个月里以数倍的回报被挣回来——因为它让环境不至于积累"死重",而那些死重如果不清,会悄悄拖垮整个体验。

A sixth concern is the relationship between the developer's Claude Code environment and the broader ecosystem of AI-assisted development tools that is forming. Skills, in particular, are increasingly being adopted by other tools. The `SKILL.md` format is being honored by Cursor, by various open-source IDE extensions, and by several command-line tools that are not Claude Code. A skill written for Claude Code today often works without modification in three or four other tools. This cross-compatibility is not accidental; it reflects the convergence of the industry around a small set of

第六件事,是开发者的 Claude Code 环境与正在形成的、更广的"AI 辅助开发工具"生态之间的关系。Skills 尤其正在被越来越多的其他工具采纳。`SKILL.md` 这套格式,正在被 Cursor、被各种开源 IDE 扩展、被若干"不是 Claude Code"的 CLI 工具所尊重。今天为 Claude Code 写的一条 skill,常常不加修改就能在三四个其他工具里跑。这种跨工具的兼容性不是偶然的——它反映了整个行业正在向一小套

← 上一节

下一节 →

standards, with `SKILL.md` emerging as one of those standards. The implication for the developer is that her investment in skill design pays dividends not just inside Claude Code but across whatever set of AI-assisted tools she ends up using over the years. The skills travel. The environment, in this sense, is more durable than the specific tool the environment was built for.

标准收敛——`SKILL.md` 正在成为其中之一。对开发者的含义是:她在 skill 设计上的投入,收益不只是落在 Claude Code 内部,而是会跨她未来多年里会用到的各种 AI 辅助工具,持续产生回报。Skills 是会"旅行"的。从这个意义上,环境比它当初为之搭建的那个具体工具更耐久。

A seventh concern is what the developer can expect from the ecosystem in the months and years ahead. The trajectory of Claude Code, and of similar tools, is toward more sophisticated automation, more capable subagents, and more richly composable hook systems. New lifecycle events will appear. New tool restrictions will become possible. New patterns of subagent communication will emerge. The developer who has internalized the architecture in this book will be well-positioned to absorb each of these changes as they arrive, because each will fit into the framework rather than disrupting it. The framework itself is not tied to any specific feature set. It is tied to the durable insight that capability, specialization, and enforcement are three different concerns that benefit from being handled by three different mechanisms. That insight will remain valid as long as AI-assisted development continues to be useful, which is to say, indefinitely.

第七件事,是开发者可以预期未来几个月到几年里,生态会朝哪些方向走。Claude Code 以及同类工具的轨迹,指向更复杂的自动化、更能干的 subagent、组合性更丰富的 hook 系统。会有新的生命周期事件出现;会有新的工具限制成为可能;会有新的 subagent 通信范式浮现。已经把本书这套架构内化于心的开发者,会处在很好的位置去吸收这些变化——因为每一项变化都会"嵌进"这套框架,而不是去颠覆它。框架本身并不绑死在任何具体的功能集上;它绑死在一条持久的洞察上——能力、专门化、强制,是三件不同的事,由三种不同的机制去处理,这件事整体上有利。这条洞察,会在 AI 辅助开发仍然有用的整个时期里持续成立;换言之,会无限期地成立。

A reflection on what the book has tried to do. The book has tried to teach a way of relating to a tool that, for most of its users, is treated as a chat interface. The way of relating that the book has taught is more like the way an engineer relates to a well-configured editor or a well-tuned shell: as a substrate that can be bent to fit the engineer's work, that rewards patient configuration with compounding returns, and that becomes, over time, an extension of the engineer's craft rather than merely a tool she uses. The shift in posture is what matters most. The specific skills, subagents, and hooks that the book has shipped are starting points. The shift in posture, once made, persists across every future tool the developer encounters, because it is a way of thinking about tooling rather than a fact about any specific tool.

回头看看,这本书究竟在尝试做什么。它在尝试教一种"和工具相处的方式"——这个工具,在它的大多数用户手里,被当成了"一个聊天界面"。本书教的那种相处方式,更接近于"工程师和一款配得好的编辑器、和一副调得顺手的 shell"之间的关系:把它当作"可以弯到自己工作形状"的底层;用耐心的配置去换复利的回报;让它随着时间,变成工程师手艺的延伸,而不是她用一下就放下的工具。这种姿态上的转变,才是关键。本书走过的那些具体 skills、subagents、hooks,只是起点。姿态一旦转换,就会在开发者未来遇到的每一个工具里延续——因为它是一种"关于工具该怎么想"的思维方式,而不是关于任何一个具体工具的事实。

## The Community of Practice · 实践共同体

A second reflection concerns the community of practice that is forming around the architecture this book has been teaching. The community is real, growing, and largely informal. Developers are sharing their configurations

第二条反思,是关于"围绕本书所教架构正在形成中的实践共同体"。这个共同体是真实存在的、正在成长的、很大程度上是非正式的。开发者们在分享自己的配置——

← 上一节

下一节 →

on GitHub, writing blog posts about their patterns, recording short videos of their environments in action, and trading components in private Slacks and Discords. The community is not centralized; there is no single forum that captures all of it, and no single voice that speaks for it. The decentralization is a strength, because it means that good ideas surface from many different directions and the practice evolves through real use rather than through pronouncement. The reader who wants to participate has many entry points. She can publish her own configuration. She

can contribute components to the marketplace. She can write about what has worked for her. She can simply adopt other developers' ideas and incorporate them into her own environment. The community rewards participation, even modest participation, with both reciprocal benefit and a sense of belonging to something that is moving in real time.

在 GitHub 上分享、写 blog 总结自己的范式、录短视频展示自己环境实际跑起来的样子、在私有的 Slack 和 Discord 里交换组件。这个共同体不是中心化的——没有哪个单一论坛能装下它全部,也没有哪个单一声音能代表它。分散正是它的力量所在:好点子会从很多不同方向冒出来,这套实践是经由"真实使用"演化的,而不是靠"权威宣告"。读者想参与,入口很多:可以发布自己的配置;可以向市场贡献组件;可以写文章讲自己踩过什么;也可以只是把别人的想法接过来,嵌进自己的环境。这个共同体回报"参与"——哪怕是 modest 的参与——以双向的收益、以及对"自己在参与一件正在实时向前推进的事"的归属感。

A third reflection concerns contribution upstream, to Anthropic and to the broader tooling ecosystem. The patterns this book has been teaching are not fixed. They are evolving, and the people who use them daily are the most likely to discover the patterns that should evolve next. A developer who notices a recurring failure mode in subagent design, who finds a hook event that does not exist but should, who realizes that some category of skill would benefit from a feature the format does not yet support, has the standing to surface that observation. The right channels include filing issues on the relevant tools' GitHub repositories, writing publicly about the observation, and engaging with the developer-relations teams who watch the community. Many of the features that have shipped in Claude Code over the past year originated in observations from real users. The reader who has worked through this book and developed a serious practice has the standing to contribute to the next round of those observations, and the contribution is not abstract: it shapes the tools she will be using a year from now.

第三条反思,是关于"向上游——Anthropic 和更广工具生态——做贡献"。本书一直在教的范式不是定死的;它们在演化;而每天都在用这些范式的人,最有可能发现"下一步该往哪里演化"。一位开发者,如果在 subagent 设计中注意到反复出现的失效模式,或者发现"某个 hook 事件本该有但还没有",或者意识到"某一类 skill 会受益于一个这个格式目前还不支持的能力",她就有资格把这种观察推到台面上。合适的渠道包括:在相关工具的 GitHub 仓库提 issue,把这些观察写出来公开发表,以及和"注视着这个共同体"的开发者关系团队互动。过去一年里 Claude Code 推出的许多功能,源头就是真实用户提出来的观察。认真读完本书、且已形成严肃实践的读者,有资格去贡献下一批观察;而这种贡献不是抽象的——它会塑造她明年会用到的那种工具。

A fourth reflection concerns the longevity of the architecture itself. The framework in this book has been described in terms of one specific tool, Claude Code, and one specific moment in time, the spring of 2026. The tool will change. The moment will pass. The framework, however, is durable, because it is rooted in a deeper truth about how AI-assisted development naturally divides into knowledge, specialization, and enforcement. Any future tool that handles AI-assisted work will, by the structure of the work itself, present similar concerns, and the framework will translate into

第四条反思,是关于"这套架构本身的耐久性"。本书里讲的框架,是围绕一个具体工具(Claude Code)、一个具体时点(2026 年春)展开的。工具会变;时点会过去。但框架本身是耐久的,因为它扎根于一条更深的真理:AI 辅助开发天然地分裂为"知识、专门化、强制"这三块。任何一个未来做 AI 辅助工作的工具,只要工作本身的结构没变,就会呈现同样的关切;而这套框架,会被翻译成——

← 上一节

下一节 →

whatever vocabulary that future tool adopts. A reader who has internalized the architecture is, in this sense, learning a posture that will outlast the specific tool the book was written for. The skills she develops in shaping her Claude Code environment are the same skills she will use to shape whatever tool replaces it, and whatever tool replaces that, indefinitely. The investment is not in Claude Code specifically. The investment is in the practice of engineering one's relationship with one's primary tooling, and the practice transfers.

那个未来工具所采用的任何词汇表。在这个意义上,一位把架构内化于心的读者,所学的是一种姿态——这种姿态会比"本书为之写作的那款具体工具"更耐久。她在塑造自己 Claude Code 环境时练就的这些技能,也是她用来塑造"取代它的那款工具",以及"再取代那款的工具"——一直接力下去——的同一些技能。这份投入,不是投给"Claude Code"这一个具体对象;而是投给"把自己和主力工具的关系工程化"这门实践。这门实践是可迁移的。

The reader who has worked through the book has, at this point, the framework, the patterns, and the practical libraries to build a substantial Claude Code environment for her own work. The next steps are her own. She will install the components from the chapters that fit her work most directly. She will adapt the others to her own preferences. She will build her own components for the frictions the book did not anticipate. She will commit her configuration, share what generalizes, and refine what fits only her. She will, over the months ahead, end up with an environment that no off-the-shelf product could produce, because it has been shaped by her actual work and her actual preferences, and the shaping has been deliberate.

走完本书的读者,到这里手里已经握有:框架、范式、可立即上手的实践库——可以为自己的工作搭出一份有分量的 Claude Code 环境。接下来的每一步,都是她自己的:她会那些"最贴合自己工作"的章节里的组件装上;把另一些按自己偏好改一改;为那些"本书没有料到"的摩擦,从零搭自己的组件;把配置 commit 进去,把"普适的"那部分分享出去,把"只合自己"的那部分精修;在接下来几个月里,一步步长出一份"任何现成产品都给不出来"的环境——因为它是按她实际的工作、实际的偏好被有意识地塑形出来的。

A short word about pacing the build. New readers sometimes feel an urgency to install everything in the book during the first weekend. The urgency is understandable but probably misplaced. The components that have been demonstrated across the chapters were chosen because they cover common needs, but no individual reader needs all of them on day one. The pacing that has worked best in practice is to install a small starter set in the first week, perhaps three skills, two subagents, and two hooks, and then let real use surface which additional components are worth building. A component installed in response to a felt friction is almost always more useful than a component installed because the book described it. The book is a catalog. The reader's real work is the curriculum. Trusting the curriculum, and letting the catalog inform it rather than dictate it, produces an environment that fits the reader's actual work tightly. Trusting the catalog, and forcing the curriculum to match it, produces an environment that fits no actual work in particular.

关于"搭的节奏",说几句。新读者有时会有一股冲动,想在一个周末把书里所有东西都装上。这股冲动可以理解,但多半放错了位置。各章里演示的组件,是按"覆盖常见需求"来挑的——但任何具体读者,都不需要在第一天就全装上。实践中真正管用的节奏是:第一周先装一份"小型启动包"——比如 3 个 skills、2 个 subagents、2 个 hooks——然后让真实使用告诉你"哪些额外组件值得去搭"。一个"因为感受到了某个摩擦才装"的组件,几乎总比"因为书里描述了它就装"的组件更有用。本书是 catalog;读者真实的工作才是 curriculum。信 curriculum、让 catalog 来充实它而不是支配它,会造出一份紧贴读者实际工作的环境;信 catalog、硬把 curriculum 拗成和它一致,会造出一份"贴合任何具体工作都不太对"的环境。

← 上一节

下一节(收尾)→

## The Rhythm of Building · 搭建的节奏

A related observation about the rhythm of building is worth recording. The first month of building is the period in which the most components get added, because the developer is encountering frictions she has noticed for years and finally has the tools to address. The second and third months see fewer additions but more refinement, as the components installed in the first month settle into actual use and reveal their rough edges. By the sixth month, the developer is mostly maintaining and occasionally extending, and the environment has reached a kind of steady

state in which it fits her work tightly. This rhythm is normal and healthy. A developer who is still adding components at the same rate after six months has either an unusually broad and complex working life or, more often, has fallen into the over-engineering trap that this book has warned about repeatedly. The right signal that the build is reaching maturity is that new components become rare, refinements become small, and the environment recedes into the background of the developer's daily work, doing its job without drawing attention.

关于"搭建的节奏",还有一条相关的观察值得记下。搭建的第一个月,是"加组件"加得最猛的那段时间——因为开发者正在遇上她多年来注意到、但直到现在才终于有工具去解决的摩擦。第二、第三个月,加的就少了,改的多了——第一个月装上的那些组件,在真实使用中沉淀下来,露出它们粗糙的边。第六个月时,开发者基本进入"维护 + 偶有扩展"的状态;环境已经达到一种稳态,紧密贴合她的工作。这种节奏是正常且健康的。一个开发者,六个月之后还在以同样的速度加新组件——要么她有着异常宽广和复杂的工作生活,要么更常见的,是已经掉进了"过度工程化"那个本书反复警告的坑。"搭建正在走向成熟"那条正确的信号是:新组件变少、精修变小、环境慢慢退到开发者日常工作的背景里去——做它该做的事,不再被注意。

A final word, before this book ends. The developers who invest in their tooling have always had an advantage over the developers who do not. The advantage was real with text editors. It was real with shells. It was real with build systems. It is real now with AI-assisted development environments, and the advantage is, if anything, larger, because the leverage of well-engineered configuration has grown as the tools themselves have grown more capable. The reader who finishes this book and begins her own building has, in the language of investments, made an asset allocation. The asset is her tooling. The expected return is compound, and it begins paying out from the first session in which the configuration affects her work. There is no urgency to build everything at once. There is only the discipline to build deliberately, to use what is built, and to let the environment grow alongside the work it is meant to serve. The reader who follows that discipline will find, in a year or two, that her environment has become something she would not trade for any other configuration in the world, because it has become hers.

本书收尾前的最后一句。在工具上投入的开发者,一直就比不投入的有优势。在文本编辑器上是;在 shell 上是;在 build 系统上是。在 AI 辅助开发环境上仍然如此——而且优势,如果有什么变化的话,是更大了:因为工具本身越来越能打,精心工程化的配置所产生的杠杆也越来越大。读到这里、合上本书、开始自己搭的读者,用投资的话说,已经做完了一笔资产配置。资产就是她的工具;预期收益是复利的;从"配置第一次影响她的工作"那一 session 开始,这份收益就开始兑付了。不必急着一次性搭完;真正要守的纪律只有一条:有意识地搭,把搭好的用起来,让环境跟着它要服务的工作一起长大。守住这条纪律的读者,会在一两年后发现自己手里那份环境,已经是她不愿和世界上任何其他配置交换的东西——因为它已经长成了她。

The book ends here. The work begins.

书到这里收束;工作,才刚要开始。

*Appendices · 附录*  
*Reference, Troubleshooting, and Beyond the Three Pillars*  
*参考、排查,以及"三柱之外"*

APPENDICES · 附录

**Reference, Troubleshooting, and Beyond the Three Pillars**

**参考、排查,以及"三柱之外"**

The book ended one chapter ago. What follows is what you reach for.

## APPENDIX A · 附录 A

### Troubleshooting and Debugging

### 排错与调试

*"The first half of solving a problem is being able to look at it."*

— a debugger of long standing, anonymous

*"解决问题这件事,前半段在于你能正面看着它。"*

—— 一位资深 debugger(匿名)

Real Claude Code environments break in predictable ways. A skill does not fire when it should. A subagent produces generic output that ignores the project's conventions. A hook silently fails to run. A pipeline behaves correctly in one developer's environment and inexplicably differently in another. None of these failures is mysterious once the developer has a debugging procedure in working memory; each of them traces back to one of a small number of root causes, and each of those causes has a clean diagnostic path. This appendix gives the reader the diagnostic paths.

真实的 Claude Code 环境,会以"几种可以预判"的方式坏掉:一个 skill 在该触发时没触发;一个 subagent 产出了忽略项目规约的通用输出;一个 hook 静默地没跑;一条流水线在一个开发者的环境里跑得对、在另一个开发者的环境里却表现得出奇地不一致。这些失败,只要开发者手头有"一套排查流程"挂在工作记忆里,就没有一个是神秘的——每一种都能回溯到数量不多的几种根因之一,而每一种根因都有一条干净的诊断路径。这份附录,就把这些诊断路径交到读者手里。

The procedures below assume the reader has worked through Parts Two, Three, and Four of the book and is familiar with the underlying mechanics of skills, subagents, and hooks. None of the procedures requires advanced shell knowledge, but each of them does require the reader to be willing to spend a few minutes verifying assumptions one at a time rather than trying to fix everything at once.

下面这套流程,假设读者已经走完本书的第二、三、四部分,熟悉 skill、subagent、hook 的底层机制。每一条流程都不需要高级的 shell 知识;但每一条都需要读者愿意"一次一个、逐项验证假设"——而不是一次性把什么都试着修一遍。

### Skill Problems · Skill 类问题

The most common skill failure mode is that the skill does not fire when the developer expects it to. The diagnostic procedure has four checkpoints, and the developer should walk them in order.

Skill 最常见的失效模式是"该触发时没触发"。诊断流程有 4 个检查点;读者应当按顺序走过。

First, verify the skill is installed in a location Claude Code looks at. The two valid locations are the user-level skills directory ( `~/ .claude/skills/` ) and the project-level skills directory ( `.claude/skills/` ) at the root of the current project. A skill installed anywhere

else will not be discovered.

第一,确认 skill 装在 Claude Code 会去找的位置。两个有效位置:user 级别的 skills 目录( `~/.claude/skills/` ),以及当前项目根目录下的 project 级别 skills 目录( `.claude/skills/` )。装在其他位置的 skill,不会被发现。

Second, verify the skill folder contains a file literally named `SKILL.md`, with the exact capitalization. A file named `skill.md` or `Skill.md` will not be discovered on case-sensitive filesystems.

第二,确认 skill 文件夹里恰好存在一个叫 `SKILL.md` 的文件——大小写必须严格一致。在大小写敏感的文件系统上,叫 `skill.md` 或 `Skill.md` 的文件,不会被发现。

Third, verify the YAML frontmatter is well-formed. The frontmatter is the block at the very top of the file fenced by three hyphens on a line of

第三,确认 YAML frontmatter 格式完好。Frontmatter 是文件最顶上、由一行"三个连字符"围起来的那段——

← 上一节

下一节 →

their own. If the fence is missing, indented, or includes characters that the YAML parser rejects, the skill will be silently skipped at session start.

如果围栏丢了、缩进了,或者其中包含了 YAML 解析器会拒绝的字符,那么这个 skill 在 session 启动时会被静默跳过。

Fourth, verify the description field contains language that matches the kind of request the developer is making. This is the failure mode in roughly nine cases out of ten. The fix is to expand the description with the phrasings the developer actually uses, following the four-part framework from Chapter Six (canonical statement, trigger language, boundary, input expectation).

第四,确认 description 字段里装着"和你提出的请求"匹配的语言。十次里有大致九次,问题就出在这里。修法是:把开发者实际会用的那些措辞,按第六章的四段式框架(canonical statement、trigger language、boundary、input expectation)扩进 description。

A useful diagnostic technique when the skill is installed and the description appears reasonable but the skill still does not fire is to ask Claude directly. Type into the session: *why did the [skill-name] skill not fire on my last request*. Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

当 skill 装上了、description 看起来也合理、但就是不会触发——一条很管用的诊断技巧是:直接问 Claude。在 session 里敲:为什么 `[skill-name]` 这个 skill 在我上一条请求上没触发。Claude 通常会给出一条有用的回答,把"请求和 description 之间的差距"点出来。这条回答本身,就是一段精确的"该往 description 里加什么"的指示。改完,重启 session 让改动生效,再试。两三轮之内,几乎所有的触发问题都能被"精修 description"这一招搞定。

A second skill failure mode is over-firing. The skill fires on requests that should not have triggered it. The cause is almost always a description that is too broad, lacking boundary language. The fix is to add explicit exclusions to the description, naming the kinds of requests that look superficially similar to the skill's territory but should not activate it.

Skill 的第二种失效模式是"过触发"——在不该触发的请求上也触发了。根因几乎总是 description 写得太宽,缺了"boundary"那一段。修法是:在 description 里加显式的"排除项",把"看起来和这个 skill 的领域表面相似、但不该触发它"的那几类请求点名写出来。

A third failure mode is correct firing but incorrect output. The skill activates as expected, but what it produces does not match what the body specifies. This is almost always a sign that the body is ambiguous in a way the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Side-by-side good and bad examples are extraordinarily effective for fixing this.

第三种失效模式是"触发对了,但输出不对"。Skill 确实按预期激活了,但产出的内容跟 body 的描述对不上。这几乎总是"body 里有某处歧义是写它的人当时没注意到的"那条信号。修法是:像第一次读到这份 body 一样去读它,找到"指令和输出开始分叉"的那一处,把语言收紧。在修这种问题上,"好例子 + 坏例子"并列摆出来,效果异常好。

### Subagent Problems · Subagent 类问题

Subagent problems share a structure with skill problems, with one important addition: subagents have a system prompt that does most of the shaping work, and most subagent failures trace back to that system prompt being too vague.

Subagent 的问题,结构上和 skill 的问题是同一族——多一项:subagent 有一个 system prompt 承担了大部分"塑形"工作,大多数 subagent 失败,根因都是"system prompt 太含糊"。

If the subagent does not get dispatched at all, the cause is almost always a description that does not match the request. The diagnostic is identical to the skill triggering procedure. Type the failed request again, then ask Claude why the subagent was not dispatched, and refine the description based on the answer.

如果 subagent 压根就没被派出去,根因几乎总是 description 和请求不匹配。诊断流程和 skill 触发那条一模一样——把失败的请求再敲一次,然后问 Claude 这次为什么没派;再根据它的回答去精修 description。

If the subagent gets dispatched but produces generic output, the cause is almost always a system prompt that gives the subagent too much latitude. The fix is to add specificity: name the method as a numbered procedure, name the output format down to the section headings, and add a *what-you-do-not-do* section that explicitly forbids the failure modes the developer has already encountered.

如果 subagent 被派出去了,但产出的是通用化输出,根因几乎总是 system prompt 给了 subagent 太大的自由空间。修法是:补具体性——把方法用编号化的步骤写出来,把输出格式连"小节标题"都点名定下来,再加一段 *what-you-do-not-do*(你不做什么),把开发者已经踩过过的失效模式显式禁掉。

If the subagent gets dispatched and produces output in the right shape but with the wrong substance, the cause is often a missing skill. The subagent has the role definition but not the project-specific knowledge. The fix is either to add a skill that encodes the project knowledge (which the subagent will then load when relevant) or, less ideally, to put the project knowledge directly into the subagent's system prompt.

如果 subagent 被派出去了,产出"形状对、但内容不对"——根因常常是"缺一条 skill"。Subagent 拿到了"角色定义",但没拿到"项目特有的知识"。修法是:写一条新 skill 把这些项目知识编码进去(子代理会在相关时去加载它);或者退而求其次,把这些项目知识直接塞进 subagent 的 system prompt。

If the subagent does not respect its tool restrictions, the cause is usually a typo in the `tools` field of the frontmatter. The values are case-sensitive and must match the tool names exactly: `Read`, `Write`, `Edit`, `MultiEdit`, `Bash`, `Grep`, `Glob`. A typo silently grants the full tool set.

如果 subagent 不遵守它的工具限制,根因通常是 frontmatter 的 `tools` 字段里某个值打错了。这些值是大小写敏感的,必须和工具名精确匹配: `Read`、`Write`、`Edit`、`MultiEdit`、`Bash`、`Grep`、`Glob`。一个拼错,会在静默中"开闸"到"全部工具"。

A useful introspection technique is the `/agents` slash command. Run it in a session; it lists every subagent available, shows the description and tool restrictions, and confirms the file is being read correctly. If the developer expects a subagent to exist and `/agents` does not show it, the problem is at the file or YAML level rather than at the description level.

一条很有用的"自省"技巧是 `/agents` 这个 slash command。在 session 里跑一下,它会列出所有可用的 subagent,显示 description 和工具限制,并确认该文件正在被正确读取。如果开发者预期某个 subagent 存在,但 `/agents` 没列出来——那么问题出在文件层或 YAML 层,而不是 description 层。

## Hook Problems · Hook 类问题

Hook problems are the most subtle of the three categories, because hooks fail silently more often than they fail loudly. A misconfigured hook does not produce a visible error in most cases; it simply does not run. The four-step diagnostic procedure for hooks is worth committing to memory.

Hook 类问题,是三类问题里最微妙的——因为 hook 经常"静默"地失败,而不是"响亮"地失败。配错的 hook 多数情况下不会产生可见错误,只是不跑而已。Hook 的 4 步诊断流程,值得背下来。

First, verify the hook is loaded. Run the `/hooks` command inside a Claude Code session. The interactive editor lists every event and the hooks bound to each one. If the new hook does not appear, the configuration file

第一,确认 hook 被加载了。在 Claude Code session 里跑 `/hooks` 这个命令。交互式编辑器会列出每一个事件、以及绑定到该事件上的所有 hook。如果新加的 hook 没出现在列表里,那么配置文件——

← 上一节

下一节 →

has a problem, usually a JSON syntax error or a misnamed event. Many hook problems disappear at this step.

本身有问题——通常是个 JSON 语法错误,或事件名写错。很多 hook 问题,走到这一步就消失了。

Second, verify the hook is firing. Add a small log statement to the hook itself, writing a timestamp to a file every time the hook fires. Make an action that should trigger the hook. Check the log file. If the log file has no new entry, the matcher is wrong; the hook is bound to an event or tool name that does not match what is happening. Adjust the matcher and try again.

第二,确认 hook 真的"触发"了。给 hook 自己加一行小日志——每次 hook 触发,就往一个文件里写一个时间戳。然后做一次"本应触发这个 hook"的动作,再去查看那个日志文件。如果日志里没新条目,说明 matcher 配错了——hook 被绑定到了一个和实际发生动作对不上的事件名或工具名。改 matcher,再试。

Third, verify the hook is succeeding. Read the hook's standard output and standard error in the log. If the hook is exiting with a non-zero code due to a missing tool or an unexpected JSON shape, fix the cause. A common case is that a tool the hook depends on (like `jq` or `npx`) is not installed on the developer's system, or has been moved out of the path.

第三,确认 hook 是"成功"完成的。读 hook 的标准输出和标准错误日志。如果 hook 因为"依赖工具缺失"或"JSON 形状对不上"而退出码非 0,就去修根因。常见情况是:hook 依赖的某个工具(`jq` 或 `npx` 之类)没装,或者已经不在 PATH 上了。

Fourth, verify the effect of the hook. The hook may be firing, succeeding, and still not producing the intended outcome because of a logic error in the script or because the hook's output is not being interpreted as expected. For hooks that emit JSON to control the lifecycle, validate the JSON shape against the documentation before assuming the hook is broken.

第四,确认 hook 的"效果"。Hook 可能"触发了""成功了",却仍没产出预期结果——根因可能是脚本里的逻辑错,或 hook 的输出没有被按预期解读。对于那些"靠吐 JSON 来控生命周期"的 hook,在怀疑它坏了之前,先按文档核一下 JSON 的形状。

The `/doctor` slash command is worth knowing about as a complementary diagnostic. It reports on the current installation and surfaces common configuration problems including malformed settings files, missing dependencies that hooks rely on, and version mismatches. Running `/doctor` at the start of any debugging session gives the developer a quick read on whether the problem is local to her configuration or environmental.

`/doctor` 这个 slash command 也值得记一下,作为互补的诊断手段。它会报告当前安装状态,主动把"常见的配置问题"摆到台面上:格式有误的 settings 文件、hook 依赖的缺失组件、版本不匹配等。在任何一次排错的开头先跑一遍 `/doctor`,能让开发者快速判断"问题在她自己的配置里"还是"在环境那一层"。

A specific subtlety worth naming: hook configuration changes do not affect a running session. After editing the hooks file or any `settings.json`, the developer must restart Claude Code or open a new session for the change to take effect. Many otherwise-correct hook configurations appear broken because the developer is testing them in the same session in which she made the change.

还有一条具体的"暗坑"值得明说:hook 的配置变更不会影响"已经在跑的 session"。改完 hook 文件或 `settings.json` 之后,必须重启 Claude Code 或开一个新 session,改动才会生效。很多"看着对、但似乎坏了"的 hook 配置,实际是开发者在"刚改完它的那个 session 里"测的——所以新配置还没生效。

## Pipeline Problems · 流水线类问题

When several hooks compose into a pipeline, the diagnostic complexity rises but the procedure does not change. Walk each hook in the pipeline through the four-step procedure individually. The hook that fails one of the four steps is the failing component. Pipelines almost never fail because of

当几个 hook 组合成一条流水线,诊断的复杂度上升了,但流程本身没变。逐个把流水线里的每个 hook 走一遍那 4 步流程——哪一步在哪一个 hook 上挂掉,挂掉那个就是有问题的组件。流水线几乎从来不会"因为"——

← 上一节

下一节 →

pipeline-level issues; they fail because one hook in the pipeline fails, and the failure cascades.

流水线级别的问题而失败;它们失败,是因为流水线里某一个 hook 失败,而失败又沿着链路向下传播。

A second pipeline-specific concern is order. Hooks bound to the same event run in the order they appear in the configuration. If the second hook in a pair depends on the first hook having modified the file system, and the first hook is misordered or misconfigured, the second hook will run on stale state. The fix is to verify the order matches the dependency, and to add a small assertion at the start of any hook that depends on another hook's effect.

流水线专属的第二条关切是顺序。绑在同一事件上的 hook,按它们在配置里出现的顺序执行。如果"成对 hook 里的第二个"依赖"第一个"已经动过文件系统,而第一个的顺序被排错或被配错,第二个就会跑在陈旧的状态上。修法是:确认"声明顺序"和"实际依赖顺序"一致,并在每个"依赖另一个 hook 副作用"的 hook 开头加一个小断言。

## APPENDIX B · 附录 B

### Hook Events Quick Reference

### Hook 事件速查

*"The reference is the part you reread. The argument is the part you read once."*

— a textbook author, anonymous

"参考是你回头再读的那部分;正文是你读一遍就够的那部分。"

—— 一位教科书作者(匿名)

This appendix is a compact reference to every hook event Claude Code currently supports, what each event fires for, what input it receives, and the most common pattern of use. The reference is intended to be looked up rather than read straight through. Skim it once to learn what is here, and then return to it when designing a new hook to verify which event fits.

这份附录,是 Claude Code 当前支持的所有 hook 事件的一份紧凑参考:每个事件在什么时点触发、收到什么输入、最常见的使用形态。参考是用来"查"的,不是用来"从头读到尾"的;先通览一遍知道里头有什么,设计新 hook 时再回来核对"哪个事件最贴"。

Event · 事件	Behavior, input, and use · 行为、输入与典型用法
PreToolUse	<b>Fires:</b> 紧接 Claude 调用一个工具之前触发,此时工具名和输入已可用。 <b>Input:</b> JSON 对象,包含工具名、工具输入以及 session 元数据。 <b>Use for:</b> 用于"门"——阻断、修改、或延后工具调用。可用于安全策略强制、危险命令拦截、文件路径保护。 <b>Exit 2:</b> 阻断这次工具调用。标准错误输出会被作为反馈回给 Claude。另外,通过 JSON 输出还能拿到 4 种更丰富的结局:allow / deny / ask / defer。
PostToolUse	<b>Fires:</b> 一个工具调用成功完成后立即触发。 <b>Input:</b> JSON 对象,包含工具名、输入、以及调用结果。 <b>Use for:</b> 用于"守卫"——强制一致性:formatter、linter、edit 后校验、对 Claude 每一个动作的审计日志。 <b>Exit 2:</b> 此刻无法阻断(动作已经发生);标准错误输出会作为反馈回给 Claude,可在下一轮触发修正工作。
UserPromptSubmit	<b>Fires:</b> 用户提交一条 prompt 时,Claude 处理它之前触发。 <b>Input:</b> JSON 对象,包含 prompt 文本和 session 上下文。

UserPromptSubmit (续)	<b>Use for:</b> prompt 级别的过滤、可选地往里注入额外上下文,以及对"哪些请求被允许"做策略强制。 <b>Exit 2:</b> 整条 prompt 被拒。用户会看到拒绝消息。
Stop	<b>Fires:</b> Claude 一次响应生成完毕时触发。 <b>Input:</b> JSON 对象,包含 Claude 准备停下的那一刻的 session 状态。 <b>Use for:</b> 回合末端的校验器、session 摘要写入器、回合末端的审计日志。 <b>Exit 2:</b> 拒绝停。Claude 被强制继续工作,直到一次 stop 真正成功。
SubagentStop	<b>Fires:</b> 一个 subagent 完成任务时触发,刚好在它的结果回到主 session 之前。 <b>Input:</b> JSON 对象,包含 subagent 名字和它的最终输出。 <b>Use for:</b> 把 subagent 的输出按"预期形状"做一遍校验、记录 subagent 的活动。 <b>Exit 2:</b> 拒绝该 subagent 的 stop。请谨慎使用。
SessionStart	<b>Fires:</b> 新 session 启动,或一份被保存的 session 被恢复时触发。 <b>Input:</b> JSON 对象,包含 session 元数据。 <b>Use for:</b> 把"环境级"的项目状态加载进 session 的起始上下文里。 <b>Exit 2:</b> 写到 stdout 的输出会被注入到 Claude 的起始上下文里。
SessionEnd	<b>Fires:</b> 一个 session 终止时触发。 <b>Input:</b> JSON 对象,包含 session 元数据和最终状态。 <b>Use for:</b> session 级别的审计日志、日报生成、清理任务。 <b>Exit 2:</b> 此刻没有"阻断"效果——session 本来就要结束,无论结果如何。

Notification	<b>Fires:</b> Claude 需要提醒用户某件事时触发。 <b>Input:</b> JSON 对象,包含 notification 的 payload。 <b>Use for:</b> 弹桌面通知、把提醒发到聊天平台。 <b>Exit 2:</b> 不会影响生命周期。Notification hooks 是"通知器"型 hook 的规范示例。
PreCompact	<b>Fires:</b> Claude 即将压缩上下文以腾出空间之前触发。

A practical guideline for choosing among these events: the event whose name most directly matches the moment when the work needs to happen is almost always the right one. `PreToolUse` for prevention. `PostToolUse` for correction. `Stop` for end-of-turn validation. `SessionStart` for ambient state injection. With practice, the right event becomes obvious at the moment the developer recognizes which kind of work she is trying to enforce.

在这些事件之间做选择,有一条实用的经验法则:事件名和你想干活的时点最直接对应的那一个,几乎永远是对的。要"事前拦"→ `PreToolUse` 。要"事后改"→ `PostToolUse` 。要"回合末端校验"→ `Stop` 。要"环境级状态注入"→ `SessionStart` 。练到一定熟练度,只要意识到"自己想强制的是哪一类工作",对的事件会一眼就跳出来。

PreCompact (续)	<b>Input:</b> JSON 对象,包含当前上下文状态。 <b>Use for:</b> 在重要上下文即将被压掉之前,把它"捞"出来,写一份检查点摘要。 <b>Exit 2:</b> 写到 stdout 的输出会被保留进压缩后的上下文。
----------------	----------------------------------------------------------------------------------------------------------------------------------

注:以上是 Hook 事件速查表的最后一项( `PreCompact` 的 `Input` / `Use for` / `Exit 2` 三个字段)。整张速查表覆盖了 9 个事件: `PreToolUse` 、 `PostToolUse` 、 `UserPromptSubmit` 、 `Stop` 、 `SubagentStop` 、 `SessionStart` 、 `SessionEnd` 、 `Notification` 、 `PreCompact` 。

注:以上是 Hook 事件速查表的最后一项( `PreCompact` 的 `Input` / `Use for` / `Exit 2` 三个字段)。整张速查表覆盖了 9 个事件: `PreToolUse` 、 `PostToolUse` 、 `UserPromptSubmit` 、 `Stop` 、 `SubagentStop` 、 `SessionStart` 、 `SessionEnd` 、 `Notification` 、 `PreCompact` 。

APPENDIX C · 附录 C

Beyond the Three Pillars: Plugins and Agent Teams

三柱之外:Plugins 与 Agent Teams

"The framework is the foundation. What sits on top of the foundation is the work of the next decade."

"框架是地基;建在地基之上的部分,是未来十年要做的工作。"

—— 一位软件架构师(匿名)

The three pillars covered in this book, skills, subagents, and hooks, are the durable core of Claude Code customization, and they are sufficient on their own to build the engineered environments that the case studies in Part Five demonstrated. They are not, however, the entirety of what Claude Code now supports. Two additional capabilities have emerged in the more recent releases that are worth knowing about, both because they extend the architecture in useful directions and because the developer who is fluent in the three pillars will find the additions easy to absorb. This appendix introduces them at a level sufficient to recognize when they apply and to know where to look for deeper coverage.

本书讲过的三柱——skills、subagents、hooks——是 Claude Code 定制化的"耐久核心";单凭这三柱,已经足以搭出第五部分案例里展示的那些工程化环境。但它们并不是 Claude Code 现在支持的全部。在较新的几次发布里,有两项额外的能力冒出来了,值得知道——是因为它们在有用的方向上扩展了这套架构,二是因为一位"对三柱已经很熟"的开发者去吸收它们,会发现上手很轻。这份附录把它们介绍到一个"能认出来什么场景该用、并知道去哪里查更深入资料"的水平。

The first additional capability is the plugin system. The second is agent teams.

第一项额外能力,是 plugin 系统;第二项,是 agent teams。

## Plugins · Plugins

A plugin, in the Claude Code sense, is a packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The plugin system formalizes a pattern that this book has already taught informally: the developer who builds a coherent set of components for a specific kind of work is, in effect, building a plugin, even if she has not packaged the components for distribution. The plugin system gives that pattern a formal home.

Plugin,放在 Claude Code 的语境里,就是一份"打包好的集合"——里面装着 skills、subagents、hooks 以及配套的资源——可以作为一个整体来安装。Plugin 系统把本书里已经以一种非正式方式教过的一种范式,正式化了:开发者如果为某类特定工作搭出一组"内部自治"的组件,事实上就等于在搭一份 plugin——即使她还没把这些组件打包成可分发的形式。Plugin 系统,正是给这种范式一个正式的家。

A plugin is a folder that contains a manifest file describing what the plugin provides, the components themselves arranged in their conventional subfolders, and any documentation the author wants to ship with it. The plugin can be installed from a local path, from a Git repository, or from a registry, and once installed it appears alongside the developer's hand-built

Plugin 是一个文件夹:里面有一份 manifest 文件,描述这份 plugin 提供了什么;有按规约子目录摆放的组件本身;再有作者愿意随它一起发出去的文档。Plugin 可以从本地路径装,可以从 Git 仓库装,也可以从一个 registry 装;装上之后,它就和开发者手搭的配置并列出现——

← 上一节

下一节 →

configuration. The plugin's components are activated through the same triggering and dispatch mechanisms covered in Parts Two through Four: skills fire when their descriptions match, subagents are dispatched when their roles fit, hooks bind to lifecycle events. From the model's perspective, a plugin is invisible; the components inside the plugin behave exactly as components installed directly would behave.

和开发者手搭的配置并列出现。Plugin 里的组件,激活方式和第二至第四部分讲过的触发与派发机制一致:description 匹配时 skill 触发,角色对得上时 subagent 被派,生命周期事件和 hook 绑定。从模型的角度看,plugin 是"看不见"的——plugin 里的组件,行为和直接装进环境的组件完全一致。

The strategic value of plugins is twofold. The first value is distribution. A developer who has built a thoughtful environment for a specific niche, such as technical writing, financial analysis, or React development, can package it as a plugin and share it with peers or sell it. The second value is consumption. A developer who recognizes that her work overlaps significantly with an existing plugin can install the plugin and inherit its components, saving herself the time of building the same components from scratch. Both ends of the exchange are useful, and the plugin ecosystem is in the early phase of populating with real, useful packages.

Plugin 的战略价值是双重的。第一重价值是"分发":一位开发者,只要为某个具体 niche(比如技术写作、财务分析、React 开发)搭好了一份深思熟虑的环境,就可以把它打包成 plugin,拿去和同行分享,或者出售。第二重价值是"消费":一位开发者,只要意识到"自己的工作"和某份现成 plugin 高度重叠,就可以把那份 plugin 装下来,继承它的组件,省下自己从零搭同样组件的时间。交换的两端都有用;而 plugin 生态正处在"开始装进真实、有用包"的早期阶段。

The reader who finishes this book and wants to participate in the plugin ecosystem has two natural entry points. The first is to package one of her own most successful sets of components as a plugin and publish it. The second is to browse what is already available, install one or two plugins that fit her work, and observe how the plugin author shaped the components to fit a niche. Either approach develops the fluency that makes the plugin system a real extension of the developer's environment rather than a separate concern.

走完本书、想要参与 plugin 生态的读者,有两条自然的入口。第一条:把"自己手上最成功的那组组件"打包成一份 plugin,发出去。第二条:去浏览已经上架的那些,挑一两个贴合自己工作的装上,再观察 plugin 作者是怎么把组件塑形去贴合某个 niche 的。两条路,任一条走下去,都会让 plugin 系统从"另一件和你的环境无关的事"变成"你环境的真正延伸"——这种熟稔是练出来的。

## Agent Teams · Agent Teams

The second extension is agent teams. The orchestrator pattern from Chapter Twelve introduced the idea of dispatching several subagents in parallel and synthesizing their outputs. Agent teams formalize that pattern. An agent team is a named, configured group of subagents that work together on a single class of task, with explicit coordination logic that specifies how the team communicates and how the final output is assembled.

第二项扩展,是 agent teams。第十二章的 orchestrator 范式讲过一种"并行派出多个 subagent、再综合它们的产出"的做法;agent teams 把这种范式正式化了。一个 agent team 是一组"有名字、配过"的 subagent,它们合作处理某类单一任务;团队里有显式的协调逻辑,规定团队内部怎么通信、最终输出怎么组装。

A team differs from an ad-hoc parallel dispatch in two respects. First, the team is a named, persistent configuration; the developer defines it once and then dispatches the entire team with a single invocation, rather than orchestrating individual subagents on every request. Second, the team can

一支 team 和"临时的并行派发"相比,有两点不同。第一,team 是一个"有名字、可持久化"的配置;开发者只需定义一次,之后用一次调用就能派出整支 team,而不是每次请求都要手工编排那批 subagent。第二,team 可以——

include explicit communication patterns between its members. A simple team is the parallel-fan-out pattern from Chapter Twelve, where each subagent contributes an independent perspective and a synthesizer combines them. A more elaborate team can have sequential dependencies, with one subagent's output becoming the input to the next, or even feedback loops where a reviewer subagent flags issues and a writer subagent revises in response.

包含"成员之间的显式通信模式"。一支简单的 team,就是第十二章的"并行扇出"范式——每个 subagent 提供一份独立的视角,由一个 synthesizer 合成。更复杂一点的 team,可以有"顺序依赖":一个 subagent 的输出成为下一个的输入;甚至可以有"反馈环":reviewer subagent 抛出问题,writer subagent 据此修订。

The team's capability is most valuable for workflows that the developer runs often enough to deserve their own configuration. A weekly review like the one in the solo-founder case study from Chapter Twenty-One is a natural candidate; rather than dispatching the financial-analyst, the marketing-writer, and the product-strategist subagents individually each Friday, the developer can define a weekly-review team that runs all three with a single command and produces the unified review automatically. The development cost of the team configuration is small, and the savings on every future invocation accumulate.

Team 这个能力,对那些"开发者跑得足够频繁、值得为它单写一份配置"的工作流,价值最大。第二十一章独立创业者案例里那份"周五周会回顾",就是天然的候选——与其每周五都单独派 financial-analyst、marketing-writer、product-strategist 这三个 subagent,不如直接定义一支"周会回顾" team:一条命令就把这三个跑完,自动产出一份统一的 review。搭一份 team 配置的开发成本很小;而此后每一次调用省下来的时间,会不断累积。

A reasonable rule of thumb for when to reach for an agent team rather than ad-hoc orchestration: if the same combination of subagents has been dispatched together more than three times in a month, the combination has earned its place as a named team. The team configuration captures the pattern explicitly and removes the small overhead of remembering which subagents to dispatch on every future invocation.

什么时候该伸手去抓"agent team"而不是"临时编排"?有一条合理的经验法则:同一组 subagent,如果一个月内被一起派出去超过 3 次,那这组组合就赢得了"被命名为一支 team"的资格。Team 配置把这种范式显式记下来,免去了"每次还要想一遍这次该派哪几个 subagent"那一小笔开销。

A final observation is worth making about plugins and agent teams in relation to the rest of the book. Both capabilities sit on top of the three-pillar architecture rather than replacing it. A plugin is a packaged set of skills, subagents, and hooks. An agent team is a coordinated invocation of subagents. The underlying components are the same components covered in Parts Two through Four, and the design disciplines for those components are unchanged. The reader who has internalized the three pillars will find both extensions natural to learn. The reader who has not will find both extensions confusing, because the extensions assume the underlying pillars are already understood. This is the right ordering: pillars first, extensions second, and the extensions become useful precisely because the pillars are solid.

关于 plugins 和 agent teams,值得再点一条观察:它们俩,都是"坐在三柱架构之上",而不是"取代"它。Plugin 是一组"打包好的"skills、subagents、hooks;agent team 是一次"协调过的"subagent 调用。底下的组件,和第二至第四部分讲过的是同一组;对这些组件的设计纪律也没变。已经把三柱内化于心的读者,会发觉这两项扩展都自然到几乎不用学;没把三柱内化于心的读者,则会感到这两项都很让人困惑——因为它们预设了"三柱已经懂了"这个前提。这正是正确的学习顺序:先三柱,后扩展;而扩展之所以"好用",正是三柱"够扎实"这件事带来的回报。

Both capabilities are evolving, and the official documentation is the right primary source for current details. The treatment in this appendix is meant to put the capabilities on the reader's mental map and to indicate when reaching for them makes sense. The deeper coverage will mature as the ecosystem matures, and the reader who has finished this book has the architectural foundation to absorb that coverage as it appears.

这两项能力都在持续演化;官方文档,是获取当前细节的"主信源"。本附录对它们的处理,是把它们摆到读者"心智地图"上的相应位置,并指出什么时候伸手去抓它们是合适的。更深入的覆盖,会随生态一起成熟;而读完了本书的读者,手里已经有吸收那些新覆盖所需的架构基础。

## APPENDIX D · 附录 D

### Key Terms Index

#### 关键术语索引

"A book without an index is a city without street signs."

— a librarian, anonymous

"没有索引的书,是一座没有路牌的城市。"

—— 一位图书管理员(匿名)

The index below is a curated list of the technical terms and architectural concepts the book has used most often, with a brief gloss for each one. The index is intended to serve as a quick lookup when the reader encounters a term in a later chapter and wants to recall its meaning, or when she returns to the book months later and needs to refresh her vocabulary. The list is alphabetical and the glosses are short on purpose; for the full treatment of any term, the reader should consult the chapter where the term was introduced.

下面的索引,是本书出现频率最高的技术术语和架构概念的一份精挑过的清单,每条都附有简短的释义。这份索引的用途,是供读者速查——当她在后面某一章碰到一个术语、想唤起它的含义,或者几个月后再回来、需要"刷一遍词汇表"的时候。本表按字母排序,释义刻意写得短;任何术语的完整论述,请回查"该术语最初出现的那一章"。

## A

**Agent loop.** The think-act-observe cycle that the language model executes on every turn. Skills and subagents shape what happens inside the loop; hooks shape what happens at its boundaries. See Chapter Two and Chapter Fifteen.

**Agent loop(代理循环)**。语言模型每一轮都执行的"思考—行动—观察"循环。Skills 和 subagents 塑形的是循环*内部*发生的事;hooks 塑形的是循环*边界*上发生的事。参见第二章和第十五章。

**Agent team.** A named, persistent configuration of subagents that work together on a single class of task, with coordination logic specifying how the team communicates. See Appendix C.

**Agent team(代理小队)**。一组"有名字、可持久化"的 subagent 配置,合作处理某类单一任务,内部有协调逻辑规定通信方式。参见附录 C。

**allowed-tools** . The optional frontmatter field in a `SKILL.md` file that restricts the set of tools Claude can use while the skill is active. Reaches its purpose by narrowing the model's capability surface to exactly what the skill needs. See Chapter Four.

**allowed-tools (允许工具列表)**。 `SKILL.md` 文件里一个可选的 frontmatter 字段,用来限制该 skill 激活期间 Claude 可用的工具集合。它的作用是"把模型的能力面收窄到正好是这条 skill 需要的那个范围"。参见第四章。

**Ambient versus targeted knowledge.** The architectural distinction between knowledge that is true at all times in a project (which belongs in `CLAUDE.md`) and knowledge that is true only when a specific kind of work is being done (which belongs in a skill). See Chapter Two and Chapter Eight.

**环境级 vs 触发级(Ambient versus targeted)**。一条架构上的区分:"一个项目里任何时候都为真的知识"(归 `CLAUDE.md`) vs "只在某类特定工作进行时才为真的知识"(归 skill)。参见第二章和第八章。

**.claude/** . The convention directory at the root of a project, or in the user's home folder, that holds Claude Code configuration. Inside it live the `skills/` and `agents/` subdirectories, the `settings.json` file that configures hooks,

**.claude/ 目录**。放在项目根目录、或者用户 home 目录的"约定目录",Claude Code 的配置就住在那里。里面住着 `skills/` 和 `agents/` 子目录、用来配 hooks 的 `settings.json` 文件,

← 上一节

下一节 →

and (by convention) a `hooks/` folder for the shell scripts that `settings.json` references. See Chapter Two.

以及(按约定)一个 `hooks/` 目录,装着 `settings.json` 引用的那些 shell 脚本。参见第二章。

## C

**CLAUDE.md** . The special file Claude Code reads at session start whose contents are added to the system context. The right place for ambient project knowledge that should apply to every interaction. See Chapter Two.

**CLAUDE.md**。Claude Code 在 session 启动时读入的一个特殊文件,文件内容会被加进系统上下文。它是"项目级环境知识、应当作用于每一次交互"这件事的归属地。参见第二章。

**Composition pattern.** The architectural pattern in which skills activate inside subagents, producing capability and specialization simultaneously. The deepest leverage in the three-pillar framework. See Chapter Fourteen.

**组合范式(Composition pattern)。** 一条架构范式:让 skill 在 subagent 内部被激活,从而同时获得"能力"和"专门化"。三柱框架里最深层的杠杆所在。参见第十四章。

## D

**Description.** The frontmatter field in any skill or subagent definition that determines whether the component fires or is dispatched on a given request. The single most consequential design decision in any component. See Chapter Six.

**Description(描述字段)。** 任一 skill 或 subagent 定义里 frontmatter 中的字段,决定该组件在面对某条给定请求时是否触发或被派。任何组件里"后果最严重的一条设计决策"。参见第六章。

**Deterministic outer / probabilistic inner.** The architectural pattern in which the language model operates probabilistically inside the agent loop, while hooks impose deterministic constraints at the loop's boundaries. The central design principle of the book. See Chapter Three.

**外层确定 / 内层概率(Deterministic outer / probabilistic inner)。** 一条架构范式:让语言模型在 agent loop 内部按概率性运行,而 hooks 在 loop 的边界上施加确定性约束。本书的中心设计原则。参见第三章。

## G

**Gate.** A hook design pattern in which a `PreToolUse` hook evaluates a tool call and either allows or blocks it. The mechanism by which wishes become rules. See Chapter Sixteen.

**Gate(门)。** 一种 hook 设计范式: `PreToolUse` hook 评估一次工具调用,选择放行或阻断。"愿望变成规则"的那条机制。参见第十六章。

**Guard.** A hook design pattern in which a `PostToolUse` hook runs after a tool call and enforces consistency on its result. The mechanism by which conventions become automatic. See Chapter Sixteen.

**Guard(守卫)。** 一种 hook 设计范式: `PostToolUse` hook 在一次工具调用之后跑,对其结果强制一致性。"规约变成自动"的那条机制。参见第十六章。

## H

**Hook.** A user-defined shell command that Claude Code executes automatically at a specific lifecycle event. The third pillar of the framework, providing deterministic enforcement. See Part Four.

**Hook(钩子)。** Claude Code 在某个特定生命周期事件上自动执行的一条用户自定义 shell 命令。框架里的第三柱,提供确定性强制。参

见第四部分。

**hookSpecificOutput** . The JSON output field in a hook that carries richer control instructions to Claude Code, including the four **PreToolUse** outcomes (allow, deny, ask, defer) and the modified tool input. See Chapter Fifteen.

**hookSpecificOutput (hook 专属输出)**。一条 hook 里负责把"更丰富的控制指令"送回 Claude Code 的 JSON 输出字段,包括四种 **PreToolUse** 的结局(allow、deny、ask、defer)以及被修改后的工具输入。参见第十五章。

N

**Notifier**. A hook design pattern in which a hook produces output to a system other than Claude Code itself, such as a desktop notification or an audit log entry. See Chapter Sixteen.

**Notifier(通知器)**。一种 hook 设计范式:让 hook 把输出送到"Claude Code 自己之外"的某个系统,比如桌面通知、审计日志条目。参见第十六章。

O

**Orchestrator pattern**. The multi-subagent pattern in which the main session decomposes a task, dispatches several subagents in parallel, and synthesizes their outputs into a unified result. The most common pattern of subagent composition. See Chapter Twelve.

**Orchestrator 范式**。一条多 subagent 范式:主会话拆解一个任务,并行派多个 subagent,再把它们的输出合成为一份统一结果。subagent 组合里最常用的一种形态。参见第十二章。

P

**Plugin**. A packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The formal mechanism for distributing a coherent set of components. See Appendix C.

**Plugin(插件)**。一份"打包好的"skills、subagents、hooks 以及配套资源的集合,可作为一个整体来安装。分发"一套内部自洽组件"的正式机制。参见附录 C。

**Progressive disclosure**. The loading strategy in which Claude Code reads only the descriptions of available skills at session start, then loads a skill's body only when the skill activates, and only reads bundled files when the body refers to them. The mechanism that lets large skill libraries scale without paying a context tax. See Chapter Four.

**渐进式披露(Progressive disclosure)**。一种加载策略:Claude Code 在 session 启动时只读所有可用 skill 的 description;只有在某条 skill 真正激活时,才去读它的 body;而它打包进来的文件,只在 body 引用到时才读。这条机制让大型 skill 库能扩展规模、而不用付"上下文税"。参见第四章。

## S

**Skill**. A reusable capability defined by a folder containing a `SKILL.md` file. The first pillar of the framework. See Part Two.

**Skill(技能)**。一种"可复用的能力",由一个装着 `SKILL.md` 文件的目录来定义。框架里的第一柱。参见第二部分。

**SKILL.md** . The Markdown file inside a skill folder that contains YAML frontmatter declaring the skill's metadata and a body containing the instructions Claude follows when the skill is active. See Chapter Four.

**SKILL.md** 。 skill 文件夹里的那份 Markdown 文件,顶上是一段声明该 skill 元数据的 YAML frontmatter,正文是"该 skill 激活时 Claude 要遵循的那些指令"。参见第四章。

**Subagent**. A separate Claude process with its own system prompt, restricted tool set, and isolated context window, dispatched by the main session for specific kinds of work. The second pillar of the framework. See Part Three.

**Subagent(子代理)**。一个独立的 Claude 进程:自己的 system prompt、受限的工具集、被隔离的上下文窗口,由主会话派去处理特定种类的任务。框架里的第二柱。参见第三部分。

## T

**Three-pillar framework**. The book's organizing architecture: skills for capability, subagents for specialization, hooks for enforcement. The framework that any sophisticated Claude Code environment uses in some combination. See Chapter Three.

**三柱框架(Three-pillar framework)**。本书整套论述的组织性架构:skills 负责"能力",subagents 负责"专门化",hooks 负责"强制"。任何"严肃的 Claude Code 环境"都会以某种组合形式用到它。参见第三章。

## @

**@agent-name** . The explicit invocation syntax for a subagent. Typing `@agent-name` in a prompt dispatches the named subagent immediately, bypassing description-based delegation. See Chapter Ten.

**@agent-name 语法**。显式调用 subagent 的语法:在 prompt 里敲 `@agent-name`,被点名的那个 subagent 会被立刻派出去——绕过"按 description 派发"那条路径。参见第十章。

The terms above cover the vocabulary the book has been building since Chapter One. Reading the index in one pass is a quick way to confirm that the central concepts have settled into working memory. Returning to it later, when one of the terms surfaces in a new context, is how the index earns its place in a reference book. The reader who has finished the book has, by this

上面这些术语,覆盖了本书从第一章开始一直在构建的那套词汇表。一遍通读,就能快速确认那些核心概念是否都已落入工作记忆。以后

某个术语在新的语境下冒出来时,再回来翻——这就是一份"参考书里的索引"该有的存在方式。读完本书的读者,到这里——

← 上一节

全书末页 →

point, a working command of every term above; the index is a backstop for the moments when one of them needs to be looked up rather than recalled.

已经对上面每一条术语都"可工作地掌握"了;这份索引,只是在"某条术语需要被查、而不是被想出来"那些时刻的"兜底"。

— 全书完 —